

MicroSky Horizon

ESP32-EFIS Project Reference

Indexed compilation of the current repository documentation

59 source documents

Contents

Contents	2
Document index	17
account-service/README.md	19
EFIS Account Service	19
Security boundary	19
Run	19
Current prototype API	19
Still required before production	19
Customer web portal	19
BOM.md	20
ESP32 EFIS — Bill of Materials and Purchasing Checklist	20
Procurement rules	20
Project stages	20
Purchasing status and sources	20
Order / delivery evidence	21
Stage 1 — immediate purchase/verification list	21
Stage 2 — immediate purchase/selection list	22
Stage-specific items not yet ready to order	22
Bench consumables / useful spares	22
Budget bench-power option — optional / not required to start Stage 1	22
Safety / configuration rule	23
Interface direction update — 30 September 2026	23
Remote Engine Interface Unit — planning BOM	23
Stage 7 — provisional EIU / AEF-CAN BOM addition	24
USB-C dual-role production interface — 30 September 2026	24
customer-app/android/README.md	27
EFIS Customer — Android	27
customer-app/ios/EFISCustomer/README.md	28
EFIS Customer — iOS	28
customer-app/README.md	29
EFIS Customer Mobile Apps	29
Current OTA architecture	29
docs/AHRS.md	30
AHRS / Attitude Solution	30
Purpose and current status	30
Implemented initial estimator	30
Current calculation	30
Timing and stale-data handling	30
BMI088 acquisition layer	31
Physical installation gate — not yet passed	31
Validation still required	31
Future estimator evolution	31
docs/BRANDING.md	33

Product branding	33
Working identity	33
Naming intent	33
Legal and commercial status	33
Development usage	33
Before commercial freeze	34
docs/CALIBRATION.md	35
Calibration	35
Goals	35
Gyroscope calibration	35
Accelerometer calibration	35
Axis alignment	35
Panel installation offset	35
Vibration checks	36
Stored calibration	36
docs/CAN-PROTOCOL.md	37
AEF-CAN — Aircraft Experimental Flight CAN	37
1. Purpose	37
2. Safety and operating assumptions	37
3. V1 physical/data-link baseline	37
4. Network architecture	37
5. Identifier allocation	38
6. Initial engine messages	38
7. Encoding rules	39
8. Validity and faults	39
9. Versioning and compatibility	40
10. Node identity and discovery	40
11. Producer/consumer separation	41
12. Configuration and commands	41
13. CAN FD and future transports	41
14. Machine-readable source of truth	41
15. Testing requirements	42
16. Initial implementation sequence	42
17. Design principles to preserve	42
18. V1 implementation artefacts	43
19. C codec implementation	43
20. EIU and sensor discovery / commissioning architecture — 1 October 2026	43
21. AEF-CAN remote firmware-update service — architecture	44
22. Adopted production firmware transport — 1 October 2026	45
23. Operational independence and gateway role — 1 October 2026	46
Flash persistence rule for AEF-CAN nodes — 1 October 2026	46
Firmware compatibility advertisements — 1 October 2026	46
docs/COST_MODEL.md	47
EFIS Parts Cost Model	47
Current one-off estimate	47
Cost target	47

Why prototype cost is higher	47
Expected quantity effects	48
Major remaining cost uncertainties	48
Exclusions	48
Maintenance	48
docs/CUSTOMER_ACCOUNT_AND_APPS.md	49
Customer account, licensing and mobile applications	49
Ecosystem	49
Mobile scope	49
Delivery stages	49
Ownership and licence transfer	49
Device ID at boot	50
Offline firmware carrier	51
docs/DEMONSTRATION.md	52
ESP32 EFIS — QEMU video demonstration	52
What the demonstration shows	52
Build the demonstration	52
Record it on macOS	52
Optional post-production	52
Validation meaning	53
docs/design-suggestions/README.md	54
ESP32 EFIS — Design Suggestion Image Register	54
Reference set — 16 September 2026	54
Ideas already adopted independently	54
Ideas worth carrying forward as proposals	55
Rules for using these images	55
Image installation	55
docs/DEVICE_REGISTRATION.md	56
Horizon initial registration and ownership claiming	56
Goal	56
Physical identity label	56
Factory provisioning	56
First-owner flow	56
Offline behaviour	57
Account/profile model	57
Ownership transfer	57
Security rules	57
UI direction	58
Production tests	58
Physical identity / QR label — initial mechanical specification	58
docs/DISPLAY.md	60
Display Design	60
Selected reference panel	60
Project interface	60
Current RGB timing baseline	60

ESP-IDF integration	61
ST7701S startup	61
Backlight	61
Graphics concept	61
Failure display	62
docs/ELECTRICAL_TEST_HARNESS.md	63
EFIS Electrical Power-On Self-Test (POST) and Test Harness	63
Requirement	63
Boot modes	63
Failure-code format	63
Initial failure-code register	64
Testability rules	64
Result presentation	65
Implementation stages	65
Persistent runtime fault history	65
docs/ENCLOSURE.md	66
Flight-development enclosure	66
Front geometry — unchanged	66
Rear-cover multifunction revision	66
Pressure installation	66
Remote magnetometer	66
Existing internal stack	66
CAD / STL status	66
Verification	66
Current production mechanical direction — 26 September 2026	67
Airspeed scope decision — 26 September 2026	67
Engine-sensor plug and GNSS connector revision — 30 September 2026	67
Engine interface simplification — 30 September 2026	67
Registration identity marking — 30 September 2026	67
USB-C SERVICE / GPS connector — 30 September 2026	68
docs/GNSS_DISPLAY.md	69
GNSS position and accuracy display	69
Adopted requirement	69
Position presentation	69
Accuracy colour coding	69
Validity and freshness	69
Heading and track terminology	69
Data model	70
Receiver integration	70
Simulation and tests	70
docs/HARDWARE.md	71
Hardware Design	71
Processor — optimised reference choice	71
Custom processor carrier	71
IMU — reference part frozen	72

Display — reference part frozen	72
Backlight	72
Low-speed GPIO	73
User input	73
Power	73
Enclosure integration	73
Thermal and mechanical considerations	73
Boot control, diagnostic testability and production networking	74
Airspeed scope decision — 26 September 2026	74
Engine-sensor and USB GPS interface direction — 30 September 2026	74
Remote Engine Interface Unit (EIU) direction — 30 September 2026	74
Stage 7 readiness — provisional engine monitoring	75
Single external USB-C OTG interface — 30 September 2026	75
Stage-7 architecture constraints — 1 October 2026	76
docs/images/README.md	78
ESP32 EFIS project images	78
Authoritative regenerated diagrams	78
Accuracy rule	78
Current design notes reflected in these images	78
docs/LICENSING.md	79
EFIS Product Licensing	79
1. Trust boundaries	79
2. Device identity and provisioning	79
2A. Frozen licence cryptography and wire format	79
3. Licence payload	81
4. Customer purchase and entitlement	81
5. Online installation from the EFIS	81
6. Installation through web/iPhone/Android	82
7. Offline activation	82
8. Boot and runtime verification	82
9. Firmware updates	83
10. Reset and replacement	83
11. Ownership transfer / sale	83
12. Entitlement lifecycle	84
13. Audit requirements	84
14. Failure behaviour	84
15. Security requirements before production	84
16. Required tests	85
17. Current implementation status	85
docs/MACOS_BUILD_AND_QEMU_SETUP.md	86
macOS build and QEMU setup	86
Build scripts — preferred workflow	86
One-time prerequisites	86
Repository and ESP-IDF	86
Environment script	87
Install QEMU once	87

What the QEMU build script does	.87
What the QEMU run script does	.87
What the hardware build script does	.87
Troubleshooting	.88
Safety boundary	.88
docs/OTA_IMAGE_ADMIN.md	89
ESP32 EFIS — Hosted firmware administration and definitive OTA workflow	.89
Policy	.89
Docker administrator	.89
EFIS user behaviour	.89
Administrator release workflow	.90
Persistent server data	.90
Security	.90
Validation	.90
Remote EIU firmware delivery — 1 October 2026	.90
Simulated EIU OTA implementation — 1 October 2026	.91
Two-stage download checkpoints — 1 October 2026	.91
End-to-end interruption harness — 1 October 2026	.91
Firmware transport architecture decision — 1 October 2026	.92
docs/OTA_PHYSICAL_TEST_NODE.md	93
Minimal Physical OTA Test Node	.93
Purpose	.93
Minimum hardware	.93
Two permanent test images	.93
Required OTA behaviour	.93
Minimal user interface	.94
Partitioning	.95
Suggested repository implementation	.95
Relationship to the server harness	.95
Initial acceptance tests	.96
Later destructive/negative tests	.96
Safety/design rule	.96
docs/OTA_USER_SCENARIO.md	97
ESP32 EFIS — OTA user software scenario	.97
Definitive user model	.97
Simulator	.97
Activation and rollback	.97
User-facing rules	.97
Relationship to Docker OTA server	.98
Validation status	.98
Running the simulator	.98
Boot-time Wi-Fi connection dialog	.98
Physical boot-menu control	.99
docs/PARTS_SELECTION.md	100
Reference Parts Selection and Sourcing	.100

Purchase links — UK	100
Procurement evidence — display	100
MCU — Espressif ESP32-S3-WROOM-1-N16R2	100
Custom processor carrier PCB	101
Optical window specification	101
IMU — Bosch Sensortec SHUTTLE BOARD 3.0 BMI088	101
Display — Newhaven NHD-2.1-480480AF-ASXP	101
Display adapter	102
Backlight driver — Adafruit TPS61169 PID 6354	102
GPIO expander — Microchip MCP23008-E/P	102
Rotary encoder — Bourns PEC09 reference	102
Rear electronics carrier and USB-C strain relief	102
5 V USB-C architecture	102
Still required / not recorded as ordered	103
Remaining hardware choices	103
docs/PHONE_NETWORK_AND_PUBLIC_OTA.md	104
ESP32 EFIS — phone network pairing and public OTA access	104
Network path	104
Instrument connection configuration	104
Connection test	104
Maintenance/network policy	104
Public Synology endpoint	104
Public versus private	105
Simulator	105
Physical implementation boundary	105
Validation	105
Boot integration and credential persistence — 26 September 2026	105
docs/POWER_BUDGET.md	106
ESP32 EFIS — Preliminary power budget	106
Design summary	106
Bench power supply selection	106
Preliminary 5 V budget	107
Display/backlight significance	107
ESP32-S3 and Wi-Fi	107
Approximate aircraft-side consumption	107
Prototype supply requirement	108
Hardware commissioning measurements	108
Acceptance before final aircraft power design	108
Completed-EFIS design allowance — 26 September 2026	109
docs/PROCUREMENT_STATUS_2026-09-16.md	110
Prototype procurement status — updated 17 September 2026	110
Receipt-evidence rule	110
Confirmed received — photographic evidence 17 September 2026	110
In transit	110
Stage 1 — still required	110
Stage 2 — still required	111

Source-retention rule	111
Do not buy yet	111
docs/PROJECT_STATUS.md	112
ESP32 EFIS — Project activity, validation and decision status	112
Current development gate	112
Status vocabulary	112
Completed / current activities	112
Design decision and ideas register	113
Definitive OTA policy	116
Accepted emulator sequences	116
Remaining work	116
Hardware-resumption gate	117
Documentation and decision-history rule	117
AEF-CAN / Engine Interface Unit architecture — 30 September 2026	123
docs/PROJECT_STEPS.md	125
ESP32 EFIS — Project Steps and Validation Gates	125
Stage 0 — Software baseline and requirements	125
Stage 1 — Bench hardware and single-display bring-up	125
Stage 2 — Sensors and GNSS integration	125
Stage 3 — Aircraft survey and measurements	125
Stage 4 — Physical interfaces and enclosure freeze	126
Stage 5 — Custom carrier PCB	126
Stage 6 — Aircraft installation and ground validation	126
Stage 7 — Airborne comparison and refinement	126
Cross-stage rules	126
Added boot/maintenance workstream — 26 September 2026	126
Airspeed scope decision — 26 September 2026	127
docs/REMOTE_UPDATES.md	128
ESP32 EFIS — Remote firmware update and fallback design	128
Purpose and policy	128
A/B architecture	128
Download versus activation	128
Activation / fallback	128
State model	129
Release/distribution service	129
Security	129
Persistent configuration compatibility	129
Network model	130
Recovery	130
Validation plan	130
Current status	130
Boot/update entry and Wi-Fi — 26 September 2026	130
Licence compatibility	130
Offline phone-carried OTA — 1 October 2026	131
Multi-node release compatibility and dependency management — 1 October 2026	131

docs/SAFETY.md	133
Safety and Airworthiness Notes	133
Status of this project	133
Key hazards	133
Design rules	133
Power	133
Enclosure material	134
Flight testing	134
Change control	134
docs/SENSORS.md	135
Sensor architecture	135
Attitude sensor — BMI088	135
Pressure sensor — frozen choice	136
Magnetometer — frozen choice	136
Heading fusion	136
Bench development	137
docs/SERVER_PLATFORM.md	138
Server and Container Platform	138
Purpose	138
Budget and hardware target	138
Why a modern mini-PC rather than an old rack server	138
Virtualisation baseline: Proxmox VE (no-subscription)	139
Initial VM layout	139
RAM policy	141
Database placement	141
Storage	141
Networking	141
Docker first, Kubernetes deliberately	141
Future resilience	142
Procurement checkpoint	142
Hypervisor cost and alternative	142
Native Proxmox LXC containers	142
docs/SIMULATION.md	145
Bench and QEMU simulation	145
Instrument acceptance status	145
Artificial Horizon acceptance record	145
Altimeter acceptance record	145
Compass acceptance record	145
Full QEMU demonstration mode	146
Swift simulator synchronisation	146
Build, run and record	146
Display resolution and graphics-quality stage	147
docs/STAGE-7-EIU.md	148
Provisional Stage 7 — Engine Sensor / EIU Integration	148
Purpose	148

Stage boundary	148
Horizon readiness requirement	148
Provisional EIU channels	148
Gates	149
Sensor commissioning and automatic fault detection	149
Sensor power / excitation	149
EIU firmware update architecture — 1 October 2026	149
USB-C factory/service programming	150
Flash-endurance architecture — 1 October 2026	151
Architecture review decisions — 1 October 2026	152
docs/SUPPORT_SERVICE.md	154
MicroSky Horizon customer support chatbot	154
Objective	154
Core design decision	154
Architecture	154
Knowledge lifecycle	154
Data/privacy	154
Production gates	154
Grounded LLM answering — 27 September 2026	155
Semantic/vector retrieval — 27 September 2026	155
Answer status and customer resolution — 27 September 2026	155
Customer interaction modes — 27 September 2026	155
Voice/chat implementation checkpoint — 27 September 2026	156
docs/TESTING.md	157
Testing Plan	157
Principle	157
Stage 0 — Build, emulator and simulator regression — CURRENT SOFTWARE STAGE	157
Stage 1 — Electronics bench test — PAUSED / PENDING HARDWARE	157
Stage 2 — Static orientation tests — PENDING HARDWARE	157
Stage 3 — Dynamic hand/fixture tests — PENDING HARDWARE	158
Stage 4 — Acceleration-rejection tests — PENDING HARDWARE	158
Stage 5 — Altitude/static-pressure tests — PENDING HARDWARE	158
Stage 6 — Heading tests — PENDING HARDWARE	158
Stage 7 — Vibration testing — PENDING HARDWARE	158
Stage 8 — Thermal and sunlight tests — PENDING HARDWARE	158
Stage 9 — Aircraft ground installation test — PENDING HARDWARE	158
Stage 10 — Airborne experimental comparison — FUTURE / BLOCKED	158
Fault injection	158
Bootable full electrical test harness — 26 September 2026	159
docs/user-guides/ALTIMETER.md	160
Altimeter panel — user guide	160
Purpose	160
Presentation	160
Controls	160
Pressure source	160
Validity / failure	160

Bench checks	160
docs/user-guides/COMPASS.md	161
Compass panel — user guide	161
Purpose	161
Presentation	161
Controls	161
Heading source	161
Heading versus track	161
Failure behaviour	161
Bench/aircraft checks	161
docs/user-guides/HORIZON.md	162
Artificial Horizon / PFD panel — user guide	162
Purpose	162
Presentation	162
Controls	162
PFD auxiliary data	162
Failure behaviour	162
Bench checks	162
docs/user-guides/README.md	163
Instrument user guides	163
Panel guides	163
Common PEC09 control model	163
docs/WIRING.md	164
Wiring Plan	164
Power	164
Processor module	164
USB-C	164
3.3 V regulator	164
Display electrical mode	165
Frozen ESP32 GPIO map	165
Reserved ESP32 pins	166
MCP23008 allocation	166
Shared SPI behaviour	166
LCD supply	167
Starting RGB timings	167
PCB implementation rule	167
Authoritative schematic files	167
efis-web-simulator/README.md	168
Horizon EFIS Web Simulator	168
Scope	168
Run	168
Service integration	168
Controls	168
Safety	168
Multi-node offline OTA simulation	168

firmware/DEPLOYMENT.md	170
Deploying the ESP32 EFIS firmware	170
Development shell	170
Hardware connection	170
Normal hardware build	170
Find the USB device	170
Flash and monitor	171
Clean normal rebuild	171
QEMU is not a deployment image	171
Expected physical bring-up checks	171
Erase flash	171
Troubleshooting	171
Deployment discipline	172
firmware/README.md	173
ESP-IDF Firmware	173
Safety and data validity	173
Source layout	173
Physical display configuration	173
QEMU backend	173
macOS shell setup	174
Normal hardware build	174
Clean QEMU build and run	174
Current implementation direction	174
Planned boot state machine and maintenance functions	174
Persistent operational fault log	175
Product licence subsystem — planned	175
hardware/kicad/carrier_rev_a/README.md	176
Carrier PCB Revision A	176
Testability and production follow-on	176
hardware/pcb/README.md	177
Custom Carrier PCB — Revision A requirements	177
Board role	177
Mechanical definition	177
Preferred layer stack	177
Placement zones	177
Routing priorities	178
EMI/noise separation	178
Connector philosophy	179
Test points	179
Revision-A exclusions	179
Fabrication review checklist	179
Production-interface reservations (post-Revision-A)	180
Production mechanical direction	180
hardware/schematics/CARRIER_PCB_SCHEMATIC.md	181
ESP32-S3 Carrier PCB — schematic definition	181

1. Processor module	181
2. Board mechanical target	181
3. 5 V input and USB-C	181
4. 3.3 V regulator	182
5. ESP32 power and reset	183
6. Display connector	183
7. BMI088 interface	184
8. MCP23008 low-speed GPIO	185
9. Rotary encoder connector	185
10. Backlight driver	185
11. Programming/debug	186
12. Frozen GPIO map	186
13. PCB layout priorities	187
14. First-revision build strategy	187
15. Bring-up order	187
16. References	187
license-signer/BUILD-AND-DEPLOY.md	189
Licence Signer — Complete Build and Deployment Guide	189
1. Security boundary	189
2. Repository assets	189
3. Pull on Synology	189
4. Create signer configuration	189
5. Generate DEVELOPMENT signing key	189
6. Validate configuration without exposing secrets	190
7. Build	190
8. Start/recreate	190
9. Health test	190
10. Check isolation	190
11. Logs	190
12. Stop/restart	190
13. Rebuild after source changes	191
14. Repository helper	191
15. Backup/recovery	191
16. Validation gate before account integration	191
17. Production gate	192
license-signer/README.md	193
EFIS Licence Signer	193
Boundary	193
Development bring-up	193
ota-server/README.md	194
ESP32 EFIS OTA image server	194
Services and trust boundary	194
Persistent repository	194
Build and start on Synology	194
Rebuild from scratch and functional test	194
Stage a newly built EFIS binary	195

Publish for EFIS consumption	195
Public deployment	195
Verify public consumption	196
Diagnostics	196
Security	196
Validation status	196
Release sets and offline mobile delivery	196
ota-server/release-set.example.json	197
protocol/aef-can.yaml	198
protocol/golden-vectors.txt	205
AEF-CAN V1 golden vectors	205
Format: CAN_ID DLC DATA_HEX ; interpretation	205
README.md	206
ESP32 EFIS	206
Current development status — 16 September 2026	206
Instrument pages	206
Hardware baseline	207
Firmware and development environment	207
Simulator and acceptance	207
Display baseline	207
Enclosure	208
Development principles	208
Boot, maintenance and diagnostics — 26 September 2026	208
Pressure scope — 26 September 2026	209
Working product identity — 27 September 2026	209
Customer support chatbot — 27 September 2026	209
Server platform — 29 September 2026	209
AEF-CAN aircraft data bus — 30 September 2026	209
simulator/README.md	210
ESP32 EFIS iPhone/iPad Simulator	210
Scope	210
Full demonstration / video recording mode	210
OTA behaviour represented	210
Instrument synchronisation	211
Generate and run	211
OTA scenario checks	211
Safety and validation boundary	211
Required boot-menu parity — 26 September 2026	211
support-service/README.md	212
MicroSky Horizon Support Service	212
Principle	212
Data flow	212
Current MVP	212
Bring-up	212
Product-document ingestion	213

Learning/validation model	213
Safety boundary	213
Grounded natural-language answers	213
Semantic retrieval	213
Answer provenance and resolution feedback	214
Voice and keyboard interaction	214
Customer web chat and speech	214

Document index

account-service/README.md
BOM.md
customer-app/android/README.md
customer-app/ios/EFISCustomer/README.md
customer-app/README.md
docs/AHRS.md
docs/BRANDING.md
docs/CALIBRATION.md
docs/CAN-PROTOCOL.md
docs/COST_MODEL.md
docs/CUSTOMER_ACCOUNT_AND_APPS.md
docs/DEMONSTRATION.md
docs/design-suggestions/README.md
docs/DEVICE_REGISTRATION.md
docs/DISPLAY.md
docs/ELECTRICAL_TEST_HARNESS.md
docs/ENCLOSURE.md
docs/GNSS_DISPLAY.md
docs/HARDWARE.md
docs/images/README.md
docs/LICENSING.md
docs/MACOS_BUILD_AND_QEMU_SETUP.md
docs/OTA_IMAGE_ADMIN.md
docs/OTA_PHYSICAL_TEST_NODE.md
docs/OTA_USER_SCENARIO.md
docs/PARTS_SELECTION.md
docs/PHONE_NETWORK_AND_PUBLIC_OTA.md
docs/POWER_BUDGET.md
docs/PROCUREMENT_STATUS_2026-09-16.md
docs/PROJECT_STATUS.md
docs/PROJECT_STEPS.md
docs/REMOTE_UPDATES.md
docs/SAFETY.md
docs/SENSORS.md
docs/SERVER_PLATFORM.md
docs/SIMULATION.md
docs/STAGE-7-EIU.md
docs/SUPPORT_SERVICE.md
docs/TESTING.md
docs/user-guides/ALTIMETER.md
docs/user-guides/COMPASS.md
docs/user-guides/HORIZON.md

docs/user-guides/README.md
docs/WIRING.md
efis-web-simulator/README.md
firmware/DEPLOYMENT.md
firmware/README.md
hardware/kicad/carrier_rev_a/README.md
hardware/pcb/README.md
hardware/schematics/CARRIER_PCB_SCHEMATIC.md
license-signer/BUILD-AND-DEPLOY.md
license-signer/README.md
ota-server/README.md
ota-server/release-set.example.json
protocol/aef-can.yaml
protocol/golden-vectors.txt
README.md
simulator/README.md
support-service/README.md

account-service/README.md

Repository source: account-service/README.md

EFIS Account Service

Prototype container service for customer accounts, EFIS registration and licence entitlements.

Security boundary

This service does **not** store card numbers/CVV and does **not** contain the private licence-signing key. Card/wallet entry must use provider-hosted/tokenized checkout (Stripe, PayPal/Braintree, Adyen etc.). Provider webhooks must be signature-verified before production use. Apple Pay/Google Pay/PayPal/cards are payment adapters, not changes to the EFIS licence protocol.

Primary device identity is a provisioned EFIS Device ID. ESP32 MAC may be retained as a secondary fingerprint only.

Run

Copy `.env.example` to `.env`, replace every `CHANGE_ME` with strong random values, then:

```
docker compose -f account-service/compose.yml --env-file account-service/.env up -d --build
curl http://127.0.0.1:8091/healthz
```

The service binds to NAS loopback only. Publish it only through an authenticated TLS reverse proxy.

Current prototype API

- GET `/healthz`
- POST `/v1/users`
- POST `/v1/devices`
- GET `/v1/devices/{device_id}/entitlements`
- POST `/internal/payment-event` — normalized internal prototype endpoint, **not a public production webhook**

Still required before production

Login/session or OIDC, email verification/password reset/MFA, CSRF/rate limiting, admin RBAC, migrations/backups, verified payment-provider webhook adapters, device challenge authentication, private licence signer integration, privacy/retention controls, tests and security review.

Customer web portal

The container now includes a responsive owner portal at `/` with account registration/sign-in, session-backed dashboard, registered-instrument display and EFIS Device ID registration. Payment and licence-generation controls remain deliberately inactive until verified provider-webhook and private signer integrations are implemented.

Production hardening still requires CSRF protection, rate limiting, email verification/password reset/MFA, session expiry/revocation, admin RBAC and end-to-end security tests before public exposure.

BOM.md

Repository source: BOM.md

ESP32 EFIS — Bill of Materials and Purchasing Checklist

Evolving purchasing BOM for the **ESP32 EFIS** supplementary/non-primary multifunction flight instrument. Stock and prices change; recheck before ordering. Quantities below are prototype quantities, not production quantities.

Procurement status updated: 24 September 2026 from RS, DigiKey and Mouser delivery-note/packaging photographs, subsequent delivery-status updates, The Pi Hut invoice evidence, and photographic confirmation of delivered parts. **RECEIVED** is used when arrival is confirmed by a supplied delivery-note photograph or other explicit physical-arrival confirmation.

Procurement rules

1. Keep an explicit purchase source against every BOM line that is ready to buy.
2. Manufacturer/supplier part number is authoritative; do not substitute a similar-looking part without checking electrical and mechanical compatibility.
3. A delivery-note photograph that clearly identifies the part and quantity is sufficient evidence to change that quantity to **RECEIVED**.
4. An order confirmation, invoice, dispatch notice or statement that an item is on the way changes status to **ORDERED / IN TRANSIT**, not **RECEIVED**.
5. For generic workshop items, the listed supplier is a practical UK source rather than a frozen manufacturer choice.
6. Recheck price, stock, VAT and carriage immediately before purchase.

Project stages

- **Stage 1 — Bench hardware / bring-up:** power, MCU, single display, controls and basic wiring required to begin physical development.
- **Stage 2 — Sensor + GNSS integration / instrument validation:** attitude, pressure, magnetic and GNSS sources; validate GPS latitude/longitude, fix/freshness and reported horizontal-accuracy handling as well as attitude/altitude/heading.
- **Stage 3 — Aircraft survey / measurements:** measure static system, installation clearances, remote magnetometer location/routing and GNSS antenna/receiver installation requirements.
- **Stage 4 — Physical interfaces / enclosure freeze:** optical window, static fittings, magnetometer harness, GNSS antenna/receiver interface, mounting and final enclosure details.
- **Stage 5 — Custom carrier PCB:** fabricate/populate the integrated carrier only after bench and physical interfaces are mature.
- **Stage 6 — Aircraft installation / validation:** final aircraft power protection, retained wiring/connectors, GNSS installation hardware, mounting hardware and installation-specific items.
- **Stage 7 — PROVISIONAL Engine Sensor / EIU integration:** optional remote engine monitoring over AEF-CAN. Does not block Stages 1–6; see docs/STAGE-7-EIU.md.

Purchasing status and sources

| Item | Exact/reference choice | Stage | Needed | Received | Status | Purchase source / reference |

|---|---|---|---|---|---|

| MCU module | **ESP32-S3-WROOM-1-N16R2** | 1 | 2 | 2 | **RECEIVED — delivery-note photo confirmed** | DigiKey UK — 5407-ESP32-S3-WROOM-1-N16R2CT-ND |

| Temporary MCU carrier/programmer | **Espressif ESP-Module-Prog-1**, specifically Prog-1 not Prog-1R | 1 | 1 | 1 | **RECEIVED — DigiKey packaging/photo confirmed 24 Sep 2026** | DigiKey UK 1965-ESP-MODULE-PROG-1-ND; manufacturer ESP-MODULE-PROG-1 |

| Round display | **Newhaven NHD-2.1-480480AF-ASXP** | 1 | 1 | 1 | **RECEIVED — Mouser packaging/delivery document photo confirmed 21 Sep 2026** | Mouser P/N 763-2.1-480480AFASXP; manufacturer P/N NHD-2.1-480480AF-ASXP |

| Display bench adapter | **Newhaven NHD-FFC40** | 1 | 1 | 1 | **RECEIVED — RS invoice/photo confirmed; invoice dated 15 Sep 2026** | RS UK stock 723-891; invoice line: "40 pin FFC to thru hole adapter"; £9.01 ex VAT |

| Display interconnect | **Integral 40-way display flex tail into NHD-FFC40** | 1 | 1 | 1 | **RECEIVED / PHYSICALLY IDENTIFIED — no separate FFC required** | Supplied as part of Newhaven display; verify orientation/continuity before power |

| Backlight driver | **Adafruit TPS61169 PID 6354** | 1 | 1 | 1 | **RECEIVED** | DigiKey UK 1528-6354-ND |

| Rotary/push control | **Bourns PEC09-2320F-T0015** | 1 | 1 | 1 | **RECEIVED** | DigiKey UK PEC09-2320F-T0015-ND |

| GPIO expander | **Microchip MCP23008-E/P** | 1 | 1 | 1 | **RECEIVED** | DigiKey UK MCP23008-E/P-ND |

| Bench PSU | Current-limited regulated bench PSU suitable for prototype bring-up | 1 | 1 | 1 | **AVAILABLE / USER CONFIRMED 21 Sep 2026** | Already owned |

| Bench PSU lead kit | Banana leads plus croc/bare-wire/header leads | 1 | 1 set | 1 set | **AVAILABLE / USER CONFIRMED 21 Sep 2026** | Already owned |

| USB-C bench cable | Data + power USB-C cable | 1 | 1 | 0 | **NEEDED unless owned** | RS UK / CPC Farnell |

| Solderless breadboard | **Mounted full-size breadboard, aluminium plate & binding posts** | 1 | 1 | 1 | **RECEIVED — item photo confirmed 18 Sep 2026** | **The Pi Hut product 103521, invoice #1622980; item £7.00; order total £10.80 inc shipping/VAT** |

| Prototyping wire | 22–26 AWG solid-core assortment | 1 | 1 set | 1 set | **AVAILABLE / USER CONFIRMED 21 Sep 2026** | Already owned |

| Header pins | 2.54 mm breakaway male/female | 1 | 1 set | 1 set | **AVAILABLE / USER CONFIRMED 21 Sep 2026** | Already owned |

| Attitude IMU | **Bosch Shuttle Board 3.0 BMI088** | 2 | 1 | 1 | **RECEIVED** | DigiKey UK 828-SHUTTLEBOARD3.0BMI088-ND |

| BMI088 bench interposer | Verified **1.27 mm-pitch** mating solution | 2 | 1 | 0 | **NEEDED — verify connector** | Supplier/part to freeze after physical verification |

| Pressure module | **Adafruit BMP585 Baro+Temp Sensor PID 6413** | 2 | 1 | 1 | **RECEIVED — packaging/photo confirmed 22 Sep 2026** | Adafruit PID 6413; DigiKey UK 1528-6413-ND; The Pi Hut/Pimoroni alternate |

| Pressure cable | **Adafruit STEMMA QT/Qwiic JST-SH 4-pin, 150 mm, PID 4397** | 2 | 2 | 2 | **RECEIVED — qty 2, DigiKey packaging/photo confirmed 24 Sep 2026** | DigiKey UK 1528-4397-ND |

| QT breadboard adapter | **Adafruit PID 5961** | 2 | 1 | 1 | **RECEIVED — DigiKey packaging/photo confirmed 24 Sep 2026** | DigiKey UK 1528-5961-ND |

| Magnetic heading sensor | **PNI RM3100-CB P/N 14754** | 2 | 1 | 0 | **NEEDED** | PNI Sensor direct SKU 14754 |

| GNSS receiver | Position/fix/freshness + explicit horizontal accuracy | 2 | 1 | 0 | **SELECTION REQUIRED** | Source after receiver selection |

| GNSS bench antenna | Compatible with selected receiver | 2 | 1 | 0 | **BLOCKED by receiver selection** | Select with receiver |

Order / delivery evidence

Received

RS and DigiKey delivery-note photographs confirm receipt of the NHD-FFC40, two ESP32-S3-WROOM-1-N16R2 modules, TPS61169 board, Bourns encoder, MCP23008 and BMI088 Shuttle Board. Additional RS invoice evidence supplied on 22 September 2026 confirms the NHD-FFC40 adapter as RS stock 723-891, described as “40 pin FFC to thru hole adapter”, quantity 1, invoiced 15 September 2026 at £9.01 ex VAT. A photograph supplied on 18 September 2026 confirms physical receipt of The Pi Hut mounted full-size breadboard, product 103521, quantity 1. Photographs supplied on 21 September 2026 confirm receipt of 1 × Newhaven NHD-2.1-480480AF-ASXP display from Mouser (Mouser P/N 763-2.1-480480AFASXP, quantity 1). A photograph supplied on 22 September 2026 confirms physical receipt of 1 × Adafruit BMP585 Baro+Temp Sensor, product/PID 6413, including loose header strip. Photographs supplied on 24 September 2026 confirm physical receipt of 1 × Espressif ESP-MODULE-PROG-1, 2 × Adafruit STEMMA QT/Qwiic 150 mm cables PID 4397, and 1 × Adafruit QT/STEMMA QT breadboard adapter PID 5961.

Ordered / in transit

DigiKey order dated 21 Sep 2026, invoice evidence supplied 22 Sep 2026: 1 × Espressif ESP-MODULE-PROG-1 (1965-ESP-MODULE-PROG-1-ND), 2 × Adafruit STEMMA QT/Qwiic 150 mm cables PID 4397 (1528-4397-ND), and 1 × Adafruit QT/STEMMA QT breadboard adapter PID 5961 (1528-5961-ND) are recorded as ordered/shipped. The same invoice also shows 1 × BMP585 PID 6413, which is already separately confirmed RECEIVED by physical photo. Hook-up wire and jumper kit on the invoice are bench consumables and are not separately tracked as outstanding BOM items.

Stage 1 — immediate purchase/verification list

1. **ESP-Module-Prog-1, Newhaven display and NHD-FFC40 received.** The display has an integral 40-way flex tail; no separate FFC cable is required. Verify connector orientation, pin numbering and continuity before applying power.
2. **ESP-Module-Prog-1 is physically received** (1965-ESP-MODULE-PROG-1-ND); Stage 1 MCU programming/bring-up is no longer blocked by the programmer.
3. **Current-limited PSU, leads, headers, hookup wire and breadboard are available.**
4. Confirm a suitable USB-C data cable is available before MCU programming.

Stage 1 gate: safe/current-limited 5 V power, programmable/mounted single MCU, working single-display connection/backlight and encoder/GPIO-expander bring-up.

Stage 2 — immediate purchase/selection list

1. Verify and obtain the 1.27 mm mating adapter/interposer for the BMI088 Shuttle Board 3.0.
2. **Adafruit BMP585 PID 6413 received.** Retain for Stage 2 pressure/temperature bench integration.
3. **2 × STEMMA QT/Qwiic 150 mm cables PID 4397 received.**
4. **Adafruit PID 5961 QT breadboard adapter received.**
5. Buy 1 × PNI RM3100-CB P/N 14754.
6. Select the GNSS receiver, then its compatible bench antenna.

Stage-specific items not yet ready to order

Stage 3+ aircraft static bulkhead fitting and pneumatic tubing/tee, final RM3100 harness/mount, final GNSS installation hardware, optical window, final enclosure hardware, custom carrier PCB production, and aircraft 12 V input/protection hardware remain gated by bench validation and aircraft surveys.

Bench consumables / useful spares

Useful bench stock: 0.1 uF, 1 uF and 10 uF ceramic capacitors; 4.7 kOhm and 10 kOhm resistors; heat-shrink; small cable ties/lacing; and suitable non-magnetic/stainless M2/M2.5/M3 hardware. Preferred general sources: DigiKey, Mouser, RS, CPC Farnell and Accu as appropriate.

Budget bench-power option — optional / not required to start Stage 1

The existing current-limited bench PSU remains the preferred source for first power-up. The following low-cost parts form a useful separate 12 V bench-power rig and a stepping stone toward later aircraft-power development. **Do not mark these items ordered or received without purchase/arrival evidence.**

Item	Practical choice	Qty	Status	Indicative budget	Notes / source
------	------------------	-----	--------	-------------------	----------------

$$| \text{---} | \text{---} | \text{---} : | \text{---} | \text{---} : | \text{---} |$$

| 12 V source | Regulated 12 V DC brick, **3 A minimum** | 1 | OPTIONAL / BUY IF NEEDED | £12-£20 | Reputable UK electronics/model supplier; 5.5x2.1 mm barrel is convenient |

| 5 V buck | Adjustable **LM2596-class** step-down module, preferably multi-turn adjustment / display | 1 | OPTIONAL | £3-£8 | Set to **5.00 V** before connecting **Horizon**; a displayed LM2596 3 A module is a convenient bench choice |

Spare adjustable buck	LM2596-class adjustable module	1	OPTIONAL	£2–£5	Useful for experiments; never assume factory output setting
-----------------------	--------------------------------	---	----------	-------	---

Input fuse holder	Inline ATO/ATC blade-fuse holder	1	OPTIONAL	£2–£5	Fit an appropriately small bench fuse; start conservatively
-------------------	----------------------------------	---	----------	-------	---

5 V branch fuse	Inline holder + 1 A fuse	1	OPTIONAL	£2–£5	Prototype protection; revise if measured startup/current demands require it
-----------------	---------------------------------	---	----------	-------	---

Master switch	DC-rated SPST toggle/rocker, ≥ 3 A at 12 V	1	OPTIONAL	£2–£4	Switch 12 V input before the converters
---------------	---	---	----------	-------	---

5 V enable/switch	DC-rated SPST	1	OPTIONAL	£1–£3	Lets logic be isolated independently
-------------------	---------------	---	----------	-------	--------------------------------------

Voltage/current indication Small DC volt/ammeter or USB/DC inline meter 1 OPTIONAL £4-£8 Diagnostic convenience, not a calibrated instrument
--

| Screw terminals / DC socket / wire | 5.5x2.1 mm socket, terminal blocks, 20–22 AWG wire, heat-shrink | 1 set | OPTIONAL | £3–£6 | Keep polarity and rails clearly labelled |

Budget: about £20–£30 if a suitable 12 V source, switches, fuse parts or meter are already in the workshop; approximately £30–£50 if buying the complete rig from scratch. Prices are indicative and should be rechecked before purchase.

Bench topology

```

graph TD
    A[12 V regulated source] --> B[fuse]
    B --> C[master switch]
    C --> D[+]
    D -.- E[+]

```

LM2596 buck -> verified 5.00 V	future/experimental branch
1 A branch fuse	+> separate converter as required
Horizon 5 V bench rail	
regulated 3.3 V rail -> ESP32/sensors as designed	

Display backlight: use the dedicated TPS61169 backlight-driver path already in the BOM;
do not substitute an arbitrary 6 V rail for the designed LED current driver.

Bring-up rules

1. First power-up should still use the owned **current-limited bench PSU**.
2. Adjust each buck **with no Horizon electronics attached**, verify with a multimeter, power-cycle it, and verify again.
3. Never rely on the module's printed/displayed voltage alone for first connection.
4. Start with a conservative current limit/fuse and monitor rail voltage/current for abnormal draw.
5. Keep a common 0 V reference only where required by the verified wiring design; do not improvise aircraft grounding from the bench rig.
6. Espressif recommends 3.3 V and at least 500 mA capability for an ESP32-S3 single-supply design, with local bulk/decoupling capacitance. The Horizon design should retain its documented decoupling rather than relying on the buck module alone.
7. This low-cost bench rig is **not** the production aircraft power-input design and does not replace reverse-polarity, surge/transient, filtering and other aircraft-side protection.

Safety / configuration rule

This remains an experimental **supplementary/non-primary** flight instrument. Bench simulation must be unmistakably identified and disabled for aircraft-use firmware. Missing, stale or implausible data must invalidate the relevant indication rather than freezing a plausible value. GNSS loss must not invalidate otherwise-valid attitude or magnetic heading, and GNSS validity must not imply those sources are valid.

See docs/PROCUREMENT_STATUS_2026-09-16.md, docs/GNSS_DISPLAY.md, docs/POWER_BUDGET.md, docs/PROJECT_STATUS.md, docs/SENSORS.md, docs/ENCLOSURE.md, docs/SIMULATION.md, hardware/ and docs/user-guides/.

Interface direction update — 30 September 2026

- **Engine sensors:** add a keyed, positively retained enclosure-accessible multiway connector/harness provision for CHT, EGT, oil temperature, oil pressure and coolant/water temperature. Connector family, sender-specific conditioning and exact parts are **SELECTION REQUIRED**; nothing is ordered by this decision.
- **GNSS:** supersede the earlier dedicated GNSS connector/bench-antenna direction with a **USB GPS mouse/receiver — SELECTION REQUIRED**. It must be validated for ESP32-S3 USB-host operation, protocol support, power demand and mechanical retention before procurement/freeze. A USB GPS mouse is an active USB-powered GNSS receiver, not a passive electrical device.

Remote Engine Interface Unit — planning BOM

The direct engine-sensor connector at Horizon is superseded by a separate **Engine Interface Unit (EIU)**. For initial planning assume a conventional Rotax 912-series installation, while verifying the actual engine variant/serial/configuration before freezing sender curves, ranges or limits.

| EIU item/function | V1 direction | Status |

|---|---|---

| EIU enclosure + sensor connectors | Compact black-plastic ULM/microlight-headset-style locking circular connectors; splash/water-resistant installation, not necessarily waterproof | SELECTION REQUIRED |

| EIU processor | Small MCU with watchdog, CAN and adequate diagnostics | SELECTION REQUIRED |

| Analogue conversion | External precision ADC / sender-specific front ends | SELECTION REQUIRED |

| EGT channels | 2 × K-type thermocouple front ends with cold-junction compensation and open-sensor detection | SELECTION REQUIRED |

| Head/coolant temperature | 2 channels, exact Rotax sender/transfer function selected by installation | SELECTION REQUIRED |

| Oil temperature | Rotax-compatible sender conditioning | SELECTION REQUIRED |

| Oil pressure | Sender-specific input; exact installed Rotax sender generation/range must be verified | SELECTION REQUIRED |

| EIU ↔ Horizon bus | **Protected non-isolated Classical CAN** with explicit reference/ground strategy; galvanic isolation only if installation testing justifies it | ADOPTED BASELINE |

| EIU power | Protected aircraft supply with local regulation/filtering | DESIGN REQUIRED |

| Diagnostics | Open/short/plausibility, sequence/freshness, raw + engineering values | DESIGN REQUIRED |

The EIU is monitoring-only. It must not become necessary for engine operation and must not disturb an existing engine-control or required indication circuit. No EIU parts are marked ordered by this planning update.

Stage 7 — provisional EIU / AEF-CAN BOM addition

These items are **provisional** and are not instructions to purchase yet. Exact transceiver/isolation/protection and sensor-front-end parts will be frozen after electrical design and installed-engine survey.

| Item | Provisional requirement | Location | Qty | Status |

|---|---|---|---|---|

| Horizon CAN transceiver | 3.3 V Classical CAN transceiver compatible with ESP32-S3 TWAI; protected interface | Horizon carrier | 1 | SELECTION REQUIRED |

| CAN termination | 120 ohm, switchable/jumper-selectable on Horizon | Horizon carrier | 1 | DESIGN REQUIRED |

| CAN protection | ESD/transient protection appropriate to final interface | Horizon carrier | 1 set | DESIGN REQUIRED |

| CAN connector | keyed locking CAN-H/CAN-L/reference/shield provision; coordinate with final Horizon enclosure interface | Horizon enclosure | 1 | SELECTION REQUIRED |

| CAN cable | twisted pair, installation length after aircraft survey | aircraft harness | as required | DEFERRED |

| EIU MCU | MCU with watchdog + Classical CAN; ESP32-S3 remains candidate but must be compared with suitable industrial/deterministic alternatives before PCB freeze | EIU | 1 | DESIGN REVIEW REQUIRED |

| EIU CAN transceiver | 3.3 V compatible protected Classical CAN transceiver; non-isolated baseline, isolation only if testing/installation requires it | EIU | 1 | SELECTION REQUIRED |

| EIU termination | 120 ohm if EIU is opposite physical bus end | EIU | 1 | DESIGN REQUIRED |

| EIU protected power | aircraft input protection, filtering and local rails | EIU | 1 set | DESIGN REQUIRED |

| Precision ADC/front ends | sender-specific analogue conversion | EIU | as required | DESIGN REQUIRED |

| K-type thermocouple front ends | cold-junction compensation + open-sensor detection | EIU | provision for 4 EGT | DESIGN REQUIRED |

| Temperature sender interfaces | installed Rotax sender-specific | EIU | as required | SURVEY REQUIRED |

| Oil-pressure interface | installed sender generation/range-specific | EIU | 1 | SURVEY REQUIRED |

| RPM conditioner | protected interface appropriate to selected RPM source | EIU | 1 | SURVEY REQUIRED |

| EIU J1 ENGINE TEMP panel socket | Connector/contact count derived from actual sender/harness survey including thermocouple termination requirements; earlier 12-way concept is illustrative only | EIU | 1 | SURVEY + SELECTION REQUIRED — DO NOT ORDER |

| J1 mating cable plug | Matching black-plastic locking cable plug, contacts/termination and strain relief/boot as required | engine harness | 1 | SELECTION REQUIRED — DO NOT ORDER |

| EIU J2 ENGINE AUX panel socket | Connector/contact count derived from actual sender/harness survey; earlier 12-way concept is illustrative only | EIU | 1 | SURVEY + SELECTION REQUIRED — DO NOT ORDER |

| J2 mating cable plug | Matching black-plastic locking cable plug, contacts/termination and strain relief/boot as required | engine harness | 1 | SELECTION REQUIRED — DO NOT ORDER |

| EIU J3 AEF-CAN panel socket | Keyed connector for CAN-H/CAN-L/reference and only those installation conductors deliberately selected after power/harness design; earlier 7-way concept is illustrative only | EIU | 1 | DESIGN + SELECTION REQUIRED — DO NOT ORDER |

| J3 mating cable plug | Matching black-plastic locking cable plug, contacts/termination and strain relief/boot as required | EIU-to-Horizon harness | 1 | SELECTION REQUIRED — DO NOT ORDER |

| Connector environmental accessories | Boots, heat-shrink, strain relief, sealing/grommet hardware as needed for splash/water-resistant installation; IP67 not required by current concept | EIU/harness | 1 set | DESIGN REQUIRED |

| EIU enclosure | vibration/temperature/moisture appropriate; connector orientation should minimise direct splash ingress and permit drip loops | EIU | 1 | SELECTION REQUIRED |

The Horizon-side CAN provision is part of making the core EFIS **Stage-7-ready**. The EIU and engine-sensor hardware remain optional/provisional.

USB-C dual-role production interface — 30 September 2026

| Item | Requirement | Status |

|---|---|---|

| External USB-C connector | Single robust rear connector, labelled SERVICE / GPS | DESIGN REQUIRED |

| USB-C CC/role circuitry | Correct host/device role handling for ESP32-S3 OTG architecture | DESIGN REQUIRED |

| USB host VBUS switch | Protected/current-limited 5 V supply for validated GPS receiver; off when Horizon is USB device | DESIGN REQUIRED |

| USB ESD protection | Appropriate low-capacitance protection and PCB layout | DESIGN REQUIRED |

| GPS cable retention | Vibration-resistant installed connection without adding a second GNSS connector | DESIGN REQUIRED |

| Internal programming/test pads | Factory/recovery access independent of external connector condition | DESIGN REQUIRED |

| USB GPS mouse/receiver | USB-host compatible receiver/protocol and acceptable power demand | SELECTION REQUIRED |

One external USB-C port serves both GPS host operation and computer service/programming. Routine field firmware updating is intended to use signed OTA, but USB programming/recovery remains a production requirement.

Stage 7 connector procurement note — 1 October 2026

The EIU connector visual/mechanical target is the **small black-plastic locking circular style used on ULM/microlight headset installations**. Metal-bodied Binder/aerospace-style connectors and industrial M12 connectors shown in earlier concept work are not the current preferred production appearance.

Do **not** purchase the provisional 12-way/12-way/7-way connector set yet. Those pin counts are packaging concepts only. Freeze the connector family and contact counts after confirming the actual Rotax 912 sensor set, EGT thermocouple termination strategy, wire gauges, contact ratings, temperature/vibration requirements and the final CAN/power pin allocation. The installation needs sensible splash/water resistance, not full waterproof/IP67 performance.

EIU sensor-power BOM rule — 1 October 2026

Do not add a generic +5 V engine-sensor supply as a frozen BOM requirement. EGT thermocouples are self-generating; resistive temperature senders use EIU measurement excitation; oil-pressure and RPM power/interface requirements depend on the actual installed sender/source. Any required regulated sender supply is generated, protected and fault-contained locally within the EIU. The EIU itself should preferably use an independently protected aircraft-supply branch rather than the EFIS regulated electronics. Sender-specific regulator/current-limit/protection components remain **DESIGN/SURVEY REQUIRED**.

EIU USB-C SERVICE / factory-programming additions — 1 October 2026

| Item | Requirement | Status |

|---|---|---|

| EIU USB-C SERVICE receptacle | External USB-C connector for factory flashing, development, diagnostics and recovery | DESIGN REQUIRED |

| EIU USB CC/device circuitry | Correct ESP32-S3 USB-device/service role handling; native USB preferred | DESIGN REQUIRED |

| EIU USB ESD protection | Low-capacitance protection and appropriate PCB layout at external service connector | DESIGN REQUIRED |

| EIU USB bench-power path | Permit USB 5 V to power required service/digital circuitry for bench programming | DESIGN REQUIRED |

| EIU power-path isolation | Reverse-current blocking/ORing/current limiting so USB VBUS and aircraft supply cannot back-feed each other | DESIGN REQUIRED |

| EIU USB/service-present detection | Allow firmware/manufacturing harness to identify service/bench operation where useful | DESIGN REQUIRED |

| EIU internal recovery pads | Fallback GND/reset/boot plus serial/debug/programming access appropriate to selected MCU | DESIGN REQUIRED |

| EIU factory programming cable | USB-C data-capable cable for development/production station | ACQUIRE LATER |

| EIU factory provisioning script | Flash bootloader/partition/factory app, provision identity, reboot/interrogate and record test result | SOFTWARE REQUIRED |

The external USB-C SERVICE connector is now the preferred factory/development programming route. Pogo/test pads are retained as fallback/recovery rather than the primary operator interface. Routine production firmware upgrades remain signed Horizon -> AEF-CAN -> EIU updates.

Do not freeze a USB power mux/ideal-diode/current-limit part until the EIU aircraft-input regulator and total bench-service power requirement are known. Likewise, do not enable irreversible ESP32 security/eFuse settings in the ordinary development flash process; production security provisioning requires a separate reviewed procedure.

Stage-7 architecture review — 1 October 2026

The EIU and Horizon use independent protected aircraft-power branches; EIU power is not sourced from Horizon regulated rails. The CAN baseline is now protected **non-isolated** Classical CAN with deliberate reference/ground design. Galvanic isolation is retained as an option only when common-mode/noise/installation testing justifies its additional complexity.

ESP32-S3 is not frozen for the EIU. Perform an MCU comparison before PCB freeze. Likewise, the former 12/12/7 connector counts are packaging concepts only and must not drive the sensor/harness design. Actual Rotax installation/sender survey and thermocouple wiring requirements determine contacts and connector family.

V1 commissioning uses explicit channel role/sender-profile assignment plus automatic compatibility/fault detection, not semantic auto-identification. Horizon is a maintenance gateway rather than an AEF-CAN operational master.

customer-app/android/README.md

Repository source: customer-app/android/README.md

EFIS Customer — Android

Native Kotlin/Jetpack Compose customer companion. Initial skeleton uses mock data until authenticated API contracts are frozen.

Production authentication/session tokens should use Android Keystore-backed secure storage. No card PAN/CVV storage. Checkout uses provider-hosted/SDK flows including Google Pay where enabled.

customer-app/ios/EFISCustomer/README.md

Repository source: customer-app/ios/EFISCustomer/README.md

EFIS Customer — iOS

Native SwiftUI customer companion, target iOS 17+. Initial skeleton uses mock data until authenticated account API contracts are frozen.

Planned secure storage: Keychain for account/session tokens. No card PAN/CVV storage. Checkout will use provider SDK/hosted flows (including Apple Pay where enabled).

customer-app/README.md

Repository source: customer-app/README.md

EFIS Customer Mobile Apps

Native customer companion apps. They are account/licensing clients, not flight instruments.

- ios/EFISCustomer: SwiftUI, iOS 17+
- android/: Kotlin + Jetpack Compose

Shared functions: account access, instrument registration, licence status, provider-hosted payment, licence retrieval/offline activation and re-provisioning. No raw card PAN/CVV or private signing key is stored in either app. See docs/CUSTOMER_ACCOUNT_AND_APPS.md.

Current OTA architecture

The iOS and Android prototypes now model the adopted multi-node/offline workflow: download/cache a complete signed release set while online, show offline readiness, connect to Horizon's local maintenance Wi-Fi at the aircraft, and transfer the cached set to Horizon. Horizon remains the update/security authority and delivers EIU/future-node firmware over AEF-CAN according to the release-set dependency order. The phone never signs firmware or directly programs the EIU.

The current screens are architecture prototypes; network transfer, cryptographic verification and production account integration remain to be implemented and device-tested.

docs/AHRS.md

Repository source: docs/AHRS.md

AHRS / Attitude Solution

Purpose and current status

The BMI088 provides raw angular-rate and specific-force measurements; it does not directly provide a trustworthy aircraft attitude. An initial attitude estimator is now implemented in `firmware/main/attitude_estimator.c` and compiles for the ESP32-S3 target. **It has not been physically validated and is deliberately not connected to the live Horizon until the Shuttle Board axis/sign mapping is demonstrated on hardware.**

Physical sensor work is currently paused because no prototype hardware is available.

Implemented initial estimator

The present implementation is a deliberately simple **pitch/roll complementary estimator**, not the quaternion estimator described in the earlier design proposal. That distinction is important: documentation must describe the code that exists, while quaternion/Mahony/Madgwick/error-state approaches remain possible later evolutions after real BMI088 data are available.

Input is a `bmi088_sample_t` already expressed in aircraft body axes. Required body convention is:

- **+X forward**
- **+Y right**
- **+Z down**

Sensor-to-aircraft axis/sign transformation is intentionally outside the estimator and has not yet been frozen.

Outputs in `attitude_estimator_t` are pitch, roll, last-sample timestamp, initialization state and validity.

Current calculation

On first usable sample, gravity-derived roll and pitch initialize the estimator. Subsequent samples propagate:

- roll from gyro X rate
- pitch from gyro Y rate

When acceleration magnitude is considered gravity-usable, the gyro-propagated result is corrected toward accelerometer-derived attitude with a complementary weight of **0.98 gyro / 0.02 accelerometer per update**.

Accelerometer attitude is calculated only when total acceleration magnitude lies between **0.70 g and 1.30 g**. Outside this range, gyro propagation can continue but the accelerometer is not blindly treated as gravity.

The present pitch validity limit is $\pm 89^\circ$ and roll $\pm 180^\circ$. These are implementation guards, not claims about validated flight envelope.

Timing and stale-data handling

The estimator rejects:

- null/non-finite samples
- initialization without a plausible gravity vector
- zero sample interval
- sample interval greater than **100 ms**
- non-finite output

- output beyond the current pitch/roll implementation limits

`attitude_estimator_check_stale()` marks an initialized attitude invalid when the latest sample is more than **250 ms** old. A failed/stale real source must ultimately result in `attitude_valid=false`; it must never leave a frozen plausible Horizon.

These thresholds are initial software values and require physical/dynamic validation before aircraft use.

BMI088 acquisition layer

The current BMI088 SPI driver:

- uses separate accelerometer and gyroscope chip selects
- verifies accelerometer ID 0x1E and gyro ID 0x0F
- configures ± 6 g accelerometer range
- configures ± 500 °/s gyro range
- handles the accelerometer SPI dummy byte
- returns six engineering-unit values plus a monotonic millisecond timestamp

An optional `CONFIG_EFIS_BMI088_DIAGNOSTICS` commissioning mode logs acceleration X/Y/Z, acceleration magnitude and gyro X/Y/Z at approximately 5 Hz. It is OFF by default, unavailable in QEMU and never feeds the display.

Physical installation gate — not yet passed

Before connecting BMI088 measurements to the Horizon:

1. connect the actual Shuttle Board to the ESP32-S3 prototype
2. verify both chip IDs and stable sample acquisition
3. establish which physical sensor axes/signs correspond to aircraft +X forward, +Y right, +Z down
4. confirm approximately 1 g stationary magnitude in known orientations
5. record stationary gyro bias/noise
6. verify positive/negative pitch and roll rate signs by deliberate hand/fixture rotation
7. encode the explicit sensor-to-body mapping
8. only then feed mapped samples into the estimator and `instrument_data_t`

This gate prevents an assumed PCB orientation from producing a convincing but reversed attitude indication.

Validation still required

Static tests: level, $\pm 10^\circ/\pm 20^\circ$ pitch and $\pm 30^\circ/\pm 60^\circ$ roll against a physical reference.

Dynamic tests: smooth/abrupt rotations, latency, overshoot, drift after stopping and timing behaviour.

Acceleration rejection: translational acceleration without intended rotation.

Fault/stale tests: communication failure, delayed samples, invalid values and disconnect; indication must become conspicuously invalid.

Vibration/thermal tests: characterize gyro/accelerometer noise and estimator stability under representative conditions.

Only after these tests should estimator weighting or a more sophisticated quaternion/filter architecture be selected on evidence from real data.

Future estimator evolution

Candidate later approaches remain quaternion complementary filtering, Mahony, Madgwick or an error-state/EKF approach if measurements justify the additional complexity. Gyro bias estimation, calibration storage, confidence weighting based on angular rate/vibration, and potentially other aiding sources remain future work.

Absolute heading is a separate fusion problem using the remote RM3100 plus validated attitude for tilt compensation. GNSS course over ground, if introduced, is **TRK**, not HDG.

docs/BRANDING.md

Repository source: docs/BRANDING.md

Product branding

Status: WORKING BRAND / NOT LEGALLY CLEARED

Adopted for development: 27 September 2026

Working identity

The project will use the following working commercial identity during development:

- **Brand / manufacturer identity:** MicroSky Avionics
- **Product name:** Horizon
- **Full product reference:** MicroSky Avionics Horizon
- **Short customer-facing reference:** MicroSky Horizon

A simple instrument presentation may use:

```
MicroSky Avionics  
HORIZON  
Flight Display
```

The product name **Horizon** is distinct from the master brand. Future products should therefore be capable of using the same MicroSky Avionics identity without being tied to the Horizon name.

Naming intent

MicroSky is intended to suggest microlight/light aviation without using technical electronics terminology. **Avionics** makes the business/product field explicit. **Horizon** describes the initial flight-display product in simple aviation language.

The branding should remain approachable and non-geeky. Avoid incorporating development-platform terms such as ESP32, AHRS, MCU or similar implementation details into customer-facing product branding.

Legal and commercial status

These names are **working names only**. They are not recorded here as registered trademarks, legally cleared marks, available company names or guaranteed domain names.

Initial collision searching was encouraging enough to continue development under **MicroSky Avionics**, but formal trademark/company/domain clearance must be completed before commercial launch, packaging, significant marketing expenditure or trademark claims.

The word **Horizon** has existing aviation/avionics usage, so the distinctive **MicroSky Avionics / MicroSky** brand should be presented prominently with the product name rather than relying on HORIZON alone.

Development usage

From this checkpoint:

- documentation and new concept material may refer to the product as **MicroSky Horizon**;
- boot/splash-screen concepts may show **MicroSky Avionics** and **HORIZON**;
- repository/internal technical identifiers do **not** need to be renamed from ESP32-EFIS yet;
- existing Device ID conventions such as EFIS- . . . remain unchanged unless a separate identity-schema decision is made;

- no existing URLs, Docker services, signing identities, OTA product identifiers or cryptographic/licensing identifiers should be renamed merely for branding at this stage.

This separation prevents a provisional marketing name from unnecessarily destabilising technical interfaces.

Before commercial freeze

Complete and document:

1. UK trademark search and professional clearance where appropriate;
2. relevant international/EU searches if those markets are intended;
3. company/trading-name checks;
4. domain-name and appropriate social/account-name checks;
5. aviation/avionics product-name collision review;
6. final wordmark/logo and usage rules;
7. decision on whether the legal manufacturer identity is MicroSky Avionics or a separate legal entity trading under that brand.

Until those checks are complete, use **WORKING BRAND** rather than TM or ® claims.

docs/CALIBRATION.md

Repository source: docs/CALIBRATION.md

Calibration

Goals

Calibration must address both sensor errors and mechanical alignment.

Gyroscope calibration

At minimum:

1. hold the instrument completely stationary
2. collect several seconds of gyro data
3. calculate zero-rate bias for X/Y/Z
4. store the offsets
5. repeat at different temperatures during development to assess drift

The startup sequence may perform a fresh stationary bias estimate, but it must not silently assume the aircraft is stationary if motion is detected.

Accelerometer calibration

A six-position static calibration should be implemented:

- +X up
- -X up
- +Y up
- -Y up
- +Z up
- -Z up

This can be used to estimate bias and scale errors for each axis.

Axis alignment

The breakout-board axes may not match the desired aircraft axes. The firmware must define and document an explicit transform between:

- sensor axes
- instrument enclosure axes
- aircraft axes

The target aircraft convention should be documented before flight testing, e.g. longitudinal, lateral and vertical directions and positive rotations.

Panel installation offset

Even a correctly printed enclosure may not sit perfectly level in the aircraft panel. A deliberate installation-alignment routine may therefore be useful.

Any user zero/cage feature must be carefully designed so it cannot mask a serious mounting or sensor fault.

Vibration checks

Calibration on a quiet bench is not sufficient. The system should later be tested with representative engine/airframe vibration while stationary to determine:

- gyro noise increase
- accelerometer vibration spectrum
- filter sensitivity
- mechanical resonances

Stored calibration

Calibration values should be stored in ESP32 non-volatile storage with:

- version number
- timestamp or sequence counter if useful
- checksum/validation
- sane-range checks on load

Corrupt or implausible calibration data should force a safe fallback or recalibration request.

docs/CAN-PROTOCOL.md

Repository source: docs/CAN-PROTOCOL.md

AEF-CAN — Aircraft Experimental Flight CAN

Status: ADOPTED architecture; V1 core wire contract frozen for implementation

Scope: MicroSky Horizon / ESP32-EFIS and future experimental aircraft nodes

Role: Secondary, supplementary, non-certified instrumentation

1. Purpose

AEF-CAN is the project CAN application protocol. It transports aircraft measurements and node health between independent modules such as an Engine Interface Unit (EIU), Horizon display, data logger, future air-data module, fuel module or second display.

The protocol describes **aircraft data, not screen layout**. Producers publish measurements and validity; consumers decide how to display, log or alert on them.

AEF-CAN is intentionally not named after ESP32. A future node may use another MCU without changing the aircraft-level protocol.

2. Safety and operating assumptions

This project is experimental supplementary/non-primary instrumentation. AEF-CAN is not a certified avionics data bus and this specification does not claim certification, integrity assurance or suitability as a sole source of flight or engine information.

Rules:

- stale or invalid data must not remain presented as trustworthy live data;
- no consumer may silently replace failed real data with simulated data;
- simulation traffic must be explicitly identifiable in development builds;
- a producer reports measurements and sensor state, not UI colours;
- alert thresholds are consumer/configuration policy unless a future message is explicitly defined as a source-native alarm;
- loss of a node or message family must be detectable by timeout.

3. V1 physical/data-link baseline

AEF-CAN V1 uses **ISO 11898-1 Classical CAN**, initially at **500 kbit/s**, with standard 11-bit identifiers and payloads of 0–8 bytes.

For the current ESP32-S3 hardware:

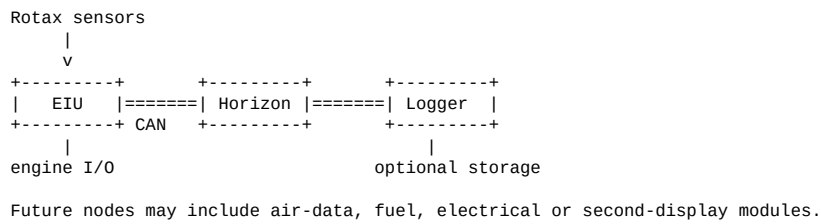
- ESP32-S3 TWAI provides the Classical CAN-compatible controller;
- an external CAN transceiver is required;
- ESP32-S3 native TWAI is not CAN FD capable;
- CAN-H/CAN-L use a twisted pair;
- the physical bus is terminated with 120 ohms at each physical end;
- topology should be a linear trunk with short stubs rather than a star.

CAN FD is a possible future transport, but V1 does not require it. Application code must not assume that the conceptual AEF data model can only ever be carried in 8-byte frames.

4. Network architecture

Nodes are peers on a multi-master broadcast bus. The EIU does not send data "to the EFIS"; it publishes data onto the bus.

Example:



No normal measurement publisher depends on a particular display being present.

5. Identifier allocation

V1 reserves the 11-bit identifier space by function:

CAN ID range	Purpose
0x000–0x07F	Network control / highest-priority exceptional traffic
0x080–0x0FF	Node management and discovery
0x100–0x17F	Engine measurements
0x180–0x1FF	Electrical
0x200–0x27F	Air data
0x280–0x2FF	Navigation / GNSS
0x300–0x37F	Attitude / motion
0x380–0x3FF	Fuel
0x400–0x47F	Environmental
0x480–0x4FF	Aircraft state
0x500–0x57F	Alerts/events
0x580–0x5FF	Configuration/service
0x600–0x67F	Diagnostics
0x680–0x6FF	Logging/time synchronization
0x700–0x77F	Experimental/development
0x780–0x7FF	Reserved

Lower CAN identifiers have higher arbitration priority. Allocation within ranges must therefore be deliberate; IDs are not assigned merely in chronological order.

6. Initial engine messages

The first EIU implementation reserves:

ID	Name	Nominal rate	Purpose
0x090	NODE_ANNOUNCE	boot + 1 Hz	identity/protocol/capabilities summary
0x100	ENGINE_FAST_V1	10 Hz	RPM, oil pressure, oil temperature
0x101	CHT_V1	5 Hz	CHT 1/2
0x102	EGT_12_V1	5 Hz	EGT 1/2
0x103	EGT_34_V1	5 Hz	EGT 3/4
0x104	ENGINE_COOLING_V1	5 Hz	coolant temperature
0x180	EIU_ELECTRICAL_V1	2 Hz	EIU supply voltage/current
0x600	SENSOR_STATUS_V1	2 Hz	sensor validity/fault bitmap
0x601	EIU_HEALTH_V1	1 Hz	EIU health/reset/internal state

| 0x091 | NODE_HEARTBEAT_V1 | 1 Hz | node alive/protocol version |

Exact byte layouts are defined by `protocol/ae-f-can.yaml`, which is the machine-readable source of truth.

7. Encoding rules

7.1 Integers, not floating point

Wire values use fixed-width scaled integers. Examples:

- 4.23 bar -> integer 423 at 0.01 bar/LSB;
- 96.4 degC -> integer 964 at 0.1 degC/LSB;
- 4870 RPM -> integer 4870 at 1 RPM/LSB.

7.2 Canonical wire units

| Quantity | Canonical representation |

|---|---|

| temperature | 0.1 degC |

| pressure | 0.01 bar |

| voltage | 0.01 V |

| current | 0.01 A |

| rotational speed | 1 RPM |

| altitude (future) | 0.1 m |

| speed (future) | 0.01 m/s |

| fuel quantity (future) | 0.01 L |

Consumers convert canonical values into user-selected display units. Unit preference never changes the wire format.

7.3 Endianness

All multi-byte V1 signal values are **little-endian**. This is a protocol rule, not an assumption inherited from a processor.

7.4 Reserved values/bytes

Reserved bytes and bits must be transmitted as zero and ignored by receivers unless a later compatible revision assigns them. Receivers must not fail solely because a reserved field becomes defined in a compatible minor revision.

8. Validity and faults

A numeric value alone is insufficient. Consumers must know whether it is trustworthy.

V1 sensor status defines these semantic states:

- VALID
- NOT_INSTALLED
- OPEN_CIRCUIT
- SHORT_CIRCUIT
- OUT_OF_RANGE
- STALE
- CALIBRATING
- SENSOR_FAULT

Where the compact measurement frame has no room for per-signal state, status is carried by the associated `SENSOR_STATUS` message and freshness is also enforced locally by each consumer.

A consumer must mark data stale when its message timeout expires, even if the last numeric value looked plausible.

Initial timeout guidance:

- 10 Hz data: stale after 500 ms;
- 5 Hz data: stale after 1 s;
- 2 Hz data: stale after 2 s;
- 1 Hz health/heartbeat: node considered missing after 3 s.

These are protocol defaults and may be tightened after bench testing.

8.1 SENSOR_STATUS_V1 scalable layout

The V1 status frame uses explicit logical sensor identifiers rather than a Rotax-specific fixed bitmap:

Byte	Meaning
0	producer node instance
1	status page
2	sensor ID A
3	state A
4	sensor ID B
5	state B
6	sensor ID C
7	state C

A producer sends as many pages as required. Sensor IDs are stable protocol identifiers (for example RPM, oil pressure, CHT1 or EGT4); they are not ADC channel numbers. This permits different engines, sensor counts and multiple producer nodes without changing the state encoding. A sensor omitted from all current pages is not automatically assumed valid or installed; capabilities/configuration determine expectation.

9. Versioning and compatibility

AEF-CAN uses protocol **major.minor** versioning.

- **Major** changes may be incompatible.
- **Minor** changes are additive/backward compatible.
- Existing message byte meanings are never silently changed.
- A materially different payload receives a new message definition/name and, where necessary, a new CAN ID.
- Unrecognised messages are ignored.
- Missing optional messages are not automatically faults.
- Consumers declare which protocol major versions they support.

Firmware version and protocol version are independent.

10. Node identity and discovery

Nodes must not assume that there is exactly one EIU or one display.

A node announcement exposes at least:

- protocol major/minor;
- node type;
- logical node instance;

- hardware revision;
- firmware version or build identifier;
- capability bitmap.

Capabilities describe data a node can produce or services it can provide. This permits, for example, a 2-CHT Rotax installation and a future 6-cylinder engine monitor to coexist with the same protocol architecture.

A future extended discovery/service mechanism may carry human-readable product/serial information. V1 keeps periodic discovery compact to avoid wasting bus bandwidth.

11. Producer/consumer separation

Producers publish physical facts and source health. They do not publish presentation decisions.

Correct:

```
oil_pressure = 1.72 bar
oil_pressure_status = VALID
```

Not part of the measurement protocol:

```
oil_pressure_colour = RED
```

The Horizon configuration owns thresholds, colours, unit conversion and page layout.

12. Configuration and commands

V1 engine measurement operation is broadcast-first and does not require request/response polling.

The 0x580–0x5FF range is reserved for future controlled services such as:

- identify node;
- read calibration metadata;
- set authorised calibration;
- request diagnostics;
- select optional sample rates;
- invoke a sensor test.

Measurement delivery must not depend on these services being used.

Configuration writes should eventually use explicit acknowledgement, validation and persistence rules; they must not be introduced ad hoc.

13. CAN FD and future transports

The AEF data model is separated from the current Classical CAN encoding. A future node may use an external CAN-FD controller or a later MCU with native CAN FD.

Migration rules:

1. Classical CAN V1 remains a supported baseline for V1 nodes.
2. A CAN-FD extension must not be placed on a Classical-CAN-only bus without an explicit gateway/segmentation design.
3. Semantic signal names, units, validity and versioning should survive transport changes.
4. Generated application models should not expose a hard-coded eight-byte assumption beyond the Classical CAN codec.

14. Machine-readable source of truth

`protocol/aef-can.yaml` is authoritative for IDs, signals, units, scaling, rates, versions and timeouts.

Long-term generation targets:

- C/C++ encode/decode structures for ESP-IDF;
- Swift models/codecs for the iPhone simulator;
- Python decoders and log-analysis tools;
- Markdown protocol tables;
- deterministic test vectors;
- bus simulators/fuzz tests.

Generated files must carry a "do not edit; generated from aef-can.yaml" header.

15. Testing requirements

Before an AEF-CAN implementation is called validated, test at least:

- encode/decode round trips at min/nominal/max values;
- endianness;
- signed temperatures;
- missing frames and stale transitions;
- each sensor fault state;
- node disappearance/reappearance;
- unknown IDs;
- newer compatible minor-version traffic;
- bus-off/recovery behaviour;
- duplicate node-instance detection;
- realistic bus-load calculation;
- EIU-to-Horizon bench operation through real transceivers and termination.

Simulation/codec tests do not constitute physical CAN validation.

16. Initial implementation sequence

1. ~~Freeze V1 YAML schema and core message byte layouts.~~ **DONE for core V1.**
2. ~~Add a schema validator/linter.~~ **DONE:** `scripts/validate-aef-can.py`.
3. ~~Generate or hand-verify initial golden test vectors.~~ **DONE:** `protocol/golden-vectors.txt`; **initial vectors independently byte-checked.**
4. Implement a transport-neutral AEF data model.
5. Implement Classical CAN codec.
6. Add EIU simulator traffic.
7. Add Horizon consumer with stale/fault behaviour.
8. Bench-test two physical transceivers with 120-ohm end termination.
9. Add logging/diagnostic tooling.
10. Only then expand configuration services or additional node families.

17. Design principles to preserve

Stable semantics over clever packing. Save bandwidth, but never at the expense of ambiguous data.

Publish facts, not UI. Producers do not know how consumers present data.

Validity is first-class data. A believable stale value is worse than an obvious failure.

Add, don't reinterpret. New versions extend the protocol without changing old meanings.

Hardware independence. AEF-CAN belongs to the aircraft system architecture, not to ESP32.

One specification. Firmware, simulator, tests and documentation derive from the same machine-readable definition.

18. V1 implementation artefacts

The core V1 wire contract is now frozen sufficiently to start codecs and simulators.

- `protocol/aef-can.yaml` — authoritative machine-readable protocol definition.
- `protocol/golden-vectors.txt` — deterministic example frames for cross-language codec tests.
- `scripts/validate-aef-can.py` — structural validation of IDs, field overlap and Classical CAN payload bounds.

Changing the meaning, offset, scaling, signedness or unit of a frozen V1 field is an incompatible change and must not be committed as an in-place edit. Additive reserved-field use must obey the compatibility rules; otherwise define a new message/version.

The next implementation layer is a transport-neutral codec/model shared semantically across ESP-IDF C/C++, Swift and Python, followed by simulated EIU publication and Horizon freshness/fault consumption.

19. C codec implementation

The first executable V1 codec is now in `firmware/main/aef_can_codec.[ch]`. It is deliberately independent of ESP-IDF/TWAI and can therefore be compiled both into Horizon firmware and as ordinary host C.

Initial implemented codecs cover `ENGINE_FAST_V1`, two-temperature frames used by CHT/EGT, and `SENSOR_STATUS_V1`. Decoders enforce DLC and V1 reserved-byte/state constraints rather than accepting malformed frames silently.

Host regression is in `tests/aef_can_codec_test.c`; `scripts/test-aef-can-codec.sh` builds it with strict C11 warnings and compares encoded data with the frozen golden byte patterns. This is codec validation only, not TWAI/transceiver/bus validation.

Next implementation increment: complete the remaining V1 message codecs, add a small AEF frame dispatcher/freshness model, then connect that abstraction to simulated EIU publication before adding the ESP-IDF TWAI transport.

20. EIU and sensor discovery / commissioning architecture — 1 October 2026

AEF-CAN shall support **device discovery and capability enumeration** rather than assuming a permanently hard-coded EIU node/address. Discovery is layered because conventional analogue and thermocouple senders cannot electronically identify their semantic role.

Discovery levels

1. **Device discovery.** An EIU announces node identity, device class, hardware/firmware/protocol versions and channel capabilities. Horizon can therefore discover a compatible EIU without a pre-created per-device configuration.
2. **Electrical discovery.** The EIU safely characterises each input as supported by its hardware: for example open/inactive, resistive, voltage, thermocouple or frequency/pulse. It reports raw diagnostic information and confidence/compatibility information where useful.
3. **Semantic assignment.** Electrical characteristics alone generally cannot prove that a resistive sender is oil temperature rather than another compatible temperature sender, nor can a thermocouple identify itself as EGT 1 versus EGT 2. Horizon therefore presents compatible roles/profiles during commissioning and requires explicit user confirmation.

Persistent commissioning

After confirmation, the channel-to-role/profile mapping is stored persistently with enough identity/version information to detect incompatible hardware or configuration changes. Normal subsequent boots should require no commissioning interaction when the discovered hardware matches the stored configuration.

A stored mapping conceptually binds:

- EIU identity / compatible device class;
- physical channel;
- electrical input type;
- sender/profile identifier;
- semantic aircraft role (for example OIL_TEMP or EGT_1);
- calibration/configuration revision where applicable.

Change and fault detection

The EIU continuously performs non-disruptive plausibility diagnostics appropriate to each configured channel. It shall distinguish, where electrically possible, valid measurement, open circuit, short circuit, out of range, missing sensor and unsupported/ambiguous state. Horizon must never turn these states into a plausible numeric indication.

If an input previously stored as empty later contains a plausible sensor, the EIU reports a **new/unassigned sensor** condition and Horizon offers commissioning for that channel. If a commissioned sensor disappears or changes incompatibly, Horizon reports the fault/change rather than silently remapping it.

Thermocouple discovery is deliberately limited: the EIU may detect a plausible thermocouple circuit/open circuit and its configured front-end type, but semantic cylinder/EGT assignment remains a commissioning decision.

Published measurement contract

Normal AEF-CAN measurements should carry or be associated with:

- semantic sensor/channel identity;
- engineering value and defined units/scaling;
- validity/diagnostic state;
- sequence/freshness information;
- sufficient raw/diagnostic data for maintenance where the message family defines it.

Horizon shall invalidate stale data when freshness limits are exceeded. EIU reboot, sequence discontinuity, node loss/reappearance and configuration revision changes must be observable rather than hidden.

Extensibility

The same mechanism is intended to support future EIU channels and other AEF-CAN sensor nodes (for example fuel pressure/level, manifold pressure or additional temperature modules) without embedding each physical product into Horizon UI logic. Producers publish identity, capabilities, facts and validity; Horizon owns semantic commissioning and presentation.

The machine-readable AEF-CAN schema will require additive discovery/capability/configuration message definitions before this feature is considered wire-protocol complete.

21. AEF-CAN remote firmware-update service — architecture

AEF-CAN shall provide a versioned maintenance service for remote node firmware delivery. This is separate from normal engine telemetry and must not reinterpret existing V1 measurement frames.

Required transaction semantics include: target-node identity and hardware compatibility; update request/ready refusal states; image metadata (product/hardware compatibility, version, size and cryptographic digest/signature metadata); ordered data transfer with explicit progress/acknowledgement and bounded retry; end-to-end complete-image verification; activation request; reboot/reappearance; running-version report; and success/rollback status.

Classical CAN bandwidth is acceptable because updates are infrequent and reliability is preferred over speed. The exact segmentation/windowing scheme and CAN identifiers are **not yet frozen** and must be added additively to the machine-readable protocol schema.

The EIU bootloader independently verifies firmware authenticity before execution. CAN transport integrity, Horizon verification and cryptographic image authenticity are distinct checks.

Firmware update mode must not produce stale-but-plausible engine data: Horizon explicitly marks EIU engine indications unavailable/stale while the node is in maintenance/update/reboot state.

Executable V1 maintenance transport — 1 October 2026

The first executable transport model is now implemented in `ota-server/admin/eiu_can_ota.py`. The canonical schema reserves 0x5C0-0x5FF for firmware maintenance and defines FW_CONTROL_V1 (0x5C0), FW_META_V1 (0x5C1), FW_DATA_V1 (0x5C2) and FW_STATUS_V1 (0x5C3).

The V1 baseline deliberately uses simple stop-and-wait segmentation: each Classical CAN DATA frame contains a uint16 sequence number and up to six firmware bytes. The simulated EIU rejects out-of-order blocks and oversized data, acknowledges accepted sequence numbers, verifies the complete image digest before activation, retains the RPM=0 activation gate and exercises first-boot confirmation/configuration preservation. This simple baseline favours deterministic recovery and testability; a later additive windowed transport may improve throughput without redefining V1.

`test_eiu_can_ota.py` is part of the normal Docker rebuild/test runner. This remains protocol simulation, not physical TWA/CAN validation.

Resumable firmware checkpoints — 1 October 2026

Firmware delivery is resumable at both stages: release service -> Horizon staging and Horizon -> EIU. Progress is represented by an absolute byte offset, never by percentage on the wire.

For the EIU leg, V1 uses a durable checkpoint after each complete **4 KiB** block. Individual Classical CAN DATA frames remain sequence checked; the checkpoint records transfer identity, image identity/digest, committed byte offset and block integrity. After interruption or power loss Horizon queries the EIU, compares transfer/image identity and resumes at the EIU's last committed offset. A mismatched image or transfer does not inherit an old checkpoint.

Integrity is layered: CAN sequencing detects missing/out-of-order frames; checkpoint CRC detects block corruption; final SHA-256 and firmware signature authenticate/verify the complete image before activation. Checkpoints always concern the inactive A/B slot. The currently running known-good slot is never modified by an incomplete transfer.

The executable simulator uses 4 KiB durable checkpoints and regression-tests restart/resume, absolute-offset recovery, complete-image verification and rejection of checkpoint reuse by a different image. Physical flash durability and CAN interruption behaviour remain hardware-validation gates.

22. Adopted production firmware transport — 1 October 2026

The original six-byte DATA / per-frame stop-and-wait transport was a useful executable prototype, but it is **not the production design**. Its per-frame acknowledgement overhead and two-byte sequence field are unnecessarily expensive for firmware images of order 1–2 MB.

The adopted production direction is:

- Classical CAN at 500 kbit/s remains the baseline;
- each FW_DATA_V1 frame carries a **1-byte local sequence plus 7 firmware bytes**;
- firmware is transferred in **4 KiB blocks**;
- each completed block is checked with **CRC32** and receives one block ACK or NACK;
- a failed/incomplete block is retransmitted; individual DATA frames are not normally ACKed;
- durable resume metadata is committed nominally every **64 KiB**, not every 4 KiB;
- a restart may therefore retransmit up to one 64 KiB durable-checkpoint interval;
- final SHA-256 and cryptographic signature verification remain mandatory before activation;

- all transfer writes target the inactive A/B application slot;
- operational AEF-CAN traffic has priority over firmware maintenance traffic and firmware transfer must yield/throttle rather than monopolise the bus.

The 4 KiB and 64 KiB mechanisms deliberately solve different problems. A **4 KiB transfer block** bounds retransmission and detects corruption. A **64 KiB durable checkpoint** bounds restart recovery while avoiding unnecessary persistent metadata writes/flash wear.

Absolute byte offsets are authoritative for restart. Percentage is UI-only. Transfer ID plus image identity/digest prevents a partial image from being resumed as a different release.

The simulator must be revised to model block ACK/NACK, corruption inside a block, retransmission, interruption between durable checkpoints, restart from the last 64 KiB committed offset, and bus-yield behaviour. The existing stop-and-wait simulator remains historical/prototype evidence until replaced; it must not be described as the production wire implementation.

23. Operational independence and gateway role — 1 October 2026

AEF-CAN remains peer-to-peer/multi-master. Horizon/EFIS may provide maintenance, firmware and commissioning gateway services, but it is **not the operational bus master**. Measurement producers publish independently of Horizon presence. Loss/reboot/update of Horizon must not stop an EIU, logger or other producer from continuing its normal protocol role.

The baseline physical interface is protected **non-isolated** Classical CAN with an explicit reference/ground design. Galvanic isolation is an installation-driven option, not a V1 default; add it only where grounding/common-mode/noise validation establishes a need. This avoids unnecessary isolated power, PCB area and component/failure complexity while preserving the option for installations that require isolation.

Firmware-maintenance traffic remains lower priority than operational measurements and health traffic.

Flash persistence rule for AEF-CAN nodes — 1 October 2026

AEF-CAN operational traffic is RAM-first. Receipt/transmission of a CAN frame, sensor sampling, heartbeat generation, freshness tracking, diagnostic counters and ordinary changing measurement state must not trigger persistent flash writes.

For EIU firmware transfer, 4 KiB transfer-block CRC/ACK state is volatile; durable resume metadata is nominally committed every 64 KiB using an append-only rotating checkpoint journal (or demonstrably equivalent wear-levelled storage), never a repeatedly erased fixed metadata sector. A/B application slots remain the firmware rollback mechanism.

Persistent storage is reserved for intentionally durable configuration/calibration/identity, wear-aware OTA metadata, and explicitly justified infrequent events. This rule applies to future AEF-CAN nodes as well as the initial EIU.

Firmware compatibility advertisements — 1 October 2026

AEF-CAN discovery/capability information is the runtime basis for multi-node firmware compatibility. Nodes expose their own firmware identity, AEF-CAN protocol major/minor and supported capabilities; consumers must not infer feature support solely from a peer's firmware version.

Compatibility rules:

- protocol-major incompatibility is fail-closed for functions requiring that protocol;
- additive functionality is capability-negotiated;
- older capability/message behaviour required during a supported OTA transition remains available until the release-set rollback/transition window has closed;
- unknown optional capabilities are ignored safely;
- required missing capabilities block the dependent feature/update rather than producing plausible-but-invalid operation.

The signed OTA release-set manifest supplies deployment constraints and ordering; AEF-CAN discovery supplies the observed installed state against which those constraints are evaluated.

docs/COST_MODEL.md

Repository source: docs/COST_MODEL.md

EFIS Parts Cost Model

Status: planning estimate; update as the BOM and production design are frozen.

This document records the current approximate parts-cost target for one completed ESP32-EFIS. It separates repeatable production parts from development equipment such as programmers, breadboards, adapters and spares.

Current one-off estimate

Production item	Approximate cost
--- ---	
Newhaven 2.1-inch 480×480 display	£25–35
ESP32-S3-WROOM-1-N16R2	£5–6
BMI088 IMU	£20–30
BMP585 static/barometric pressure sensor	£15–20
RM3100 magnetometer	£35–50
GNSS receiver + antenna	£20–35
Backlight driver + control electronics	£10–15
Aircraft power conversion/protection/filtering	£15–25
Custom 4-layer PCB, one-off allocation	£15–30
Connectors, PITOT/STATIC fittings, USB/data etc.	£15–25
PEC09 encoder, passives and internal wiring	£10–15
Enclosure, optical window and hardware	£20–35
Estimated total	 approximately £205–£285

Use **approximately £250 per finished EFIS** as the present planning figure and **£300 as a useful contingency target** until the remaining major items are frozen.

Cost target

Design target: aim for a repeatable production BOM of **£250 or less at modest quantities (approximately 10–25 units)** where this can be achieved without compromising the adopted supplementary-instrument safety, reliability, environmental or serviceability requirements.

The cost target is subordinate to correct engineering. A cheaper component must not replace a validated component merely to meet the target.

Why prototype cost is higher

Development expenditure is not equivalent to the per-unit production BOM. Prototype work also includes items such as:

- ESP-Module-Prog-1 programmer;
- NHD-FFC40 bench adapter;
- breadboards, jumper leads and test wiring;
- sensor development/breakout boards;
- spare ESP32 modules;
- development fixtures and test equipment;
- prototype PCB/enclosure iterations.

These are reusable development assets and should not automatically be allocated in full to each finished instrument.

Expected quantity effects

At 10–25 units, component quantity pricing, consolidated PCB manufacture/assembly and removal of development-only breakout/adaptor boards should reduce unit cost. The eventual carrier PCB should integrate appropriate production components directly where technically sensible.

Do not assume a volume saving until supplier quotations and the production BOM are available.

Major remaining cost uncertainties

The current estimate should be revisited when these are frozen:

1. GNSS receiver and antenna;
2. RM3100 production implementation;
3. custom carrier PCB and assembly method;
4. protected aircraft 12 V power input;
5. STATIC bulkhead fitting and tubing;
6. final enclosure/window manufacturing process;
7. connectors and external data interfaces;
8. production test/calibration hardware allocation.

Exclusions

The figure is a **parts/BOM estimate**, not a retail price. It excludes engineering/development labour, software development, assembly labour, calibration, production testing, scrap/yield, tooling, certification/approval work, packaging, documentation, warranty/support, payment/platform fees, VAT, shipping, distributor minimum-order effects and commercial margin.

The EFIS remains an experimental supplementary/non-primary instrument unless and until a separate applicable approval/certification path is completed.

Maintenance

Use BOM.md as the authoritative component/procurement list. Update this cost model whenever a major BOM component, enclosure manufacturing method or production quantity assumption changes.

docs/CUSTOMER_ACCOUNT_AND_APPS.md

Repository source: docs/CUSTOMER_ACCOUNT_AND_APPS.md

Customer account, licensing and mobile applications

Status: architecture adopted; account/web prototype implemented; native mobile prototypes staged; production security/payment/licence signer pending.

Ecosystem

Web portal, iPhone app and Android app use the same versioned HTTPS API from efis-account. efis-account owns users, provisioned EFIS Device IDs and entitlements; PostgreSQL stores account/device/audit metadata; payment adapters handle provider-hosted card, Apple Pay, Google Pay and PayPal checkout; a separate private efis-license-signer issues signed licences. Normal EFIS operation remains offline-capable after licence installation.

The permanent EFIS Device ID is the primary commercial identity. ESP32 MAC is secondary fingerprint only.

Mobile scope

Both apps provide account sign-in, registered instrument list, Device ID registration, licence status, payment/entitlement management, signed licence retrieval, offline activation assistance, reset/re-provision request and support/update status. They never contain licence-signing private keys or store raw PAN/CVV.

Delivery stages

1. Native account/device UI with mock service.
2. Authenticated API and Keychain/Android Keystore token storage.
3. Device registration/challenge.
4. Provider-hosted checkout and entitlement refresh.
5. Signed licence/offline activation.
6. Reset/re-provision/support.
7. Accessibility, privacy, security and App Store/Play release hardening.

No mobile build or store-readiness claim is made until native toolchains and tests have run.

Ownership and licence transfer

A registered EFIS must support a controlled transfer when the aircraft/instrument is sold.

Transfer modes

1. **Transfer device and transferable licence** — normal sale. The seller starts a transfer for a specific EFIS Device ID. The service records the pending transfer, issues a short-lived one-time transfer code or invitation, and the buyer accepts it while signed into their own account. On acceptance, ownership of the device changes atomically and every entitlement explicitly marked transferable moves with it.
2. **Transfer device only** — used where an entitlement is non-transferable or the seller elects to retain a separately portable entitlement. The service must clearly show what will and will not move before confirmation.
3. **Administrative recovery transfer** — for lost account access, deceased owner, dealer/manufacture intervention or other exceptional cases. This requires stronger evidence and an audited privileged workflow; it must not be an ordinary customer bypass.

Safety and security rules

- Device ID never changes because ownership changes.
- Seller must be authenticated and currently own the device.

- Buyer must accept the transfer; knowing a Device ID alone is never sufficient.
- Require re-authentication for transfer confirmation; add MFA when available.
- Transfer tokens are random, single-use, short-lived and stored hashed server-side.
- Do not expose seller/buyer personal details beyond what is necessary.
- A pending transfer can be cancelled by the seller until accepted.
- Acceptance is atomic: the device cannot belong to two customer accounts.
- Record immutable audit events for initiation, cancellation, acceptance and administrative intervention.
- Payment-provider customer/payment instruments never transfer to the buyer.
- Historical invoices/payment records remain with the original purchaser subject to retention/privacy rules.
- Installed signed licence remains usable during a reasonable sale/transfer window; normal EFIS startup must not require Internet access.
- After acceptance, issue a fresh signed licence bound to the same Device ID and the buyer's current entitlement state. The old installed licence enters a **bounded ownership-transfer grace period** rather than being invalidated immediately. During the grace period the EFIS remains fully usable offline while the buyer completes account registration and installs the replacement signed licence. At grace expiry, the superseded licence follows the product's defined post-expiry behaviour; safety/fail-obvious core instrument behaviour must never be corrupted or misleading.

The exact duration is configurable server-side and shown clearly to seller and buyer. Initial product default: **30 days from buyer acceptance of the transfer**. Installing the buyer's replacement signed licence ends the transfer grace state early. The grace period must not be shortened retrospectively for an already accepted transfer.

- Refund, chargeback, subscription and non-transferable-feature policy must be explicit per entitlement.
- A factory reset or licence reset does not itself change registered ownership.

Client UX

Web, iOS and Android instrument-detail pages should expose **Sell / Transfer EFIS**. The seller sees the Device ID, transferable entitlements, exclusions and consequences before confirming. The buyer uses **Accept transferred EFIS**, entering/scanning the one-time code or following an authenticated invitation. Both receive a final transfer receipt/status.

This workflow is account/licensing administration only; it must never alter flight data, calibration or instrument safety configuration as a side effect.

Device ID at boot

The permanent provisioned **EFIS Device ID must be displayed during every boot** so an owner can identify/register the instrument without entering a maintenance screen.

Boot presentation requirement:

```
EFIS BOOT
EFIS-00001247

> START EFIS
  FULL TEST
  FAULT LOG
  LICENSE
  FIRMWARE UPDATE
```

Rules:

- show the Device ID on the normal boot menu, before any account/network/licence action;
- keep it visible while the boot menu is displayed and readable at normal installed viewing distance;
- the Device ID is not a secret and may also appear in LICENSE, diagnostics and customer registration workflows;
- it is immutable under normal customer operations, licence reset, Wi-Fi reset, factory reset and ownership transfer;
- it must come from the authoritative provisioned device identity, never a generated UI placeholder or MAC address;
- if device identity is missing/corrupt, display a clear identity fault rather than inventing an ID, and record the fault in the persistent fault log;

- displaying the ID must not require Wi-Fi, account access or a valid licence.

Offline firmware carrier

The iPhone and Android apps shall support pre-caching signed firmware for a registered Horizon installation while Internet access is available. Cached release sets may contain the compatible Horizon image plus EIU/future-node images needed for that installation.

The instrument-detail/update UI shall clearly distinguish server availability from **offline readiness**, including cached version, target Device ID/hardware compatibility, package integrity/signature metadata status and whether all required packages are present.

At the aircraft the app connects directly to Horizon's local maintenance Wi-Fi and transfers the cached Horizon image without requiring Internet access. Horizon remains the verifier and update authority; the app cannot sign firmware, bypass compatibility/security checks, select the active partition directly, or override maintenance/activation gates. Horizon subsequently distributes compatible peripheral firmware over AEF-CAN using the normal EIU update architecture.

Mobile firmware cache contents are replaceable distribution artifacts, not secrets. Account/session credentials remain protected by Keychain/Keystore, and the local maintenance session must authenticate/bind the intended instrument.

docs/DEMONSTRATION.md

Repository source: docs/DEMONSTRATION.md

ESP32 EFIS — QEMU video demonstration

This repository can produce a video demonstration of the **actual ESP32 firmware renderer running in Espressif QEMU**. The demonstration is synthetic and is not evidence of physical sensor, display or aircraft validation.

What the demonstration shows

The demonstration is intended to show the implemented/accepted display functions in one continuous sequence:

- Artificial Horizon: level, pitch up/down, bank left/right, combined pitch/bank, ATT FAIL, recovery
- Altimeter: representative low/mid/high altitudes, three pointers, digital altitude, progressive 9-o'clock >10,000-ft hatching, compact 3-o'clock Kollsman scale, ALT FAIL, recovery
- Compass: cardinal/intercardinal headings, 350→010 north crossing, continuous rotation, selected 060° heading bug, HEADING FAIL, recovery
- permanent red SIM indication and deterministic on-screen test captions

The QEMU framebuffer is 480×480 RGB565, matching the native pixel dimensions of the selected Newhaven display. The recording therefore demonstrates firmware graphics rather than a separately drawn promotional mock-up.

Build the demonstration

From the repository root:

```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
source scripts/efis-env.sh
zsh scripts/build-qemu.sh --clean
```

Record it on macOS

Run:

```
zsh scripts/record-qemu-demo.sh
```

The script starts QEMU and then invokes the built-in macOS screen recorder in window-selection mode. Click the **QEMU instrument window** when prompted. Recording stops automatically after the configured demonstration duration and is written under docs/media/ as a .mov file.

macOS may request **Screen Recording** permission for Terminal on the first run. If so, grant permission in System Settings and rerun the script.

To choose a different output file:

```
zsh scripts/record-qemu-demo.sh ~/Desktop/ESP32-EFIS-Demo.mov
```

To override the automatic recording duration:

```
EFIS_DEMO_SECONDS=120 zsh scripts/record-qemu-demo.sh
```

The QEMU console is saved separately as firmware/qemu-demo.log so the recorded sequence can be correlated with firmware scenario transitions.

Optional post-production

The .mov is deliberately a clean source recording. It can subsequently be trimmed and captioned in iMovie, Final Cut Pro, DaVinci Resolve or another editor without changing the underlying instrument imagery. A useful final title is:

> ESP32 EFIS — Firmware/QEMU Demonstration

> Synthetic development data · supplementary/non-primary instrument

Do not remove the on-screen red SIM marker or edit the video in a way that suggests real sensor/aircraft validation.

Validation meaning

The video demonstrates functions already accepted in QEMU. It does **not** demonstrate the physical Newhaven LCD, BMI088, BMP585, RM3100, MCP23008/encoder, PSRAM, power electronics, enclosure or aircraft installation. Those activities remain pending physical hardware and are tracked in `PROJECT_STATUS.md`.

docs/design-suggestions/README.md

Repository source: docs/design-suggestions/README.md

ESP32 EFIS — Design Suggestion Image Register

This directory is for **design inspiration and reference material only**. Images here are not authoritative engineering drawings, wiring diagrams, BOM definitions, PCB layouts, mechanical dimensions or accepted flight-display specifications unless a specific idea is separately adopted in the project requirements/status documentation.

The authoritative project remains a **single physical Newhaven NHD-2.1-480480AF-ASXP 2.1-inch 480×480 round display** with selectable instrument pages. Current hardware, wiring, enclosure and sensor decisions are governed by the corresponding project documents, especially docs/WIRING.md, docs/SENSORS.md, docs/GNSS_DISPLAY.md, docs/ENCLOSURE.md, BOM.md and docs/PROJECT_STATUS.md.

Reference set — 16 September 2026

The following eight images were supplied as design suggestions. Store the original image files under docs/design-suggestions/images/ using the filenames below.

| # | Filename | Classification | Useful ideas | Important conflicts / limitations |

|---|---|---|---|---|

| 01 | 01-electronics-associated-circuitry.jpeg | Electronics / PCB presentation | PCB documentation style, functional blocks, labelled aircraft connector, power-protection presentation, enclosure overview | Shows two displays, BMP388, different display/interface/memory assumptions and unfrozen aircraft-power details. Reference only. |

| 02 | 02-wiring-diagram-dual-display.jpeg | Wiring/interconnection presentation | Colour-coded signal groups, connector pinout box, optional-interface styling, installation overview | Dual displays; SPI displays; BMP388; development board; RTC/microSD; particular GNSS implementation. Does not supersede docs/WIRING.md. |

| 03 | 03-bench-prototype-wiring.jpeg | Bench prototype wiring/layout | Single-display bench arrangement, short display wiring, labelled harnesses, common power distribution, remote RM3100, encoder/MCP23008 relationship | GPIO table is illustrative rather than the frozen map; temporary development-board representation; GNSS choice not frozen. |

| 04 | 04-bench-system-architecture.jpeg | Bench prototype system architecture | Strong separation of functional architecture from exact wiring; single-display architecture; clear supplementary/non-primary labelling | Simplified power; RM3100 interface not frozen; statement that GNSS is not required conflicts with the newer Stage-2 GNSS requirement. |

| 05 | 05-combined-system-enclosure-pcb-bench.jpeg | Combined presentation concept | Four-panel documentation layout, front/side enclosure views, architecture-to-bench progression, PSU comparison, eventual actual bench photograph | Uses a 4.3-inch rectangular display and corresponding enclosure; PCB/wiring are conceptual and are not current hardware. |

| 06 | 06-single-display-exploded-enclosure.jpeg | Single-display enclosure concept | Bezel → display → mounting plate → electronics → body → rear-cover assembly concept; useful exploded-view presentation | Ø66 mm body, 57 mm cut-out, 62 mm PCD, M3 hardware, ~38 mm depth, M12 gland and PCB geometry are not all physically validated. docs/ENCLOSURE.md/CAD remain authoritative. |

| 07 | 07-single-display-prototype-wiring.jpeg | Single-display wiring concept | Clear signal grouping, bench-power block, USB presentation, prototype-vs-aircraft warning | Example 3.3 V regulator conflicts with adopted TPS62162-Q1; TPS61169 supply depiction conflicts with adopted 5 V feed; display/GPIO wiring is not authoritative; RM3100 interface remains to be frozen. |

| 08 | 08-display-concepts-gnss-accuracy.jpeg | Horizon/PFD + Compass/HSI display concepts | GNSS position and reported-horizontal-accuracy presentation on both pages; useful information hierarchy and accuracy legend | Two round images represent **two pages of one physical display**, not two displays. IAS/TAS, navigation bearing/distance/ETA and some symbology require valid data sources before adoption. TRK remains distinct from HDG and GS must not be presented as IAS. |

Ideas already adopted independently

The following ideas visible in the reference set correspond to decisions already adopted elsewhere in the project and therefore do not depend on these pictures for authority:

- one physical 2.1-inch 480×480 round Newhaven display;
- selectable Horizon/PFD, Altimeter and Compass pages;
- supplementary/non-primary instrument status and fail-obvious invalid/stale data;
- BMI088 attitude source, BMP585 static-pressure source and remote RM3100 magnetic source;
- MCP23008 with PEC09 rotary/push control;
- TPS61169 display-backlight driver;

- GNSS latitude/longitude on Horizon/PFD and Compass with coordinate values colour-coded by **receiver-reported horizontal accuracy**;
- GNSS stale/no-fix state must not retain plausible frozen coordinates;
- GNSS TRK/GS semantics remain distinct from heading (HDG) and airspeed (IAS).

Ideas worth carrying forward as proposals

The reference set suggests several presentation practices worth considering without changing the frozen engineering design: colour-coded harness/signal documentation; a clearly labelled aircraft-interface connector once its pinout is validated; separate architecture and exact-wiring drawings; front/side/exploded enclosure documentation after physical measurements; an actual photograph of the validated bench prototype; and a compact combined project-overview sheet for builders.

These remain **reference/proposed presentation ideas** until explicitly adopted in docs/PROJECT_STATUS.md or the relevant authoritative document.

Rules for using these images

1. Never derive GPIO numbers, connector pins, supply voltages, mechanical dimensions or component substitutions from a reference image when authoritative project documentation differs.
2. Never infer that an image represents completed hardware or validation.
3. Preserve the one-physical-display architecture. Side-by-side Horizon/Compass pictures are page comparisons only.
4. Treat generated or illustrative PCB/enclosure renderings as concepts until checked against the real components, CAD, schematic and aircraft measurements.
5. When a useful idea is adopted, record the decision in the authoritative project documentation rather than relying on the image itself.

Image installation

The GitHub text connector cannot upload the binary JPEGs from the chat directly. After exporting/copying the eight supplied originals to the repository, place them in:

```
docs/design-suggestions/images/
```

using the filenames in the register above, then commit them normally. The register is intentionally committed first so the purpose and limitations of the images are already documented before the binaries enter the repository.

docs/DEVICE_REGISTRATION.md

Repository source: docs/DEVICE_REGISTRATION.md

Horizon initial registration and ownership claiming

Status: **ADOPTED DESIGN DIRECTION / IMPLEMENTATION PENDING**

Date: 30 September 2026

Goal

Associate a genuine MicroSky Horizon EFIS with a customer account while keeping normal EFIS startup independent of Internet, account, payment and licensing services. Registration establishes ownership/account association; it does not determine whether the flight display can start.

Physical identity label

Fit a durable QR/data label to a non-prominent accessible part of the EFIS body/rear enclosure. Also print the human-readable immutable Device ID (for example EFIS-00001247) and product/model identification beside it.

The QR code is an identifier/onboarding pointer, **not proof of ownership by itself**. It must contain no password, Wi-Fi credential, licence private material, signing key, customer PII or reusable bearer secret.

Preferred QR payload is an HTTPS registration URL carrying only a public device identifier and format/version, for example conceptually:

```
https://register.<future-domain>/v1/claim?device=EFIS-00001247&v=1
```

Final production domain is deliberately not frozen here.

Provide the same QR on an owner/setup card inside the packaging for convenience, but the durable body label is authoritative for identifying the physical unit.

Factory provisioning

Before sale each EFIS receives:

- immutable Device ID;
- cryptographic device identity/credential distinct from licence signing;
- manufacturing record: product, hardware revision, manufacture date/batch and claim state;
- no customer identity required at manufacture.

The device private credential never appears in the QR code.

First-owner flow

1. Customer scans the EFIS QR with a phone.
2. Registration page/app opens and asks the customer to sign in or create a MicroSky account.
3. Backend validates that the Device ID is genuine and in a claimable state.
4. Customer is shown the product and Device ID and chooses **Claim this Horizon**.
5. System requires physical-possession confirmation. Preferred mechanism: on Horizon choose **LICENSE / REGISTRATION** → **REGISTER DEVICE** (or equivalent setup item), which opens a short registration window and displays a short-lived challenge/confirmation code.

6. Customer confirms the matching short-lived code on the phone. Alternative implementation may have the EFIS authenticate the challenge to the backend during a deliberately initiated maintenance Wi-Fi session.
 7. Backend atomically binds the Device ID to the account, records ownership history/audit information and closes the claim token/window.
 8. Phone shows **Registration complete** and the EFIS can show owner/registration status on its maintenance/status page.
 9. Registration may then lead into licence acquisition/installation, warranty details and support, but these remain separate states.
- Scanning a static QR alone must never be enough to steal/claim a unit.

Offline behaviour

Horizon remains a supplementary/non-primary flight instrument and normal **START EFIS** must not require registration or network access. If the owner cannot get online at installation, registration can be deferred. Maintenance Wi-Fi is entered deliberately and does not silently reconnect during normal flight operation.

Account/profile model

Minimum owner profile:

- account ID;
- display/name fields required for customer service;
- verified email;
- locale/units/preferences;
- owned Horizon Device IDs;
- licence/support/warranty associations;
- ownership role and sharing permissions;
- audit timestamps.

Do not store unnecessary aircraft/location information as a prerequisite to ownership.

A device has one primary owner at a time. Additional users may later be invited with explicit roles without changing primary ownership.

Ownership transfer

Use the already adopted explicit transfer workflow rather than allowing a second account to rescan the static QR and take ownership. Seller initiates transfer; buyer accepts; backend changes ownership atomically and preserves audit history. Device-only/admin recovery remains available under the documented recovery policy. The static Device ID/QR remains unchanged across owners.

Security rules

- Static QR = discovery/identification only.
- Physical possession must be proven with a short-lived challenge or device-authenticated claim.
- Claim challenges are single-use, expire quickly and are rate-limited.
- Never encode reusable credentials or PII in the QR.
- Device identity and licence signing remain separate trust domains.
- Account service cannot obtain licence private signing keys.
- Record claim/transfer/recovery security events.
- Registration reset must not change immutable Device ID.
- Factory/service recovery requires an auditable privileged process.

- Avoid exposing the QR prominently where casual photographs could capture it.

UI direction

Boot menu remains focused on instrument operation. Registration belongs under LICENSE/maintenance rather than blocking boot.

Suggested maintenance view:

```
DEVICE REGISTRATION

Device: EFIS-00001247
Status: NOT REGISTERED

> REGISTER DEVICE
  REGISTRATION STATUS
  BACK
```

During a claim:

```
REGISTER DEVICE

Scan QR on EFIS body
Sign in on your phone

Confirmation code:
  482 731

Expires in 04:32

> CANCEL
```

The exact six-digit format and timeout are illustrative until threat modelling is complete.

Production tests

Test at minimum: valid first claim; photographed/static QR without physical device; expired/replayed challenge; already-owned device; offline/deferred registration; interrupted claim; backend retry/idempotency; ownership transfer; factory/service recovery; account deletion/privacy handling; device identity corruption; and registration state surviving firmware OTA without changing Device ID.

Physical identity / QR label — initial mechanical specification

ADOPTED DIRECTION / ARTWORK AND MATERIAL VALIDATION PENDING.

Place the primary device identity label on the **rear enclosure**, clear of the STATIC fitting, 12 V power, EIU/CAN connector, USB service/GPS connection, mounting fasteners and cable bend/connector release zones. It should remain readable with the instrument removed from the panel but should not be prominent from the cockpit or visible through the normal panel opening.

Preferred label content

Keep the permanent label deliberately sparse:

```
MicroSky Avionics
HORIZON
Flight Display

Device ID: EFIS-00001247

[ QR CODE ]

Scan to register
HW: V1
```

The product serial identity is the immutable Device ID. Do not add customer name, email, licence status, Wi-Fi details, passwords or other PII. Regulatory/compliance markings may be added separately when requirements are known rather than crowding the registration label.

QR payload

Use a versioned HTTPS registration URI containing the immutable public Device ID only. The production domain remains unfrozen. The QR must not contain a reusable claim token or device private credential. Backend records determine whether the device is unclaimed, owned, transferred, retired or under service recovery.

The human-readable Device ID must always accompany the QR so registration/support remains possible if the QR is damaged or a camera cannot read it.

Marking method

Preferred production direction is a durable high-contrast laser-marked plate/label or similarly permanent industrial marking suitable for the enclosure material. A prototype adhesive polyester/polyimide label is acceptable during development. Final material/adhesive/markings must be checked for heat, moisture, cleaning agents, abrasion and expected enclosure surface/finish.

Do not freeze exact QR size until representative labels are printed and scanned using several current phones under poor lighting and at awkward rear-panel angles. Preserve a quiet zone around the QR and avoid placing it across a seam, curvature, screw recess or textured feature that impairs scanning.

Secondary setup card

Ship a second copy of the Device ID and registration QR on a removable setup/owner card. This is a convenience copy only. The rear-body identity marking remains the physical unit's durable identity reference.

Service/manufacturing data

Manufacturing may maintain additional machine-readable internal records (hardware revision, batch, test record and cryptographic identity references), but the customer-facing QR should remain stable and simple. Do not expose factory secrets or cryptographic private material on an external label.

Prototype validation

Before artwork freeze:

1. Generate representative labels for multiple Device IDs.
2. Verify QR decoding against the exact intended URI format.
3. Test phone scanning before and after enclosure installation.
4. Test low light, oblique angle and modest surface contamination.
5. Verify the human-readable Device ID without scanning.
6. Confirm no connector, cable, mounting screw or static tube obscures the label.
7. Check abrasion/cleaner/temperature resistance of the selected marking.
8. Confirm a photograph of the static QR alone cannot complete a device claim.

docs/DISPLAY.md

Repository source: docs/DISPLAY.md

Display Design

Selected reference panel

Reference display: **Newhaven Display NHD-2.1-480480AF-ASXP**.

Manufacturer characteristics:

- 2.1-inch round IPS TFT
- 480 × 480 pixels
- 1000 cd/m² luminance
- ST7701S controller/driver
- no touch layer
- 40-pin, 0.5 mm-pitch FFC
- active area: 53.28 × 53.28 mm
- outline: 58.18 × 60.71 × 2.26 mm
- TFT supply: 2.5–3.3 V, project uses 3.3 V
- backlight: approximately 6.0 V / 100 mA
- operating temperature: -20 °C to +70 °C
- EMI-shielded FPC
- anti-glare construction

The panel is marketed as an 18-bit parallel RGB / 1-lane MIPI DSI display. The ST7701S itself supports 16-bit RGB input mode, which this project uses to reduce ESP32-S3 GPIO demand.

Project interface

The project uses:

- **16-bit RGB565** pixel bus
- DE-mode parallel RGB
- PCLK + HSYNC + VSYNC + DE
- 3-wire / 9-bit serial control for ST7701S startup
- physical mode straps IM0=0, IM1=1, IM2=0

The RGB565 mapping is:

- ESP D0..D4 -> panel B1..B5
- ESP D5..D10 -> panel G0..G5
- ESP D11..D15 -> panel R1..R5
- panel B0 -> GND
- panel R0 -> GND

The ST7701S COLMOD RGB pixel-format field is set to VIPF=101 for 16-bit/pixel mode.

Current RGB timing baseline

The current Newhaven datasheet gives these recommended **RGB** values:

- pixel clock: **30 MHz**
- HFP: **50** clocks
- HBP: **50** clocks
- HS pulse width: **4** clocks
- VFP: **50** lines
- VBP: **50** lines
- VS pulse width: **2** lines

An earlier project note used 18 MHz with 40/60/20 and 10/10/6 porches/sync widths. Those values belong to Newhaven's **MIPI timing table**, not the RGB table, and are no longer the firmware baseline.

ESP-IDF integration

The firmware uses Espressif's native RGB LCD driver:

- `esp_lcd_new_rgb_panel()`
- `data_width = 16`
- two full framebuffers
- framebuffers allocated in PSRAM

Two 480×480 RGB565 framebuffers consume 921,600 bytes, which fits comfortably in the N16R2 module's 2 MB Quad PSRAM.

ST7701S startup

The firmware retains Newhaven's published panel-specific power/gamma initialization sequence and changes the example's RGB pixel-format setting from 18-bit to the ST7701S-defined 16-bit RGB value.

This is a bench-validation item. The first hardware test must verify:

- colour order
- line/column alignment
- no horizontal or vertical tearing
- stable 30 MHz pixel clock operation
- correct HSYNC/VSNC/DE polarity
- correct RGB565 mapping

If any of those fail, the display electrical configuration must be corrected before AHRS rendering is introduced.

Backlight

The 5 V instrument rail feeds the dedicated TPS61169 constant-current boost driver. The ESP32-S3 controls brightness through the driver control/PWM input.

The proof-of-life firmware keeps the backlight off throughout panel initialization and framebuffer preparation, then enables it only after a complete static horizon image exists.

Graphics concept

The artificial horizon will use:

- blue sky
- brown ground
- white horizon line
- bank-angle scale
- pitch ladder
- fixed aircraft reference symbol
- prominent invalid-attitude overlay

The current firmware contains a static version of this scene purely for display bring-up.

Failure display

A failed or stale attitude solution must never leave a believable frozen horizon. The eventual live renderer must replace or obscure the attitude image with:

```
ATTITUDE  
INVALID
```

This requirement applies regardless of whether the fault originates in the BMI088, SPI link, AHRS timing or validity logic.

docs/ELECTRICAL_TEST_HARNESS.md

Repository source: docs/ELECTRICAL_TEST_HARNESS.md

EFIS Electrical Power-On Self-Test (POST) and Test Harness

Status: STAGED DESIGN — 26 September 2026

Role: supplementary/non-primary flight-development instrument.

Requirement

The EFIS firmware shall provide a complete electrical/component test harness that can be selected at boot. It shall exercise every component/interface for which a meaningful automated test is possible, display an unambiguous **PASS / FAIL / NOT TESTED** result, and report stable documented failure codes.

The harness is a maintenance/commissioning facility. A successful POST does **not** establish airworthiness and does not replace electrical, environmental, vibration, installation or airborne validation.

Boot modes

Normal boot remains the default. A deliberate boot gesture/service command shall enter **FULL TEST** before normal instrument operation. The exact gesture is to be frozen after the physical encoder is validated.

FULL TEST shall:

1. identify firmware/build and hardware revision;
2. run tests in a deterministic order;
3. show the test currently running;
4. retain all failures rather than stopping at the first failure where safe;
5. finish with overall PASS or FAIL and a list of failure codes;
6. make results available to the service/debug interface;
7. permit safe re-run without rewriting calibration or operational settings.

A short automatic POST may later run on every normal boot. Destructive/stress tests belong only in FULL TEST.

Failure-code format

Codes use ECCC - NN:

- E = electrical/component test failure;
- CCC = subsystem;
- NN = specific failure.

Subsystems:

| Prefix | Subsystem |

|---|---|

| PWR | power / rails |

| MCU | ESP32 / memory / reset |

| DSP | display / RGB interface |

| BKL | backlight |

| CTL | encoder / MCP23008 / controls |

| IMU | BMI088 |

| BAR | BMP585 / static pressure |

| MAG | RM3100 magnetometer |

| GNS | GNSS |

| USB | USB-C / service interface |

| DAT | external avionics data interface |

| NVS | non-volatile storage |

| OTA | OTA partitions / boot state |

| NET | Wi-Fi/network maintenance interface |

| TMP | thermal/temperature monitoring |

Initial failure-code register

| Code | Meaning | Initial diagnostic action |

|---|---|---|

| EPWR-01 | required input supply outside permitted range | verify source, connector and protection stage |

| EPWR-02 | 5 V rail outside tolerance | inspect DC/DC stage/load |

| EPWR-03 | 3.3 V rail outside tolerance | inspect regulator/load/short |

| EMCU-01 | ESP32 internal startup/self-test failure | capture reset reason/log |

| EMCU-02 | PSRAM unavailable or memory test failed | verify exact module/configuration |

| EMCU-03 | flash read/integrity test failed | verify flash/image |

| EDSP-01 | display controller initialization failed | verify FFC/orientation/init bus |

| EDSP-02 | RGB/display timing test failed or no expected response where detectable | verify RGB/timing/power |

| EBKL-01 | backlight control test failed | verify TPS61169/control path |

| ECTL-01 | MCP23008 not detected | verify I2C/power/address |

| ECTL-02 | encoder input test failed | verify encoder/wiring/GPIO expander |

| EIMU-01 | BMI088 accelerometer not detected | verify SPI/power/CS |

| EIMU-02 | BMI088 gyroscope not detected | verify SPI/power/CS |

| EIMU-03 | BMI088 identity/configuration invalid | verify device/configuration |

| EIMU-04 | BMI088 data implausible/stuck during test | inspect sensor/mount/interface |

| EBAR-01 | BMP585 not detected | verify I2C/power/cable |

| EBAR-02 | BMP585 identity/configuration invalid | verify device/configuration |

| EBAR-03 | pressure/temperature reading implausible | inspect sensor/static plumbing |

| EMAG-01 | RM3100 not detected | verify remote sensor/interface |

| EMAG-02 | magnetic sensor data implausible/stuck | inspect sensor/cable/interference |

| EGNS-01 | configured GNSS device not detected | verify selected GNSS/interface |

| EGNS-02 | GNSS data stream malformed/unsupported | verify protocol/baud/USB driver |

| EGNS-03 | GNSS present but no fix | informational/environmental failure; check antenna/view of sky |

| EUSB-01 | USB service interface initialization failed | verify USB PHY/connector/config |

| EDAT-01 | RS-232 avionics interface self-test failed | verify transceiver/UART |

| EDAT-02 | RS-485 interface self-test failed (if fitted) | verify transceiver/UART |

| EDAT-03 | CAN interface self-test failed (if fitted) | verify transceiver/controller |

| ENVS-01 | NVS read/write scratch test failed | inspect flash/NVS partition |

| ENVS-02 | configuration CRC/schema invalid | recover defaults only with explicit maintenance action |

| EOTA-01 | OTA partition table/state invalid | inspect partition table/otadata |

| EOTA-02 | active image integrity/version metadata invalid | service firmware |

| ENET-01 | Wi-Fi hardware/driver initialization failed | inspect firmware/radio state |

| ETMP-01 | monitored temperature outside test limit | stop stress test and inspect thermal condition |

Testability rules

A test must distinguish **component failure** from **test unavailable**. For example, lack of satellites must not be reported as a failed GNSS receiver if valid GNSS messages are being received; it is a no-fix/environmental condition. Optional/unfitted hardware reports NOT FITTED/NOT TESTED rather than FAIL.

Where practical, production PCB design should add testability features: rail sensing/test points, loopback capability for protected serial interfaces, device identity reads, and safe diagnostic hooks. No test mode may drive an aircraft-connected line into an unsafe state.

Display and backlight tests require human-visible patterns/brightness steps in addition to any electrical checks because the ESP32 cannot prove correct pixels merely by successfully writing the interface.

Result presentation

Example:

```
EFIS FULL ELECTRICAL TEST
HW: Rev A      FW: 0.X.X

PWR  PASS
MCU  PASS
DSP  PASS / VISUAL CHECK REQUIRED
BKL  PASS / VISUAL CHECK REQUIRED
CTL  PASS
IMU  FAIL   EIMU-02
BAR  PASS
MAG  NOT FITTED
GNS  PASS   NO FIX
USB  PASS
DAT  NOT FITTED
NVS  PASS
OTA  PASS

OVERALL: FAIL
FAILURES: EIMU-02
```

The final screen remains displayed until acknowledged and the full structured result should also be emitted over the service/debug interface.

Implementation stages

1. **Stage A:** software harness framework, result model, code registry, simulated pass/fail tests.
2. **Stage B:** Phase-1 hardware tests for MCU/flash/PSRAM/display/backlight/MCP23008/encoder.
3. **Stage C:** BMI088 and BMP585 tests.
4. **Stage D:** RM3100 and selected GNSS tests.
5. **Stage E:** production power sensing and protected avionics-interface loopback tests.
6. **Stage F:** regression harness and manufacturing/maintenance test procedure.

Every implemented test requires both a known-good PASS case and an injected/realistic FAIL case before being marked validated.

Persistent runtime fault history

The electrical test code registry also provides stable identifiers for faults detected during normal operation where applicable. Runtime hardware/software faults are persisted in a bounded, wear-conscious non-volatile fault log and reviewed from **FAULT LOG** on the rotary boot menu.

Store fault transitions/significant events rather than continuously rewriting the same active fault. Entries retain fault code, subsystem/source, severity, occurrence count, firmware/build identity and non-secret diagnostic context. Add trustworthy timestamps only when a valid time source exists; do not invent a wall-clock time when it does not. Credentials are prohibited from the log.

The boot viewer must allow scrolling through entries, viewing details and deliberately clearing the log with confirmation. Power cycling, normal boot and firmware update do not silently erase fault history. Persistent logging supplements, but never delays or replaces, immediate fail-obvious runtime annunciation.

docs/ENCLOSURE.md

Repository source: docs/ENCLOSURE.md

Flight-development enclosure

This document records the current 3D-printable enclosure for the ESP32 supplementary/non-primary multifunction flight instrument.

Front geometry — unchanged

Adding Altimeter and Compass does not enlarge the face. The design remains conventional 3 1/8-inch format: 80.30 mm reference panel opening, 79.60 mm locating body, 88 × 88 mm flange, 58 mm depth target, 62.9 mm four-hole pattern, Newhaven round LCD, 55 mm clear aperture and PEC09 front control.

The front bezel, body, optical window, display carrier, BMI088 carrier and 68 mm PCB fit-gauge geometry remain valid.

Rear-cover multifunction revision

The Bosch BMP585 static-pressure system and remote PNI RM3100-CB require rear services. The OpenSCAD source now adds **two reinforced 12 mm diameter × 5 mm bosses**, each with an intentionally undersize **3 mm pilot hole**: upper-left STATIC and upper-right MAG.

The final fitting/gland is not guessed in CAD. Enlarge the pilot only after purchased hardware is measured. At this stage only rear-cover geometry changes; regenerate its STL from the current SCAD before printing the multifunction cover.

Pressure installation

First plumbing development uses the **Adafruit BMP585 Ported breakout PID 6413**, avoiding an improvised sensor plenum during early tests. The module/tubing must be supported independently and connected through the STATIC service. Leak-test the system, prevent cabin-pressure leakage, avoid kinks/excessive pneumatic volume and select the final fitting to match the aircraft's actual static tubing.

After bench/static testing, decide whether the final article retains a separately mounted ported BMP585 module or integrates the bare BMP585 with a purpose-designed sealed plenum. That decision will trigger the next carrier/rear-cover CAD revision automatically.

Remote magnetometer

The reference heading sensor is **PNI RM3100-CB**, deliberately remote from display/ESP32/DC-DC/backlight/steel panel hardware. The MAG boss is a cable service only. The sensor needs a separate rigid non-magnetic bracket with permanent FWD/UP/lateral-axis marks and strain relief at both cable ends. Keep its harness away from high-current wiring.

Existing internal stack

Front to rear remains bezel/PEC09 → 62 × 2 mm hard-coated AR window → LCD → display carrier → rigid BMI088 cradle → 68 mm custom carrier PCB on 60 mm PCD → rear cover. USB-C remains at the lower rear with service slot and cable strain relief. The 58 mm depth remains the target until physical fit proves otherwise.

CAD / STL status

Authoritative source: enclosure/source/ESP32_Artificial_Horizon_Flight_Development_Case.scad. Current targets are body, front bezel, display carrier, BMI088 carrier, custom PCB fit gauge, revised rear cover and USB strain-relief clamp. **Older rear-cover STLs do not contain STATIC/MAG bosses and must not be treated as current multifunction geometry.**

Verification

Before aircraft use: verify display/IMU/USB fit; leak-test static plumbing; check pressure response/lag/hysteresis; prove RM3100-CB mounting cannot move; characterize magnetic interference with electrical loads on/off; verify sensor-axis transforms; deliberately disconnect/stale each source and confirm unmistakable invalid indication.

This remains a supplementary/non-primary instrument under flight development.

Current production mechanical direction — 26 September 2026

The production concept places the EFIS body **behind the binnacle**. A minimal number of small cap-head Allen mounting bolts enter from the front of the binnacle and engage recessed stainless captive nuts in the enclosure; loose rear mounting nuts are not intended. Enclosure assembly hardware should also be minimised.

Production power is nominal **12 V aircraft input** through an appropriate protected/filtering conversion stage to internal 5 V and 3.3 V rails; USB-C becomes primarily a service/programming interface. The rear/interface concept also reserves GNSS and protected DATA/EXPANSION connectivity. Exact connector families, mounting pattern and dimensions remain unfrozen pending physical validation. Earlier renders/CAD remain conceptual where they conflict with these later requirements.

Airspeed scope decision — 26 September 2026

ASI/IAS capability is removed from the EFIS scope. The enclosure requires only the **STATIC** pneumatic connection for the BMP585 altitude/barometric-pressure system. There is no PITOT connection and no differential-pressure sensor. Preserve the established round-display/front-bezel/PEC09/behind-binnacle/captive-nut enclosure concept. The previously generated rectangular multi-button image remains rejected and is not a design reference.

Engine-sensor plug and GNSS connector revision — 30 September 2026

ADOPTED DESIGN DIRECTION / CONNECTOR FAMILY TO BE FROZEN. Provide externally accessible, keyed and positively retained plug access on the enclosure for a grouped engine-sensor harness. The intended initial measurement set is CHT, EGT, oil temperature, oil pressure and coolant/water temperature, with signal conditioning/protection appropriate to each sender type. Do not expose raw ESP32 GPIO/ADC pins at this connector. Final pin count, connector family, shielding/grounding, thermocouple treatment and whether conditioning lives on the main carrier or a separate engine-interface PCB remain engineering decisions pending sender identification and electrical validation.

The previously reserved dedicated **GNSS connector is removed**. GNSS is instead intended to use an external **USB GPS mouse/receiver** connected through the instrument USB interface. This requires USB-host support, suitable receiver/protocol selection and a mechanically retained service/flight connection; do not assume an arbitrary USB GPS device is compatible until validated. The GPS unit may be physically passive from the pilot's perspective, but a USB receiver is an active powered electronic device.

Engine interface simplification — 30 September 2026

The preferred engine-monitoring architecture now moves sender termination and analogue/thermocouple conditioning into a separate **Engine Interface Unit (EIU)**. Accordingly, the Horizon enclosure should not reserve a large direct sensor block. Reserve a compact, positively retained connector for the EIU digital bus and appropriate power/ground only. V1 preference is an isolated CAN link; connector family and pinout remain to be frozen. The EIU itself receives the grouped Rotax 912 sensor harness. This supersedes the earlier concept of bringing CHT/EGT/oil/coolant analogue signals directly into the Horizon enclosure.

Registration identity marking — 30 September 2026

Reserve a non-prominent, readable area on the **rear enclosure** for the permanent Horizon identity/registration marking: MicroSky Avionics / HORIZON, immutable human-readable Device ID, versioned registration QR, and hardware revision. Keep it clear of STATIC, 12 V, EIU/CAN, USB, mounting fasteners and connector/cable service zones. The QR is identification/onboarding only and contains no reusable credential or customer PII. Exact label dimensions/material remain subject to physical scan and environmental testing. See docs/DEVICE_REGISTRATION.md.

USB-C SERVICE / GPS connector — 30 September 2026

ADOPTED. Provide one rear USB-C connector labelled **SERVICE / GPS**. It replaces separate external programming and GNSS connectors. In normal installed operation it accepts the validated USB GPS mouse/receiver; for factory/service/recovery the GPS is disconnected and a computer uses the same port. Provide suitable mechanical retention/protection for the flight GPS connection. Internal PCB programming/test pads remain available for factory/recovery access and do not require another enclosure opening.

docs/GNSS_DISPLAY.md

Repository source: docs/GNSS_DISPLAY.md

GNSS position and accuracy display

Adopted requirement

The **Horizon/PFD** and **Compass** pages shall both include GNSS position information when a valid GNSS source is available. Latitude and longitude are shown in WGS84 decimal degrees together with fix state, horizontal accuracy estimate and satellites used where available.

This is supplementary information. GNSS position does not replace the independent attitude, barometric-altitude or magnetic-heading sources.

Position presentation

The compact 480×480 layout shall reserve a lower information strip for GNSS without obscuring the primary attitude or compass symbology. The minimum useful presentation is:

```
GPS  53.9621°N  2.0143°W  
3D   ACC  2.4m   8 SAT
```

The exact typography may be compressed after real-panel readability testing, but both Horizon/PFD and Compass must expose the same position-quality semantics.

Accuracy colour coding

Colour applies to the **latitude/longitude numbers and accuracy value/status marker**, not merely a decorative icon:

| Horizontal accuracy estimate | Colour | Meaning |

|---:|---|---|

| <= 1 m | green | Excellent |

| > 1 m to 3 m | light green | Good |

| > 3 m to 10 m | yellow | Fair |

| > 10 m to 30 m | orange | Weak |

| > 30 m | red | Poor |

| no valid fix / stale / non-finite accuracy | red | Invalid |

These thresholds are project UI categories, not certification or integrity limits. They describe the receiver-reported horizontal accuracy estimate. They must not be presented as a guaranteed error bound.

Validity and freshness

GNSS display validity is independent of attitude, altitude and magnetic heading. A failed GPS must not invalidate the artificial horizon or compass heading. Conversely, a valid GPS must not make an invalid attitude or magnetic-heading solution appear valid.

A position is display-valid only when the GNSS source reports a valid navigation fix, latitude/longitude are finite and in range, the horizontal accuracy estimate is finite/non-negative, and the sample is fresh. The implementation must define and test a stale timeout appropriate to the selected receiver/update rate. When invalid, coordinates are removed/obscured and the strip shows GPS INVALID or GPS STALE in red; the last plausible position must not remain displayed as though current.

Heading and track terminology

Magnetic/true heading remains **HDG** and comes from the RM3100/AHRS heading pipeline. GNSS course over ground is **TRK**, never HDG. GNSS ground speed is **GS**, never IAS. If TRK/GS are later shown, they inherit GNSS validity/freshness but do not substitute for magnetic heading or airspeed.

Data model

The shared instrument data model includes at least:

- `gps_latitude_deg`
- `gps_longitude_deg`
- `gps_horizontal_accuracy_m`
- `gps_satellites_used`
- `gps_fix_type`
- `gps_valid`
- `gps_stale`

The renderer uses the common `gnss_quality` classifier so Horizon and Compass cannot silently diverge in their colour thresholds.

Receiver integration

The final GNSS receiver/interface is not yet frozen. Prefer a receiver/protocol that provides an explicit fix-valid indication and horizontal accuracy estimate rather than deriving a colour only from satellite count or HDOP. Receiver selection, antenna placement, RF coexistence, update rate and wiring remain hardware-design tasks.

Simulation and tests

QEMU and Swift simulation must exercise at least: no fix, stale fix, ≤ 1 m, 1–3 m, 3–10 m, 10–30 m and > 30 m accuracy. Synthetic positions remain clearly marked as simulation data. Tests must verify that colour changes follow the shared thresholds and that GNSS failure does not freeze coordinates or alter unrelated instrument validity.

docs/HARDWARE.md

Repository source: docs/HARDWARE.md

Hardware Design

Processor — optimised reference choice

The reference processor is now the **Espressif ESP32-S3-WROOM-1-N16R2** module mounted on a project-specific carrier PCB.

Key reasons for this choice:

- 16 MB Quad flash
- 2 MB Quad PSRAM
- enough PSRAM for double-buffered 480×480 RGB565 graphics
- Quad rather than Octal PSRAM, preserving GPIO35–37 for the pin-heavy display/IMU design
- active module with good current distributor availability
- smaller and mechanically cleaner than designing the instrument around a development board
- native USB available directly from the ESP32-S3

The processor handles:

- BMI088 acquisition over SPI
- calibration and alignment correction
- quaternion AHRS/filter execution
- validity monitoring
- 480×480 horizon graphics
- rotary encoder input through MCP23008
- display brightness PWM

The earlier **ESP32-S3-DevKitC-1-N8R2** remains useful for bench firmware development if already available, but is no longer the reference final hardware because that exact DevKit variant is obsolete at major distributors.

Why not an 8 MB Octal-PSRAM ESP32-S3

The display requires 16 RGB data signals plus timing and sensor/control interfaces. Espressif documents GPIO35–37 as part of the Octal memory interface on Octal-PSRAM configurations. Losing those three pins makes the current design significantly harder.

For this project, **2 MB Quad PSRAM is a better system-level choice than 8 MB Octal PSRAM** because GPIO availability is more valuable than the extra memory.

Two RGB565 frame buffers require:

$$480 \times 480 \times 2 \text{ bytes} \times 2 \text{ buffers} = 921,600 \text{ bytes}$$

That fits comfortably inside 2 MB PSRAM.

Custom processor carrier

The final PCB should carry the N16R2 module directly and provide:

- regulated 5 V instrument input via USB-C
- 3.3 V regulator sized for ESP32-S3 transient current plus peripherals

- local bulk and high-frequency decoupling
- native USB D-/D+ routing to GPIO19/GPIO20
- EN/reset network
- BOOT access on GPIO0
- test/programming pads
- RGB display connector
- shared LCD-configuration/BMI088 SPI connector
- I²C/MCP23008 connector
- TPS61169 PWM/output wiring
- deliberate grounding and short high-speed return paths

Normal flight firmware should not require Wi-Fi or Bluetooth. They should remain disabled during ordinary attitude-display operation unless deliberately enabled for a maintenance function.

IMU — reference part frozen

The reference IMU board is the **Bosch Sensortec SHUTTLE BOARD 3.0 BMI088** using SPI.

Approximate board dimensions are 22 × 14 mm with a 1.27 mm-pitch connector. The accelerometer and gyroscope are separate logical SPI devices with separate chip selects. Firmware must explicitly perform the accelerometer's documented SPI-mode transition after reset.

The sensor is rigidly mounted and its axes are explicitly mapped to aircraft longitudinal, lateral and vertical axes. Soft foam suspension is not used as the primary structural mounting method.

Display — reference part frozen

The reference display is the **Newhaven NHD-2.1-480480AF-ASXP**:

- 2.1-inch round IPS TFT
- 480 × 480 pixels
- 1000 nit typical luminance
- ST7701S controller/driver
- no touch layer
- project pixel interface: **16-bit RGB565 parallel RGB**
- ST7701S configuration: 9-bit serial/SPI-style initialization
- 40-pin 0.5 mm FFC
- active area 53.28 × 53.28 mm
- outline 58.18 × 60.71 × 2.26 mm
- TFT supply approximately 3.0–3.3 V
- backlight approximately 6.0 V / 100 mA
- operating temperature -20 °C to +70 °C

RGB565 is chosen instead of the panel's full 18-bit mode to save two direct ESP32 GPIOs.

Backlight

The prototype uses the **Adafruit TPS61169 constant-current boost converter, PID 6354**, configured for approximately 100 mA maximum LED current. Brightness is controlled by ESP32 PWM.

The backlight is not powered from an ESP32 GPIO or from the ESP32's 3.3 V regulator output.

Low-speed GPIO

The **Microchip MCP23008** handles low-speed functions including:

- LCD configuration chip select
- LCD hardware reset
- rotary encoder A
- rotary encoder B
- rotary encoder push switch

Its interrupt output is connected directly to the ESP32.

User input

The reference control family is **Bourns PEC09**, using an incremental rotary encoder with push switch. Exact shaft length/knob suffix remains to be physically frozen.

Likely functions:

- rotate: brightness/menu selection
- short press: acknowledge/select
- deliberate long press: cage/calibration action subject to final safety logic

Power

Development power is a regulated **5 V input via USB-C**. Aircraft 12 V conversion, surge suppression and reverse-polarity protection remain outside the enclosure during this phase.

The custom carrier will regulate 5 V down to 3.3 V for the ESP32-S3 and logic. The 5 V rail also supplies the dedicated backlight boost/current driver.

Enclosure integration

The 3 1/8-inch enclosure now provides:

1. optical-window/front-bezel assembly
2. Newhaven display carrier
3. rigid BMI088 carrier
4. rear-service electronics carrier
5. rotary encoder control pod
6. USB-C cable strain relief
7. removable rear cover

The existing electronics carrier was deliberately designed with edge-location rails so it can be revised for the final custom N16R2 PCB without redesigning the main enclosure.

Thermal and mechanical considerations

The 1000-nit LCD backlight is likely to dominate heat generation. The complete assembly must be tested at elevated ambient temperature and under representative solar loading.

Electronics that affect attitude measurement must not move relative to the airframe. IMU reference alignment is more important than cosmetic display alignment.

Boot control, diagnostic testability and production networking

The PEC09 rotary/push control is the normal physical controller for the boot menu and maintenance dialogs: rotate to move selection and short-press to enter/confirm. The boot menu offers **START EFIS**, **FULL TEST** and **FIRMWARE UPDATE**, with **START EFIS** selected by default.

Production hardware should be designed for the bootable electrical test harness: preserve labelled rail/test points and, where practical, permit safe identity, rail and protected-interface diagnostics. Display/backlight testing still requires visual confirmation. External aircraft-connected interfaces must never be driven into an unsafe state by test mode.

Wi-Fi credentials are persistent in ESP-IDF NVS. Production configuration requires NVS encryption for stored credentials. Wi-Fi remains maintenance-only and is explicitly entered from the firmware-update path; it is not silently enabled during normal flight presentation.

Airspeed scope decision — 26 September 2026

ASI/IAS is not part of the EFIS. Do not fit a pitot/differential-pressure sensor. BMP585 remains the sole pressure sensor and connects to aircraft STATIC for altitude/barometric pressure. No PITOT pneumatic input is required.

Engine-sensor and USB GPS interface direction — 30 September 2026

ADOPTED / DETAILED ELECTRICAL DESIGN PENDING. Reserve an enclosure-accessible multiway engine-sensor connector for CHT, EGT, oil temperature, oil pressure and coolant/water temperature. Inputs require sender-specific protection, filtering and conversion; EGT/thermocouple inputs require a proper thermocouple front end and cold-junction compensation. Prefer a dedicated precision conversion/interface stage rather than routing aircraft sender wiring directly to ESP32 ADC/GPIO pins. Exact sender types and ranges must be identified before the analogue design is frozen.

Remove the dedicated GNSS connector from the carrier/enclosure concept. GNSS will use a validated external USB GPS mouse/receiver. The ESP32-S3 design must therefore support the selected USB-host architecture and receiver protocol while retaining a practical service/programming path. USB power budget, ESD/transient protection, connector retention and coexistence with service USB must be resolved before PCB freeze. Do not describe a USB GPS mouse as electrically passive: it is an active USB-powered GNSS receiver even though it needs no separate aircraft data interface.

Remote Engine Interface Unit (EIU) direction — 30 September 2026

PREFERRED ARCHITECTURE / DETAILED DESIGN PENDING. For the initial engine-monitoring design assume a conventional carburetted Rotax 912-series installation and terminate the engine senders at a separate remote Engine Interface Unit rather than carrying multiple low-level analogue/thermocouple circuits to the Horizon enclosure. The EIU performs sender excitation where required, protection, filtering, thermocouple cold-junction compensation, precision analogue-to-digital conversion, calibration/linearisation, open/short plausibility diagnostics and local acquisition. A single robust digital link carries measurements plus validity/diagnostic metadata to Horizon.

Initial EIU channels should cover the applicable Rotax 912 head/coolant temperature channels, oil temperature, oil pressure and provision for two EGT thermocouples. Exact sensor transfer functions and limits must be selected by engine variant/serial/configuration from current Rotax documentation; Horizon must not assume all 912 installations use identical head/coolant or oil-pressure senders.

Preferred EIU-to-Horizon physical bus for V1: isolated CAN. Use a short application protocol with node identity, channel identity, engineering value, raw value/diagnostic state, sequence counter and freshness/CRC handling. Horizon must mark stale, missing or implausible engine data invalid rather than retain a plausible old value. The EIU is not an engine-control device and must not be placed in series with any sensor required by another engine-control system. Where an existing sender must also feed another instrument, loading/isolation must be engineered and validated rather than simply paralleling it.

Locate the EIU near enough to the engine/sensor harness to keep thermocouple and analogue runs controlled, but in an installation environment compatible with its temperature, vibration and moisture ratings. Keep the Horizon enclosure interface to protected

power/ground plus the digital bus rather than a large bundle of analogue sensor wires.

Stage 7 readiness — provisional engine monitoring

The core Horizon design shall be **AEF-CAN ready** without making engine monitoring a dependency. Reserve ESP32-S3 TWAI pins and carrier-board space for a 3.3 V Classical CAN transceiver, interface protection, connector and selectable 120-ohm end termination. The exact transceiver/isolation implementation remains to be frozen before carrier-PCB release.

Firmware shall accept AEF-CAN through a transport boundary and maintain an optional engine-data state. Missing or stale EIU traffic invalidates engine data only; it must not impair attitude, altitude, GNSS or other core Horizon functions. The initial implementation is in `aef_can_codec.[ch]` and `aef_engine_input.[ch]`.

The remote EIU and its analogue sensor electronics are **Provisional Stage 7**; see `docs/STAGE-7-EIU.md`.

Single external USB-C OTG interface — 30 September 2026

ADOPTED PRODUCTION DIRECTION. Horizon shall use one externally accessible USB-C port for both service/programming and the external USB GNSS receiver. The ESP32-S3 USB interface is used in dual-role/OTG fashion: Horizon is a USB **device** when attached to a service computer and a USB **host** when operating the selected GPS mouse/receiver.

Routine production firmware updates should use the signed OTA path, but USB programming/recovery remains a required service capability for initial factory programming, development, diagnostics and recovery when OTA is unavailable. Retain internal factory/recovery programming/test pads on the carrier PCB as a second service path if the external connector or USB firmware path is damaged.

The final circuit must correctly implement USB-C role detection and protected VBUS switching. Horizon must not source VBUS while attached as a USB device to a computer; in host mode it must provide a protected/current-limited 5 V supply adequate for the validated GPS receiver. ESD protection, USB signal integrity, receiver power budget, connector retention and host/device firmware behaviour are PCB-release gates.

No second external programming connector and no dedicated GNSS connector are required. The rear-panel designation is **USB-C — SERVICE / GPS**. Exact USB GPS receiver/protocol remains selection/validation required.

EIU connector concept — 1 October 2026

PROVISIONAL MECHANICAL/INTERCONNECT DIRECTION. The preferred external EIU connectors are compact **black-plastic circular locking connectors in the style used on ULM/microlight headset systems**. The intent is a lightweight keyed connector with positive push/twist or equivalent locking, rather than a large metal aerospace connector or an industrial M12 connector.

The installation does **not** require a fully waterproof/IP67 connector. It should instead be designed for reasonable water/splash resistance appropriate to the selected EIU mounting position, using sensible connector orientation, enclosure lips/seals, cable strain relief, heat-shrink/boots and drip loops as appropriate. Do not treat IP40 alone as water resistance; final connector selection must be checked against the actual installation environment.

Provisional enclosure interface:

- **J1 — ENGINE TEMP:** multiway ULM-style black-plastic locking circular connector for EGT/CHT and related temperature channels.
- **J2 — ENGINE AUX:** multiway ULM-style black-plastic locking circular connector for oil pressure, oil temperature, coolant/water temperature and spare/provisional sensor channels.
- **J3 — EFIS CAN:** smaller/differently keyed ULM-style black-plastic locking circular connector carrying protected power/ground and the EIU-to-Horizon CAN bus, with spare conductors only if justified.

Use different pin counts and/or mechanical keying where practical so sensor and CAN/power harnesses cannot be cross-connected. Exact contact count, connector manufacturer/family and part numbers remain **TBD** until the Rotax 912 sender set, thermocouple wiring requirements, conductor sizes, current ratings, temperature/vibration limits and required degree of splash resistance are frozen.

The visual target is the small black plastic connector style familiar from ULM headset installations. Earlier metal-bodied Binder-style illustrations are **not** the intended EIU production appearance.

EIU sensor power and excitation architecture — 1 October 2026

Sensor power is **sender-dependent**. Do not assume that every engine sensor requires, or may safely receive, a +5 V feed.

- **K-type EGT thermocouples:** self-generating millivolt sources; no sensor power is supplied. The EIU provides the thermocouple analogue front end, cold-junction compensation, open-sensor diagnostics and appropriate protection.
- **Resistive temperature senders (CHT/head/coolant/oil temperature where applicable):** no separate power wire. The EIU supplies only the controlled low-level measurement excitation through a precision resistance-measurement network and measures the resulting voltage/resistance.
- **Oil pressure:** power/interface requirements depend on the actual installed sender. A three-wire electronic sender may require a regulated excitation supply (for example 5 V), while other sender types require different conditioning. No supply voltage is frozen until the installed sender is identified.
- **RPM:** excitation/interface depends on the selected Rotax/tacho pickup or any added Hall/frequency sensor. Do not assume +5 V.
- Any powered sensor supply shall be generated and protected **locally inside the EIU**, with current limiting/fault containment appropriate to the final sender.

Preferred system power architecture: the EIU receives its **own protected aircraft-supply feed** rather than being powered from the EFIS regulated electronics. The EIU locally provides its MCU/ADC/front-end rails and any required sensor excitation. This limits the ability of an EIU or engine-sensor wiring fault to pull down the core EFIS.

The AEF-CAN connection carries digital communications; whether aircraft power is physically bundled with that cable or supplied independently at installation remains a harness/topology decision. In either case, the EFIS core electronics must not be the unprotected source of EIU/sensor power.

J1 ENGINE TEMP is therefore primarily passive thermocouple/resistive measurement wiring. J2 ENGINE AUX may contain sender-specific excitation only where the selected sensor requires it. Exact J1/J2 power/excitation pins remain **TBD / DO NOT FREEZE** until the installed Rotax 912 sender set and RPM source have been surveyed.

EIU USB-C SERVICE interface — 1 October 2026

The EIU shall have an externally accessible **USB-C SERVICE** port as its primary factory programming, development, diagnostics and recovery interface. For an ESP32-S3 implementation, use the MCU native USB capability where practical rather than adding a USB-to-UART bridge solely for programming.

The USB-C SERVICE port is distinct from the normal production firmware-delivery path:

- **normal field update:** signed firmware service -> Horizon/EFIS -> AEF-CAN -> EIU inactive application slot;
- **factory/development/recovery:** service computer -> USB-C SERVICE -> EIU.

USB service shall support initial blank-board/factory flashing, development flashing, bootloader recovery, diagnostic console/manufacturing test functions and device interrogation as implemented. It is not a substitute for the AEF-CAN field-update architecture.

USB-C 5 V may power the EIU digital/service electronics for bench programming without an aircraft 12 V supply. The PCB must implement a deliberate protected power-path arrangement so USB VBUS cannot back-feed the aircraft supply and the aircraft supply cannot drive USB VBUS. Final implementation shall consider current limiting, reverse-current blocking/ORing, ESD and USB-C CC/device-role requirements. Firmware should be able to detect USB/service presence where useful.

The production enclosure interface set therefore includes **J1 ENGINE TEMP, J2 ENGINE AUX, J3 AEF-CAN and USB-C SERVICE**.

Retain small internal programming/recovery/test pads even with USB-C. At minimum provide access appropriate to the selected MCU for ground, reset/enable, boot/download control and a fallback serial/debug/programming route. These pads are a last-resort manufacturing/recovery path if USB is unavailable or misconfigured.

Factory provisioning should be scriptable over USB and may flash the bootloader, partition table and factory application; provision non-secret manufacturing identity such as serial number/hardware/PCB revision; reboot/interrogate the EIU; and run production self-tests. Security/eFuse provisioning must be a separate deliberate production step and must not be performed accidentally by ordinary development flashing.

Stage-7 architecture constraints — 1 October 2026

- **Independent power:** Horizon and EIU use separately protected aircraft-supply branches. Do not power the EIU from Horizon regulated rails.
- **CAN baseline:** protected non-isolated Classical CAN plus deliberate reference/ground design. Galvanic isolation is optional and must be justified by installation/common-mode/noise testing.
- **Fault containment:** sensor, EIU, CAN and USB/service faults must be locally contained and must not disable the core Horizon instrument or destroy the known-good recovery path.
- **EIU MCU:** ESP32-S3 is a candidate, not a frozen selection. Perform an ESP32-S3 versus suitable industrial/deterministic MCU review before EIU PCB freeze.
- **Connectors:** previous 12/12/7 pin counts are conceptual only. Freeze only after the installed engine/sender/harness survey, including thermocouple termination requirements.
- **USB-C SERVICE:** retain as local factory/workshop interface, preferably recessed/protected. USB bench power is scoped to service/digital operation; it need not power the full sensor environment.
- **Boot architecture:** use mature platform A/B OTA/rollback facilities where they meet the required known-good/candidate/trial/confirm behaviour rather than inventing a bespoke bootloader.

docs/images/README.md

Repository source: docs/images/README.md

ESP32 EFIS project images

This directory contains project documentation images that can be regenerated or revised as the design changes.

Authoritative regenerated diagrams

- `efis-system-architecture.svg` — functional bench-prototype architecture showing the selected ESP32-S3-WROOM-1-N16R2, Newhaven NHD-2.1-480480AF-ASXP, BMI088, BMP585, RM3100-CB, MCP23008/PEC09 controls, TPS61169 backlight driver, native USB and current-limited bench power. It is deliberately **not** a connector-level wiring diagram; use docs/WIRING.md and docs/HARDWARE.md for authoritative signal assignments.
- `project-stage-roadmap.svg` — visual form of the BOM procurement stages: bench bring-up, sensor integration, aircraft survey, physical-interface freeze, custom PCB and aircraft installation/validation.
- `../../../../enclosure/images/flight-development-case-preview.svg` — existing generated enclosure preview maintained with the enclosure sources.

Accuracy rule

Generated/illustrative images must never override the text engineering records. Exact GPIO assignments, connector pins, electrical limits and component variants come from the current project documentation and source files. If an image conflicts with BOM.md, docs/HARDWARE.md, docs/WIRING.md, docs/POWER_BUDGET.md or the applicable component datasheet, the image is wrong and must be corrected.

Earlier conversational concept artwork is not treated as authoritative because some versions contained guessed wiring, obsolete display details or illustrative components. Those images should not be restored unchanged. Replacement images should be generated from the current repository state.

Current design notes reflected in these images

- MCU baseline: ESP32-S3-WROOM-1-N16R2, 16 MB Quad flash / 2 MB Quad PSRAM.
- Display baseline: Newhaven NHD-2.1-480480AF-ASXP, 480 × 480, RGB565 plus 9-bit startup/configuration.
- Sensor baseline: BMI088 attitude, BMP585 pressure/altitude, remote RM3100-CB magnetic heading.
- Development power: regulated current-limited bench supply; Korad KA3005P and Siglent SPD3303X-E remain BOM alternatives.
- Final aircraft power stage and custom carrier PCB remain later-stage work and must not be inferred from bench illustrations.
- Instrument remains supplementary/non-primary.

docs/LICENSING.md

Repository source: docs/LICENSING.md

EFIS Product Licensing

Status: architecture adopted; implementation and production security validation pending.

This document is the authoritative end-to-end product-licensing process for ESP32-EFIS. Licensing is a commercial entitlement mechanism, not a flight-safety dependency. Normal startup and safe/fail-obvious behaviour must not depend on Internet, account or payment availability.

1. Trust boundaries

The system separates four trust domains:

1. **EFIS device** — stores immutable Device ID, public licence-verification key, protected installed licence and local licence state.
2. **efis-account** — customer accounts, device ownership, entitlements, payment-provider references and audit history.
3. **efis-license-signer** — private service/worker permitted to use the licence-signing private key. It is not publicly exposed.
4. **Payment provider** — handles card/wallet credentials. Raw PAN/CVV never enters EFIS apps/services.

OTA signing and licence signing are separate trust domains and use separate keys.

2. Device identity and provisioning

Each production unit receives an immutable Device ID, e.g. EFIS-00001247. This is the primary licensing identity. ESP32 factory/eFuse MAC may be recorded only as a secondary fingerprint.

Provisioning must:

- allocate a unique Device ID;
- write it into protected persistent device identity storage;
- establish any device authentication credential required for online licence retrieval;
- provision the public licence-verification key/trust anchor;
- record manufacturing/provisioning audit metadata server-side;
- verify the Device ID can be read back correctly.

The Device ID survives licence reset, Wi-Fi reset, factory reset, firmware update and ownership transfer. If missing/corrupt, firmware reports an identity fault and logs it; it never invents a replacement.

Every normal boot displays the Device ID:

```
EFIS BOOT
EFIS-00001247

> START EFIS
FULL TEST
FAULT LOG
LICENSE
FIRMWARE UPDATE
```

2A. Frozen licence cryptography and wire format

ADOPTED design choice: use **Ed25519** digital signatures over a **deterministic CBOR** licence payload.

Rationale:

- Ed25519 gives compact 32-byte public keys and 64-byte signatures, strong modern asymmetric authentication and straightforward offline verification.

- CBOR is compact, binary, implementation-friendly on embedded systems and can be encoded deterministically so the exact bytes being signed are reproducible.
- The EFIS stores only the Ed25519 public verification key. The private signing key exists only in the isolated licence-signer/key-storage boundary.
- This choice keeps licensing independent of HTTPS and OTA image signing.

Signed envelope v1

The transport artefact is a CBOR map containing:

- `v`: envelope version, initially 1;
- `alg`: fixed algorithm identifier Ed25519;
- `kid`: verification-key identifier supporting controlled key rotation;
- `payload`: byte string containing the complete deterministic-CBOR licence payload;
- `sig`: 64-byte Ed25519 signature over exactly the `payload` bytes.

The payload v1 contains integer-keyed fields for compactness and deterministic encoding:

1. schema version;
2. product ID;
3. immutable EFIS Device ID;
4. unique licence ID;
5. monotonically useful issuance sequence/identifier;
6. issued-at timestamp where server time is authoritative;
7. licence class;
8. enabled feature/entitlement set;
9. optional not-before;
10. optional expiry;
11. optional transfer/grace metadata;
12. optional support/maintenance metadata.

Unknown optional fields must be safely ignored only where the schema explicitly permits forward compatibility. Unknown mandatory schema versions are rejected.

Canonicalization and verification

The signer serializes the payload using deterministic CBOR and signs those exact bytes. The verifier never reconstructs a JSON/string representation for signature verification; it verifies the original payload byte string. After a valid signature, firmware parses the payload and validates schema, product, Device ID, entitlement and applicable time/grace policy.

Base64url may be used only as an outer text transport representation for portals/manual tooling. It is not the signed representation.

Key management

- Initial signing algorithm is Ed25519 only; algorithm agility is explicit through `alg` and schema version, not automatic fallback.
- `kid` selects a trusted public key. Firmware contains a small trusted-key set so keys can be rotated without immediately invalidating licences signed by a previous still-trusted key.
- Private keys are generated and retained in controlled server-side key storage accessible only to `efis-license-signer`; never Git, Docker images, database rows, customer apps or EFIS.
- Key rotation uses overlap: ship new public key in validated firmware first, then begin signing with the new private key, retain old public key for the defined compatibility window, and remove it only after migration policy allows.
- A compromised signing key requires an explicit incident/recovery process; do not silently accept an alternate algorithm or unsigned licence.

Implementation libraries

Use maintained, widely reviewed implementations rather than custom cryptography. Server-side Python should use a mature Ed25519 implementation from a mainstream cryptographic library and a CBOR library supporting deterministic/canonical encoding. ESP32 firmware should use an audited Ed25519 implementation compatible with the selected ESP-IDF baseline and a bounded CBOR decoder. Exact libraries and versions are pinned during implementation and validated with cross-platform test vectors.

Size and parser limits

Define strict maximum envelope/payload sizes, collection lengths, string lengths and feature counts before firmware parsing. Reject duplicate map keys, malformed/non-deterministic encodings where required, excessive nesting and trailing/unexpected data. Treat all downloaded/imported licence bytes as hostile until signature and structural checks succeed.

3. Licence payload

A signed licence should contain, at minimum:

- schema/version;
- product identifier;
- Device ID;
- unique licence identifier;
- entitlement/features;
- issue time/sequence;
- licence type;
- optional expiry/support metadata;
- ownership-transfer/grace metadata where applicable;
- key/signature algorithm identifier;
- digital signature over a canonical representation.

Do not put customer passwords, payment credentials, private signing material or unnecessary personal data into the licence.

Serialization/signature are frozen above: deterministic CBOR payload in a versioned envelope signed with Ed25519. Private key never leaves controlled signer/key storage.

4. Customer purchase and entitlement

1. Customer creates/verifies an account.
2. Customer registers the EFIS Device ID using the approved device-registration/challenge flow.
3. Server proves/validates the registration rather than accepting knowledge of Device ID alone.
4. Customer selects a licence/product.
5. Checkout is created through the payment adapter.
6. Card/Apple Pay/Google Pay/PayPal data goes directly to the payment provider.
7. Provider sends a webhook.
8. Server verifies webhook authenticity, event identity and replay/idempotency.
9. Only a verified successful payment changes entitlement to active.
10. Account service requests a licence from the private signer.
11. Signer validates the authorized request and entitlement snapshot, signs a licence and returns the signed artefact.
12. Account/audit store records issuance without storing the signing private key.

5. Online installation from the EFIS

Physical rotary/push workflow:

```
EFIS BOOT
-> LICENSE
-> Get / Refresh Licence
-> Wi-Fi Connection
-> authenticate device
-> retrieve signed licence
-> verify signature locally
-> verify Device ID/product/schema/time/state
-> store candidate
-> atomic activation
-> show ACTIVE
```

Rules:

- user account password is never entered/sent by the EFIS;
- device uses a provisioned credential/challenge mechanism;
- TLS protects transport, but licence signature is independently verified;
- invalid signature, wrong Device ID/product/schema or malformed licence is rejected;
- current valid licence is retained until the candidate has been fully verified and committed;
- licence is stored in protected/encrypted NVS in production;
- network failure never prevents START EFIS;
- Wi-Fi is maintenance-oriented and does not silently reconnect during normal flight display operation.

6. Installation through web/iPhone/Android

Customer clients show registered devices and entitlement state. They may request/retrieve a signed licence after authenticating the customer and satisfying device/entitlement policy.

Apps use normal OS secure token storage (Apple Keychain; Android Keystore-backed storage). They do not contain the signing key. A downloaded licence is already signed and untrusted until the target EFIS verifies it.

7. Offline activation

For an EFIS without usable Internet:

1. LICENSE screen shows Device ID and a short registration/challenge value.
2. Customer signs into web/iPhone/Android.
3. Customer selects **Offline Activation** and supplies/scans the challenge.
4. Account service validates device ownership and entitlement.
5. Private signer issues a signed response/licence bound to that Device ID and challenge/state.
6. Customer transfers it using the supported service/USB mechanism or a compact manual response.
7. EFIS verifies signature, Device ID, challenge/state, schema and entitlement locally.
8. Verified licence is atomically installed into protected storage.
9. EFIS reports ACTIVE and records a non-secret audit/fault-log event as appropriate.

Do not require rotary entry of a large cryptographic payload; prefer QR/service/USB or a deliberately compact challenge/response design.

8. Boot and runtime verification

Licence verification is local. At boot the firmware:

1. reads immutable Device ID;
2. reads installed licence;
3. validates structural integrity/schema;

4. verifies digital signature against embedded/provisioned public key;
5. confirms licence Device ID/product;
6. evaluates entitlement and any expiry/grace rules using trustworthy time only;
7. exposes licence status to the UI;
8. logs corruption/signature/state faults without secrets.

No network call is required for a valid offline licence.

If trustworthy wall-clock time is unavailable, firmware must not invent a timestamp or silently make an expiry decision based on false time. The time/expiry model must be explicitly designed before expiring subscriptions are shipped.

9. Firmware updates

Firmware updates normally preserve the installed valid licence. Licence storage/schema migration is versioned and rollback-safe. OTA compatibility testing includes old/current licence schemas. Firmware must not silently strand a legitimately licensed device.

10. Reset and replacement

LICENSE menu provides Status, Get/Refresh, Install/Replace and Reset.

Licence Reset:

- requires explicit user action and confirmation;
- removes installed licence/activation state;
- does not alter Device ID, device ownership, Wi-Fi credentials or calibration unless a separately confirmed operation says so;
- is audit/fault logged without secrets;
- leaves the device ready to install/retrieve a valid licence.

Corrupt/invalid licence is fail-obvious and replaceable; it does not cause identity regeneration.

11. Ownership transfer / sale

Seller chooses **Sell / Transfer EFIS** in web/iOS/Android. Seller re-authenticates and sees the Device ID, transferable entitlements, exclusions and consequences.

Normal transfer:

1. server verifies seller owns the Device ID;
2. seller initiates transfer;
3. server creates random, single-use, short-lived transfer credential stored hashed;
4. buyer signs into their own account and accepts invitation/code;
5. server atomically moves device ownership and transferable entitlements;
6. payment instruments never move;
7. historical purchase records remain with original purchaser subject to retention/privacy policy;
8. signer issues a fresh buyer licence for the unchanged Device ID;
9. both accounts receive transfer status/receipt.

Pending transfer can be cancelled before acceptance. Device cannot belong to two accounts.

Transfer grace

Default grace is **30 days from buyer acceptance**. The seller-era installed licence continues to work offline during this bounded period so sale/registration cannot abruptly disable the instrument. The duration is server-configurable but cannot be shortened retrospectively after transfer acceptance.

Installing the buyer's replacement licence ends transfer grace early. Exact post-grace product behaviour must be frozen before commercialization and must never produce misleading flight indications.

Exceptional recovery transfers require privileged, audited administrative evidence/workflow and cannot bypass normal ownership controls casually.

12. Entitlement lifecycle

State model should distinguish at least:

NONE -> PENDING_PAYMENT -> ACTIVE -> TRANSFER_PENDING -> TRANSFER_GRACE/SUPERSEDED, plus product-specific EXPIRED, REFUNDED, CHARGEBACK, REVOKED where policy permits.

A server entitlement state and an already-installed offline licence are not identical. Revocation cannot be assumed instantaneous on an offline EFIS. Policies must reflect that technical reality.

Subscription, refund/chargeback, revocation and non-transferable-feature policies remain to be frozen before sale.

13. Audit requirements

Server audit events include account/device registration, entitlement creation/change, payment webhook outcome, licence issuance/reissue, reset request, transfer initiation/cancel/accept, administrative intervention and security-relevant failures.

Do not log passwords, card PAN/CVV, signing keys, Wi-Fi credentials, raw authentication secrets or unnecessary licence secrets.

EFIS persistent fault/diagnostic log records stable non-secret codes for identity/licence verification/storage failures and significant licence transitions, wear-consciously.

14. Failure behaviour

- Account/payment/licence server unavailable: existing valid licence and START EFIS continue.
- Wi-Fi/DNS/TLS failure: report maintenance error; START EFIS remains.
- Invalid downloaded signature: reject candidate, retain current valid licence.
- Wrong Device ID/product: reject.
- Storage failure: fail obvious; retain prior valid copy where technically possible.
- Missing/corrupt Device ID: identity fault; never fabricate.
- Signer unavailable: entitlement remains recorded; retry issuance safely/idempotently.
- Duplicate payment webhook: idempotent; never duplicate entitlement/payment effects.

15. Security requirements before production

- isolated signer and controlled key storage/rotation/recovery;
- authenticated, authorized account and device APIs;
- email verification, password reset, MFA/re-authentication for sensitive actions;
- CSRF/rate limiting/session expiry/revocation;
- signed and replay-resistant device challenge protocol;
- verified/idempotent payment webhooks;
- protected/encrypted device NVS;
- database migrations, encrypted backups and restore tests;
- privacy/retention/export/deletion policy;
- audit monitoring and administrative RBAC;

- licence signature test vectors and malformed-input/fuzz testing;
- penetration/security review.

16. Required tests

Test known-good and negative cases for provisioning, duplicate IDs, account registration, ownership proof, payment success/failure/replay, signer failure, valid/invalid/wrong-device licences, NVS corruption, power loss during replacement, offline installation, firmware migration/rollback, licence reset, transfer/cancel/accept, 30-day grace boundaries, buyer replacement, clock unavailable/incorrect, and recovery/admin flows.

Physical ESP32 tests are required. Simulator/API tests alone do not establish production readiness.

17. Current implementation status

Implemented/staged: account-service prototype, PostgreSQL account/device/entitlement model, web portal prototype, iOS/Android customer skeletons, Device ID architecture, payment-provider abstraction requirements, transfer/grace policy and this process specification.

Pending: production device provisioning mechanism, authenticated customer/device API, registration challenge, signer container/key management, signed licence format/crypto selection, verified payment adapters, online/offline installation firmware, protected NVS licence store, transfer implementation, security hardening and physical validation.

docs/MACOS_BUILD_AND_QEMU_SETUP.md

Repository source: docs/MACOS_BUILD_AND_QEMU_SETUP.md

macOS build and QEMU setup

This is the authoritative setup/runbook for building and running ESP32 EFIS firmware on Apple Silicon macOS. The project targets **ESP32-S3** and is pinned to **ESP-IDF v5.4.4**. QEMU and synthetic data are development-only; aircraft-target builds must keep simulation disabled.

Build scripts — preferred workflow

The repository now contains the complete routine build/run workflow under `scripts/`. Use these scripts instead of repeatedly typing the underlying `idf.py` commands:

- `scripts/efis-env.sh` — prepares the pinned ESP-IDF/Python/QEMU shell environment. **Source** this script.
- `scripts/build-qemu.sh` — builds the QEMU configuration in `firmware/build-qemu`. Pass `--clean` to delete and completely reconfigure it first.
- `scripts/run-qemu.sh` — starts graphical QEMU and automatically saves console output to `firmware/qemu.log`.
- `scripts/build-hardware.sh` — builds the real ESP32-S3 configuration in the separate `firmware/build` directory. Pass `--clean` for a fresh hardware configuration.

From a new Terminal, the normal emulator workflow is:

```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
source scripts/efis-env.sh
zsh scripts/build-qemu.sh
zsh scripts/run-qemu.sh
```

After QEMU/Kconfig/default changes, an ESP-IDF/Python mismatch, or whenever a genuinely clean emulator build is required:

```
zsh scripts/build-qemu.sh --clean
zsh scripts/run-qemu.sh
```

For normal hardware firmware:

```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
source scripts/efis-env.sh
zsh scripts/build-hardware.sh
```

Use `zsh scripts/build-hardware.sh --clean` when a fresh hardware configuration is required. **Never flash** `build-qemu` to hardware.

The scripts deliberately do not source `efis-env.sh` themselves: environment activation must modify the current interactive shell and therefore `efis-env.sh` is sourced once at the start of a development session.

One-time prerequisites

Install Apple's command-line tools and Homebrew as required. The Espressif QEMU installation used here requires:

```
xcode-select --install
brew install libgcrypt sdl2
```

Observed library locations include `/opt/homebrew/opt/libgcrypt/lib/libgcrypt.20.dylib` and `/opt/homebrew/opt/sdl2/lib/libSDL2-2.0.0.dylib`.

Repository and ESP-IDF

The normal checkout is `~/Documents/Xcode/ESP32-EFIS`. The pinned ESP-IDF installation is `~/espressif/v5.4.4/esp-idf`.

Install/repair the ESP32-S3 tools with:

```
~/espressif/v5.4.4/esp-idf/install.sh esp32s3
```

Environment script

Run this at the beginning of each new Terminal development session:

```
cd ~/Documents/Xcode/ESP32-EFIS
source scripts/efis-env.sh
```

It activates ESP-IDF v5.4.4, forces `~/espressif/tools/python/v5.4.4/venv/bin/python`, removes competing Espressif Python environment bin directories from PATH, defines `idf.py` through the pinned interpreter, exports `EFIS_ROOT` and `EFIS_FIRMWARE_DIR`, locates the installed Espressif QEMU binary, and fails early if the active Python is inconsistent.

This prevents the observed CMake mismatch between `~/espressif/python_env/idf5.4_py3.14_env/bin/python` and `~/espressif/tools/python/v5.4.4/venv/bin/python`. Existing CMake build directories remember their configuring interpreter, so use the appropriate build script with `--clean` after such a mismatch.

Install QEMU once

```
source scripts/efis-env.sh
python "$IDF_PATH/tools/idf_tools.py" install qemu-xtensa
source scripts/efis-env.sh
```

Verify with `which qemu-system-xtensa` and `qemu-system-xtensa --version`. The original installation was `qemu-xtensa@esp_develop_9.2.2_20250817`; the environment script searches installed versions rather than hard-coding that release.

What the QEMU build script does

`zsh scripts/build-qemu.sh` changes to `$EFIS_FIRMWARE_DIR` and builds with:

```
idf.py -B build-qemu \
-D SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.qemu.defaults" \
build
```

If `build-qemu` has not yet been configured, or `--clean` is supplied, it first runs:

```
idf.py -B build-qemu \
-D SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.qemu.defaults" \
set-target esp32s3
```

The QEMU defaults enable `CONFIG_EFIS_BENCH_SIMULATION` and `CONFIG_EFIS_QEMU` and explicitly contain `#CONFIG_SPIRAM` is not set. This PSRAM override is essential because the physical N16R2 has external Quad PSRAM but the QEMU configuration does not provide that physical device. Without the override, boot asserted in `s_psram_chip_init` before `app_main()`.

What the QEMU run script does

`zsh scripts/run-qemu.sh` verifies that `build-qemu` exists, removes the previous `firmware/qemu.log`, and runs:

```
idf.py -B build-qemu qemu --graphics monitor 2>&1 | tee qemu.log
```

This opens the 480×480 RGB565 virtual display and retains the console log for diagnosis. Exit with `Ctrl-]`. The current QEMU acceptance stage locks the display to the Artificial Horizon/PFD and shows the test name/countdown directly on-screen.

For panic diagnosis, inspect the saved log with:

```
grep -B 60 -A 15 -E "Guru|panic|abort|assert|Backtrace|Rebooting" qemu.log | tail -200
```

What the hardware build script does

`zsh scripts/build-hardware.sh` uses only the normal `firmware/build` directory and normal defaults. It never references `sdkconfig.qemu.defaults`. The physical build retains the ESP32-S3-WROOM-1-N16R2 PSRAM configuration and simulation remains OFF by default.

The underlying build is:

```
idf.py -B build set-target esp32s3 # first/clean configuration
idf.py -B build build
```

Before any aircraft use, verify the boot log reports `BENCH SIMULATION: OFF`.

Troubleshooting

`qemu-system-xtensa` **not found**: source `scripts/efis-env.sh`; it handles the nested Espressif QEMU installation path.

Python environment mismatch: source `scripts/efis-env.sh`, confirm the pinned Python shown in its summary, then run `zsh scripts/build-qemu.sh --clean` or `zsh scripts/build-hardware.sh --clean` as appropriate.

QEMU reboots in `esp_psram_init`: pull the current repository and run `zsh scripts/build-qemu.sh --clean`. QEMU must use `sdkconfig.qemu.defaults` with SPIRAM disabled.

`dyld: Library not loaded`: install the named Homebrew dependency; the observed requirements were `libgcrypt` and `sdl2`.

QEMU build does not exist: run `zsh scripts/build-qemu.sh --clean` before `zsh scripts/run-qemu.sh`.

Simulation unexpectedly enabled in a hardware build: stop. Run `zsh scripts/build-hardware.sh --clean` and verify `BENCH SIMULATION: OFF` before hardware/aircraft use.

Safety boundary

QEMU and bench simulation are development facilities only. Synthetic data must never activate as a fallback for a missing, failed or stale real sensor. A real-source failure must produce an unmistakable invalid indication. Keep `build` and `build-qemu` separate. Never flash `build-qemu` to aircraft hardware. The ESP32 EFIS remains a supplementary/non-primary experimental instrument.

docs/OTA_IMAGE_ADMIN.md

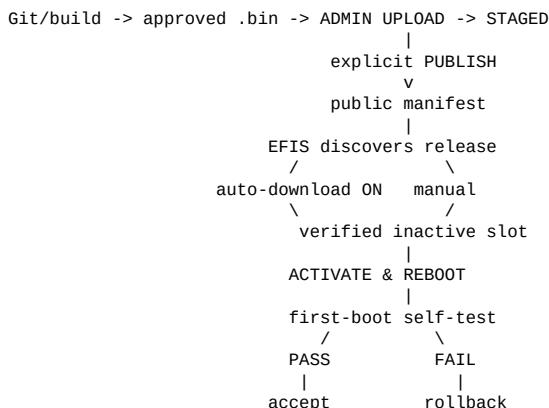
Repository source: docs/OTA_IMAGE_ADMIN.md

ESP32 EFIS — Hosted firmware administration and definitive OTA workflow

Status: admin workflow and simulator implemented; Synology/physical ESP32 validation pending.

Policy

ESP32 EFIS separates **Upload**, **Publish**, **Download**, and **Activate**. These are independent decisions.



Publishing never causes an EFIS to reboot. Automatic download never means automatic activation.

Docker administrator

The private browser dashboard manages the actual firmware images hosted by the OTA Docker server. Uploading an approved .bin creates a **STAGED** release and records version, build, minimum allowed version, release notes and server-calculated SHA-256 in per-release metadata. Upload does not change the public manifest.

An explicit **Publish** action atomically changes manifest.json. The previously published image remains hosted and becomes archived/staged. Any retained older version can be deliberately republished, providing an administrative release rollback. The currently published version cannot be deleted.

The public nginx service remains read-only. The admin service alone has read/write access to ota-server/public. Admin port 127.0.0.1:8090 remains private; public EFIS downloads use the HTTPS reverse proxy to the read-only origin on 127.0.0.1:8080.

EFIS user behaviour

The normal user has one persistent preference:

```
Automatically download next update  ON/OFF
```

Default is **ON**. Automatic activity is permitted only in the maintenance/network-update context, never as an unannounced flight operation.

With automatic download ON, a compatible administrator-published release is downloaded to the inactive OTA slot and verified. It is then left dormant. The user sees a deliberately simple decision:

```
SOFTWARE UPDATE READY
Version 0.4.2
Downloaded and verified

[ ACTIVATE & REBOOT ]
  Later
```

There is no automatic activation or automatic reboot. With automatic download OFF, the user first sees the available release and selects **Download**; after verification the same **ACTIVATE & REBOOT** decision is presented.

Activation selects the candidate OTA slot and reboots. The candidate remains pending until the mandatory first-boot self-test succeeds. Success marks it known-good. Failure/crash/reset before confirmation invokes the A/B rollback design and restores the previous known-good application.

Administrator release workflow

1. Build/test an intended firmware image separately.
2. Open the private OTA Admin dashboard.
3. Upload the approved .bin; it becomes STAGED only.
4. Review version/build/release notes/SHA-256.
5. Select **Publish** deliberately.
6. Confirm the public HTTPS manifest identifies the intended release.
7. EFIS units may now discover/download it according to their preference.
8. Each user still explicitly selects **ACTIVATE & REBOOT**.

A bad publication can be withdrawn by republishing a retained previous image. Devices that already downloaded a newer candidate must still apply compatibility/policy checks before activation; production implementation should support invalidating a withdrawn candidate when the manifest changes.

Persistent server data

Back up:

```
ota-server/public/efis/manifest.json
ota-server/public/efis/releases/
ota-server/public/efis/release-metadata/
```

Containers remain disposable/rebuildable from Git.

Security

Staging/publishing is not firmware signing. Production images still require the signed-image trust model in REMOTE_UPDATES.md. Signing private keys never belong in the Docker admin container. The admin UI stays behind LAN/VPN/private authenticated reverse proxy; its Flask session secret is not user authentication.

Before any Internet-facing multi-user admin deployment add authentication, CSRF protection, audit history, upload limits and preferably signed-image verification before Publish.

Validation

The Docker admin code now implements separate staging and publishing, per-release metadata, SHA-256 calculation, republishing and protected deletion. The Swift simulator implements automatic-download preference, manual download, verified-ready state, explicit **ACTIVATE & REBOOT**, first-boot success and rollback simulation.

Not yet validated: Docker build/runtime on Synology, public reverse-proxy deployment, physical ESP32 download/flash, real A/B boot selection, interruption recovery or signed-image verification. Simulator behaviour must not be represented as physical OTA validation.

Remote EIU firmware delivery — 1 October 2026

Horizon is the update gateway for the Stage-7 EIU. Production flow is:

signed release service -> Horizon OTA client -> local verified staging -> AEF-CAN maintenance transfer -> EIU inactive A/B slot -> EIU verification -> explicit activation -> health confirmation/rollback.

The EIU is not provisioned with Internet/Wi-Fi credentials. Release metadata must distinguish Horizon images from EIU images and declare target product, supported hardware/board revision, minimum bootloader/protocol compatibility, version, size, digest and signature information.

Horizon verification before CAN transfer is required but is not the final trust boundary: the EIU independently authenticates the image before boot. EIU persistent sensor mappings/calibration/identity remain outside the application slots.

Extend the existing simulator/admin harness with an emulated EIU target and at least two visibly/version-distinct test images. Exercise successful update, wrong-hardware rejection, corrupt image/hash failure, interrupted transfer, activation, first-boot failure and rollback, EIU disappearance/reappearance, and preservation of commissioned configuration. Keep these results explicitly labelled simulation until real transceivers, real flash A/B behaviour and interruption tests pass on hardware.

Simulated EIU OTA implementation — 1 October 2026

The first executable transport-neutral EIU update state machine is in `ota-server/admin/eiu_ota_sim.py`, with regression coverage in `ota-server/admin/test_eiu_ota.py`. The normal `scripts/rebuild-and-test.sh` runner now executes this suite inside the admin image after the existing isolated OTA-admin tests.

Current simulation covers discovery/version reporting, target-hardware rejection, segmented transfer, end-to-end SHA-256 corruption detection, inactive-slot staging, RPM=0 activation gate, successful first-boot confirmation, failed-first-boot rollback and preservation of commissioned sensor configuration. Test payloads deliberately identify two distinct simulated releases (EIU-V2-GREEN-LED and EIU-V3-BLUE-LED).

This remains a software model. It does not yet implement AEF-CAN frame segmentation, cryptographic signature verification, resume after interruption, ESP32 partition-table flashing or physical bootloader rollback. Those are subsequent gates and must not be described as hardware-validated.

Two-stage download checkpoints — 1 October 2026

Both firmware-delivery legs are checkpointed.

Release service -> Horizon: Horizon stages firmware persistently and records absolute downloaded offset/image identity so an interrupted network download can resume rather than restart. The complete staged image must pass digest/signature verification before it is offered to the EIU.

Horizon -> EIU: AEF-CAN DATA remains sequence checked and the EIU CRC-verifies the inactive-slot candidate in 4 KiB transfer blocks and records durable resume state nominally every 64 KiB. Each durable checkpoint binds the transfer/image identity to the committed byte offset and block integrity. On restart Horizon queries the EIU and resumes only when identities match.

The UI may display percentage, but the stored/protocol state is always an absolute byte offset. Final whole-image verification remains mandatory even when every intermediate checkpoint passed.

End-to-end interruption harness — 1 October 2026

The executable OTA harness now covers the complete simulated path:

release service -> persistent Horizon/EFIS staging -> verified staged EIU image -> AEF-CAN -> EIU inactive slot -> activation/confirmation.

`end_to_end_ota_sim.py` models persistent EFIS staging with 64 KiB network-download checkpoints. `test_end_to_end_ota.py` deliberately interrupts the server-to-EFIS download, reconstructs the EFIS-side object from persistent state, resumes at the committed byte offset and requires complete SHA-256 verification before the image can enter the CAN stage.

The same test then interrupts EFIS-to-EIU delivery after multiple prototype 4 KiB EIU checkpoints (this harness predates the adopted 4 KiB-block/64 KiB-durable-checkpoint production transport and is scheduled for revision), recreates the volatile EIU/CAN endpoint while preserving durable checkpoint state, resumes from the EIU's reported committed offset, verifies the complete image,

confirms that the old EIU application remained active throughout transfer, and finally exercises explicit activation and first-boot confirmation.

The normal Docker rebuild/test script executes this end-to-end suite. These tests validate software recovery semantics only; actual HTTP range requests, ESP32 flash wear/durability, physical power interruption, TWAI/CAN faults and cryptographic signature verification remain hardware/integration gates.

Firmware transport architecture decision — 1 October 2026

Following engineering review, the stop-and-wait CAN transport is retained only as prototype evidence. It is not the intended production protocol because acknowledging every six-byte firmware frame creates excessive frame count, acknowledgement traffic and idle wait time.

Production EIU delivery uses **7 firmware bytes + 1 local sequence byte per DATA frame**, grouped into **4 KiB CRC32-verified transfer blocks** with one block ACK/NACK. Durable restart metadata is committed every **64 KiB** rather than every block. This separates short-range corruption/retransmission recovery from persistent power-loss recovery and reduces metadata flash writes.

On restart, transfer ID, image identity/digest and absolute committed byte offset are compared. Matching transfers resume from the last durable 64 KiB checkpoint; otherwise the partial candidate is not reused. The final candidate still requires complete SHA-256 and signature verification before activation.

Firmware maintenance traffic must yield to normal AEF-CAN operational traffic. The next simulator revision must replace per-frame ACK behaviour with block ACK/NACK, inject within-block corruption/loss, verify retransmission, exercise restart between durable checkpoints and demonstrate that maintenance transfer does not starve higher-priority traffic.

docs/OTA_PHYSICAL_TEST_NODE.md

Repository source: docs/OTA_PHYSICAL_TEST_NODE.md

Minimal Physical OTA Test Node

Purpose

Provide the smallest possible physical ESP32-S3 setup for exercising the ESP32-EFIS OTA lifecycle independently of the EFIS display, sensors, GNSS, CAN/EIU and other aircraft hardware.

This is a development and regression-test fixture. It is not an aircraft instrument.

Minimum hardware

- One ESP32-S3 development board
- USB cable for power, initial programming and serial diagnostics
- Wi-Fi network with access to the ESP32-EFIS OTA service
- One visible status LED
- Prefer the board's controllable onboard LED where suitable.
- Otherwise use one external LED, approximately 330 ohm series resistor, and a spare GPIO.

No display, AHRS, pressure sensors, GNSS receiver, CAN interface or EIU is required.

Two permanent test images

Maintain two deliberately trivial firmware builds. They should differ visibly and identify themselves over serial and the device status page.

| Image | Example version | Visible indication | Serial identity |

| --- | --- | --- | --- |

| OTA-A | 0.1.0-A | slow LED blink, approximately 1 Hz | ESP32-EFIS OTA TEST - IMAGE A |

| OTA-B | 0.1.0-B | fast LED blink, approximately 4 Hz | ESP32-EFIS OTA TEST - IMAGE B |

A and B should remain permanently buildable so OTA regression tests can repeatedly alternate A -> B -> A without depending on production EFIS firmware.

Required OTA behaviour

Publishing an image must NOT automatically install or activate it on a device.

The physical test node must preserve distinct user-controlled states:

1. **Running** - current active image.
2. **Available** - newer/different image discovered in the published manifest.
3. **Downloaded/staged** - image has been pulled into the inactive OTA partition and verified.
4. **Activation requested** - user explicitly authorises switching to the staged image.
5. **Rebooting** - boot selection changes and the device restarts.
6. **Healthy** - new image completes its startup checks and confirms itself.
7. **Rollback** - later negative tests deliberately fail health confirmation and prove recovery.

The basic experiment is:

Image A running

|

v

publish Image B

|

v

CHECK FOR UPDATE

|

v

B shown as available

|

v

DOWNLOAD

|

v

B written to inactive OTA partition

|

v

SHA-256 / image verification succeeds

|

v

ACTIVATE & REBOOT

|

v

Image B boots

|

v

fast LED + version B confirms success

Repeat in the opposite direction to test B -> A.

Minimal user interface

For the first physical harness, host a very small status/control web page on the ESP32 so a Mac, iPhone or other browser on the test network can operate it.

Example:

ESP32-EFIS OTA TEST NODE

Running firmware: 0.1.0-A

Published firmware: 0.1.0-B

Downloaded firmware: none

Wi-Fi: Connected

OTA server: Reachable

[CHECK FOR UPDATE]

Update 0.1.0-B available

[DOWNLOAD]

Firmware downloaded

SHA-256: OK

Ready for activation

[ACTIVATE & REBOOT]

The interface must make it obvious that **DOWNLOAD** and **ACTIVATE & REBOOT** are separate user actions.

Serial logging should expose the same state transitions for diagnosis and automated testing.

Partitioning

Use an ESP-IDF OTA-capable partition table with OTA metadata and two application slots (for example ota_0 and ota_1). The test project should use the same fundamental A/B update mechanism intended for production ESP32-EFIS firmware.

Exact partition sizes should be selected when the test firmware is implemented, with enough headroom to test images representative of expected production firmware.

Suggested repository implementation

Create a self-contained test project, for example:

```
tools/  
ota-device-test/  
README.md  
CMakeLists.txt  
sdkconfig.defaults  
partitions.csv  
main/  
CMakeLists.txt  
main.c  
wifi.c  
ota.c  
webui.c
```

The implementation should be intentionally small, but the OTA state machine and verification rules should be reusable or representative of the production implementation.

Relationship to the server harness

The physical node complements the existing Synology OTA server/admin test harness.

ESP32-S3 physical test node

|

| Wi-Fi / HTTPS

v

ESP32-EFIS OTA service

|

+-- manifest
+-- Image A
+-- Image B
+-- image hashes / metadata

Server-side tests prove staging, publishing, manifests and downloads. The physical node proves that a real ESP32 can discover, pull, verify, stage and activate those published images.

Initial acceptance tests

The first hardware milestone is complete when all of the following have been demonstrated:

- Image A can be installed initially by USB.
- A visibly identifies itself with the slow LED pattern.
- B can be published through the OTA administration workflow.
- A discovers B without automatically installing it.
- User can explicitly download B.
- Downloaded B is stored in the inactive OTA slot.
- Image integrity is verified before activation.
- A continues running after download until activation is requested.
- User can explicitly request ACTIVATE & REBOOT.
- B boots and visibly identifies itself with the fast LED pattern.
- Device reports B as the running version.
- The process can be reversed by publishing and installing A again.

Later destructive/negative tests

After A/B operation is reliable, add a third intentionally unsuitable test image (for example OTA-C-BAD) solely for controlled bench testing. It can deliberately withhold successful startup confirmation so ESP-IDF rollback behaviour can be verified.

Later tests should cover:

- interrupted download
- loss of Wi-Fi during download
- corrupt image/hash mismatch
- malformed or unavailable manifest
- power interruption before activation
- power interruption during reboot/first boot where practical
- failed post-boot health check
- automatic rollback
- inability to activate an unverified image
- repeated A/B/A regression cycles

Safety/design rule

OTA publication, download and activation are separate events.

The production design must never interpret publication alone as permission to change the running EFIS firmware. The physical test node exists specifically to make that lifecycle observable and testable before the mechanism is integrated with the full EFIS application.

docs/OTA_USER_SCENARIO.md

Repository source: docs/OTA_USER_SCENARIO.md

ESP32 EFIS — OTA user software scenario

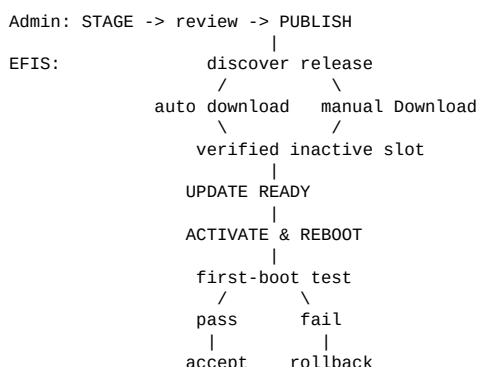
Status: definitive interaction implemented in Swift simulation; physical ESP32 OTA remains pending.

Date: 16 September 2026

Definitive user model

Administrator publication and instrument activation are separate controls. A firmware binary is first **STAGED** on the Docker OTA server, then explicitly **PUBLISHED** by the administrator. Only the published manifest is consumable by EFIS units.

On the instrument, **Automatically download next update** defaults ON. This means discovery/download/verification may occur while the EFIS is deliberately in its maintenance/network-update context. It never means automatic activation or reboot.



Simulator

simulator/ESP32EFISimulator/OTAFlowView.swift implements this user-facing policy. The page is permanently labelled **OTA USER-FLOW SIMULATION** and performs no real networking, flash writes or reboot.

The maintenance page exposes the persistent auto-download preference. With it ON, checking a published update proceeds to simulated download and stops at **Update ready**. With it OFF, the user sees **Update available** and must select **Download**. Both paths converge on the same verified state:

```
SOFTWARE UPDATE READY
Version <candidate>
Downloaded and verified

[ ACTIVATE & REBOOT ]
  Later
```

There is no auto-activation mode. Selecting **Later** leaves activation for a future maintenance session in the production design.

Activation and rollback

ACTIVATE & REBOOT is the explicit boundary between having a verified dormant candidate and attempting to run it. The production ESP32 implementation will select the inactive OTA slot and reboot. The new image remains pending until mandatory first-boot self-test succeeds. Failure, crash or reset before confirmation must restore the previous known-good image.

The simulator's **Simulate first-boot failure** switch exercises the user-visible rollback path. This is behavioural simulation, not evidence that ESP-IDF rollback has been hardware validated.

User-facing rules

- software update functions belong to maintenance, not normal flight operation;

- only an administrator-published release is offered;
- auto-download may fetch/verify a candidate but may not activate it;
- manual mode requires Download;
- activation always requires local **ACTIVATE & REBOOT**;
- current software remains recoverable while writing the inactive slot;
- no normal-looking flight page is presented while first-boot verification is incomplete;
- failure explicitly reports restoration of previous known-good software;
- no synthetic sensor value may become a fallback flight-data source.

Relationship to Docker OTA server

The private Docker admin UI owns staging/publishing. The public nginx origin exposes only the current manifest and versioned binaries. A new upload is invisible to EFIS units until Publish is deliberately selected. Republish of a retained older release provides administrative withdrawal/rollback of what is advertised.

Validation status

IMPLEMENTED — SWIFT USER-FLOW SIMULATION / NOT PHYSICAL OTA VALIDATION. Still required are the real A/B partition table, maintenance Wi-Fi, HTTPS client, manifest parser, OTA writer, signed-image verification, boot-slot selection, pending-verify self-test, mark-valid/rollback calls, interruption testing and USB recovery testing.

Running the simulator

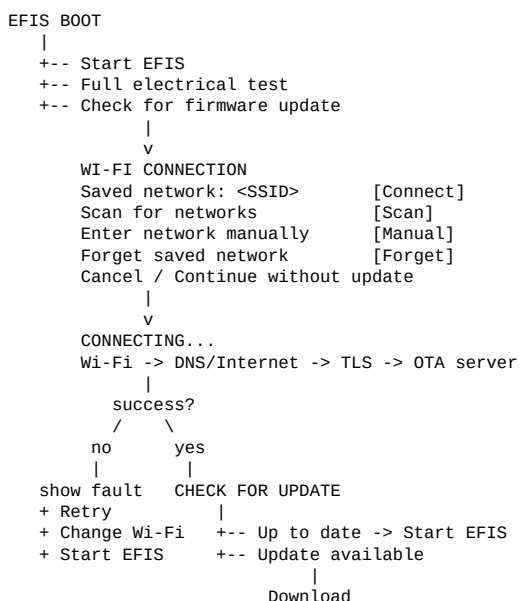
```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
cd simulator
xcodegen generate
open ESP32EFISimulator.xcodeproj
```

Exercise: auto-download ON success, auto-download OFF/manual Download, Later , successful Activate & Reboot, and simulated first-boot failure/rollback. Repository implementation is not by itself an Xcode compile/runtime validation claim.

Boot-time Wi-Fi connection dialog

When boot offers **Check for update**, the update path must include an explicit Wi-Fi connection dialog before any network request. The EFIS must not silently enable/reconnect Wi-Fi during normal instrument operation.

Recommended boot/update sequence:



```

      |
verify inactive slot
      |
ACTIVATE & REBOOT

```

The dialog shall support an iPhone Personal Hotspot and ordinary WPA2/WPA3 Personal networks supported by the ESP32. It should show SSID and connection state but never display a stored password after entry. Credentials are stored persistently in a dedicated ESP-IDF NVS namespace after deliberate connection/save behaviour. Production units use NVS encryption for stored Wi-Fi credentials; ordinary NVS is permitted only during early prototype bring-up. The password is never redisplayed after entry and must never be logged, included in diagnostics/test results, or sent to the OTA server. **Forget network** erases the saved credentials.

Failure must be recoverable: inability to establish Wi-Fi, Internet, TLS or OTA-server connectivity must **not prevent the existing known-good EFIS firmware from starting**. The dialog should identify the failed layer and offer **Retry**, **Change Wi-Fi**, and **Start EFIS**.

Firmware activation remains separate from networking. A successful Wi-Fi connection only permits Check/Download; a verified candidate still requires the explicit local **ACTIVATE & REBOOT** action.

Physical boot-menu control

The **Bourns PEC09 rotary encoder with push switch** is the sole normal control for the boot menu. No touchscreen is assumed.

At boot the round display presents a high-contrast menu such as:

```

EFIS BOOT

> START EFIS
  FULL TEST
  FIRMWARE UPDATE

Rotate = select
Press  = enter

```

Control semantics:

- rotate clockwise/counter-clockwise: move the highlight one menu item;
- short press: select/enter the highlighted item;
- in sub-dialogs, rotation moves between fields/options and short press confirms/edits;
- long press is reserved for **Back/Cancel** where required, subject to physical encoder validation;
- firmware update enters the Wi-Fi Connection dialog, which is also completely operable using rotate + press;
- Full Test enters the documented electrical/component test harness;
- Start EFIS immediately continues normal startup.

The default highlighted item is **START EFIS**. A configurable boot-menu timeout may later auto-select Start EFIS, but no timeout value is frozen until physical usability testing. Firmware update and Full Test must never start merely because of a timeout.

The UI must remain usable if networking is absent. A failed update/network attempt always provides a rotary-selectable **Start EFIS** escape path.

docs/PARTS_SELECTION.md

Repository source: docs/PARTS_SELECTION.md

Reference Parts Selection and Sourcing

Checked: 16 September 2026.

This document records the reference hardware selected for the ESP32 artificial-horizon prototype. Availability and prices are time-sensitive and should be rechecked before ordering.

Purchase links — UK

Use the **manufacturer part number** as the authoritative identifier before ordering.

| Part | Preferred purchase link | Procurement status |

|---|---|---|

| ESP32 module | DigiKey UK — ESP32-S3-WROOM-1-N16R2 | **Ordered — 2 units, 14 Sep 2026** |

| Bosch BMI088 Shuttle Board 3.0 | DigiKey UK — SHUTTLE BOARD 3.0 BMI088 | **Ordered — 14 Sep 2026** |

| Newhaven NHD-2.1-480480AF-ASXP | Mouser — MFG P/N NHD-2.1-480480AF-ASXP | **Ordered and shipped — 1 unit; ordered 14 Sep, shipped 15 Sep 2026** |

| Newhaven NHD-FFC40 adapter | RS UK — NHD-FFC40 | **Ordered** |

| Adafruit TPS61169 PID 6354 | DigiKey order | **Ordered — 14 Sep 2026** |

| Microchip MCP23008-E/P | DigiKey order | **Ordered — 14 Sep 2026** |

| Optical front window | Diamond Coatings — hard-coated AR polycarbonate | Preferred final window source; not recorded as ordered |

| Compact rotary encoder | Bourns PEC09-2320F-T0015 | **Ordered — 14 Sep 2026** |

Procurement evidence — display

Mouser invoice 92509947 records one NHD-2.1-480480AF-ASXP, quantity ordered 1, shipped 1, pending 0. Order date was 14 September 2026 and ship date 15 September 2026. The merchandise price was £18.37 before freight/VAT. This confirms the selected display is already in transit and must not be treated as an outstanding purchase.

MCU — Espressif ESP32-S3-WROOM-1-N16R2

Frozen reference choice: ESP32-S3-WROOM-1-N16R2.

Why this is preferred over the earlier DevKitC approach:

- **16 MB Quad flash** gives generous firmware and graphics headroom;
- **2 MB Quad PSRAM** is enough for two complete 480×480 RGB565 frame buffers;
- Quad PSRAM preserves GPIO35–37, unlike Octal-PSRAM configurations that consume those signals;
- the bare module is active and widely stocked;
- the final instrument can use a compact project-specific carrier PCB rather than carrying unused development-board hardware;
- native ESP32-S3 USB can be routed directly to the rear USB-C connector.

Two 480×480 RGB565 frame buffers consume approximately **921,600 bytes**, leaving useful PSRAM headroom.

Why not N8R8 / Octal PSRAM

The extra PSRAM would be attractive in isolation, but the current instrument is **GPIO-constrained rather than memory-constrained**. The 16-bit RGB display, timing signals, BMI088 SPI, PWM and control functions make GPIO availability more valuable than an additional 6 MB of PSRAM.

Espressif documents GPIO35, GPIO36 and GPIO37 as part of the Octal memory interface. The current project uses those GPIOs for the shared LCD-configuration/BMI088 SPI path, so Octal-PSRAM variants are a poor fit without a wider GPIO redesign.

Bench-development option

If an ESP32-S3-DevKitC-1-N8R2 is already on hand, it remains a useful **bench-only development board** because it has the same ESP32-S3 core and 2 MB Quad PSRAM. It is no longer the preferred final hardware because the exact DevKit variant is obsolete at major distributors.

Custom processor carrier PCB

The final PCB should mount the N16R2 module directly and provide:

- 5 V USB-C input
- low-noise 3.3 V regulation
- bulk and local decoupling
- native USB D-/D+ on GPIO19/GPIO20
- EN/reset circuit
- BOOT access on GPIO0
- test/programming pads
- display RGB/timing connector
- shared LCD-init/BMI088 SPI connector
- I²C connector for MCP23008
- TPS61169 PWM connection
- short high-speed traces and continuous ground reference

The current 3D-printed electronics carrier should therefore be treated as a development holder. Its edge rails will be revised around the final custom PCB dimensions once the PCB outline is frozen.

Optical window specification

The front protective window is specified as **2.0 mm optical clear polycarbonate with a scratch-resistant hard coat and anti-reflective coating on both sides**. The CAD freezes the prototype disc at **62.0 mm diameter**.

Preferred sources:

- Diamond Coatings hard-coated polycarbonate, AR both sides, AFP one side
- Itotek AR-coated acrylic/polycarbonate display windows

IMU — Bosch Sensortec SHUTTLE BOARD 3.0 BMI088

Selected part: Bosch Sensortec SHUTTLE BOARD 3.0 BMI088.

The rigid IMU cradle is based on the approximately 22 × 14 mm external PCB envelope and avoids assuming an unverified mounting-hole pattern.

Display — Newhaven NHD-2.1-480480AF-ASXP

Procurement: one display is ordered and shipped from Mouser.

Key characteristics:

- 2.1-inch round IPS

- 480 × 480
- 1000 nit typical luminance
- ST7701S
- active area 53.28 × 53.28 mm
- outline 58.18 × 60.71 × 2.26 mm
- 40-pin 0.5 mm FFC
- VDD 2.5–3.3 V
- backlight approximately 6.0 V / 100 mA

The project uses **16-bit RGB565** for pixels and 9-bit serial initialization for the ST7701S.

Display adapter

Prototype adapter: Newhaven NHD-FFC40 — ordered.

For the final compact PCB, Newhaven's datasheet names **Molex 54104-4031** as the example 40-pin 0.5 mm FFC connector.

Backlight driver — Adafruit TPS61169 PID 6354

Selected for the prototype because it accepts the 5 V rail, regulates LED current and supports PWM dimming. **Ordered 14 September 2026.**

GPIO expander — Microchip MCP23008-E/P

An 8-bit I²C GPIO expander is used for low-speed controls and LCD reset/chip-select handling. **Ordered 14 September 2026.**

Rotary encoder — Bourns PEC09 reference

Reference family: **Bourns PEC09**, incremental encoder with push switch.

Reference prototype part: PEC09-2320F-T0015 — **ordered 14 September 2026.**

The front control pod is sized around the compact 9 mm class body and 7.2 mm mounting-hole requirement. Exact physical dimensions should be confirmed when the ordered part arrives before final printing.

Rear electronics carrier and USB-C strain relief

The rear-service assembly provides a removable electronics carrier, generic support zones, harness tie points, rear USB-C service slot and cable-jacket strain-relief clamp.

The processor carrier geometry will be revised around the final **custom N16R2 PCB**, not around a DevKit board.

5 V USB-C architecture

```

5 V USB-C
|
+--> custom processor carrier
|   +--> ESP32-S3-WR00M-1-N16R2
|   +--> 3.3 V BMI088
|   +--> 3.3 V LCD logic
|   +--> RGB565 + timing
|   +--> shared SPI for LCD init + BMI088
|   +--> I2C to MCP23008
|
+--> TPS61169
    +--> Newhaven backlight (~100 mA)

```

Still required / not recorded as ordered

- Adafruit BMP585 Ported breakout, PID 6413, for the first static-pressure prototype
- PNI RM3100-CB P/N 14754 remote magnetometer (can follow the initial display/attitude/altitude bench bring-up)
- a practical temporary carrier/breakout for the bare ESP32-S3-WROOM-1-N16R2, unless a suitable board is already available
- prototype interconnect materials as required: headers/sockets, short harness leads/perfboard and suitable connectors
- 5 V USB-C bench supply/cable if not already available
- optical front window can wait for enclosure/optical evaluation

Remaining hardware choices

- freeze the custom MCU carrier PCB outline and connector placement
- physically measure the chosen PEC09 encoder when received before final print
- select the exact USB-C cable and set the strain-relief groove diameter
- replace prototype MCP23008 wiring with the compact project carrier PCB
- verify actual aircraft-panel spacing around the front control pod

docs/PHONE_NETWORK_AND_PUBLIC_OTA.md

Repository source: docs/PHONE_NETWORK_AND_PUBLIC_OTA.md

ESP32 EFIS — phone network pairing and public OTA access

Status: workflow/simulator implemented; physical ESP32 Wi-Fi and public Synology deployment unvalidated

Date: 16 September 2026

Network path

```
ESP32 EFIS --Wi-Fi--> iPhone Personal Hotspot --cellular--> Internet
-> public HTTPS :443 -> Synology TLS reverse proxy -> 127.0.0.1:8080 -> read-only OTA nginx
```

The phone is only a network gateway. It does not receive/flash firmware. The private OTA administrator is a separate service on 127.0.0.1:8090 and is never part of the EFIS device path.

Instrument connection configuration

The physical 480×480 maintenance UI provides **Network Connection** with hotspot SSID, masked password, HTTPS update-server profile/manifest URL, Test, Save and Forget. The production rotary encoder will provide character entry; Swift uses native text entry to test workflow.

Persist in NVS: SSID, password, approved manifest URL/profile and schema version. Password must never be logged, included in diagnostics/manifest, committed, sent to OTA server or shown unmasked by default. Forget erases stored credentials. Support secured WPA2/WPA3 Personal as available; do not use open Wi-Fi for OTA.

Connection test

Test is separate from Save, Download and Activate. Report layers independently: phone association/IP, Internet/DNS, TLS certificate/hostname validation, and OTA /healthz + syntactically compatible manifest. Failure messages must identify the layer without exposing credentials.

The Test action itself never downloads or activates firmware.

Maintenance/network policy

Wi-Fi remains maintenance-oriented and normally off during flight presentation. Entering the network/update maintenance context permits association and, if the user preference **Automatically download next update** is ON, discovery/download/verification of the administrator-published release.

This is the only intended automatic OTA activity. **Auto-download never selects the boot partition, activates firmware or reboots the EFIS.** A completed candidate remains dormant until local ACTIVATE & REBOOT.

With auto-download OFF, the user explicitly selects Download. Loss of hotspot/cellular service during transfer leaves the running known-good application untouched and the incomplete inactive-slot candidate unusable.

Public Synology endpoint

ota-server/compose.yml binds nginx to NAS loopback 127.0.0.1:8080. Create a public DNS hostname (for example efis-updates.example.net), a valid TLS certificate, and a DSM HTTPS :443 reverse proxy to http://127.0.0.1:8080. Router/firewall exposes only TCP 443 where inbound hosting is possible. Never expose 8080, 8090, DSM admin or Docker management.

If behind CGNAT, use a deliberately configured trusted HTTPS tunnel/reverse proxy while preserving the stable trusted device-facing hostname.

External test from outside the LAN:

```
curl -fsS https://efis-updates.example.net/healthz
curl -fsS https://efis-updates.example.net/efis/manifest.json
```

Also verify public :8080 and :8090 are unreachable.

Public versus private

Published manifest/binaries may be publicly readable. URL secrecy is not trust. Production trust is TLS certificate validation plus signed application verification. Never embed GitHub PATs or signing private keys in the EFIS. Private admin access should use LAN/VPN/SSH or a properly authenticated private reverse proxy.

Simulator

NetworkSetupView.swift models SSID/password, password masking, manifest URL, staged Phone → Internet → OTA Server test, Save/Forget and persistence. OTAFlowView.swift separately models auto/manual download and explicit activation. Neither performs real networking/flashing.

Physical implementation boundary

ESP32 firmware still requires maintenance Wi-Fi station scanning/association, NVS credential storage, connection reporting/shutdown, HTTPS manifest/download handling and integration with the A/B update manager. Network state must never alter attitude/altitude/heading validity.

Validation

SWIFT WORKFLOW IMPLEMENTED / PHYSICAL NETWORKING AND PUBLIC DEPLOYMENT UNVALIDATED. Bench validation requires real ESP32-S3 + iPhone hotspot + cellular Internet + public DNS/TLS, plus wrong password, hotspot/cellular loss, DNS/TLS/server failure, interrupted downloads and recovery. See OTA_USER_SCENARIO.md, OTA_IMAGE_ADMIN.md and REMOTE_UPDATES.md.

Boot integration and credential persistence — 26 September 2026

The maintenance network flow is entered from the rotary-controlled boot menu by selecting **FIRMWARE UPDATE**. Before manifest access, the EFIS presents a Wi-Fi dialog operable using rotate + press: connect to saved network, scan, manual entry, Forget, retry/change network, or Start EFIS without updating. A failed Wi-Fi/Internet/DNS/TLS/OTA-server step never prevents normal startup.

SSID and credentials persist in a dedicated ESP-IDF NVS namespace. Production units require **NVS encryption** for Wi-Fi credentials; ordinary NVS is acceptable only during early prototype bring-up. The password is never redisplayed after entry, logged, included in diagnostics/test-harness output or transmitted to the OTA server. **Forget network** erases the stored credentials.

docs/POWER_BUDGET.md

Repository source: docs/POWER_BUDGET.md

ESP32 EFIS — Preliminary power budget

Status date: 16 September 2026

This document records the preliminary electrical power/current budget for the ESP32 EFIS prototype. Values are **design estimates, not measured prototype results**. They must be replaced or supplemented with measured figures during hardware commissioning.

Design summary

For the current single-display EFIS architecture, a reasonable preliminary expectation is approximately **250–350 mA from the 5 V rail in normal operation** with Wi-Fi inactive, or about **1.25–1.75 W**.

Wi-Fi activity, particularly OTA operation/transmission, can create substantially higher short-duration ESP32-S3 demand. The prototype therefore uses a **good regulated 5 V source with at least 2 A capability** during integrated bench work. This is intentional engineering margin and is **not** a prediction that the EFIS continuously consumes 2 A.

The final aircraft input stage should likewise not be tightly sized around the expected average consumption. A preliminary design allowance of **at least 1 A continuous capability from the nominal 12 V aircraft supply** leaves substantial margin for startup, converter losses, full backlight, Wi-Fi/OTA transients and future peripherals. Final converter/fuse/wiring choices require measured prototype data and aircraft electrical-system requirements.

Bench power supply selection

Two bench-supply options are recorded in BOM.md. They are **alternatives; only one is required**.

Option A — Korad KA3005P

Preferred economical choice: Korad KA3005P, adjustable 0–30 V / 0–5 A, single-output programmable linear bench supply.

This is sufficient for most EFIS work because normal integration should power the instrument from its intended single input rather than independently forcing every internal rail. Its adjustable current limit is especially valuable for first power-up and fault finding.

Typical use:

| Test | Suggested starting setup |

|---|---|

| First 5 V subassembly power-up | 5.00 V with a deliberately low current limit appropriate to the connected load; increase only after checking for shorts/heating |

| Integrated EFIS prototype | 5.00 V; current limit raised progressively with measured consumption, while retaining margin for transients |

| Direct 3.3 V subassembly test | 3.30 V only where the circuit is explicitly intended for direct 3.3 V input |

| Aircraft-input-stage development | Sweep roughly 10–15 V initially, later extending only to the formally defined aircraft input range |

Never apply the supply directly to a rail merely because the voltage is available: first confirm the specific module/connector pinout and permissible voltage.

Option B — Siglent SPD3303X-E

Premium multi-output choice: Siglent SPD3303X-E, triple-output programmable bench supply.

Choose this instead when simultaneous independent rails materially improve development—for example, when characterising a regulator stage separately from a load or investigating interactions between subsystems. It is more capable than required for ordinary EFIS operation, but gives useful flexibility for a general electronics bench.

During end-to-end EFIS testing, prefer powering the instrument through the intended input architecture. Simultaneously forcing internal rails can hide regulator, sequencing, grounding or transient problems that the integrated design needs to expose.

Interchangeable bench leads

Use 4 mm banana terminals as the common PSU interface and maintain clearly identified leads/adapters for:

- banana to crocodile clips;
- banana to bare wire/test header;
- suitable low-voltage PCB/header connections;
- USB-C **power** breakout/adaptor where appropriate.

Keep low-voltage leads short and polarity-marked. A passive USB-C breakout is not automatically a USB Power Delivery source/sink controller; only use voltage arrangements supported by the actual breakout and target hardware.

Preliminary 5 V budget

| Load | Preliminary 5 V input-equivalent allowance | Notes |

|---|---|---|

| ESP32-S3-WROOM-1-N16R2 + PSRAM, active graphics | ~70–120 mA | Workload-dependent; excludes high Wi-Fi TX peaks. |

| Newhaven LCD logic | ~20–30 mA | Preliminary allowance pending measurement. |

| Newhaven backlight via TPS61169, 100% | ~130–145 mA | Display backlight is approximately 6 V / 100 mA; 5 V input figure includes a converter-loss allowance. |

| BMI088 | ~5–10 mA allowance | Measure with actual configured accel/gyro rates. |

| BMP585 | <2 mA allowance | Small relative to display/processor load. |

| RM3100 | ~5–15 mA allowance | Configuration-dependent; remote sensor. |

| MCP23008 + PEC09 encoder | ~1–3 mA allowance | Encoder itself is passive apart from pull-up current. |

| Miscellaneous conversion/control losses | ~20–40 mA | Preliminary system allowance. |

| **Expected normal total, Wi-Fi inactive** | **~250–350 mA @ 5 V** | **~1.25–1.75 W; to be measured.** |

| **Development/OTA transient design region** | **~500–700+ mA @ 5 V** | Not a continuous-consumption prediction. |

These values are deliberately approximate because the actual ESP32 clock/workload, display brightness, regulator efficiencies, sensor configurations and wiring will affect the result.

Display/backlight significance

The selected Newhaven NHD-2.1-480480AF-ASXP has a backlight requirement of approximately **6.0 V / 100 mA** at the specified operating point. This is about **0.6 W delivered to the backlight**. Because the prototype starts from 5 V, the TPS61169 boosts the voltage and therefore draws more than 100 mA from the 5 V source at full brightness after conversion losses are included.

Backlight brightness is consequently one of the useful variables to measure. It is also likely to offer a meaningful reduction in normal power consumption when full daytime brightness is unnecessary.

ESP32-S3 and Wi-Fi

Normal graphics/sensor operation and Wi-Fi/OTA operation must be treated separately in the power measurements. Wi-Fi transmission can cause short-duration current peaks well above the normal CPU/graphics demand.

The maintenance-only networking policy therefore has an electrical benefit as well as an operational one: normal instrument operation does not require continuous Wi-Fi activity. OTA tests must nevertheless verify that Wi-Fi/transmit/flash activity does not cause rail droop, display corruption, resets or sensor faults.

Approximate aircraft-side consumption

For a first-order example only, if the completed EFIS consumes **1.5 W** and a 12 V → 5 V aircraft converter is **90% efficient**:

$$\begin{aligned}\text{Aircraft input current} &= 1.5 \text{ W} / (12 \text{ V} \times 0.90) \\ &\approx 0.139 \text{ A}\end{aligned}$$

This suggests an eventual normal aircraft-side current of roughly **0.12–0.18 A** is plausible for the present architecture, with higher short-duration demand during maintenance/Wi-Fi activity.

This is **not yet a measured or validated aircraft current specification**. The final aircraft power design must use measured worst-case current, supply-voltage range, startup/inrush behaviour, wiring drop, converter temperature/derating, protection requirements and appropriate fuse/circuit protection.

Prototype supply requirement

For bench development use:

- regulated **5 V DC** for normal integrated prototype operation;
- supply capability of **at least 2 A** for comfortable development/transient margin;
- adjustable current limiting for staged bring-up;
- good-quality short power wiring;
- common ground for processor, display/backlight and sensors unless a specific isolated test requires otherwise;
- do not introduce the aircraft 12 V supply until the 5 V prototype is stable and instrumented.

A 2 A-capable source provides useful margin for development. It does not remove the need to check voltage at the electronics during worst-case load.

Hardware commissioning measurements

Record current and rail voltage for at least these states:

Test	Required observation
--- ---	
Power-on/inrush	Peak input current and minimum 5 V/3.3 V rail voltage.
Backlight off	Baseline processor/display-logic/sensor consumption.
Backlight 25%	Steady 5 V current/power.
Backlight 50%	Steady 5 V current/power.
Backlight 75%	Steady 5 V current/power.
Backlight 100%	Steady 5 V current/power and driver temperature.
Normal Horizon rendering	Current with normal renderer and all available sensors.
Altimeter/Compass pages	Confirm page rendering does not materially alter load.
Wi-Fi association/connection test	Peak and steady current; watch rail stability.
OTA download	Peak/average current while Wi-Fi and flash operations are active.
OTA activation/reboot	Startup transient and rail stability.
Sensor fault/disconnect tests	Confirm electrical faults do not destabilise common rails.
Aircraft-input sweep (later power stage)	Record current, 5 V/3.3 V regulation and temperature across defined input range.

Measurements should preferably include both average current and captured transient peaks. A slow bench-supply display alone may miss ESP32 Wi-Fi peaks; use suitable instrumentation if unexplained resets or rail disturbances appear.

Acceptance before final aircraft power design

Do not freeze the aircraft DC/DC converter, input fuse/protection or final PCB power layout from this estimated budget alone. First establish measured:

1. maximum steady 5 V consumption at full backlight;
2. worst observed startup/Wi-Fi/OTA transient;

3. minimum stable 5 V and 3.3 V rails during that transient;
4. actual 12 V → 5 V converter efficiency across the aircraft supply range;
5. thermal performance in the enclosure;
6. adequate design margin after measurement.

The resulting measured figures become the authoritative power budget and should be recorded in this file with the test configuration and date.

Completed-EFIS design allowance — 26 September 2026

Until physical measurements replace the estimates above, use the following **system-level design allowance** for the completed production architecture, including the nominal 12 V aircraft input conversion stage, GNSS and protected external avionics interfaces:

| Condition | Preliminary system allowance |

|---|---|

| Expected normal operation, Wi-Fi inactive | **~1.5–2 W** |

| Worst sustained design allowance | **~3–4 W** |

| Short transient capability to provide | **at least 5 W** |

The 5 W figure is **power-system headroom, not predicted continuous consumption**. It provides margin for full display backlight, ESP32 Wi-Fi transmission, OTA flash activity, GNSS, conversion losses and future protected interfaces.

These allowances do not supersede the present estimated normal prototype consumption of **250–350 mA at 5 V (1.25–1.75 W)**. They are sizing targets for the completed system until commissioning provides measured maximum steady-state, startup/inrush, Wi-Fi/OTA transient and thermal data.

The FULL TEST/commissioning programme should measure power with the backlight at 0/25/50/75/100%, normal sensor acquisition, Wi-Fi association and OTA activity. Those measurements will replace these estimates before the aircraft DC/DC converter, fuse/protection and final thermal design are frozen.

docs/PROCUREMENT_STATUS_2026-09-16.md

Repository source: docs/PROCUREMENT_STATUS_2026-09-16.md

Prototype procurement status — updated 17 September 2026

This checkpoint records delivery evidence supplied for Stage 1 and Stage 2 prototype parts. BOM.md remains the authoritative purchasing checklist and includes explicit purchase locations/references for all items that are ready to buy.

Receipt-evidence rule

A supplied photograph of a delivery note that clearly identifies a BOM part and delivered quantity is accepted as evidence that the stated quantity has **arrived**. BOM.md should then be updated to RECEIVED without requiring a second confirmation. Order/dispatch evidence or a statement that an item is on the way is recorded as ORDERED / IN TRANSIT, not RECEIVED.

Confirmed received — photographic evidence 17 September 2026

The newly supplied photographs show the actual component packages together with the supplier paperwork, confirming physical arrival of the following parts.

DigiKey UK order 101607569 / invoice 132701189

- **Adafruit TPS61169 constant-current LED driver, PID 6354**, DigiKey P/N 1528-6354-ND, quantity **1 RECEIVED**. The component bag and packing list are both visible.
- **Bourns PEC09-2320F-T0015 rotary encoder**, DigiKey P/N PEC09-2320F-T0015-ND, quantity **1 RECEIVED**. The labelled component bag and packing list are both visible.
- **Microchip MCP23008-E/P I/O expander**, DigiKey P/N MCP23008-E/P-ND, quantity **1 RECEIVED**. The labelled antistatic bag and packing list are both visible.
- **Bosch Shuttle Board 3.0 BMI088**, DigiKey P/N 828-SHUTTLEBOARD3.0-BMI088-ND / manufacturer SHUTTLE_BOARD_3_0_BMI088, quantity **1 RECEIVED**. The labelled antistatic bag and packing list are both visible.
- **Espressif ESP32-S3-WROOM-1-N16R2**, DigiKey cut-tape P/N 5407-ESP32-S3-WROOM-1-N16R2CT-ND, quantity **2 RECEIVED**. Page 2 of the packing list and the labelled antistatic package both show quantity 2.

The photographs also confirm that the DigiKey order states **THE ORDER IS COMPLETE**.

RS UK

- **Newhaven NHD-FFC40 40-pin FFC-to-through-hole adapter**, RS stock 723891 / displayed stock format 723-891, manufacturer P/N NHD-FFC40, quantity **1 RECEIVED**. The RS delivery note and labelled package are both visible.

These receipt confirmations agree with the RECEIVED quantities already carried in BOM.md. The DigiKey MCU purchasing reference is retained as 5407-ESP32-S3-WROOM-1-N16R2CT-ND, matching the supplied packing-list/package photographs.

In transit

The project owner confirms the **Newhaven NHD-2.1-480480AF-ASXP display is on the way**. It remains **ORDERED / IN TRANSIT** and will move to RECEIVED after physical-arrival evidence/confirmation.

Stage 1 — still required

1. Await the **Newhaven NHD-2.1-480480AF-ASXP** display. Source/reference retained in BOM: RS UK stock 286-351; DigiKey is the alternate.
2. After the display arrives, verify contact orientation and length and buy **2 × 40-position 0.5 mm FFC/FPC cables**. Do not freeze a supplier part until that physical check is complete.

3. Buy **1 × Espressif ESP-Module-Prog-1** temporary solderless module carrier/programmer. Preferred source: **Mouser UK** 356-ESPMODULEPROG1. Do not substitute Prog-1R without checking the spring-pin layout.
4. Buy one current-limited bench PSU: **Korad KA3005P** preferred economical option or **Siglent SPD3303X-E** premium alternative.
5. Obtain bench PSU leads, USB-C data/power cable, solderless breadboard, 22–26 AWG prototype wire and 2.54 mm headers unless already available.

Stage 1 is ready for physical bring-up once the processor can be safely mounted/programmed, the display is present and connected through the verified FFC arrangement, and a current-limited 5 V bench supply/interconnect arrangement is available.

Stage 2 — still required

1. A verified **1.27 mm-pitch mating adapter/interposer for the Bosch BMI088 Shuttle Board 3.0**. Do not buy a guessed 2.54 mm adapter.
2. **Adafruit BMP585 Ported I2C/SPI PID 6413**, quantity 1. Preferred source: DigiKey UK; UK alternates The Pi Hut/Pimoroni when stocked.
3. **STEMMA QT/Qwiic JST-SH 4-pin cables**, 100–200 mm, quantity 2. Adafruit PID 4397 (150 mm) is a suitable length candidate subject to connector verification.
4. **Adafruit Qwiic/STEMMA QT breakout PID 5961**, quantity 1; DigiKey UK 1528-5961-ND.
5. **PNI RM3100-CB P/N/SKU 14754**, quantity 1; source: PNI Sensor direct, subject to UK shipping/import-cost check.
6. **GNSS receiver**, exact part still to be selected. It must provide latitude, longitude, fix validity/type, freshness and an explicit horizontal-accuracy estimate.
7. **GNSS bench antenna** compatible with the selected receiver; do not order before the receiver RF interface and active/passive antenna requirement are known.

Source-retention rule

Purchase locations are part of the BOM, not temporary research notes. When a part is selected, BOM.md retains the preferred supplier, supplier stock/order number where known, and an alternate source where useful. Receiving a part does **not** remove its purchase source; the source remains for traceability and replacement/spare ordering.

Do not buy yet

Stage 3+ aircraft static fittings, final RM3100 harness/mount, final GNSS installation hardware, optical window, final enclosure hardware, custom carrier PCB production, and aircraft 12 V input/protection hardware remain gated by bench validation and aircraft surveys.

docs/PROJECT_STATUS.md

Repository source: docs/PROJECT_STATUS.md

ESP32 EFIS — Project activity, validation and decision status

Status date: 20 September 2026

This is the authoritative project-level record of implementation, validation **and significant design decisions**. It records ideas that are adopted, proposed, parked, superseded or dismissed, including the reason, so rejected approaches are not accidentally reintroduced later.

The project remains experimental **supplementary/non-primary**. Missing/invalid/stale real data fails obviously; synthetic data is never an automatic fallback. Build/simulation success is not physical validation.

Current development gate

Physical hardware activity remains paused until prototype hardware is available. Active software work includes QEMU regression, Swift parity, documentation, pure estimator/calculation tests, static safety review, and OTA server/user-flow work.

Status vocabulary

| Status | Meaning |

|---|---|

| **ADOPTED** | Agreed architecture/design; implementation may still be pending. |

| **IMPLEMENTED** | Code/hardware artefact exists; validation is stated separately. |

| **VALIDATED** | Tested at the explicitly named validation layer. |

| **PROPOSED** | Worth considering; no final decision yet. |

| **PARKED** | Deliberately deferred until a prerequisite or later phase. |

| **SUPERSEDED** | Replaced by a later design; retained here for history. |

| **DISMISSED** | Deliberately rejected; reason retained here. |

| **BLOCKED** | Cannot proceed until a dependency is resolved. |

| **ACCEPTED — QEMU** | Deterministic firmware/emulator behaviour reviewed at 480×480. |

| **SYNCED — SWIFT** | Swift represents accepted behaviour/scenario; not authoritative pixel renderer. |

| **COMPILE-VALIDATED** | ESP32 target compiled; no physical operation implied. |

| **IMPLEMENTED — UNVALIDATED** | Code exists; required runtime/physical validation not performed. |

Completed / current activities

| Function | State | Validation |

|---|---|---|

| Safety model | supplementary/non-primary; explicit simulation; fail-obvious validity | **ADOPTED / DOCUMENTED** |

| QEMU RGB565 backend | native 480×480 virtual display | **ACCEPTED — QEMU** |

| Horizon | accepted pitch/bank geometry, ladder, bank scale, ATT FAIL | **ACCEPTED — QEMU; SYNCED — SWIFT** |

| Altimeter | three pointers, digital altitude, >10k hatch, Kollsman, ALT FAIL | **ACCEPTED — QEMU; SYNCED — SWIFT** |

| Compass | rotating card, wrap, heading bug, HEADING FAIL | **ACCEPTED — QEMU; SYNCED — SWIFT** |

| QNH/storage | 950–1050 hPa, 1 hPa, NVS-backed setting | **SOFTWARE/QEMU ACCEPTED; ENCODER PENDING** |

| Pressure calculation | validated calculation path | **SIMULATION-TESTED; BMP585 INPUT PENDING** |

| BMI088 SPI/diagnostics | engineering-unit acquisition + commissioning log path | **COMPILE-VALIDATED; HARDWARE PENDING** |

| Attitude estimator | gyro/gravity complementary estimator, plausibility/staleness gates | **COMPILE-VALIDATED; PHYSICAL TUNING PENDING** |

| BMI088 live display gate | withheld until physical axes/signs verified | **IMPLEMENTED SAFETY GATE** |

| Swift instrument simulator | adaptive instruments/manual/AUTO/failure/acceptance scenarios | **SOFTWARE OPERATIONAL; PARITY MAINTAINED** |

| Phone/network simulator | hotspot credentials, HTTPS manifest config and staged connectivity UI | **IMPLEMENTED — SWIFT; REAL ESP32 NETWORK PENDING** |

| OTA user simulator | auto-download preference, manual Download, verified Ready, explicit ACTIVATE & REBOOT, success/rollback | **IMPLEMENTED — SWIFT; PHYSICAL OTA PENDING** |

| OTA Docker origin | loopback nginx read-only public-origin backend on host 127.0.0.1:8180 | **VALIDATED — SYNOLOGY RUNTIME + PUBLIC HTTPS HEALTH PATH** |

| OTA Docker admin | private responsive maintenance dashboard on host 127.0.0.1:8090; release inventory/status, structured manifest summary, staging form, explicit Publish/delete | **VALIDATED — SYNOLOGY RUNTIME; ISOLATED STAGE/PUBLISH WORKFLOW** |

| OTA rebuild/test harness | clean Compose validation/rebuild/recreate, local health checks and isolated Stage → Publish integrity tests | **VALIDATED — SYNOLOGY RUNTIME** |

| OTA CLI staging helper | versioned binary copy + SHA-256 + metadata; cannot publish | **IMPLEMENTED — RUNTIME UNVALIDATED** |

| OTA A/B ESP32 client | dual-slot/write/boot/self-test/rollback design | **ADOPTED / NOT IMPLEMENTED ON ESP32** |

| OTA security | HTTPS + signed-image design; Secure Boot/flash encryption later | **ADOPTED; LATER HARDENING** |

| Enclosure CAD | development case/carriers/fit gauge | **DESIGNED; PHYSICAL FIT PENDING** |

| Carrier PCB | architecture/pins | **PARKED** |

Design decision and ideas register

This register records material alternatives as well as the chosen design. A later change should update the existing entry or add a new one rather than erasing the history.

Safety, role and data semantics

| ID | Idea / decision | State | Reason / replacement |

---|---|---|---

| D001 | Treat ESP32 EFIS as supplementary/non-primary | **ADOPTED** | Avoids implying certification or sole-source flight-instrument status; independent trusted instruments remain required. |

| D002 | Continue displaying the last plausible value after a sensor becomes invalid/stale | **DISMISSED** | A frozen believable attitude/altitude/heading is hazardous. Invalid/stale sources must fail obviously. |

| D003 | Substitute synthetic/simulator data when a real source fails | **DISMISSED** | Could create plausible false flight information. Synthetic data is development-only and conspicuously marked. |

| D004 | Label future GNSS course as heading | **DISMISSED** | Course over ground is not magnetic heading; if added it is labelled TRK, and groundspeed GS, never IAS. |

Display, instruments and controls

| ID | Idea / decision | State | Reason / replacement |

---|---|---|---

| D010 | Newhaven NHD-2.1-480480AF-ASXP round 480×480 high-brightness display | **ADOPTED** | Fits the compact round instrument concept; 1000-nit class display; accepted renderer uses native 480×480 geometry. |

| D011 | 18-bit RGB pixel bus | **DISMISSED** | GPIO pressure is more important than the extra colour bits; ST7701S supports the adopted 16-bit RGB565 path. |

| D012 | 16-bit RGB565 + 9-bit serial controller initialization | **ADOPTED** | Reduces GPIO demand while retaining full 480×480 framebuffer operation. |

| D013 | Earlier 18 MHz / 40/60/20 / 10/10/6 display timing | **SUPERSEDED** | Those values came from the MIPI timing table; current RGB baseline is 30 MHz with 50/50/4 and 50/50/2 timing. |

| D014 | Touchscreen user interface | **DISMISSED** | Selected panel has no touch; compact rotary/push control is the intended physical interaction. |

| D015 | Bourns PEC09 rotary encoder/push control | **ADOPTED** | Compact physical control for page selection/settings; physical behaviour still pending hardware validation. |

| D016 | Three instrument pages: Horizon/PFD, Altimeter, Compass | **ADOPTED** | Provides the agreed multifunction presentation while keeping each round page clear and testable. |

| D017 | Rectangular QNH/Kollsman box | **SUPERSEDED** | Replaced by accepted compact curved 3-o'clock Kollsman scale to use round-display space better. |

| D018 | >10,000-ft progressive altimeter hatching | **ADOPTED / ACCEPTED QEMU** | Provides an unmistakable visual cue without replacing the classic three-pointer presentation. |

| D019 | Selected-heading bug at relative bearing bug - heading | **ADOPTED / ACCEPTED QEMU** | Conventional rotating-card relationship; deterministic 060° bug used for acceptance. |

Processor, memory and carrier hardware

| ID | Idea / decision | State | Reason / replacement |

|---|---|---|---|

| D030 | ESP32-S3-WROOM-1-N16R2 bare module | **ADOPTED** | 16 MB flash, 2 MB Quad PSRAM, native USB and sufficient memory while preserving GPIO35–37. |

| D031 | ESP32-S3 N8R8 / Octal-PSRAM module | **DISMISSED** | Project is GPIO-constrained rather than memory-constrained; Octal memory consumes GPIO35–37 needed by current shared SPI design. |

| D032 | ESP32-S3 DevKitC as final installed processor board | **SUPERSEDED** | Useful for bench work but final design favours compact bare N16R2 module on project carrier; exact older DevKit variant availability also weak. |

| D033 | Project-specific carrier PCB | **ADOPTED / PARKED** | Preferred final architecture, but connector placement/physical interfaces should not be frozen before prototype hardware proves them. |

| D034 | Finalise/manufacture carrier PCB before prototype interface validation | **DISMISSED FOR CURRENT PHASE** | Risks locking incorrect display, encoder, pressure, magnetometer or enclosure interfaces. |

| D035 | Two full 480×480 RGB565 framebuffers in PSRAM | **ADOPTED** | About 921,600 bytes total, fitting the N16R2's 2 MB Quad PSRAM; physical PSRAM operation remains to be validated. |

Sensors, attitude, altitude and heading

| ID | Idea / decision | State | Reason / replacement |

|---|---|---|---|

| D050 | Bosch BMI088 Shuttle Board 3.0 for attitude sensing | **ADOPTED** | Separate accel/gyro, vibration-oriented device, available development board and implemented SPI acquisition path. |

| D051 | Infer BMI088 aircraft axes from PCB/connector orientation | **DISMISSED** | A plausible reversed attitude is unacceptable; physical +X forward/+Y right/+Z down mapping must be demonstrated first. |

| D052 | Connect BMI088 immediately to live Horizon | **BLOCKED / DELIBERATELY WITHHELD** | Physical axis/sign mapping and dynamic validation are prerequisites. |

| D053 | Complementary gyro/gravity pitch/roll estimator as initial AHRS | **ADOPTED / IMPLEMENTED** | Simple auditable first estimator; gravity correction is suppressed outside plausible acceleration. More advanced fusion remains possible after real data. |

| D054 | Treat accelerometer continuously as gravity | **DISMISSED** | Manoeuvre acceleration would corrupt attitude; adopted estimator gates correction using acceleration magnitude. |

| D055 | BMP585 as static-pressure source | **ADOPTED** | Pressure range/relative accuracy/environmental robustness suit the development static-altimeter role. |

| D056 | BMP581/BMP390/DPS310 alternatives | **DISMISSED FOR BASELINE** | BMP585 chosen for the documented combination of range, relative accuracy, environmental robustness and availability. |

| D057 | Adafruit ported BMP585 module for prototype | **ADOPTED** | Provides practical static plumbing without inventing a pressure chamber around an unported breakout. |

| D058 | Freeze bare-BMP585 sealed-plenum design now | **PARKED** | Decide only after the ported prototype/static system is characterised. |

| D059 | PNI RM3100-CB remote magnetometer | **ADOPTED** | Remote placement reduces panel magnetic interference; three-axis field supports calibrated tilt-compensated heading. |

| D060 | Mount magnetometer beside ESP32/regulator/display | **DISMISSED** | Local electronics/backlight/steel can distort magnetic field; remote non-magnetic installation is preferred. |

| D061 | Use raw magnetometer azimuth as final compass heading | **DISMISSED** | Final heading needs calibration, installation alignment and tilt compensation using validated attitude. |

| D062 | Magnetic/true reference and variation policy | **PROPOSED / OPEN** | Must be explicit before final heading behaviour is frozen. |

Simulation, testing and development workflow

| ID | Idea / decision | State | Reason / replacement |

|---|---|---|---|

| D080 | Espressif QEMU at real 480×480 framebuffer geometry as authoritative software renderer | **ADOPTED** | Exercises actual ESP32 renderer without claiming physical display validation. |

| D081 | SwiftUI iPhone/iPad simulator | **ADOPTED / IMPLEMENTED** | Faster interactive scenario/UI development; it mirrors accepted behaviour but is not pixel-authoritative. |

| D082 | Treat Swift simulator appearance as proof of ESP32 display output | **DISMISSED** | Swift is a native representation, not the RGB565 renderer. QEMU is authoritative until physical display validation. |

| D083 | Mark all synthetic firmware screens clearly SIM | **ADOPTED** | Prevents synthetic values being mistaken for flight data. |

| D084 | Continue routine physical hardware work while hardware is unavailable | **PARKED** | Hardware-dependent claims cannot be validated; active work remains software/QEMU/Swift/docs. |

| D085 | Treat a successful hardware-target compile as physical validation | **DISMISSED** | Compile proves build integrity only. |

| D086 | Automatic hardware CI builds during hardware pause | **PARKED / SUPERSEDED BY MANUAL WORKFLOW** | Routine automatic builds add little physical evidence; workflow remains manually invocable when useful. |

| D087 | Automated QEMU demonstration/video workflow | **ADOPTED / IMPLEMENTED** | Provides repeatable demonstration of implemented instrument states; recording helper itself still requires local runtime validation. |

OTA release source, hosting and administration

| ID | Idea / decision | State | Reason / replacement |

|---|---|---|---|

| D100 | GitHub main as a directly installable firmware feed | **DISMISSED** | Development source should not automatically become installable firmware; releases need an approval/publication boundary. |

| D101 | ESP32 download directly from private GitHub using a PAT | **DISMISSED** | Would place repository credentials on the instrument and couple OTA to GitHub authentication. |

| D102 | GitHub as authoritative source/build/release history + separate HTTPS distribution origin | **ADOPTED** | Separates development/release history from simple device-facing distribution and avoids GitHub credentials on EFIS. |

| D103 | Synology-hosted Docker/nginx OTA origin | **ADOPTED / IMPLEMENTED; SYNOLOGY LOCAL RUNTIME VALIDATED** | Both OTA containers build/start on the DS918+ and local health endpoints pass; public read-only HTTPS endpoint is externally health-validated on granvillehouse.synology.me:8448; physical EFIS consumption remains pending. |

| D104 | Expose Docker nginx port 8080 directly to Internet | **DISMISSED** | Origin remains loopback-only; public path is HTTPS :443 via reverse proxy. |

| D105 | Publicly readable approved manifest/binaries | **ADOPTED** | URL secrecy is not the trust boundary; TLS plus signed-image verification is the intended production trust model. |

| D106 | Put signing private key/GitHub token/Wi-Fi credentials in OTA container | **DISMISSED** | Distribution server should not hold development credentials or signing secrets. |

| D107 | Browser admin interface for hosted EFIS images | **ADOPTED / IMPLEMENTED** | Simplifies staged image inventory, metadata/hash review, publication, republishing and deletion. |

| D108 | Expose OTA admin UI on public update hostname | **DISMISSED** | Management is a higher-risk capability; keep it LAN/VPN/private authenticated reverse-proxy only. |

| D109 | Upload firmware and immediately publish it | **SUPERSEDED / DISMISSED** | Upload mistakes should not become consumable releases. Upload now creates STAGED release; Publish is a separate explicit admin action. |

| D110 | Separate Stage and Publish operations | **ADOPTED / IMPLEMENTED** | Creates deliberate administrative approval boundary. CLI staging helper cannot publish. |

| D111 | Delete currently published release | **DISMISSED / BLOCKED BY ADMIN UI** | Prevents manifest from pointing at a removed current image. |

| D112 | Republish an older retained image | **ADOPTED / IMPLEMENTED** | Provides administrative withdrawal/rollback of the advertised release independently of device A/B rollback. |

EFIS networking and update user experience

| ID | Idea / decision | State | Reason / replacement |

|---|---|---|---|

| D130 | Add a cellular modem to EFIS for OTA | **DISMISSED FOR CURRENT DESIGN** | iPhone hotspot provides temporary field connectivity without additional modem/SIM/power/integration burden. |

| D131 | iPhone Personal Hotspot as temporary EFIS Internet gateway | **ADOPTED** | Phone supplies network only; EFIS remains the HTTPS OTA client. |

| D132 | Phone itself downloads/flashs EFIS firmware | **DISMISSED** | Adds an unnecessary phone-side update protocol; adopted design keeps update verification/flash/rollback on ESP32. |

| D133 | Maintenance-only Network Connection dialog on instrument | **ADOPTED; SWIFT SIMULATED** | User needs explicit hotspot credentials/server/test controls without networking silently appearing during normal operation. |

| D134 | Store hotspot SSID/password persistently in a dedicated NVS namespace, with Forget action; use NVS encryption for production units | **ADOPTED / ESP32 IMPLEMENTATION PENDING** | Enables repeat maintenance use while keeping credentials local to the instrument. Prototype may initially use ordinary NVS; production credentials must use encrypted NVS. Password must never be displayed after entry, logged or server-sent. |

| D135 | Open Wi-Fi for OTA maintenance | **DISMISSED** | Use authenticated WPA2/WPA3 Personal as supported; no reason to accept an open maintenance network. |

| D136 | Wi-Fi silently reconnects during normal instrument operation | **DISMISSED** | Network/OTA activity is maintenance-only and should not unexpectedly alter operational behaviour. |

| D137 | Layered connection test: Phone → DNS/Internet → TLS → OTA server | **ADOPTED; SWIFT SIMULATED** | Produces useful fault isolation without downloading/installing firmware. |

OTA download, activation, rollback and security

| ID | Idea / decision | State | Reason / replacement |

|---|---|---|---|

| D150 | Single-slot in-place firmware overwrite | **DISMISSED** | Power/network/image failure could destroy the known-good application. |

| D151 | ESP-IDF A/B OTA slots + otadata | **ADOPTED / ESP32 IMPLEMENTATION PENDING** | Candidate is written to inactive slot; previous known-good remains recoverable. |

| D152 | Automatically activate/reboot whenever a new release is published | **DISMISSED** | Publication should not unexpectedly restart an instrument. Final activation remains a local explicit action. |

| D153 | Optional automatic download of the next published update | **ADOPTED; SWIFT SIMULATED** | Reduces maintenance friction while leaving activation under user control. Only allowed in maintenance/update context. |

| D154 | Default auto-download setting ON | **ADOPTED IN SIMULATOR** | Candidate can be ready before user chooses activation; does not imply automatic reboot. Physical policy still requires validation. |

| D155 | Simple ACTIVATE & REBOOT final user action | **ADOPTED; SWIFT SIMULATED** | Keeps normal user interaction simple while preserving a deliberate activation boundary. |

| D156 | Allow casual cancel after flash writing has begun | **DISMISSED** | Interruption safety should come from A/B design, not encourage power-off/cancellation during flash operations. |

| D157 | Mark candidate good immediately on reaching `app_main()` | **DISMISSED** | Candidate must pass first-boot self-test before becoming known-good. |

| D158 | Application-controlled first-boot confirmation + automatic rollback | **ADOPTED / ESP32 IMPLEMENTATION PENDING** | Failed/crashed/unconfirmed candidate restores previous known-good firmware. |

| D159 | Treat missing development sensors as automatic OTA self-test failure | **DISMISSED FOR INITIAL SELF-TEST** | Sensors may legitimately be disconnected during maintenance/development; absence should produce invalid data rather than reject otherwise bootable firmware. Stricter production profile may follow. |

| D160 | OTA-update bootloader/partition table in initial implementation | **DISMISSED FOR INITIAL SCOPE** | Initial OTA is application-only; bootloader/partition-table updates increase recovery risk. |

| D161 | SHA-256 alone as publisher authentication | **DISMISSED** | Hash detects corruption but not publisher identity; signed application images are required for production trust. |

| D162 | HTTPS certificate validation | **ADOPTED** | Protects transport/server identity and is separate from image-signature verification. |

| D163 | Signed application images | **ADOPTED / NOT YET IMPLEMENTED** | Device must verify publisher authenticity independently of public hosting. Private signing key stays off device/server repository. |

| D164 | Enable irreversible eFuse anti-rollback immediately | **PARKED** | Operational recovery and A/B rollback should be proven before irreversible version restrictions. |

| D165 | Secure Boot v2 / flash encryption | **PROPOSED FOR HARDENING** | Evaluate after baseline OTA/recovery is proven; not required to validate the first OTA mechanics. |

| D166 | Destructive config migration before candidate is accepted | **DISMISSED** | Would break rollback. Config schemas must be versioned/backward-compatible or migrations staged until candidate confirmation. |

Enclosure and installation

| ID | Idea / decision | State | Reason / replacement |

|---|---|---|---|

| D180 | 3 1/8-inch flight-development enclosure with removable bezel/carriers | **ADOPTED / CAD IMPLEMENTED** | Supports iterative prototype access and round-display instrument format. |

| D181 | Freeze final static/magnetometer/USB openings before actual fittings are measured | **PARKED / DISMISSED FOR CURRENT PHASE** | Physical connector/tube/cable dimensions must be proven before irreversible CAD/PCB geometry. |

| D182 | Hard-coated AR optical clear polycarbonate front window | **ADOPTED AS REFERENCE** | Scratch resistance and anti-reflection are desirable for cockpit readability; physical evaluation pending. |

| D183 | Aircraft integration before bench sensor/display/failure validation | **BLOCKED** | Ground/bench validation gates must pass before aircraft evaluation. |

Definitive OTA policy

Stage → Publish → Download → Activate. Admin upload/stage never publishes. Publish changes the public manifest. EFIS auto-download may download/verify only in maintenance context. **Activation/reboot is always explicit.** First boot must self-test and either mark the candidate known-good or rollback. See `OTA_IMAGE_ADMIN.md`, `OTA_USER_SCENARIO.md`, `PHONE_NETWORK_AND_PUBLIC_OTA.md`, `REMOTE_UPDATES.md` and `../ota-server/README.md`.

Accepted emulator sequences

Horizon: level, $\pm 10/\pm 20$ pitch, $\pm 30/\pm 60$ bank, combined attitude, failure/recovery. Altimeter: representative 0–12,500 ft values, sweep, failure/recovery and QNH exercises. Compass: cardinal/intercardinal headings, 350→010 wrap, rotation, failure/recovery with deterministic 060° bug. Full details remain in `SIMULATION.md`.

Remaining work

| Function | Work / validation required | Status |

|---|---|---|

| Decision-register maintenance | record every material new proposal, adoption, supersession, dismissal or parked item with reason | **ONGOING DOCUMENTATION RULE** |

| Swift parity | keep instrument/network/OTA scenarios aligned with firmware policy | **ACTIVE SOFTWARE TASK** |

| OTA server deployment | containers build/run on Synology; local health, private admin HTTPS dashboard and public read-only HTTPS health path are validated; manifest/release and physical EFIS OTA remain | **NETWORK PATH VALIDATED / RELEASE + DEVICE TESTING PENDING** |

| OTA ESP32 Phase 1 | freeze A/B partition sizes; manifest/parser/state machine; persisted auto-download policy | **ADOPTED / SOFTWARE CANDIDATE** |

| OTA ESP32 Phase 2 | maintenance Wi-Fi + HTTPS writer + explicit activation + boot confirmation | **PENDING HARDWARE** |

| OTA interruption/rollback | A↔B, Wi-Fi/power loss, corrupt image, crash/self-test rollback, NVS compatibility, USB recovery | **PENDING HARDWARE** |

| OTA security | signed releases/verification; later Secure Boot/flash encryption; eFuse anti-rollback deferred | **ADOPTED / LATER HARDENING** |

| BMI088 axes/bias/tuning/live connection | physical orientation, stationary/dynamic data, estimator tuning | **PENDING HARDWARE** |

| BMP585/static/live altitude | physical driver, pressure validation, plumbing, failure/staleness | **PENDING HARDWARE** |

| RM3100/calibration/heading | acquisition, hard/soft iron, installation alignment, tilt compensation | **PENDING HARDWARE** |

| MCP23008/PEC09/NVS | physical interaction/persistence | **PENDING HARDWARE** |

| LCD/backlight/PSRAM/power | physical bring-up, stress, thermal/electrical checks | **PENDING HARDWARE** |

| Enclosure/EMI/vibration/thermal | physical and powered installation tests | **PENDING HARDWARE** |

| Aircraft ground/airborne comparison | only after all preceding gates | **FUTURE / BLOCKED** |

| Carrier PCB | resume after prototype interfaces proven | **PARKED** |

Hardware-resumption gate

Inventory/inspection → ESP32/display/power → BMI088 axis mapping → estimator validation → BMP585/static → encoder → RM3100/calibration → integrated failure/staleness → **real OTA download/activation/interruption/rollback** → enclosure/thermal/vibration → aircraft ground testing. Never infer airborne readiness from compilation, Docker, Swift or QEMU results.

Documentation and decision-history rule

Every functional change updates relevant code documentation and project status in the same stage. Validation statements must identify the layer: compile, QEMU, Swift, Docker runtime, electronics bench, physical fixture, aircraft ground or airborne.

Every **material design idea** must also be represented in the Design Decision and Ideas Register when it affects architecture, safety, hardware selection, user workflow, testing or future work. Record the idea even when it is rejected. Use **PROPOSED**, **ADOPTED**, **IMPLEMENTED**, **PARKED**, **SUPERSEDED**, **DISMISSED** or **BLOCKED**, and retain the reason/replacement. Do not silently delete historical decisions when the design changes; mark them superseded/dismitted and add the replacement decision. This file is therefore the authoritative answer to both “**where is the project now?**” and “**what did we consider and why did we choose this design?**”.

Phase 1 bench wiring architecture image

The repository includes hardware/diagrams/ESP32EFISPhase1WiringDiagram.png as a visual architecture/reference aid for Phase 1 bench assembly. **It is not yet an electrically verified wiring schematic and must not be used as authority for GPIO numbers, display pin assignments, supply rails or connector orientation.** Before first power, the Newhaven display/NHD-FFC40 pin mapping, ESP32 GPIO allocation, backlight connections and all supply/ground connections must be checked against authoritative datasheets and the firmware pin map. A verified schematic will supersede this reference image.

Production enclosure, power, GNSS and future avionics — 26 September 2026

The current mechanical concept has evolved as follows. These are **design requirements/concepts, not frozen manufacturing dimensions or electrically verified interfaces**:

- EFIS body mounts **behind the aircraft binnacle** so its presentation matches the other panel instruments.

- Mounting bolts are inserted **from the front of the binnacle** and engage **stainless captive nuts pressed into recessed holes in the rear enclosure**. No loose mounting nuts are required.
- Use the minimum practical number of small stainless cap-head Allen fasteners for the enclosure itself. Enclosure fasteners are separate from the aircraft-panel mounting bolts.
- Production power input is **nominal 12 V aircraft power**, with internal reverse-polarity, transient/surge and filtering protection followed by regulated 5 V and 3.3 V rails. Phase-1 bench bring-up remains regulated 5 V.
- USB-C is primarily a **service/programming** interface, not the normal production aircraft-power connection.
- GNSS architecture must support either a suitable **externally powered/active GNSS antenna/receiver arrangement** or a **USB-connected GNSS mouse**. Exact receiver, connector, active-antenna bias arrangement and USB host requirements remain to be frozen in Phase 2.
- Preserve future avionics integration without making the EFIS a critical intermediary. Reserve ESP32 resources and PCB space for a protected **RS-232** interface and footprints/options for **RS-485 and CAN**. Keep GNSS and external-avionics UART resources distinct.
- A future PilotAware FX or other traffic source should be consumable as an external data source. The EFIS should listen to processed traffic/GNSS data rather than becoming part of a safety-critical transponder control path.
- Existing transponder/control-head links such as the TT21/TC20 installation remain independent. Any future connection must be verified against the applicable equipment installation manuals before wiring.
- The former generic rear-panel GPIO EXPANSION concept is superseded by a **DATA / EXPANSION** concept. Raw ESP32 pins must not be presented as aircraft-voltage-tolerant inputs. External avionics lines require appropriate transceivers/protection.
- Where no external expansion function is needed, keep low-level GPIO/I2C service access internal to reduce connectors, EMI paths and accidental misuse.

Concept renders generated during this design iteration are retained separately as project artefacts when binary repository upload is practical. They are visual design aids only: connector types, dimensions, pinouts, component placement and labels shown in generated images are **not authoritative engineering specifications**.

Bootable electrical test harness — 26 September 2026

A complete selectable-at-boot electrical/component test harness is **ADOPTED / SPECIFICATION STAGED**. It will provide deterministic component tests, PASS / FAIL / NOT TESTED results and stable documented failure codes. Full specification and initial code registry: `ELECTRICAL_TEST_HARNESS.md`.

The harness is intended for bench commissioning, manufacturing, installation and maintenance diagnosis. It must not imply airworthiness from a PASS result. Tests should continue after non-dangerous failures so all detected faults are reported in one run. Display/backlight tests include explicit human visual confirmation because successful bus writes cannot prove correct visible output. Optional/unfitted hardware is distinguished from failed hardware. Implementation begins with a simulated harness, then follows actual Phase-1 component bring-up.

Boot/update Wi-Fi interaction — 26 September 2026

The boot-time firmware-update option must invoke an explicit **Wi-Fi Connection** dialog before checking the OTA manifest. It supports saved-network connection, network scan/manual entry, changing/forgetting credentials, retry and **Start EFIS without update**. Connection diagnostics remain layered (Wi-Fi -> Internet/DNS -> TLS -> OTA server). Network failure must never block startup of the existing known-good firmware. Wi-Fi is maintenance-only and must not silently reconnect during normal instrument operation. See `OTA_USER_SCENARIO.md`.

Persistent Wi-Fi credential storage — 26 September 2026

Wi-Fi SSID and credentials are stored persistently in ESP32 flash using ESP-IDF NVS in a dedicated namespace. The saved password is never redisplayed after entry and must never appear in logs, diagnostics, test-harness output or OTA-server requests. The boot/update UI provides **Forget network**, which erases the saved credentials. Production units require **NVS encryption** for stored Wi-Fi credentials; ordinary NVS is permitted only during early prototype bring-up. Non-secret configuration such as the OTA server URL may remain ordinary configuration.

Persistent operational fault log — 26 September 2026

ADOPTED / IMPLEMENTATION PENDING. Hardware and software faults detected during normal EFIS operation must be written to a persistent fault/event log in ESP32 non-volatile storage and remain available after power loss or reboot. The rotary-controlled

boot menu gains **FAULT LOG** for review.

The log must be bounded/wear-conscious rather than an unlimited stream of flash writes. Record fault transitions and significant events, not every repeated polling failure. Each entry should include a stable fault code, subsystem/source, severity, occurrence count, first/most-recent occurrence information where a trustworthy time source is available, firmware/build identity, and useful non-secret diagnostic context. Never store Wi-Fi passwords or other credentials in the fault log.

Critical runtime faults must still produce immediate fail-obvious indications during operation; persistent logging is diagnostic history and must never substitute for live validity/failure annunciation. Clearing the log requires an explicit user action from the maintenance/boot UI and must not happen automatically on reboot or firmware update.

Product licence provisioning — 26 September 2026

ADOPTED / DESIGN PENDING IMPLEMENTATION. The EFIS must provide for product licence **generation, installation, inspection and reset/re-provisioning** without making normal instrument safety functions dependent on a live Internet connection.

Design direction:

- each production instrument has a stable device identity derived from a provisioned product/device identifier; do not use a Wi-Fi MAC address alone as the commercial identity;
- licences are generated off-device by an authorised licence-generation/admin tool and are **digitally signed**;
- the EFIS contains only the public verification key, never the private licence-signing key;
- a licence payload can carry licence/schema version, product/device ID, enabled feature set, issue information and optional expiry/support metadata;
- installation is a deliberate maintenance action available through the rotary-controlled boot/maintenance UI, with USB/service provisioning as the baseline and an authenticated network method possible later;
- verified licence state is stored persistently in protected non-volatile storage and remains usable offline;
- **LICENSE** in the boot menu shows non-secret licence/device status and supports Install/Replace and Reset;
- Reset requires deliberate confirmation, removes the installed licence/activation state, records the action in the persistent event/fault history, and returns the unit to an unlicensed/provisioning state; it must not reveal or regenerate private signing material;
- licence reset is separate from Wi-Fi credential reset and from general factory reset;
- licence corruption/signature failure must be fail-obvious in the licence UI and logged.

Safety boundary: licensing must not disable or corrupt essential fail-obvious behaviour in a way that could create misleading flight indications. The exact distinction between always-available core instrument functions and licensable optional features must be frozen before commercialisation.

Licence account service architecture — 26 September 2026

ADOPTED / DESIGN STAGED. Add a separate Docker service (working name `efis-account`) alongside OTA/admin services. It owns customer accounts, registered EFIS devices, licence entitlements and payment-provider linkage; it does not hold the licence-signing private key in the public web tier.

Device identity uses a provisioned immutable **EFIS Device ID / serial number** as the primary key. The ESP32 factory/eFuse base MAC may be recorded as a secondary hardware fingerprint and registration aid, but MAC address alone is not the licence identity because interface MACs can be derived/changed and MAC exposure is unnecessary for manual licensing.

Online boot-maintenance flow: LICENSE -> Connect Wi-Fi -> identify device -> authenticated account/licence endpoint -> fetch signed licence entitlement -> verify locally -> store in protected NVS -> continue offline. The EFIS must not send a user password; device authentication uses a provisioned device credential/challenge mechanism to be specified. Normal flight startup must not depend on account/payment/network availability.

Offline flow: show Device ID and short registration code/QR-capable text; user obtains a signed licence from the account portal on another device and enters/imports it manually or via USB service. The licence is verified locally with the embedded public verification key.

Payment integration is provider-adapter based. The account service creates checkout/customer-management requests and consumes verified payment webhooks; payment card data is handled by the payment provider, not stored by EFIS services. Payment status changes entitlements; a separate private signing worker/service generates signed licence payloads. Define explicit grace/revocation policy before subscriptions are enabled.

Suggested containers: efis-account API/web portal; PostgreSQL account/device/entitlement store; private efis-license-signer with tightly restricted signing-key access; existing efis-ota and efis-ota-admin. Payment provider secrets live only in server-side secret storage.

Payment-method requirements — 26 September 2026

The account/licensing portal must use a payment-provider abstraction rather than hard-code one gateway. Initial customer-facing methods should include:

- major credit/debit cards (customer enters card details into provider-hosted/tokenized fields; EFIS services never store raw PAN/CVV);
- PayPal;
- Apple Pay;
- Google Pay;
- optional Pay Later/BNPL only if commercially appropriate;
- regional methods can be enabled later without changing the EFIS licence protocol.

Preferred first implementation is a hosted/tokenized checkout from a PCI-compliant payment provider so card data is submitted directly to the provider and never traverses or persists in efis-account, its logs or database. Store only provider customer/payment/transaction identifiers, payment status, amount/currency and entitlement/audit metadata needed for reconciliation.

The server payment adapter must allow provider replacement/addition (for example Stripe, PayPal/Braintree or Adyen) without changing device licensing. Verified provider webhooks update the entitlement state; only then may the private licence signer issue/refresh the corresponding signed licence.

Account-service prototype — 26 September 2026

IMPLEMENTED IN REPOSITORY / BUILD AND SECURITY VALIDATION PENDING. account-service/ now contains a Dockerized FastAPI + PostgreSQL prototype for customer accounts, provisioned EFIS Device IDs, secondary MAC fingerprints and licence entitlements. It binds to loopback port 8091 and includes build/health-test scripts.

The prototype intentionally does not store payment-card PAN/CVV and does not hold the private licence-signing key. The internal normalized payment-event endpoint is scaffolding only: production Stripe/PayPal/Braintree/Adyen adapters require provider webhook signature verification. Authentication/session management, email verification/reset/MFA, admin RBAC, device challenge authentication, signer integration, migrations/backups, privacy controls and security tests remain pending.

No claim of a successful Synology build/deployment is made until the supplied harness is run there.

Customer mobile applications — 26 September 2026

ADOPTED / NATIVE SKELETONS STAGED / BUILD VALIDATION PENDING. Native customer companion applications are now staged under customer-app/: SwiftUI for iPhone/iPad and Kotlin/Jetpack Compose for Android. Both are clients of the same future versioned efis-account HTTPS API and cover account access, registered instruments, Device ID registration, licence/entitlement status, provider-hosted payment, signed licence retrieval/offline activation and reset/re-provisioning.

The apps are not flight instruments. They must never store raw payment PAN/CVV or licence-signing private keys. Account/session tokens will use Keychain on Apple platforms and Keystore-backed storage on Android. Current screens are prototypes with payment/licence actions disabled until the authenticated API, verified payment adapters and private signer are implemented. No iOS/Android build success is claimed yet.

Device ownership and licence transfer — 26 September 2026

ADOPTED / IMPLEMENTATION PENDING. Customer account architecture now requires controlled transfer of an EFIS when sold. Supported policy model includes device + transferable licence, device-only transfer where entitlement policy permits, and audited administrative recovery. Seller initiates, buyer accepts using a short-lived single-use transfer credential, ownership changes atomically, payment instruments never transfer, and accepted transfers produce a fresh signed licence for the unchanged EFIS Device ID. Web/iOS/Android clients will expose Sell / Transfer EFIS and Accept transferred EFIS workflows. Ownership-transfer licence grace is now ADOPTED: default 30 days from buyer acceptance, configurable server-side, with the old installed licence

remaining usable while the buyer installs a replacement signed licence. Installing the replacement ends transfer grace early. Subscription, refund/chargeback and non-transferable entitlement policies remain to be frozen.

Device ID boot display — 26 September 2026

ADOPTED / FIRMWARE + SIMULATOR IMPLEMENTATION PENDING. Every normal boot menu must display the permanent provisioned EFIS Device ID above the menu. It is available offline, survives resets/licensing/ownership transfer, and is sourced from authoritative provisioned identity rather than MAC. Missing/corrupt identity is fail-obvious and logged; no placeholder identity may be invented.

Authoritative licensing process — 26 September 2026

DOCUMENTED / IMPLEMENTATION PARTIAL. docs/LICENSING.md is now the authoritative end-to-end licensing specification. It covers trust boundaries, immutable Device ID provisioning/display, signed payload requirements, purchase/entitlement flow, online and offline installation, local boot verification, firmware compatibility, reset/replacement, ownership transfer, default 30-day transfer grace, lifecycle/audit/failure behaviour, production security gates and required tests. Existing account/web/mobile artefacts are staged; signer, cryptographic format, device challenge/API, payment adapters and physical ESP32 implementation remain pending.

Licence cryptography and format — 26 September 2026

ADOPTED / IMPLEMENTATION PENDING. Licence signing is frozen on **Ed25519** over a **deterministic CBOR** payload, wrapped in a versioned CBOR envelope carrying algorithm ID, key ID, payload bytes and signature. Base64url is permitted only as outer text transport. EFIS holds public verification keys only; private keys remain isolated in the licence signer/key-storage boundary. Key rotation uses an explicit key ID and overlapping trusted public keys. Parsers require strict bounds and hostile-input handling. Cross-platform signer/ESP32 test vectors are mandatory before production.

Licence-signer container — 26 September 2026

IMPLEMENTED IN REPOSITORY / SYNOLOGY BUILD VALIDATION PENDING. license-signer/ now contains a private FastAPI signer using the adopted Ed25519 + deterministic-CBOR envelope. Docker deployment binds to loopback port 8092, runs non-root/read-only, drops capabilities, uses no-new-privileges and mounts the signing key at runtime rather than baking it into the image. Development key generation and build scripts are included; key files/secrets are Git-ignored. Initial bearer-token service authentication is scaffolding pending stronger production service/network controls. Account-service integration, production key custody/rotation, cross-platform test vectors and ESP32 verification remain pending. The signer must never be exposed by the public reverse proxy/router.

Production parts cost model — 26 September 2026

PLANNING TARGET / NOT A QUOTATION. Current one-off completed-instrument parts estimate is approximately **£250–£365**, with **£300** used as the working planning figure and **£350** as a contingency target while major selections remain open. Aim for a repeatable production BOM of **£300 or less at approximately 10–25 units** where engineering requirements permit. Development equipment/labour and commercial/certification costs are excluded. docs/COST_MODEL.md is the maintained cost model; BOM.md remains authoritative for components/procurement.

ASI/IAS capability — 26 September 2026

REMOVED FROM SCOPE. The earlier 180 kt IAS requirement, MS4525DO differential-sensor proposal and PITOT enclosure connection are superseded. The EFIS retains BMP585 static/barometric altitude sensing through one labelled STATIC pneumatic connection. No PITOT input or ASI/IAS calculation/display is required.

Working brand and product name — 27 September 2026

ADOPTED AS WORKING IDENTITY / FORMAL CLEARANCE PENDING. Brand: **MicroSky Avionics**. Product: **Horizon**. Customer-facing shorthand: **MicroSky Horizon**. Keep the existing ESP32-EFIS repository and technical identifiers stable for now; branding must not trigger unnecessary changes to Device IDs, OTA/licensing identifiers or service names. docs/BRANDING.md is the authoritative branding note.

Customer support chatbot — 27 September 2026

ARCHITECTURE ADOPTED / MVP STAGED / SYNOLOGY VALIDATION PENDING. support-service/ adds FastAPI + PostgreSQL support infrastructure with product-document ingestion, retrieval, interaction/feedback storage, repeated-question candidate aggregation and explicit human promotion to a validated Q&A database. Customer text/model output never becomes product truth automatically. LLM generation, semantic/vector retrieval, private reviewer UI, production auth/RBAC, privacy/retention and public deployment remain gated. See docs/SUPPORT_SERVICE.md.

Support-service LLM capability — 27 September 2026

IMPLEMENTED IN REPOSITORY / RUNTIME VALIDATION PENDING. MicroSky Horizon Support now accepts natural-language questions and can produce polished grounded answers through a configurable OpenAI-compatible LLM endpoint. Retrieval of approved product documentation occurs before generation; unsupported questions must fail conservatively. Generated responses remain candidates until human validation and cannot automatically enter the validated Q&A knowledge base. Provider credentials remain runtime secrets. Semantic retrieval and production security/privacy evaluation remain pending.

Support semantic retrieval — 27 September 2026

IMPLEMENTED IN REPOSITORY / RUNTIME VALIDATION PENDING. Support document retrieval now uses PostgreSQL pgvector and configurable embeddings for natural-language semantic matching, with lexical fallback. This feeds the existing grounded LLM/human-validation pipeline. Embedding-model changes require controlled re-indexing; retrieval quality and prompt-injection resistance remain production gates.

Support answer provenance and customer validation — 27 September 2026

ADOPTED / MVP IMPLEMENTED. Support interactions can surface validated answers and retained invalidated/superseded answers, with status made explicit to the customer. Generated/retrieved unreviewed answers are likewise labelled as not yet validated. Each response asks whether it resolved the customer's question; that resolution signal is stored for quality/prioritisation but is explicitly not a substitute for technical/human validation. Invalidated answers remain auditable rather than being silently deleted.

Support voice/keyboard UX — 27 September 2026

ADOPTED / IMPLEMENTATION PENDING. Horizon customer support will support keyboard and voice through one common knowledge/validation pipeline. Voice uses STT to visible/editable text and optional TTS for answers; keyboard remains the exact-string/fallback path. Provenance/status and resolution feedback are preserved in both modes. Raw microphone audio is not retained by default. Provider-neutral STT/TTS adapters and client UI remain to be implemented and tested.

Customer support voice/chat implementation — 27 September 2026

IMPLEMENTED IN REPOSITORY / RUNTIME VALIDATION PENDING. Responsive web chat, keyboard input, microphone capture, editable STT transcript, optional TTS playback, answer-status/source display and Yes/No resolution feedback are now staged in support-service/. Provider-neutral speech configuration is runtime-only; raw audio is not persisted by the application. Browser/Synology/provider validation remains pending.

Engine sensor connector and USB GPS — 30 September 2026

ADOPTED DESIGN DIRECTION / IMPLEMENTATION PENDING. Horizon will provide externally accessible plug connectivity for a grouped engine-sensor harness covering CHT, EGT, oil temperature, oil pressure and coolant/water temperature. Sender types/ranges, connector family/pinout, conditioning, precision conversion, thermocouple cold-junction compensation, protection and calibration remain to be frozen. Raw aircraft sensor wiring will not connect directly to ESP32 ADC/GPIO.

The earlier dedicated GNSS connector is **REMOVED FROM THE DESIGN DIRECTION**. GNSS will instead use a validated external USB GPS mouse/receiver. USB-host support, receiver/protocol selection, power budget, retention, ESD/protection and service-port coexistence must be proven before PCB/enclosure freeze. A USB GPS mouse is an active powered receiver, despite being operationally plug-and-play.

Remote Engine Interface Unit — 30 September 2026

PREFERRED ARCHITECTURE / IMPLEMENTATION PENDING. Replace the direct multi-sensor connection at Horizon with a separate Engine Interface Unit (EIU), initially targeting conventional Rotax 912-series sensors. The EIU terminates/conditions CHT or coolant/head temperature as applicable, EGT, oil temperature and oil pressure, performs precision conversion and sensor diagnostics, and sends values plus validity metadata to Horizon over one robust digital link. **Protected non-isolated Classical**

CAN is the V1 baseline; galvanic isolation is added only where installation/noise/common-mode testing demonstrates a need. Engine variant/serial/configuration determines exact sender definitions and limits; do not assume all 912 variants are electrically identical. The EIU must remain monitoring-only and must not compromise any engine-control or existing required indication circuit. Horizon invalidates stale/lost EIU data.

AEF-CAN / Engine Interface Unit architecture — 30 September 2026

| ID | Idea / decision | State | Reason / replacement |

|---|---|---|---|

| D200 | Separate EIU near engine sensors with digital bus to Horizon | **ADOPTED** | Keeps analogue/thermocouple/sensor conditioning local and provides an extensible digital aircraft interface. |

| D201 | AEF-CAN application protocol independent of ESP32 hardware | **ADOPTED** | Allows future MCU, display, logger and sensor-node implementations without redefining aircraft data semantics. |

| D202 | Classical CAN at 500 kbit/s, 11-bit IDs for V1 | **ADOPTED** | Matches ESP32-S3 native TWAI capability and is ample for initial engine traffic. |

| D203 | Require CAN FD for V1 | **DISMISSED** | ESP32-S3 native TWAI is Classical CAN only; V1 does not need FD bandwidth. Architecture remains transport-neutral for later FD-capable hardware. |

| D204 | Broadcast measurement producers; no EFIS master polling required | **ADOPTED** | Decouples producers/consumers and permits logger, second display and future modules. |

| D205 | Encode display colour/warning state in normal sensor measurements | **DISMISSED** | Producers publish physical facts and validity; consumer configuration owns units/thresholds/presentation. |

| D206 | Fixed canonical units and scaled integers on wire | **ADOPTED** | Deterministic cross-platform encoding without floating-point representation ambiguity. |

| D207 | Explicit validity/fault/freshness semantics | **ADOPTED** | Prevents stale plausible measurements being treated as live. |

| D208 | Major/minor protocol version independent of firmware version | **ADOPTED** | Supports additive evolution while preserving existing message meaning. |

| D209 | Node discovery and capability announcement | **ADOPTED** | Avoids assuming one EIU, one display, a fixed engine or a fixed sensor count. |

| D210 | Reserve functional 11-bit CAN-ID blocks before expansion | **ADOPTED** | Prevents arbitrary ID growth and preserves priority/functional organization. |

| D211 | Machine-readable protocol/aef-can.yaml as protocol source of truth | **ADOPTED** | Enables common C/C++, Swift, Python, documentation and deterministic test-vector generation. |

| D212 | CAN FD as future extension, not current requirement | **ADOPTED** | Preserves a migration route without putting FD frames onto Classical-only nodes. |

| D213 | SENSOR_STATUS_V1 paged sensor-ID/state layout | **ADOPTED / FROZEN V1** | Explicit node instance + page + sensor-ID/state pairs scale across different engines, sensor counts and multiple EIUs without a Rotax-specific bitmap. |

Implementation status: docs/CAN-PROTOCOL.md and protocol/aef-can.yaml exist. Firmware codec, simulator CAN traffic, schema validation, generated test vectors and physical transceiver testing remain pending. Simulation success will not be recorded as physical CAN validation.

AEF-CAN V1 implementation checkpoint — 30 September 2026

Core V1 protocol engineering is now **IMPLEMENTATION-READY / PHYSICAL VALIDATION PENDING**. The scalable sensor-status layout is frozen; the YAML is marked adopted-v1; a structural validator and deterministic golden frame vectors have been added. The golden vectors were independently checked against the specified little-endian scaled-integer encodings.

Still pending: C/C++ codec, Swift codec, Python codec/tooling, automated execution of the validator in CI, EIU traffic simulator, Horizon stale/fault consumer, bus-load test, and real two-transceiver/termination bench validation.

Provisional Stage 7 — Engine Sensor / EIU integration

PROVISIONAL / CORE EFIS MADE INTERFACE-READY. Engine monitoring is now an additional Stage 7 and does not block Stages 1–6. Horizon reserves an AEF-CAN acceptance path; firmware now has a transport-independent optional engine state with stale invalidation. Final carrier hardware is to reserve TWAI-to-CAN transceiver/protection/connector/termination provision. The remote EIU, Rotax sender front ends and aircraft harness remain provisional pending bench development and installed-engine survey. See docs/STAGE-7-EIU.md.

Device registration and QR ownership claiming — 30 September 2026

ADOPTED DESIGN DIRECTION / IMPLEMENTATION PENDING. Fit a durable QR/data label plus human-readable immutable Device ID to a non-prominent accessible part of the Horizon body. The QR is discovery/identification only and contains no reusable

credential, PII or private key. First-owner claiming requires account sign-in plus physical-possession proof using a short-lived device challenge or authenticated maintenance-session confirmation; scanning the static QR alone cannot transfer ownership. Registration is separate from licensing and never blocks normal START EFIS. Existing explicit ownership-transfer rules remain authoritative for resale. See docs/DEVICE_REGISTRATION.md.

Single USB-C service/GPS interface — 30 September 2026

ADOPTED / DETAILED PCB VALIDATION PENDING. Horizon will have one external USB-C OTG connector labelled **SERVICE / GPS**. ESP32-S3 operates as USB device for factory programming, service, diagnostics and recovery, and as USB host for the external GPS receiver. Routine production updates use signed OTA, but USB recovery is retained. Carrier hardware must include correct USB-C role handling, protected/current-limited host VBUS switching, ESD/signal-integrity provisions and internal factory/recovery programming/test pads. No separate external programming or dedicated GNSS connector is planned.

Containerised EFIS boot/service simulator — 1 October 2026

MVP IMPLEMENTED IN REPOSITORY / RUNTIME VALIDATION PENDING. `efis-web-simulator/` provides a Dockerised browser emulator of the Horizon round display and rotary/push boot menus. It covers START EFIS, FULL TEST, FAULT LOG, LICENSE and FIRMWARE UPDATE, with synthetic device state, Wi-Fi online/offline injection, licence status/install/reset, public OTA manifest checking, firmware staging and explicit ACTIVATE & REBOOT. The browser talks only to the simulator backend; service secrets must never be exposed to client JavaScript. Default mock mode is safe and requires no live services. Upstream OTA and licence base URLs are configurable for integration testing; mutating production services is deliberately disabled by default. Runtime/Synology validation and final licence-service API adapters remain pending.

Offline phone-carried firmware updates — 1 October 2026

ADOPTED / IMPLEMENTATION PENDING. iPhone and Android apps become offline-capable firmware carriers. While Internet is available they can cache the signed release set compatible with a registered Horizon installation, including Horizon plus EIU/future-node packages. At an aircraft with no network coverage, the phone transfers the cached Horizon package over a local Horizon maintenance Wi-Fi connection. Horizon independently verifies and stages it using the same A/B/signature/rollback path as Internet OTA, then acts as the existing AEF-CAN maintenance gateway for compatible EIU/node updates. The phone is not a signing authority and cannot bypass Horizon maintenance/safety gates. Bluetooth is not the V1 bulk firmware transport; BLE may later assist discovery/pairing. Local Wi-Fi authentication/onboarding and offline-transfer simulator/physical tests remain implementation gates. See docs/REMOTE_UPDATES.md.

Multi-node firmware dependency model — 1 October 2026

ADOPTED / IMPLEMENTATION PENDING. Horizon, EIU and future AEF-CAN nodes retain independent firmware versions. Compatibility is based on firmware identity, AEF-CAN major/minor, advertised capabilities and a signed release-set manifest containing compatible artifacts, constraints and update ordering. Mobile offline cache stores the evaluated release set rather than independently combining latest node images. Normal multi-node releases must keep every committed intermediate and rollback state compatible; otherwise the change requires a special migration procedure rather than normal OTA. Horizon will re-discover/validate node protocol and capabilities after each update step before continuing. Initial `ota-server/release-set.example.json`, `generic-ota-server/admin/release_resolver.py` and dependency transition tests are now implemented in source control. Runtime execution/CI integration, signed-manifest verification, richer schema validation and integration with the OTA/mobile simulators remain pending.

Architecture-alignment pass — 1 October 2026

SOURCE UPDATED / RUNTIME VALIDATION PENDING. The customer iOS and Android prototypes, Dockerised web/boot simulator, OTA public container, OTA admin container and automated OTA simulations have been aligned with the adopted offline multi-node release-set architecture. Mobile prototypes now expose complete release-set caching/offline readiness, Horizon maintenance-Wi-Fi transfer and EIU->Horizon dependency sequencing. The web simulator models cached release sets, Horizon/EIU versions and capability-gated activation. The public OTA nginx configuration reserves `/efis/release-set.json`; the admin image includes resolver/simulator test code; GitHub CI now runs dependency, offline-phone and existing OTA simulations.

EIU CAN simulator migration completed: it now models 7-byte FW_DATA, 4 KiB CRC/ACK blocks, 64 KiB append-only durable checkpoints, corrupt/incomplete journal rejection, power-loss retransmission from the last durable offset, inactive-slot staging, explicit stationary activation and trial-boot confirmation. Remaining validation is physical: mobile networking/signature verification, Docker runtime execution, real CAN transceivers/timing/load, ESP32 flash behaviour and bootloader rollback.

docs/PROJECT_STEPS.md

Repository source: docs/PROJECT_STEPS.md

ESP32 EFIS — Project Steps and Validation Gates

Status date: 16 September 2026

This document defines the ordered development path for the ESP32 EFIS. The instrument has **one physical Newhaven 2.1-inch 480×480 display** and three selectable pages: Horizon/PFD, Altimeter and Compass. The project remains experimental **supplementary/non-primary**.

The newly adopted GNSS requirement is part of the Horizon/PFD and Compass design: both pages will show latitude/longitude and position quality. The coordinate numbers are colour-coded from the GNSS receiver's reported horizontal-accuracy estimate. GNSS is independent of attitude, barometric altitude and magnetic heading; loss of one source must not falsely invalidate or validate another.

Stage 0 — Software baseline and requirements

Current state: active / substantially complete for existing instruments; GNSS display extension now active.

Maintain the accepted 480×480 QEMU renderer, Swift simulator, safety/failure model, documentation and deterministic tests. Add the GNSS data model and shared accuracy classifier, then implement the position overlay on both Horizon/PFD and Compass.

GNSS display acceptance states must include valid positions at each accuracy band, stale data, no fix and invalid data. Thresholds are: ≤1 m green; >1–3 m light green; >3–10 m yellow; >10–30 m orange; >30 m red. Stale/no-fix must not leave believable frozen coordinates. TRK means GNSS course over ground; GS means GNSS groundspeed; neither may be presented as HDG or IAS.

Gate to Stage 1: requirements documented; QEMU/Swift scenarios defined; no software-only result described as physical validation.

Stage 1 — Bench hardware and single-display bring-up

Inspect received parts. Establish current-limited bench power. Bring up the ESP32-S3-WROOM-1-N16R2, the **single** Newhaven display through NHD-FFC40/verified FFC, TPS61169 backlight, MCP23008 and PEC09. Verify RGB565 colour/order/timing, PSRAM framebuffer operation, brightness control and physical page selection.

Gate to Stage 2: stable power, single display and controls operate reliably on the bench; measured power/current recorded; obvious display/control failures understood.

Stage 2 — Sensors and GNSS integration

Integrate BMI088, ported BMP585, remote-development RM3100 and a selected GNSS receiver. The GNSS receiver must provide latitude, longitude, fix state/freshness and an explicit horizontal-accuracy estimate; receiver/antenna hardware is not yet frozen.

Validate BMI088 axes/signs before connecting it to the live Horizon. Validate BMP585 pressure/altitude and stale/failure behaviour. Validate RM3100 acquisition, calibration approach and tilt-compensated heading. Validate GNSS update/freshness, accuracy classification and display on Horizon/PFD and Compass. Exercise simultaneous and independent failures so valid GNSS cannot make invalid attitude/heading appear valid and GNSS failure does not invalidate otherwise-valid attitude/heading.

Gate to Stage 3: all four source interfaces are stable enough that installation measurements are meaningful; GNSS display behaviour has passed QEMU/Swift acceptance and bench GNSS data tests; physical sensor results are explicitly recorded.

Stage 3 — Aircraft survey and measurements

Survey the Skyranger without committing final installation hardware. Measure static-system tube OD/ID/material, tee location, run and panel clearance. Survey magnetically quiet RM3100 positions and routing. Survey GNSS receiver/antenna positions for sky view, cable/module routing and interference. Confirm enclosure/panel clearances and service access.

Gate to Stage 4: static, magnetometer and GNSS installation requirements are measured rather than guessed.

Stage 4 — Physical interfaces and enclosure freeze

Use Stage 3 measurements to freeze the static fitting/plumbing, remote magnetometer harness/connector/gland, GNSS antenna/receiver interface and routing, optical window and enclosure openings. Conduct physical fit trials before irreversible geometry is treated as final.

Gate to Stage 5: external interfaces and mechanical constraints are sufficiently stable to commit connector positions and PCB geometry.

Stage 5 — Custom carrier PCB

Resume the parked carrier PCB. Complete and review the KiCad schematic and recommended 4-layer layout around the validated display, controls, BMI088, BMP585, RM3100 and GNSS interfaces. Include power protection/decoupling appropriate to the validated bench design. Generate manufacturing files, BOM and placement data; order a small prototype batch only after review.

Populate/test one board first. Repeat display, sensor, GNSS, power, thermal, failure/staleness and interference tests before treating the board revision as suitable for installation development.

Gate to Stage 6: integrated carrier passes bench functional/electrical/thermal/fault tests, including GNSS reception/interference and RM3100 magnetic-interference checks.

Stage 6 — Aircraft installation and ground validation

Freeze aircraft 12 V conversion/protection, fuse/circuit protection, wiring/connectors and mounting from measured prototype data. Install the instrument, static connection, remote RM3100 and GNSS receiver/antenna using the validated interfaces.

Perform aircraft-ground tests with radios, engine/electrical loads, display brightness and other relevant equipment in representative states. Confirm attitude sense, static altitude, heading/magnetic interference, GNSS position/freshness/accuracy indication, power stability, thermal behaviour and failure annunciation.

Gate to Stage 7: no unresolved ground-test issue that could create a plausible but incorrect flight indication.

Stage 7 — Airborne comparison and refinement

Only after the preceding gates, perform cautious airborne comparison against independent trusted instruments/sources. Record discrepancies and environmental effects. Do not use the ESP32 EFIS as the sole source of attitude, altitude, heading or navigation information.

Any material change discovered here loops back to the relevant earlier stage; it does not bypass bench validation.

Cross-stage rules

Every functional change updates code, relevant documentation, PROJECT_STATUS.md decision history where material, and this step plan when sequencing changes. Every validation claim identifies its layer: compile, QEMU, Swift, Docker runtime, electronics bench, physical fixture, aircraft ground or airborne.

No missing/stale source may leave a plausible frozen indication. Simulation is conspicuously marked and is never an automatic fallback. PCB and irreversible aircraft-interface decisions remain parked until their prerequisite measurements and bench validations are complete.

See GNSS_DISPLAY.md, PROJECT_STATUS.md, TESTING.md, SENSORS.md, WIRING.md, ENCLOSURE.md, POWER_BUDGET.md and ../BOM.md.

Added boot/maintenance workstream — 26 September 2026

- implement rotary-controlled boot menu with START EFIS (default), FULL TEST and FIRMWARE UPDATE;

- implement simulated electrical-test result model and documented failure codes before physical test claims;
- integrate physical component tests as each hardware interface is commissioned;
- implement boot-time Wi-Fi dialog and persistent NVS credentials;
- enable encrypted NVS for production Wi-Fi credentials;
- preserve a Start EFIS escape path for every network/update failure;
- keep activation explicit after download/verification and validate A/B first-boot rollback.

Airspeed scope decision — 26 September 2026

ASI/IAS development has been removed. Stage 3 surveys only the STATIC pressure plumbing required by the BMP585; Stage 4 freezes only the STATIC enclosure fitting/plumbing. No PITOT connection or differential-pressure sensor is required.

docs/REMOTE_UPDATES.md

Repository source: docs/REMOTE_UPDATES.md

ESP32 EFIS — Remote firmware update and fallback design

Design status: server/admin and Swift user-flow implemented; physical ESP32 OTA not implemented or hardware validated

Baseline: 16 September 2026

Target: ESP32-S3-WROOM-1-N16R2, 16 MB flash

Purpose and policy

Provide controlled remote application firmware updates while preserving a known-working application. The definitive policy separates four actions: **Stage** → **Publish** → **Download** → **Activate**.

The Docker administrator stages and explicitly publishes a release. The EFIS may optionally auto-download the next published compatible release while in its deliberate maintenance/network-update context, but **activation and reboot always require local ACTIVATE & REBOOT**. There is no unattended activation policy.

A/B architecture

Use ESP-IDF native dual-slot OTA:

```
HTTPS published manifest + signed application
      |
      v
ESP32-S3 bootloader
  ■■■ ota_0      current known-good or candidate
  ■■■ ota_1      candidate or previous known-good
  ■■■ otadata    boot selection/OTA state
NVS/config/calibration remains separate
```

Download always targets the inactive slot. The running image is never overwritten in place. A completed candidate must pass transport/image/signature/compatibility verification before it can become **UPDATE READY**.

Download versus activation

Auto-download ON means: discover the administrator-published release, download it to the inactive slot and verify it. It must stop there. The candidate remains dormant until the user chooses **ACTIVATE & REBOOT**.

Auto-download OFF means the user first chooses Download. The same verification and ready state follows. Network/download failure leaves the current application selected and usable.

If an administrator withdraws a release by republishing another manifest before activation, the production client must re-check publication/compatibility policy before activating a previously downloaded candidate and invalidate a withdrawn candidate where policy requires.

Activation / fallback

After explicit activation, select the candidate boot slot and reboot. Enable ESP-IDF application rollback with application-controlled confirmation. The candidate starts `ESP_OTA_IMG_PENDING_VERIFY` and is not accepted merely because `app_main()` starts.

Minimum first-boot checks:

1. image/partition state sane;
2. NVS and required schema readable or safely migrated;
3. core render/data tasks alive for validation interval;
4. no watchdog/reset loop/fatal allocation condition;

5. physical display framebuffer allocation succeeds on hardware;
6. required software invariants pass.

Missing development sensors normally produce invalid data rather than rejecting an otherwise bootable firmware image; stricter production hardware-profile checks may be introduced only deliberately.

Pass:

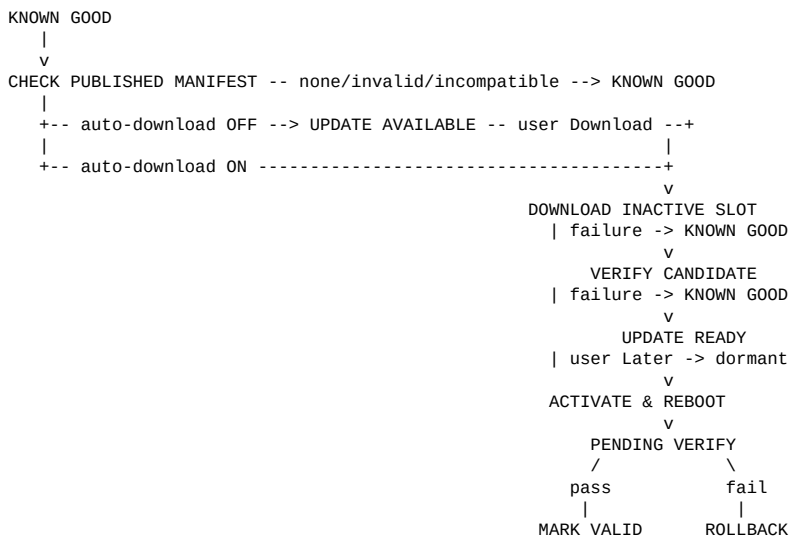
```
esp_ota_mark_app_valid_cancel_rollback();
```

Mandatory failure:

```
esp_ota_mark_app_invalid_rollback_and_reboot();
```

Crash/reset/power loss before confirmation must cause return to the previous known-good image.

State model



Release/distribution service

GitHub remains source/build history. The Docker OTA server is distribution. Its private admin service stages binaries and metadata; explicit Publish atomically changes the public manifest. Public EFIS clients never receive GitHub credentials and never access the admin endpoint.

Manifest identifies product, version, build, hardware profile, IDF baseline, immutable image URL, SHA-256, minimum allowed version and release notes. Production releases additionally require signed application images.

Security

Minimum: valid HTTPS server certificate, product/hardware/version compatibility, complete-image integrity, logging of current/candidate/result. Production: signed application verification. Signing private keys never enter Git, Docker image, EFIS filesystem or downloadable artefacts; only verification material belongs on the device.

Do not initially enable irreversible eFuse anti-rollback during prototype work. Operational A/B rollback must remain available. Secure Boot v2, flash encryption and eventual security-version anti-rollback are later hardening decisions after update/recovery is proven.

Persistent configuration compatibility

Application rollback is incomplete if new firmware irreversibly mutates shared NVS. Version every persistent config/calibration schema, prefer additive/backward-compatible changes, retain old calibration until the candidate is confirmed, and stage incompatible migrations so the previous known-good image can still boot safely.

Network model

Wi-Fi is maintenance-oriented. The EFIS may use an iPhone Personal Hotspot as its Internet gateway to the public HTTPS OTA origin. Credentials are stored locally in NVS. Auto-download is constrained to the maintenance/network-update context; normal flight presentation must not silently initiate update traffic.

Recovery

Preserve automatic A/B rollback, USB serial/JTAG service recovery, and a manual diagnostic procedure identifying running/boot/last-invalid partitions and versions. Initial OTA updates application image only; bootloader/partition-table OTA is excluded.

Validation plan

Software: manifest/parser, state machine, auto/manual download decisions, withdrawn-candidate policy, self-test decisions, schema migration/rollback and maintenance UI. Physical bench: A→B/B→A, corrupt/truncated/wrong-product/wrong-hardware/wrong-signature rejection, Wi-Fi loss, power loss at multiple phases, deliberate first-boot crash/watchdog/self-test failure, exact rollback, NVS compatibility, repeated cycles and USB recovery.

QEMU/Swift success is not evidence of physical flash/power-loss recovery.

Current status

Implemented in source control: Docker read-only origin, private image admin, staged metadata, explicit Publish, stage-only CLI helper, network/OTA Swift user scenarios. Not implemented/validated on ESP32: A/B partition table, maintenance Wi-Fi client, HTTPS OTA writer, signed-image pipeline/verification, boot confirmation and physical rollback/interruption tests.

Boot/update entry and Wi-Fi — 26 September 2026

OTA is entered deliberately from the physical rotary boot menu: **FIRMWARE UPDATE** -> **Wi-Fi Connection** -> Check -> Download/Verify -> explicit **ACTIVATE & REBOOT**. The rotary/push control must operate the complete instrument-side flow. START EFIS remains available if networking fails.

Saved Wi-Fi credentials persist in ESP-IDF NVS and require encrypted NVS in production. Wi-Fi/network failure must not alter the current known-good application or prevent it starting. Auto-download, where enabled, still applies only inside maintenance/update context and never implies automatic activation.

Licence compatibility

Firmware/OTA and product licensing are separate trust domains. OTA image signing and licence signing must use separate keys. Firmware updates must preserve a valid installed licence unless an explicit migration is required and tested. Licence verification is local/offline using a public verification key embedded/provisioned in the EFIS; the private licence-generation key must never be present on the instrument or OTA public server.

Licence schema/version compatibility must be checked during update testing so an otherwise valid update cannot silently strand a licensed instrument.

Licence account service architecture — 26 September 2026

ADOPTED / DESIGN STAGED. Add a separate Docker service (working name `efis-account`) alongside OTA/admin services. It owns customer accounts, registered EFIS devices, licence entitlements and payment-provider linkage; it does not hold the licence-signing private key in the public web tier.

Device identity uses a provisioned immutable **EFIS Device ID / serial number** as the primary key. The ESP32 factory/eFuse base MAC may be recorded as a secondary hardware fingerprint and registration aid, but MAC address alone is not the licence identity because interface MACs can be derived/changed and MAC exposure is unnecessary for manual licensing.

Online boot-maintenance flow: LICENSE -> Connect Wi-Fi -> identify device -> authenticated account/licence endpoint -> fetch signed licence entitlement -> verify locally -> store in protected NVS -> continue offline. The EFIS must not send a user password; device authentication uses a provisioned device credential/challenge mechanism to be specified. Normal flight startup must not depend on account/payment/network availability.

Offline flow: show Device ID and short registration code/QR-capable text; user obtains a signed licence from the account portal on another device and enters/imports it manually or via USB service. The licence is verified locally with the embedded public verification key.

Payment integration is provider-adapter based. The account service creates checkout/customer-management requests and consumes verified payment webhooks; payment card data is handled by the payment provider, not stored by EFIS services. Payment status changes entitlements; a separate private signing worker/service generates signed licence payloads. Define explicit grace/revocation policy before subscriptions are enabled.

Suggested containers: efis-account API/web portal; PostgreSQL account/device/entitlement store; private efis-license-signer with tightly restricted signing-key access; existing efis-ota and efis-ota-admin. Payment provider secrets live only in server-side secret storage.

Offline phone-carried OTA — 1 October 2026

ADOPTED. iPhone and Android applications are first-class Horizon firmware delivery clients for locations where the aircraft has no usable Internet connection.

When Internet is available, the mobile app may download and retain the signed firmware packages compatible with the customer's registered Horizon installation. The cache should include the Horizon image and compatible EIU/future-node images required for the selected release set. The app shall show whether the installation is ready for an offline update before the user travels to the aircraft.

At the aircraft, the phone transfers the cached Horizon package directly to Horizon over a local maintenance connection. The V1 preferred bulk transport is a temporary Horizon-hosted/local Wi-Fi maintenance link; Bluetooth is not the primary multi-megabyte firmware transport. BLE may later assist discovery/pairing but is not required for V1.

The delivery paths are deliberately equivalent:

```
OTA service -> Horizon
or
OTA service -> phone cache -> Horizon

Horizon -> AEF-CAN -> EIU/future nodes
```

There is one firmware trust model and one signed artifact format. The mobile app is a carrier/cache, **not a firmware signing authority**. Horizon independently verifies product, hardware compatibility, version policy, complete-image digest and cryptographic signature before staging an image received from either the Internet or a phone. EIU/future nodes retain their own independent candidate-authentication requirement before execution.

Offline delivery does not weaken maintenance gates. Horizon remains responsible for engine-stopped/stationary policy, adequate power, inactive A/B target selection, explicit activation, trial boot, confirmation and rollback. The phone cannot override these checks.

The local maintenance protocol must authenticate/bind the intended Horizon, reject unsolicited image injection, provide transfer integrity/resume or clean-restart behaviour, and avoid exposing normal flight operation to an unauthenticated maintenance endpoint. Exact Wi-Fi onboarding/authentication is an implementation design gate.

Validation must cover: pre-download/cache without the aircraft, airplane-mode/offline phone-to-Horizon transfer, wrong-device/wrong-hardware/wrong-signature rejection, interrupted local Wi-Fi transfer, duplicate/replayed package handling, Horizon A/B rollback, and subsequent offline Horizon-to-EIU AEF-CAN update.

Multi-node release compatibility and dependency management — 1 October 2026

ADOPTED. Horizon, EIU and future AEF-CAN node firmware versions are independently versioned. They are not required to share version numbers or be updated merely because another node has a newer build.

Compatibility is determined at four levels:

1. **Firmware version** identifies the concrete software build.
2. **AEF-CAN protocol major/minor** identifies the wire-protocol contract. Incompatible semantic changes require a new major version; additive compatible evolution uses minor/capability changes.
3. **Capabilities** advertise the concrete functions/messages a node implements. Feature dependencies should normally be expressed as required capabilities rather than arbitrary peer firmware-version comparisons.
4. **Signed release-set manifest** records the firmware artifacts, compatibility constraints, tested combinations and required update sequence for a coordinated product release.

A release-set manifest shall identify each artifact by product/node type, hardware compatibility, firmware version, image digest/signature metadata, AEF-CAN requirements/provisions and capability requirements/provisions. It may specify minimum peer firmware only where a capability/protocol constraint cannot adequately express the dependency.

The release set is the unit cached by the mobile app for offline maintenance. "Latest Horizon" and "latest EIU" shall not be independently combined without compatibility evaluation.

Safe sequencing and rollback invariant

A coordinated update may define an explicit node order; there is no hard-coded rule that EIU or Horizon always updates first. The release tooling chooses an order whose intermediate states are supported.

Publication invariant: every normal multi-node OTA release must preserve a compatible, usable installation at every committed intermediate state and after rollback of any candidate that can still roll back independently.

New firmware should therefore retain sufficient backward compatibility for at least the supported transition/rollback window. For example, if a new EIU capability is required by new Horizon firmware, the EIU can be upgraded first while continuing to provide the old capability; Horizon is upgraded only after the EIU confirms the new capability.

If a change cannot satisfy this invariant, it is not an ordinary OTA release. It requires an explicit migration procedure with its own recovery plan and must not be published through the normal one-click/offline release-set workflow.

Runtime enforcement

Before staging or activation, Horizon discovers installed AEF-CAN nodes and their protocol/capability advertisements and evaluates them against the release-set constraints. Missing required nodes, incompatible protocol major versions, absent required capabilities, wrong hardware, or an unsafe update sequence block activation with a specific diagnostic.

After each node update/reboot, Horizon re-discovers and verifies the node before proceeding to the next dependency step. A failed verification stops the sequence and preserves/recovers the last known compatible state according to the manifest rollback plan.

The same dependency engine and signed release-set manifest apply whether artifacts arrive directly from the OTA service or through the offline mobile cache.

docs/SAFETY.md

Repository source: docs/SAFETY.md

Safety and Airworthiness Notes

Status of this project

This is an **experimental electronics project**, not a certified or approved primary flight instrument.

The device must not be relied upon as the sole source of attitude information in flight unless and until the complete system has been independently demonstrated, accepted and approved for that role under the rules applicable to the aircraft.

Key hazards

Potential hazards include:

- incorrect attitude due to gyro bias or accelerometer misinterpretation
- misleading attitude during sustained acceleration or manoeuvring
- stale frozen display after sensor or software failure
- vibration-induced connector or mounting failure
- display washout in sunlight
- overheating in a closed cockpit
- power interruption or brownout
- software crashes or invalid arithmetic
- incorrect installation alignment
- EMI/RFI interaction with other avionics

Design rules

The project should follow these principles:

1. **Fail visibly.** A bad or stale solution must show ATTITUDE INVALID.
2. **Never freeze a plausible horizon silently.**
3. **Log faults and reset causes where possible.**
4. **Keep the IMU mechanically rigid and its orientation documented.**
5. **Use secure wiring and strain relief for any aircraft-installed prototype.**
6. **Do not treat bench success as evidence of airborne reliability.**
7. **Use independent attitude references during testing.**
8. **Retain a clearly experimental/non-primary status until proven otherwise.**

Power

The prototype is powered from a regulated 5 V USB-C source. The project currently excludes direct connection of raw aircraft 12 V to the instrument.

Any future direct-aircraft power design would need its own electrical-protection review, including transient, reverse-polarity, noise and overcurrent considerations.

Enclosure material

PLA is not preferred for a sun-heated cockpit because of its relatively low heat resistance and creep risk. A more suitable engineering filament such as ASA or ABS should be evaluated, along with inserts/fasteners and real thermal testing.

Flight testing

Airborne evaluation should only occur after bench, motion, vibration, thermal and aircraft-ground testing are satisfactory. During flight testing, the unit should remain a secondary experimental display and must be compared with an independent trusted reference.

Change control

Hardware, firmware, calibration, filter tuning and enclosure changes can all affect attitude performance. Important changes should therefore be committed to source control and documented before flight evaluation.

docs/SENSORS.md

Repository source: docs/SENSORS.md

Sensor architecture

This project is a supplementary/non-primary flight instrument. Sensor validity, freshness and installation effects must be explicit; missing data must never be replaced by plausible stale data.

Attitude sensor — BMI088

Bosch Sensortec BMI088 is the reference six-axis inertial source for pitch and roll. It combines independent 16-bit triaxial accelerometer and gyroscope sections and is specifically intended for vibration-heavy applications including drones and robotics.

The ESP32-S3 uses SPI with separate accelerometer and gyroscope chip selects. Firmware verifies both chip IDs before accepting the device, explicitly configures ± 6 g acceleration and ± 500 °/s gyro ranges, observes the accelerometer SPI dummy-byte behaviour, and waits for accelerometer startup before samples are accepted.

`bmi088_read_sample()` now supplies engineering-unit acceleration in g and angular rate in degrees/second plus a monotonic sample timestamp. `attitude_estimator` is a separate body-frame estimator: gyro integration supplies short-term motion while gravity-derived pitch/roll provides low-frequency correction through a complementary filter. Accelerometer correction is suppressed when total acceleration is outside 0.70–1.30 g so manoeuvre acceleration is not blindly interpreted as gravity.

The estimator is fail-obvious. It rejects non-finite samples, refuses initialization without a plausible gravity vector, invalidates an excessive sample interval, rejects impossible/non-finite attitude output and provides a 250 ms stale-data invalidation check. A failed or stale attitude must set `attitude_valid=false`; the renderer must never continue to show a frozen plausible attitude.

Installation-axis gate

The estimator requires aircraft body axes **+X forward, +Y right, +Z down**. The Bosch shuttle-board axes have not yet been physically related to the final EFIS enclosure/aircraft installation. Therefore raw BMI088 samples are **not yet connected to the live instrument_data_t attitude fields**. Before that connection is enabled, the actual mounted shuttle board must be bench-tested in known nose-up, nose-down, left-bank and right-bank orientations and an explicit sensor-to-body axis/sign mapping recorded in firmware and this document. This deliberately prevents an unverified axis assumption from becoming a plausible but reversed flight indication.

The first hardware attitude acceptance sequence is: verify chip IDs and raw stationary values; establish axis/sign mapping; confirm approximately 1 g magnitude when stationary; characterize stationary gyro bias; exercise known $\pm 10^\circ/\pm 20^\circ$ pitch and $\pm 30^\circ/\pm 60^\circ$ roll positions; then enable the estimator-to-display path and verify stale/disconnect failure. Calibration and vibration testing follow before any aircraft evaluation.

BMI088 commissioning diagnostic

A dedicated build option, `CONFIG_EFIS_BMI088_DIAGNOSTICS`, is available for bench axis commissioning. It is **OFF by default**, unavailable in the QEMU build, and does not connect BMI088 measurements to the instrument display. When enabled on hardware it logs approximately five samples per second in this form:

```
BMI088 DIAG t=12345ms A[g] X=+0.001 Y=-0.006 Z=+1.002 |A|=1.002 G[dps] X=+0.08 Y=-0.03 Z=+0.11
```

Use the diagnostic with the Shuttle Board stationary and then rotate it deliberately through known orientations. Record which sensor acceleration axis carries approximately ± 1 g for each orientation and which gyro axis/sign responds to each positive rotation. The required final mapping is aircraft **+X forward, +Y right, +Z down**. Do not enable the estimator-to-display path until that mapping has been physically demonstrated rather than inferred from connector or PCB orientation.

For a stationary board, `|A|` should be close to 1 g and gyro rates should remain near zero apart from bias/noise. Large stationary departures, unstable readings, SPI errors or implausible gyro output are commissioning failures to resolve before attitude work continues.

To enable the diagnostic temporarily:

```
cd ~/Documents/Xcode/ESP32-EFIS/firmware
idf.py menuconfig
```

Select **ESP32 EFIS development options** → **Enable BMI088 axis commissioning diagnostics**, save, then build/flash/monitor. Disable the option again after commissioning. The startup log explicitly states when diagnostics are enabled and that the diagnostic data are not feeding the display.

Pressure sensor — frozen choice

Bosch Sensortec BMP585 is the reference static-pressure sensor for the Altimeter/PFD.

Why BMP585 rather than BMP581/BMP390/DPS310:

- 300–1250 hPa full-accuracy range
- ± 6 Pa typical relative accuracy and 0.08 Pa RMS pressure noise
- ± 0.5 Pa/K typical temperature coefficient
- gel-filled cavity / enhanced resistance to water, dust and harsh environments
- I2C/I3C/SPI and 3.3 V-compatible supplies
- active component with strong UK stock
- same modern BMP58x family while giving this aircraft installation useful environmental robustness

Its maximum absolute accuracy is specified at ± 50 Pa in Bosch's current data, so QNH/reference-pressure setting remains essential as with any barometric altimeter.

Prototype module

Use **Adafruit BMP585 Ported I2C/SPI breakout, PID 6413** for bench and first flight-development plumbing. It provides a practical ported module and is available through DigiKey. This is preferable to inventing a pressure chamber around an unported bench breakout.

For a later custom carrier, decide after pressure tests whether to retain a separately mounted ported BMP585 module or integrate the bare BMP585 with a purpose-designed sealed plenum. Do not freeze the bare-sensor pneumatic geometry until the ported prototype has been characterized.

Static plumbing

The enclosure provides a reinforced STATIC service boss with a 3 mm pilot hole. Enlarge it only for the final bulkhead/tube fitting after the Skyranger static tubing has been measured. Support tubing independently, leak-test it, avoid kinks/excessive volume, and prevent cabin pressure leaking into the static line.

Firmware applies pilot-selected QNH to measured static pressure. Startup, range, rate-of-change, temperature and stale-data checks must pass before `altitude_valid` becomes true.

Magnetometer — frozen choice

PNI RM3100-CB (P/N 14754) is the reference absolute magnetic-field source.

Reasons: three-axis magneto-inductive architecture; ± 1100 μ T range; 13 nT sensitivity / ~ 15 nT noise at the documented high-resolution setting; I2C/SPI; -40 to $+85$ °C; compact $14.22 \times 15.75 \times 7.14$ mm board; documented rugged/vibration performance; UK supply through Solsta.

The RM3100-CB is remote-mounted away from the ESP32, DC/DC converter, backlight current, steel fasteners and other panel magnetic disturbances. It needs a rigid non-magnetic mount with known FWD/UP/lateral axes and a locking strain-relieved cable through the rear MAG service boss.

Heading fusion

The compass does not use raw magnetometer azimuth alone. Heading is produced from calibrated 3-axis magnetic field plus BMI088 attitude, including tilt compensation, hard-iron offset, soft-iron matrix and installation alignment. Magnetic/true reference and variation handling must be explicit. GNSS course over ground is **TRK**, not HDG.

Bench development

Until sensors are physically connected and their installation mappings are verified, bench simulation supplies clearly synthetic values. Simulation must be disabled for aircraft-use firmware.

docs/SERVER_PLATFORM.md

Repository source: docs/SERVER_PLATFORM.md

Server and Container Platform

Status: proposed infrastructure baseline — 29 September 2026

This document records the current hardware and virtualisation direction for the server-side services supporting **MicroSky Horizon / ESP32-EFIS**. It is deliberately separate from the aircraft EFIS hardware baseline. Nothing in this document is flight-critical hardware.

Purpose

The platform is intended to host the project's growing Docker workloads, including:

- `efis-ota` and OTA administration
- Horizon customer support/chat services
- account/device/licensing services
- PostgreSQL data stores
- monitoring and administration services
- development, integration and regression-test workloads
- future scalable services where Kubernetes adds a demonstrable benefit

The current Synology-hosted services can remain in use during migration and can continue to provide bulk/backup storage.

Budget and hardware target

The current budget ceiling is **£750** for the server platform.

Target specification:

Component Target
--- ---
CPU modern AMD Ryzen 7/9 class, approximately 8 cores / 16 threads
RAM 64 GB DDR5 preferred; 32 GB is the absolute minimum
Primary storage 1 TB NVMe SSD minimum
Expansion at least one additional M.2/NVMe slot preferred
Networking dual 2.5 GbE preferred
Virtualisation AMD-V/VT-x capable; Proxmox VE no-subscription baseline (£0)
Power profile suitable for quiet 24x7 operation
UPS desirable if it can be accommodated within/alongside budget

Candidate mini-PC platforms discussed include the **GMKtec K8 Plus** (Ryzen 7 8845HS class) and **Minisforum UM890 Pro** (Ryzen 9 8945HS class). The preferred purchasing pattern is a barebones or sensibly configured unit plus known-quality 64 GB RAM and 1 TB NVMe, provided the complete system remains within the £750 ceiling.

These are candidate platforms, not frozen procurement choices. Price, warranty, UK availability, RAM/SSD compatibility and NIC support must be checked at purchase time.

Why a modern mini-PC rather than an old rack server

For this workload the priorities are:

- strong multi-core performance
- 64 GB RAM capability
- fast NVMe storage
- low idle power for 24x7 operation
- low noise
- small physical footprint
- multiple Ethernet interfaces
- enough expansion for a second SSD

Used enterprise rack hardware remains an option where ECC, IPMI, many disks or enterprise RAID are required, but those features do not currently justify the higher power/noise/space cost for this project.

Virtualisation baseline: Proxmox VE (no-subscription)

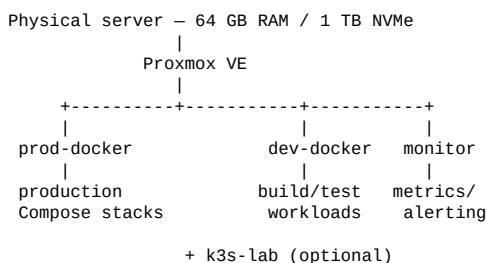
The proposed host layer is **Proxmox VE using the no-subscription repository**, with a software subscription cost of **£0** for the initial self-managed platform.

A paid Proxmox subscription is not required to run the hypervisor. The no-subscription repository is suitable for this development/self-hosted baseline, while a supported Enterprise subscription can be reconsidered later if MicroSky Horizon becomes a commercial production service requiring vendor support and the more heavily tested enterprise package stream.

The principal fully open-source alternative is **Incus**, which supports both system containers and QEMU virtual machines and can scale to clustered hosts. Incus should remain documented as the fallback/alternative rather than changing the initial architecture, because Proxmox provides the more convenient integrated VM, storage, snapshot, backup and web-management environment for this deployment.

Proxmox provides VM isolation, snapshots, backup/restore, resource allocation, networking and a web management interface. Application containers should not normally run directly on the Proxmox host; they run inside dedicated Linux VMs.

Initial architecture:



Initial VM layout

1. prod-docker

Purpose: stable production Docker/Compose workloads.

Initial allocation:

- 6–8 vCPU
- approximately 24 GB RAM
- approximately 300 GB virtual disk
- Debian or Ubuntu Server LTS
- Docker Engine + Docker Compose

Candidate workloads include OTA, support service, account/licensing APIs, web services and project databases.

Existing Compose deployments should be migrated with minimal architectural change first. Kubernetes is not a prerequisite for migration.

2. dev-docker

Purpose: builds, integration tests, regression tests, simulators/test harness services and pre-production container validation.

Initial allocation:

- 4–6 vCPU
- approximately 12 GB RAM
- approximately 150 GB virtual disk

Expected flow:

```
source -> build -> container build -> unit tests -> integration tests
      -> simulation/regression tests -> PASS -> production deployment
```

This prevents experimental builds from destabilising production services.

3. monitor

Purpose: independent infrastructure monitoring.

Initial allocation:

- 2 vCPU
- approximately 4 GB RAM
- 50–100 GB virtual disk

Likely stack:

- Prometheus
- Grafana
- Alertmanager
- host/container exporters

Monitoring should remain outside prod-docker so a complete production-VM failure can still be observed and alerted.

4. k3s-lab

Purpose: optional Kubernetes learning and migration environment.

Initial allocation:

- approximately 4 vCPU
- approximately 12 GB RAM
- 100–150 GB virtual disk

k3s is a lightweight Kubernetes distribution. It should initially be treated as an experimental environment, not the default production platform.

A small non-critical service should be migrated first to exercise Deployments, Services, Ingress, persistent volumes, Secrets and ConfigMaps. Kubernetes adoption should then be based on demonstrated operational benefit rather than migration for its own sake.

If k3s does not provide sufficient value, this VM can simply be removed and its resources returned to the host.

RAM policy

Do not allocate all 64 GB permanently. A representative starting point is:

```
| Use | RAM |
|---|---|
| Proxmox/host reserve | ~4 GB |
| prod-docker | 24 GB |
| dev-docker | 12 GB |
| monitor | 4 GB |
| k3s-lab | 12 GB |
| remaining operational headroom | ~8 GB |
```

Allocations are starting values, not hard capacity requirements. Actual utilisation should drive later tuning.

Database placement

Initially, PostgreSQL/PostGIS may remain within the production Docker environment to minimise migration complexity.

A later dedicated database VM is reasonable if operational needs justify it:

```
Proxmox
+-- prod-docker
+-- database
|   +-- PostgreSQL
|   +-- PostGIS (where required)
+-- dev-docker
+-- monitor
+-- k3s-lab
```

Database separation should be driven by backup, performance, security and failure-domain requirements rather than done automatically.

Storage

Start with a good-quality **1 TB NVMe** drive. Prefer a platform with a second M.2 slot.

A future second 2–4 TB NVMe drive can provide additional database/container storage, VM storage or local backup staging. The Synology can continue to provide independent bulk storage and backups.

Important data must not rely solely on VM snapshots. Establish tested off-host backups.

Networking

Dual 2.5 GbE is preferred because it permits later separation of management and service/storage traffic, bonding or other topology changes without replacing the server.

Do not expose Proxmox management directly to the public Internet. Public EFIS services should continue to use controlled reverse-proxy/firewall/TLS boundaries.

Docker first, Kubernetes deliberately

The migration principle is:

1. install Proxmox;
2. create the production Docker VM;
3. migrate existing Compose stacks with minimal changes;
4. create independent monitoring;

5. establish backup/restore and deployment testing;
6. create the development/test VM;
7. introduce k3s only as a controlled lab;
8. migrate services to Kubernetes only where resilience, scaling or operational management clearly improves.

Docker Compose remains entirely valid for a single production host. k3s becomes substantially more useful if the platform later expands to multiple physical nodes.

Future resilience

A single Proxmox server improves isolation and manageability but is still a **single physical failure domain**.

True host-level high availability requires additional physical nodes and suitable replicated/shared storage and network design. A future three-node k3s or Proxmox cluster can therefore be considered separately from this first £750 server.

Procurement checkpoint

Before purchase:

- verify current UK price and warranty;
- confirm two SODIMM slots and 64 GB support;
- confirm included/barebones RAM arrangement;
- confirm 1 TB NVMe compatibility and available second M.2 slot;
- confirm both Ethernet controllers are supported by the chosen Proxmox release;
- confirm BIOS virtualisation options;
- confirm cooling/noise characteristics for sustained 24x7 use;
- reserve budget for backup and preferably UPS protection.

The immediate objective is not a Kubernetes cluster. It is a reliable, expandable virtualisation host that can run the existing Docker estate cleanly today and provide a low-risk path to k3s/multi-node infrastructure later.

Hypervisor cost and alternative

The initial hypervisor/software budget is **£0**:

- **Proxmox VE no-subscription**: selected baseline; no paid subscription required for the self-managed installation.
- **Incus**: principal open-source alternative if a future decision favours a more Linux-native container/VM management model.
- **Debian + KVM/libvirt**: viable lower-level alternative, but would require more manual assembly and administration.

The £750 platform budget should therefore be spent on hardware, RAM, NVMe storage and power protection rather than a hypervisor licence.

Native Proxmox LXC containers

Proxmox VE supports **LXC containers directly on the hypervisor** alongside full KVM virtual machines. Lightweight infrastructure services should use LXC where they do not require a separate kernel or stronger VM security/failure boundary.

The default should be **unprivileged LXC containers**.

Initial proposed LXC services:

Container	OS/userspace	CPU	RAM	Disk	Purpose
---	---	---	---	---	---

| dns01 | Debian | 1 core | 512 MB–1 GB | 8 GB | Internal DNS, e.g. AdGuard Home |
| utility01 | Debian | 1 core | ~1 GB | 8–16 GB | Optional lightweight network/admin tools |

A representative host layout is therefore:

```
Proxmox VE
|
+-- LXC dns01
|   +-- Debian userspace
|   +-- AdGuard Home
|
+-- LXC utility01          [optional]
|
+-- VM prod-docker
|   +-- Debian 13
|   +-- Docker/Compose
|
+-- VM dev-docker
|   +-- Debian 13
|   +-- Docker/Compose
|
+-- VM monitor
|   +-- Debian 13
|
+-- VM k3s-lab
|   +-- Debian 13
|   +-- k3s
```

LXC guests share the Proxmox Linux kernel, so they have substantially less overhead than full VMs and start quickly. Full VMs remain preferable for the production Docker estate, development/test Docker environment, k3s and workloads requiring a stronger isolation boundary.

Docker policy

Do **not** make Docker-inside-LXC the default architecture. Although nested Docker in LXC is possible, it adds nesting, permission and kernel-feature dependencies that are unnecessary on a 64 GB host. Docker environments should remain in full Debian VMs unless a specific measured benefit justifies changing this policy.

DNS container

The initial internal DNS service can run directly as dns01 under Proxmox rather than consuming a complete VM.

Suggested starting configuration:

```
Hostname:      dns01
Type:          unprivileged LXC
Userspace:     Debian
CPU:           1 core
RAM:           512 MB initially
Swap:          512 MB
Disk:          8 GB
Network:       vmbr0
Address:       static LAN address
Start at boot: enabled
```

The local DHCP service/router should advertise dns01 as a resolver only after the DNS configuration has been tested.

Use the reserved .home.arpa namespace for private home-network names rather than inventing a public-looking internal domain.

Example:

```
proxmox.home.arpa
prod.home.arpa
dev.home.arpa
monitor.home.arpa
nas.home.arpa
```

DNS resilience

A single dns01 container on Proxmox would disappear whenever the physical host is rebooted or unavailable. The preferred later design is therefore two DNS instances in separate physical failure domains:

```
dns01 -> new Proxmox server
dns02 -> Synology or another independent device
```

Clients can then be given both resolver addresses through DHCP. DNS availability should not depend solely on the new Proxmox host.

Public authoritative DNS for MicroSky/Horizon customer-facing services should remain with a hosted DNS provider; the internal LXC DNS service is not intended to become a public authoritative DNS server.

docs/SIMULATION.md

Repository source: docs/SIMULATION.md

Bench and QEMU simulation

Simulation is development-only and must never be used as a fallback for failed flight data. CONFIG_EFIS_BENCH_SIMULATION defaults OFF. Every synthetic presentation carries a permanent red **SIM** annunciation. Aircraft builds must keep simulation and QEMU disabled; missing or stale real data must fail visibly.

Instrument acceptance status

The **Artificial Horizon / PFD, Altimeter and Compass graphics passes are accepted and parked**. The complete 480×480 instrument graphics-quality stage is accepted. Physical hardware work is currently paused because prototype hardware is unavailable; software/QEMU/simulator work may continue without claiming physical validation.

Graphics work must preserve the accepted indication direction, mathematics and failure behaviour.

Artificial Horizon acceptance record

The accepted attitude suite covers level, $\pm 10^\circ$ and $\pm 20^\circ$ pitch, $\pm 30^\circ$ and $\pm 60^\circ$ bank, combined $+10^\circ$ pitch/ $+30^\circ$ right bank, attitude failure and explicit recovery. Positive pitch moves the horizon down, negative pitch moves it up, and the attitude sphere moves opposite aircraft bank. ATT FAIL prevents a failed attitude from remaining plausibly usable.

The accepted graphics keep PITCH_PIXELS_PER_DEG at **6.8 px/degree**, use a regularised horizon line, differentiated $5^\circ/10^\circ$ pitch marks with centre gaps, and a fixed yellow aircraft symbol with black contrast outline. The centre datum remains at exact display centre.

The accepted fixed bank-angle scale marks **10° , 20° , 30° , 45° and 60° on both sides** plus zero. The 30° and 60° marks are stronger. A white triangular roll pointer with black contrast outline moves with measured roll: left bank moves left/counter-clockwise and right bank moves right/clockwise.

Altimeter acceptance record

The accepted presentation uses the circular EFIS bezel, 0–9 dial numerals, differentiated hundreds/thousands/ten-thousands hands, digital altitude readout, compact mechanical-style Kollsman pressure scale at 3 o'clock, the 9 o'clock $>10,000$ -ft hatching sector and fail-obvious ALT FAIL state.

The Kollsman display is a curved circumferential sector centred at **3 o'clock**, spanning approximately **8 hPa total** at 2° per hPa. Selected QNH aligns with a fixed index while pressure graduations move. The exact **1013.25 hPa** datum is a heavier radial tick. Selected QNH remains 950–1050 hPa in integer 1 hPa increments.

The $>10,000$ -ft hatching is a 60° annular sector centred at 9 o'clock. It is absent at/below 10,000 ft, progressively appears from 10,000 to 11,000 ft and remains fully visible thereafter.

The accepted functional QNH test held simulated static pressure at 927.0 hPa while changing QNH through 1013, 1003, 1023, 950 and 1050 hPa. Lower QNH produced lower indicated altitude, higher QNH higher indicated altitude, and invalid pressure produced ALT FAIL.

The formal Altimeter graphics suite used fourteen states: 0, 500, 1,000, 2,500, 5,000, 9,500, 9,900, 10,000, 10,100, 10,500 and 12,500 ft, moving sweep, ALT FAIL, and recovery.

Compass acceptance record

The accepted Compass suite used 000° , 045° , 090° , 135° , 180° , 225° , 270° , 315° , moving $350^\circ \rightarrow 010^\circ$ north crossing, continuous rotation, HEADING FAIL, and recovery to 000° .

The renderer carries large N/E/S/W labels and a three-digit heading. Increasing aircraft heading rotates the card oppositely beneath the fixed reference. The outlined yellow selected-heading bug is calculated from `heading_bug_deg - heading_deg`; the

acceptance suite held it at 060°.

HEADING FAIL must not be replaced by a frozen plausible heading.

Full QEMU demonstration mode

The normal QEMU application now runs a **28-step continuous demonstration** intended both for regression viewing and video capture. Each step lasts three seconds and uses the production instrument renderer plus the existing synthetic scenario engine.

The demonstration sequence is:

- Horizon: level, pitch +10, pitch -10, bank left 30, bank right 30, combined pitch/bank, ATTITUDE FAIL, recovery
- Altimeter: 0, 2,500, 9,500, 10,000, 10,100, 10,500 and 12,500 ft, altitude sweep, ALT FAIL, recovery
- Compass: north, north-east, east, south, west, north-west, 350→010 wrap, continuous rotation, HEADING FAIL, recovery north

The selected heading bug is fixed at 060° during the QEMU demonstration. The on-screen test overlay identifies the current scenario and step. The complete loop is approximately **84 seconds** and then repeats.

This concise demonstration does not replace the formal acceptance records above; it samples the accepted behaviour for presentation and regression purposes.

Swift simulator synchronisation

The native Swift iPhone/iPad simulator mirrors the accepted Horizon, Altimeter and Compass presentation. It also provides automated selectors for the complete 12-state Horizon, 14-state Altimeter and 12-state Compass acceptance suites. QEMU remains authoritative for the actual ESP32 RGB565 renderer.

The Swift simulator is a regression companion, not an independent instrument design. Any ESP32 renderer presentation change must trigger review of the corresponding Swift view and acceptance scenarios in the same development change. All simulator overlays must remain clipped to the 480×480 circular display and must not obscure the fixed aircraft symbol, pitch ladder, bank scale or each other.

Approved Swift PFD presentation

The Swift Horizon/PFD reference presentation now includes numeric pitch labels on the 10-degree ladder, numeric roll labels at 0, ±10, ±20, ±30, ±45 and ±60 degrees, a fixed centred yellow aircraft symbol, altitude without a unit suffix, and a separate bare barometric-pressure value in hPa (the display intentionally omits the text labels “QNH” and “hPa”). The GNSS overlay is a single latitude/longitude line: no qualitative word or separate GPS-accuracy line is shown. Its text colour alone indicates horizontal accuracy: green for good/excellent, yellow for fair/weak, and red for poor, stale or invalid. The simulator control panel may continue to show units and detailed synthetic test-state names for bench operation.

Build, run and record

From the repository root:

```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
source scripts/efis-env.sh
zsh scripts/build-qemu.sh --clean
zsh scripts/run-qemu.sh
```

To make a macOS video recording of the full QEMU demonstration:

```
zsh scripts/record-qemu-demo.sh
```

The recording helper starts QEMU, asks the user to click the QEMU graphics window for macOS window capture, stops automatically and writes the .mov under docs/media/. See DEMONSTRATION.md.

QEMU uses the real 480×480 RGB565 geometry but bypasses physical LCD, MCP23008, encoder, backlight and sensors. sdkconfig.qemu.defaults disables physical PSRAM emulation. Never flash build-qemu to hardware.

Display resolution and graphics-quality stage

The selected Newhaven NHD-2.1-480480AF-ASXP has a native resolution of **480×480 pixels**, so QEMU uses exactly 480×480 RGB565. Enlarging QEMU on a desktop makes primitive graphics look coarser than at the physical panel size.

The **480×480 graphics-quality stage is complete and accepted for the Artificial Horizon, Altimeter and Compass**. Sensor and physical integration remain separately gated and must not inherit an acceptance status merely from emulator success.

docs/STAGE-7-EIU.md

Repository source: docs/STAGE-7-EIU.md

Provisional Stage 7 — Engine Sensor / EIU Integration

Status: PROVISIONAL — does not block Stages 1–6

Role: Optional supplementary engine-monitoring expansion for MicroSky Horizon

Purpose

Stage 7 adds a remote Engine Interface Unit (EIU) and AEF-CAN engine-data input to Horizon. The current EFIS is to be designed so this stage can be added later without redesigning its processor architecture or application data model.

The EIU is monitoring-only. It is not required for engine operation and must not interfere with an existing engine-control or required indication circuit.

Stage boundary

Stages 1–6 continue to deliver the core Horizon attitude/altitude/navigation instrument. Stage 7 is deliberately provisional until the actual Rotax 912 variant, installed sender types, connector requirements and aircraft installation have been surveyed.

The Horizon side of Stage 7 is brought forward: firmware architecture and the future carrier PCB reserve a Classical CAN receive/transmit interface. The analogue engine-sensor electronics remain in the remote EIU.

Horizon readiness requirement

Horizon shall provide:

- ESP32-S3 TWAI controller support at 500 kbit/s Classical CAN;
- external 3.3 V CAN transceiver on the final carrier;
- protected CAN-H/CAN-L interface;
- CAN reference/ground as required by the final isolation design;
- keyed external connector with CAN-H, CAN-L and reference/shield provisions;
- switchable/optional 120-ohm termination so Horizon may be a physical end of the bus;
- AEF-CAN V1 codec/dispatcher;
- freshness tracking and explicit stale/missing EIU state;
- ability to operate normally with no EIU present.

No engine-data absence may invalidate the core EFIS attitude/altitude/navigation functions.

Provisional EIU channels

Initial target:

- RPM;
- oil pressure;
- oil temperature;
- CHT/head/coolant temperature channels appropriate to installed Rotax 912;
- EGT 1–4 provision;
- EIU supply voltage/current;
- sensor/open/short/out-of-range diagnostics;

- EIU health/heartbeat.

Exact analogue front ends remain conditional on the installed sender types.

Gates

Stage 7A — Horizon CAN readiness: codec, receiver abstraction, stale handling, simulated EIU.

Stage 7B — CAN bench: two transceivers, correct termination, fault/loss tests.

Stage 7C — EIU bench prototype: sensor front ends and simulated/bench senders.

Stage 7D — engine/aircraft survey: freeze actual sender interfaces and harness.

Stage 7E — installed supplementary evaluation.

No Stage 7 item is treated as flight-validated merely because it passes simulation or bench CAN tests.

Sensor commissioning and automatic fault detection

V1 uses explicit commissioning rather than attempting semantic auto-identification of conventional analogue engine sensors. The user/installer assigns each physical channel a semantic role and verified sender/profile (for example EGT1, CHT1 or oil temperature), and that mapping is persisted.

The EIU automatically discovers its own identity/capabilities and monitors each configured channel for presence, electrical compatibility, open/short/out-of-range/plausibility and stale/fault conditions. It must not infer or silently change semantic roles merely because an electrical reading resembles a sensor class.

Subsequent starts are automatic when the stored EIU/channel configuration remains valid. New, missing or electrically incompatible sensors generate an explicit commissioning/change/fault state rather than a silent remap.

Sensor power / excitation

The EIU owns sender excitation and analogue conditioning. Horizon/EFIS shall not directly power individual engine sensors.

Passive/self-generating channels (notably K-type thermocouples) receive no supply from the EIU. Resistive temperature channels use controlled low-level measurement excitation internal to the EIU rather than a generic sensor-power rail. Powered electronic senders receive only the supply required by their verified datasheet/installed configuration, generated and protected locally in the EIU.

Oil-pressure and RPM interfaces are explicitly **sender/source dependent**. Earlier concept illustrations showing generic +5 V for these channels are illustrative only and must not be treated as a frozen pinout or electrical requirement.

The preferred installation gives the EIU a protected aircraft-supply feed independent of the EFIS regulated rails. EIU power conversion, sensor excitation and faults are locally contained so an engine-monitoring failure cannot disable core Horizon functions. Final aircraft-power branching, grounding/reference, isolation and whether supply conductors share the AEF-CAN harness are deferred to the installation/electrical design gate.

EIU firmware update architecture — 1 October 2026

The preferred production update path is **firmware service -> Horizon/EFIS -> AEF-CAN -> EIU**. The EIU does not require Wi-Fi, Internet access, cloud credentials or a separate user-facing updater.

Horizon discovers the EIU hardware identity, board revision, bootloader version, running firmware version and AEF-CAN compatibility. It downloads only a compatible signed EIU image, validates package metadata/signature according to the project OTA trust model, then transfers the image over an AEF-CAN firmware-update service.

The EIU must independently authenticate the candidate before execution. Trust in Horizon or a successful CAN transfer alone is not sufficient authorization to boot firmware.

EIU flash model

The EIU shall reserve:

- protected/recoverable bootloader;
- application slot A;
- application slot B;
- separately versioned persistent configuration/calibration storage.

An update is written only to the inactive application slot. Sensor assignments, calibration, EIU identity and commissioning data are not application-image payload and must survive firmware replacement. Configuration-format migration must be transactional/recoverable.

After transfer the EIU verifies the complete candidate, marks it pending, reboots into it and performs startup self-test. The candidate becomes confirmed only after successful initialization and AEF-CAN communication/health confirmation. Failed boot, watchdog reset, failed self-test or missing confirmation causes automatic rollback to the previous known-good application.

Operational gate

EIU update activation is a deliberate maintenance action. It shall not begin while the engine is running. Horizon must clearly warn that engine indications are unavailable during transfer/reboot and require explicit user activation. The final gate may include RPM=0 plus other stationary/maintenance-state evidence; exact policy remains to be frozen and tested.

Physical recovery

The EIU PCB retains internal service/recovery pads or an equivalent programming/debug interface for bootloader recovery and manufacturing. Normal field updating does not require opening the EIU.

Test requirement

The OTA harness shall gain a simulated AEF-CAN EIU node so the complete workflow can be exercised without aircraft hardware: discovery/version report, compatibility decision, image transfer with acknowledgements/retry, interruption/resume or clean restart policy, complete-image verification, pending activation, reboot, health confirmation, rollback and configuration preservation. Simulation is not physical CAN/flash validation.

USB-C factory/service programming

The EIU external **USB-C SERVICE** port is the primary local programming/service interface. On an ESP32-S3 design it should use native USB where practical.

Factory/development flow is:

```
service computer -> USB-C SERVICE -> ROM/bootloader programming -> factory image -> reboot -> identity/self-test.
```

The factory image establishes the bootloader, partition table, initial application slot and persistent-storage layout required by the A/B update architecture. Manufacturing identity (serial number, hardware revision and PCB revision) is provisioned separately from aircraft sensor commissioning/calibration data.

USB VBUS may power the service-side EIU electronics on the bench, but the hardware must prevent back-feed between USB 5 V and the aircraft-power input. USB-powered service mode must not imply that engine sensor excitation or all aircraft-side interfaces are safe/available unless the final power design explicitly supports them.

USB-C does **not** replace normal field OTA. Production user updates remain Horizon -> AEF-CAN -> EIU. USB is for factory provisioning, development, workshop diagnostics and recovery.

Internal fallback programming/test pads remain mandatory for recovery from a USB/bootloader/configuration failure. Development units should remain recoverable; irreversible security/eFuse changes belong to a separately controlled production-provisioning operation.

Interrupted transfer / checkpoint requirement

EIU firmware transfer must survive removal of aircraft power or loss of CAN without threatening the running firmware. The inactive slot is written in durable blocks; the transfer-block interval is **4 KiB**; durable resume state is nominally **64 KiB**. A checkpoint binds transfer ID and image digest to an absolute committed byte offset plus block integrity information.

After restart Horizon queries update state. If transfer ID/image identity match, transfer resumes from the last committed offset. If they do not match, the partial candidate is not silently reused. Percentage complete is UI-only and is derived from committed offset/image size.

The active known-good slot remains bootable throughout download. Only a complete image that passes whole-image digest/signature verification can become pending for activation.

Adopted production CAN firmware transport — 1 October 2026

The Stage-7 production direction supersedes the prototype six-byte/per-frame-ACK transport. FW_DATA uses one local sequence byte and seven firmware bytes. Horizon sends a 4 KiB transfer block, the EIU validates that block with CRC32, then returns a block ACK/NACK. A bad or incomplete block is retransmitted.

A 4 KiB block is **not** a durable restart checkpoint. Durable resume state is recorded nominally every 64 KiB to reduce unnecessary metadata writes and flash wear. Following power loss, the EIU resumes from the last durable 64 KiB absolute offset; retransmission of data after that point is acceptable because the active known-good A/B slot is untouched.

Firmware-maintenance CAN traffic is deliberately lower priority than operational measurement/health traffic and must be rate-limited/yield when necessary. Complete-image SHA-256 and signature verification, explicit activation, trial boot and rollback remain unchanged.

Flash-endurance architecture — 1 October 2026

Flash endurance is now an explicit EIU design constraint. Firmware-image programming is not expected to be the dominant wear mechanism because production OTA uses alternating A/B application slots. The implementation shall instead prevent small, frequently updated metadata or operational state from concentrating erase cycles on one flash sector.

Durable OTA checkpoint journal

The nominal 64 KiB durable-resume interval is retained, but checkpoint metadata shall **not** repeatedly erase/rewrite one fixed sector. Durable checkpoints shall use an append-only rotating journal spanning multiple erase sectors, or an equivalent platform wear-levelled implementation with demonstrably comparable behaviour.

Each durable checkpoint record shall contain enough information to reject stale or mismatched state, including at minimum:

- monotonically increasing generation/record sequence;
- transfer/image identity and whole-image digest identity;
- absolute committed byte offset;
- transfer/block integrity state needed for safe resume;
- record integrity check (CRC or equivalent).

A record becomes valid atomically: an interrupted metadata write must leave either the previous checkpoint usable or the new checkpoint independently recognizable as complete. Journal reclamation/erase occurs only when required after rotating through the allocated checkpoint area. On boot, the newest valid compatible record is selected; corrupt, incomplete or wrong-image records are ignored.

The 4 KiB CAN transfer-block ACK/CRC state remains primarily volatile. A successful 4 KiB block does not require a persistent metadata write. Durable flash state is committed nominally every 64 KiB and at carefully selected lifecycle boundaries where required for correctness.

RAM-first operational state

High-frequency EIU state shall remain in RAM. Normal sensor samples, CAN frames, counters, transient diagnostics, heartbeat state and continuously changing engine values shall **not** cause flash writes.

Persistent writes are limited to deliberately durable information such as:

- commissioned sensor/channel configuration;
- calibration and manufacturing identity;
- infrequent user configuration changes;
- OTA checkpoint/activation/rollback metadata;
- selected significant fault/event records where persistence is explicitly justified.

Where frequently changing persistent state is genuinely required, it must use ESP-IDF NVS/wear levelling or a purpose-designed append-only journal rather than a fixed-sector rewrite loop.

Prohibited implementation pattern

No implementation may perform a flash/NVS commit per CAN frame, per sensor sample, per heartbeat, or on every increment of an operational counter. Rate limiting alone is not sufficient justification for such a design; persistence must be event-driven and wear-aware.

Validation requirement

The EIU test plan shall include flash-wear accounting. Tests/review must demonstrate checkpoint rotation, recovery after power loss during checkpoint creation, rejection of corrupt/stale records, journal wrap/reclamation, preservation of A/B rollback, and absence of high-frequency operational flash commits.

Adding FRAM, EEPROM or removable storage solely to address expected OTA flash wear is **not required** for V1. Such hardware should be introduced only if later requirements create a justified high-write persistent-data workload.

Architecture review decisions — 1 October 2026

The following decisions are now adopted constraints for Stage 7 rather than optional review notes.

Power and fault containment

Horizon/EFIS and the EIU shall use **independent protected branches from the aircraft electrical system**. The EIU shall not be powered from the EFIS regulated rails. Whether aircraft power conductors and AEF-CAN share a physical harness remains an installation decision; electrically, the branches remain separately protected and a fault in one unit must not remove power from the other.

The baseline CAN implementation is **protected non-isolated Classical CAN with an explicit reference/ground strategy**. Galvanic isolation is not a default requirement because it adds isolated power, components, PCB area and failure modes. Isolation shall be added only if aircraft grounding/common-mode/noise testing or a later installation requirement demonstrates a need.

A formal fault-containment requirement applies: EIU failure must not disable core Horizon; a sensor short must be locally contained; CAN faults must not reboot either node; USB/service faults must not propagate into aircraft power/CAN; bad firmware/configuration must remain recoverable; update failures must leave a known-good application bootable.

EIU processor design gate

ESP32-S3 is **not frozen as the EIU MCU**. It remains a strong candidate because shared ESP-IDF, native USB, OTA/security tooling and project knowledge reduce development cost. Before EIU PCB freeze, compare it against suitable deterministic/industrial MCU alternatives (for example an STM32-class device) for CAN/timers/ADC support, boot/rollback facilities, development complexity, component count and long-term maintainability. Select on system merit, not processor-family inertia.

Sensor commissioning simplification

V1 shall not attempt semantic auto-identification of conventional analogue engine sensors. Physical channels are commissioned/configured with explicit roles and sender profiles (for example EGT1, oil temperature). Firmware then performs automatic **electrical compatibility, presence and fault detection** against that configuration. Device discovery and EIU capability discovery remain automatic. No channel is silently remapped from electrical inference.

Connector design gate

The previous 12-way / 12-way / 7-way connector arrangement is a packaging illustration only. Pin counts and connector family shall not be frozen until the actual engine/sender installation has been surveyed. Sensor electrical interface, thermocouple termination/material requirements, wire count/gauge, grounding/reference, current rating and environmental/mechanical requirements determine connector selection.

Network role

Horizon/EFIS is an **AEF-CAN maintenance gateway, not an operational bus master**. The EIU publishes measurements independently and continues doing so if Horizon disappears. A logger, second display or future compatible consumer may receive EIU data directly. Firmware/configuration gateway functions do not create an operational dependency on Horizon.

USB-C service scope

EIU USB-C remains the preferred local service/programming interface, but is not an everyday operational interface. It should be mechanically recessed/protected as appropriate. USB bench power need only support the service/digital domain required for programming and diagnostics; it is not required to reproduce the complete aircraft-powered sensor environment. Internal recovery access remains required.

Platform OTA facilities

The required firmware behaviour is known-good -> candidate -> trial -> confirmed/rollback. Implement this using mature MCU/platform OTA/boot facilities where suitable rather than creating a bespoke boot manager without need. The EIU still independently verifies firmware authenticity before execution.

Phone-carried offline release sets

The customer mobile app may pre-cache a compatible signed release set containing both Horizon and EIU firmware before travelling to an aircraft with no network coverage. The phone transfers the Horizon package to Horizon over the local maintenance connection; Horizon then remains the AEF-CAN maintenance gateway for EIU delivery.

The phone does not directly program the EIU in V1. This preserves one CAN maintenance authority/path, keeps EIU transport/recovery policy inside Horizon, and avoids requiring a second phone-to-EIU radio/service protocol. The EIU independently authenticates its candidate exactly as it does for an Internet-originated update.

EFIS/EIU firmware dependency contract

EIU firmware is independently versioned from Horizon firmware. Normal compatibility is negotiated from AEF-CAN protocol version and advertised capabilities, with the signed release-set manifest defining tested combinations and deployment order.

The EIU must preserve capabilities required by the previous supported Horizon state during any normal coordinated-update transition in which Horizon could roll back to that state. Likewise, a Horizon release must not begin using a new mandatory EIU capability until discovery confirms that capability after the EIU update/reboot.

There is no universal "EIU first" rule. The release manifest specifies an order proven to keep each intermediate and rollback state compatible. A dependency transition that cannot meet this property requires a special migration/recovery procedure and is excluded from normal OTA/offline one-click updates.

docs/SUPPORT_SERVICE.md

Repository source: docs/SUPPORT_SERVICE.md

MicroSky Horizon customer support chatbot

Status: ARCHITECTURE ADOPTED / MVP STAGED / RUNTIME VALIDATION PENDING

Date: 27 September 2026

Objective

Provide a Docker-hosted MicroSky Horizon customer support service with access to approved product documentation and a controlled learning loop that builds a useful, **validated Q&A knowledge base** from real customer interactions.

Core design decision

Customer conversations are learning input, **not authority**. The system learns question frequency, wording, gaps and feedback automatically. New answers remain candidates until a reviewer validates them against current product documentation.

This is deliberately a human-in-the-loop knowledge system rather than an autonomously self-training chatbot.

Architecture

- support: FastAPI chat/retrieval/feedback/review API.
- db: PostgreSQL persistent document, interaction, candidate and validated-Q&A store.
- Product-document ingestion tied to Git revision.
- Retrieval layer; lexical MVP, semantic/vector retrieval planned.
- Future LLM provider adapter receives only retrieved approved context plus validated Q&A.
- Future customer web widget/mobile integration calls versioned support API.
- Admin/reviewer UI remains private and authenticated.

The service is separate from OTA, licence signer and account services. It must not receive licence private keys, payment-card data, OTA signing secrets or aircraft/device credentials.

Knowledge lifecycle

approved docs -> retrieve -> answer -> interaction -> candidate -> review -> validated Q&A -> answer

When source documentation changes, affected validated answers should be marked for revalidation rather than assumed current.

Data/privacy

Collect the minimum conversation/account metadata required for support. Define retention/deletion/export policy before public launch. Redact secrets and unnecessary personal data. Customer text must never be copied into the authoritative product-document corpus.

Production gates

Before public exposure: authenticated admin/RBAC, CSRF where relevant, rate limiting/abuse controls, TLS/reverse-proxy policy, PII retention controls, backup/restore test, migrations, audit trail, semantic retrieval evaluation, grounded LLM adapter, prompt-injection tests, source citations, stale-document invalidation, support escalation, monitoring and a reviewed test set of expected answers.

No runtime/build-success claim is made until the Docker harness is run on the Synology.

Grounded LLM answering — 27 September 2026

IMPLEMENTED IN MVP / RUNTIME VALIDATION PENDING. The support API now has a provider-neutral OpenAI-compatible chat-completions adapter. Natural-language questions first retrieve approved product-document evidence. The LLM is instructed to answer only from that evidence, cite source numbers, distinguish provisional development material from validated instructions, avoid unsupported safety/certification/capability claims and explicitly escalate when evidence is insufficient.

LLM output is never promoted directly to authoritative knowledge. It is stored as an interaction and candidate Q&A and must pass the existing human validation workflow before entering `validated_qa`. Exact validated Q&A takes precedence over generation. Provider failure falls back conservatively to retrieved source material.

Provider URL/key/model are runtime secrets/configuration, not repository content. Production still requires semantic retrieval evaluation, source-level citation mapping, privacy/retention review, token/cost limits, authentication/rate limiting and prompt-injection/red-team testing.

Semantic/vector retrieval — 27 September 2026

IMPLEMENTED IN REPOSITORY / RUNTIME EVALUATION PENDING. PostgreSQL is now based on the pgvector PG17 image. Approved document sections receive embeddings through a configurable OpenAI-compatible embedding endpoint. Natural-language questions are embedded and ranked against document vectors by cosine distance before grounded LLM generation. Lexical retrieval remains the fail-safe fallback if embeddings are not configured or the provider fails.

Embedding model and dimensionality are deployment configuration and must be frozen per index generation. Model/dimension changes require controlled re-embedding/migration. Retrieval quality still requires a representative Horizon support-question test set before public launch.

Answer status and customer resolution — 27 September 2026

Every response must make its knowledge status visible to the customer. Supported states are **VALIDATED**, **GENERATED — NOT YET VALIDATED**, **RETRIEVED — NOT YET VALIDATED**, **INVALIDATED**, and **NO VALIDATED ANSWER**. Invalidated historical answers may be retrieved when relevant, but they must be conspicuously labelled as superseded/not-current guidance, carry the invalidation reason where available, and must never be presented as an approved answer.

After each answer the interaction asks “**Did this answer resolve your question?**”. The yes/no result is stored separately from optional positive/negative feedback. Customer resolution is evidence about answer usefulness, not technical validation: even many successful customer confirmations cannot promote an answer into the validated knowledge base without the formal reviewer process. Conversely, unresolved interactions should increase review priority and help identify missing documentation, weak retrieval and inadequate answers.

The future reviewer dashboard should show answer status, source/revision, number of occurrences, resolution rate, feedback, invalidation history and candidate/validated lineage. It should support explicit invalidate/supersede operations without deleting the historical answer.

Customer interaction modes — 27 September 2026

ADOPTED / IMPLEMENTATION PENDING. Customer support will support both **keyboard/text** and **voice** interaction. Both modes feed the same support session, retrieval, LLM, provenance, feedback and validation pipeline; voice is an input/output presentation layer rather than a separate knowledge system.

Keyboard

Text entry is always available and is the baseline/fallback interaction. It is preferable for serial numbers, Device IDs, firmware versions, fault codes, URLs and other exact technical strings. Answers retain their visible status (**VALIDATED**, **GENERATED — NOT YET VALIDATED**, **RETRIEVED — NOT YET VALIDATED**, **INVALIDATED** or **NO VALIDATED ANSWER**), sources and the “Did this answer resolve your question?” control.

Voice

The customer may press a microphone control to speak a question. Speech-to-text produces a visible editable transcript **before or alongside submission** so the customer can correct technical terms, identifiers and numbers. The resulting text is sent through exactly the same support API as keyboard input.

Answers are displayed as text and may optionally be read aloud using text-to-speech. Spoken output must not hide provenance: the UI continues to show answer status and sources, and for safety-significant INVALIDATED or unvalidated answers the spoken introduction should also state the status before reading the answer. Voice must never turn an uncertain transcription into an assumed Device ID, fault code, firmware version or other exact identifier without confirmation.

Conversation history stores the normalized text question/answer and interaction metadata. Raw microphone audio should **not be retained by default**; any future diagnostic/audio retention requires explicit consent, purpose and retention policy. Speech-provider configuration and credentials are runtime secrets.

The resolution question is available in both modes: tap Yes/No or answer an equivalent voice prompt. Voice-derived resolution is stored identically to keyboard/tap feedback and remains a usefulness signal, never technical validation.

Implementation direction

Add provider-neutral STT and TTS adapters so deployments can use hosted or local speech services. Web clients use browser microphone/audio capabilities where appropriate; iOS/Android customer apps use native microphone/audio APIs. Always provide keyboard fallback. Production gates include permission UX, transcription accuracy tests using MicroSky/Horizon terminology, noisy-cockpit testing where relevant, accessibility, latency/cost limits, privacy review and abuse/rate controls.

Voice/chat implementation checkpoint — 27 September 2026

IMPLEMENTED IN REPOSITORY / RUNTIME VALIDATION PENDING. The Docker support service now includes a responsive customer web chat, keyboard input, browser microphone capture, editable transcription, STT/TTS API adapters, visible answer provenance, sources, spoken-answer control and Yes/No resolution feedback. Voice and keyboard converge on the same /v1/ask knowledge pipeline. Raw microphone bytes are processed in memory and are not persisted by the application. Browser/provider runtime testing and the production security/privacy gates remain outstanding.

docs/TESTING.md

Repository source: docs/TESTING.md

Testing Plan

Principle

The ESP32 EFIS must be tested progressively. Convincing graphics, either in QEMU or on the bench, do not establish trustworthy airborne attitude, altitude or heading information. Invalid/stale sources must fail obviously rather than freeze a plausible indication.

Current gate — 15 September 2026: no physical prototype hardware is available. Hardware-dependent testing is therefore **PAUSED**. Stages 1–10 below remain the required validation programme, not completed work. A successful ESP32-S3 compile is recorded only as compile validation.

Stage 0 — Build, emulator and simulator regression — CURRENT SOFTWARE STAGE

The 480×480 graphics/presentation passes for Artificial Horizon, Altimeter and Compass have been completed and accepted in QEMU. Their accepted states and failure/recovery sequences are recorded in `SIMULATION.md` and `PROJECT_STATUS.md`. The native Swift simulator is maintained in step for interactive testing.

Start a clean shell with:

```
cd ~/Documents/Xcode/ESP32-EFIS
source scripts/efis-env.sh
cd "$EFIS_FIRMWARE_DIR"
```

Confirm the script reports ESP-IDF v5.4.4, the pinned ESP-IDF Python interpreter and a working `qemu-system-xtensa` when QEMU is installed.

For QEMU, use a clean `build-qemu` after emulator configuration changes. Confirm `sdkconfig.qemu.defaults` enables QEMU and synthetic data but disables physical SPIRAM. The emulator must reach `app_main()`, open the 480×480 virtual display and show the red **SIM** marker.

Regression requirements remain: attitude, altitude and heading invalid states must fail obviously wherever each source is presented; recovery must restore only valid synthetic data; 000/359 heading wrap must remain continuous; >10,000-ft hatching and the compact Kollsman scale must retain their accepted geometry.

While hardware is unavailable, routine hardware builds/flashing are paused. If a hardware-target compile is performed specifically to check source compatibility, record it as **COMPILE-VALIDATED only** and do not infer physical PSRAM, SPI, LCD, encoder or sensor operation.

Stage 1 — Electronics bench test — PAUSED / PENDING HARDWARE

Verify stable 5 V USB-C power, repeated ESP32-S3 boot, physical 2 MB Quad-PSRAM operation, BMI088 identity/status, sensible accelerometer/gyro samples, reliable display initialization, rotary encoder operation and absence of brownouts at maximum display brightness.

Use the disabled-by-default BMI088 commissioning diagnostic to establish physical sensor axes/signs. Do not connect BMI088 data to the live Horizon before that mapping is demonstrated.

Stage 2 — Static orientation tests — PENDING HARDWARE

Place the instrument at known attitudes and compare displayed pitch/roll with physical references. Test level, $\pm 10^\circ$ pitch, $\pm 20^\circ$ pitch, $\pm 30^\circ$ roll and $\pm 60^\circ$ roll. Record repeatability, hysteresis and indication direction.

Stage 3 — Dynamic hand/fixture tests — PENDING HARDWARE

Move the unit through smooth and abrupt rotations while logging raw IMU and estimated attitude. Check latency, overshoot, drift after motion stops, estimator stability, frame-rate stability and invalid-state behaviour during induced faults.

Stage 4 — Acceleration-rejection tests — PENDING HARDWARE

Translate the instrument without deliberately rotating it and observe whether linear acceleration creates false pitch/roll corrections. Accelerometers cannot distinguish gravity from all other specific forces, so acceleration-sensitive corrections must remain confidence weighted.

Stage 5 — Altitude/static-pressure tests — PENDING HARDWARE

Implement and commission the ported BMP585 acquisition path, then compare pressure and indicated altitude against traceable/reference pressure sources over representative QNH settings. Test pressure-port leakage/blockage, stale data and sensor disconnection. The altimeter must become conspicuously invalid rather than retain a plausible frozen altitude.

Stage 6 — Heading tests — PENDING HARDWARE

Implement and commission the remote PNI RM3100-CB path. Test hard/soft-iron calibration, aircraft installation effects, cable routing, heading repeatability, tilt compensation and turns through 000/359. Compare against an independent heading reference. GNSS track, if later displayed, must remain explicitly labelled TRK rather than being substituted for heading.

Stage 7 — Vibration testing — PENDING HARDWARE

Expose the instrument to representative vibration and compare raw gyro noise, raw accelerometer noise, estimated attitude noise, display readability and connector integrity. Include representative engine RPM ranges if this can be done safely while the aircraft is stationary.

Stage 8 — Thermal and sunlight tests — PENDING HARDWARE

Test the enclosure/display at elevated temperature and direct sunlight. Check high-nit readability, LCD colour/contrast, case distortion, ESP32 stability, IMU bias change, backlight thermal load and USB-C connector/cable behaviour.

Stage 9 — Aircraft ground installation test — PENDING HARDWARE

Before airborne evaluation, verify mechanical security, instrument/aircraft axis alignment, control/wiring clearance, electrical/magnetic interference, operation with engine off/running, static-pressure installation integrity and remote magnetometer behaviour. Compare with independent references where practical.

Stage 10 — Airborne experimental comparison — FUTURE / BLOCKED

Only after the preceding stages are satisfactory and only as a non-primary experimental display. Compare against independent trusted references and collect logs rather than relying on visual impressions alone. Include straight/level flight, normal turns, climbs/descents, acceleration/deceleration, turbulence and prolonged bank.

Fault injection

Deliberately test disconnected/stale/corrupt BMI088 data, pressure-sensor failure, magnetometer failure, display task slowdown, processor restart, power interruption and invalid calibration data. Synthetic simulation must never activate automatically as a fallback for a failed real sensor.

The required response to a real-source failure is a conspicuous invalid indication, not synthetic substitution and not a frozen plausible value.

Bootable full electrical test harness — 26 September 2026

A rotary-selectable **FULL TEST** boot option is now an adopted requirement. It runs deterministic electrical/component checks and reports PASS / FAIL / NOT TESTED plus stable documented ECCC-NN failure codes. Tests continue after non-dangerous failures to collect a complete fault list. Optional/unfitted hardware must not be reported as failed. Display/backlight tests include a human visual check.

Implementation/validation proceeds from simulated PASS/FAIL cases through Phase-1 hardware, BMI088/BMP585, RM3100/GNSS and eventual production power/serial-interface tests. Each implemented test requires both a known-good PASS and an injected or realistic FAIL case. See `ELECTRICAL_TEST_HARNESS.md` for the code registry and staged implementation plan.

docs/user-guides/ALTIMETER.md

Repository source: docs/user-guides/ALTIMETER.md

Altimeter panel — user guide

Purpose

The Altimeter page deliberately resembles a classic round aircraft altimeter: black face, high-contrast white circumference scale and three analogue hands. It remains supplementary/non-primary and does not replace the aircraft's required altimeter.

Presentation

The renderer uses 50 circumference divisions with a long 100-ft hand, medium 1,000-ft hand and short 10,000-ft hand. A digital altitude readout supplements the analogue hands.

The accepted Kollsman presentation is a compact mechanical-style curved pressure scale at the **3 o'clock** position. The selected QNH sits beneath a fixed index while the surrounding pressure graduations move with the setting. The visible sector spans approximately **8 hPa total**. Pressure numbers and hatch marks remain at constant radii from the centre of the instrument; no QNH, HPA or KOLLSMAN unit text is shown in the window. The exact **1013.25 hPa** standard-pressure datum is identified by a substantially heavier tick. QNH selection itself remains integer hPa in 1 hPa steps.

Above 10,000 ft, a separate hatched annular sector appears at **9 o'clock**. It progressively reveals between 10,000 and 11,000 ft and remains fully exposed above 11,000 ft.

Controls

- **Short press:** Compass.
- **Long press (~0.8 s):** enter/leave BARO/QNH setting.
- **Rotate:** QNH by 1 hPa per detent, currently 950–1050 hPa.
- **Short press in settings:** accept/leave settings.

QNH defaults to 1013 hPa on a fresh instrument and is stored in ESP32 NVS.

Pressure source

The reference sensor is **Bosch BMP585**. First bench/static-plumbing development uses the **Adafruit BMP585 Ported breakout PID 6413**. The gel-filled BMP585 was chosen for environmental robustness while retaining low noise and good relative pressure accuracy. The BMI088 is never used to manufacture altitude.

Until the BMP585 driver and static plumbing are physically integrated and validated, normal non-simulation firmware keeps altitude invalid. Bench simulation is explicitly synthetic.

Validity / failure

Altitude becomes valid only after pressure-sensor startup, range, freshness and plausibility checks pass. Missing, stale or invalid pressure immediately invalidates the indication rather than freezing the last plausible altitude.

Bench checks

Compare against a trusted pressure reference over multiple pressures/altitudes; exercise hand wrap points; test QNH/NVS; rapid pressure changes; power cycles; static leaks/restrictions; disconnected/stale sensor conditions. See `../SENSORS.md`.

docs/user-guides/COMPASS.md

Repository source: docs/user-guides/COMPASS.md

Compass panel — user guide

Purpose

The Compass page uses the familiar aircraft directional-gyro/HSI convention: aircraft symbol and top lubber line remain fixed while the compass card rotates beneath them. It is supplementary/non-primary.

Presentation

The renderer uses a black rotating card, white 5-degree ticks, stronger major divisions, fixed yellow lubber triangle and fixed yellow aircraft symbol. A yellow heading bug belongs to the rotating card.

A compact lower GNSS strip is now an **adopted requirement**. When GNSS is valid it shows WGS84 latitude/longitude, fix state and receiver-reported horizontal accuracy; satellites used may be shown where space permits. Latitude/longitude numbers and the accuracy indication use the common project colour bands: green ≤ 1 m, light green $>1-3$ m, yellow $>3-10$ m, orange $>10-30$ m and red >30 m. Invalid or stale GNSS is red and stale coordinates are removed/obscured rather than frozen. See `../GNSS_DISPLAY.md`.

Controls

- **Short press:** Horizon/PFD.
- **Long press (~0.8 s):** enter/leave HDG BUG setting.
- **Rotate:** heading bug 1 degree per detent through 000–359°.
- **Short press in settings:** accept/leave settings.

Heading bug and last panel are persisted in ESP32 NVS.

Heading source

The reference magnetic sensor is **PNI RM3100-CB**, mounted remotely from the instrument electronics on a rigid non-magnetic bracket. BMI088 gyro/accelerometer attitude is combined with calibrated three-axis magnetic field for tilt-compensated heading. Raw gyro yaw is not accepted as an absolute compass source.

The RM3100-CB installation requires hard-iron and soft-iron calibration, installation-axis alignment and magnetic-interference testing with aircraft electrical loads in multiple states. Until that pipeline is integrated and valid, the normal non-simulation build keeps heading invalid.

Heading versus track

GNSS course over ground is labelled **TRK**, not HDG. Magnetic/true reference and variation handling must be explicit. GNSS ground speed is **GS**, never IAS. GPS position validity is independent of magnetic-heading validity: loss of one must not make the other appear valid or invalid without cause.

Failure behaviour

Missing, stale, implausible or magnetically disturbed heading data invalidates the card rather than freezing the last plausible heading. Missing/stale GNSS removes the coordinates and shows an explicit red GPS invalid/stale indication; it does not invalidate a healthy magnetic compass.

Bench/aircraft checks

Verify card sense, 000/359 continuity, bug relationship, NVS persistence, tilt compensation, full calibration, source freshness and magnetic effects from display/backlight, radios, wiring and adjacent equipment. GNSS tests cover no fix, stale data and all accuracy-colour bands. See `../SENSORS.md` and `../GNSS_DISPLAY.md`.

docs/user-guides/HORIZON.md

Repository source: docs/user-guides/HORIZON.md

Artificial Horizon / PFD panel — user guide

Purpose

The Horizon panel is the primary page of this supplementary/non-primary multifunction instrument. A short PEC09 push cycles **Horizon/PFD** → **Altimeter** → **Compass** → **Horizon/PFD**.

Presentation

The revised design takes its cues from modern compact round EFIS instruments while retaining a deliberately uncluttered attitude display. The centre remains the dominant blue-sky/brown-ground horizon with white pitch ladder and fixed yellow aircraft symbol. An upper roll arc and fixed yellow roll reference are added. Reserved data areas allow pressure altitude and heading/track only when their independent sources are valid.

A compact lower GNSS strip is now an **adopted requirement**. When GNSS is valid it shows WGS84 latitude/longitude, fix state and receiver-reported horizontal accuracy; satellites used may be shown where space permits. Latitude/longitude numbers and the accuracy indication are colour coded from the common project thresholds: green ≤ 1 m, light green $>1-3$ m, yellow $>3-10$ m, orange $>10-30$ m and red >30 m. Invalid or stale GNSS is red and the last plausible coordinates must not remain displayed as current. See `../GNSS_DISPLAY.md`.

No unavailable value is fabricated merely to fill a box. Attitude, altitude, heading and GNSS each have independent validity.

Controls

- **Short press:** next panel.
- **Long press (~0.8 s):** enter/leave Horizon settings.
- **Rotate in settings:** brightness, 5% per detent, 10–100%.
- **Short press in settings:** accept/leave settings without changing panel.

The selected brightness and last selected panel are persisted in ESP32 NVS. Actual TPS61169 PWM application remains a hardware bring-up task.

PFD auxiliary data

Altitude is shown only from the validated static-pressure system. Heading is shown only from the validated magnetic/AHRS source. GNSS course over ground is labelled **TRK**, never HDG; GNSS groundspeed is **GS**, never IAS. GNSS position/accuracy does not substitute for attitude, barometric altitude or magnetic heading.

Failure behaviour

Stale, unavailable or implausible attitude must invalidate the normal attitude presentation. A plausible frozen horizon is unacceptable. Optional PFD fields independently show invalid/unavailable rather than retaining stale values. GNSS loss affects the GNSS strip only and must not invalidate an otherwise valid horizon.

Bench checks

Confirm screen cycling, long-press settings, persistence across power cycles, roll-scale geometry, independent field validity and obvious loss of BMI088/AHRS validity. GNSS tests must cover no fix, stale data and each accuracy-colour band while confirming that stale coordinates disappear.

docs/user-guides/README.md

Repository source: docs/user-guides/README.md

Instrument user guides

The instrument UI is deliberately documented **one panel per user-guide file** so each display can evolve independently while retaining common control behaviour.

Panel guides

- [Artificial Horizon](HORIZON.md)
- [Altimeter](ALTIMETER.md)
- [Compass](COMPASS.md)

Common PEC09 control model

The Bourns PEC09 - 2320F - T0015 is the single front-panel control.

- **Short press:** cycle to the next screen.
- **Long press (~0.8 seconds):** enter or leave settings for the current screen.
- **Rotate:** changes the current screen's setting only while settings are active.
- **Short press in settings:** exits settings and stays on the same screen.

Current setting assignments are Horizon = brightness, Altimeter = QNH, Compass = heading bug.

The current firmware intentionally marks all three flight-data presentations invalid until their live data sources are implemented. No panel is allowed to fabricate a plausible value merely to make the UI look complete.

docs/WIRING.md

Repository source: docs/WIRING.md

Wiring Plan

This document records the prototype electrical architecture and frozen GPIO allocation for the custom ESP32-S3 carrier PCB.

Power

```
regulated 5 V source / USB-C VBUS
|
+--> TPS62162-Q1 3.3 V buck
|
|   +--> ESP32-S3-WROOM-1-N16R2
|   +--> BMI088
|   +--> LCD VDD
|   +--> MCP23008
|
+--> TPS61169 constant-current backlight driver
|
+--> LED_A / LED_K --> LCD backlight (~6 V / 100 mA)
```

Raw aircraft 12 V is deliberately kept outside the prototype instrument during this phase.

Processor module

Selected module: **ESP32-S3-WROOM-1-N16R2**.

Reasons:

- 16 MB Quad flash
- 2 MB Quad PSRAM
- enough PSRAM for two 480×480 RGB565 framebuffers
- GPIO35–37 remain available
- integrated PCB antenna
- active production module rather than an obsolete DevKit

The custom carrier replaces the previous DevKit and therefore includes native USB-C, 3.3 V regulation, EN/RESET, BOOT, display connector and low-speed GPIO hardware.

USB-C

Native USB is connected directly to the ESP32-S3:

- GPIO19 = USB D-
- GPIO20 = USB D+
- 22 Ω starting series resistors on D- and D+
- 5.1 k Ω sink resistors from CC1 and CC2 to GND
- low-capacitance USB ESD protection near the connector

Keep D+/D- short, parallel and away from LCD PCLK and switching nodes.

3.3 V regulator

Reference regulator: **TI TPS62162-Q1**, fixed 3.3 V, 1 A.

Starting external network:

- 10 μ F input ceramic plus 100 nF bypass
- 2.2 μ H inductor, at least ~1.5 A preferred current rating
- 22 μ F output ceramic plus local decoupling
- power-good test point

The regulator/inductor switching loop must be kept away from the BMI088 and its SPI traces.

Display electrical mode

Selected panel: **Newhaven NHD-2.1-480480AF-ASXP**.

Final connector: **Molex 54104-4031** or verified equivalent 40-pin 0.5 mm FFC connector.

The panel is operated as:

- 480 × 480
- 16-bit RGB565 pixel bus
- DE mode
- **30 MHz** RGB pixel clock starting point
- 9-bit SPI only for ST7701S initialization

RGB + serial mode straps

- IM0 = 0
- IM1 = 1
- IM2 = 0

DCX is not required by the selected 3-wire 9-bit serial protocol because command/data state is carried in the ninth bit.

RGB565 connection

- D0..D4 -> panel B1..B5
- D5..D10 -> panel G0..G5
- D11..D15 -> panel R1..R5
- panel B0 -> GND
- panel R0 -> GND

The ST7701S initialization configures the RGB interface for 16-bit/pixel mode (VIPF=101).

Frozen ESP32 GPIO map

Function	GPIO	Notes
RGB D0 / B1	4	RGB565 bit 0
RGB D1 / B2	5	
RGB D2 / B3	6	
RGB D3 / B4	7	
RGB D4 / B5	8	
RGB D5 / G0	9	
RGB D6 / G1	10	
RGB D7 / G2	11	

RGB D8 / G3	12	
RGB D9 / G4	13	
RGB D10 / G5	14	
RGB D11 / R1	15	
RGB D12 / R2	16	
RGB D13 / R3	17	
RGB D14 / R4	18	
RGB D15 / R5	21	
LCD PCLK	1	30 MHz RGB starting point
LCD DE	2	Data enable
LCD HSYNC	38	Horizontal sync
LCD VSYNC	39	Vertical sync
Shared SPI MOSI	35	LCD SDA during init; BMI088 MOSI afterwards
Shared SPI SCLK	36	LCD SCL during init; BMI088 SCLK afterwards
BMI088 MISO	37	LCD does not require MISO
BMI088 ACC CS	40	Accelerometer chip select
BMI088 GYRO CS	41	Gyro chip select
Backlight PWM	42	TPS61169 control
MCP23008 SDA	47	I²C
MCP23008 SCL	48	I²C
MCP23008 INT	43	Encoder interrupt-on-change
Spare / debug	44	Test pad
Native USB D-	19	USB
Native USB D+	20	USB

Reserved ESP32 pins

GPIO	Reason
0	BOOT strap and button only
3	Strapping pin
45	Strapping pin
46	Strapping pin

No operational loads should be attached to those strapping pins.

MCP23008 allocation

Address: 0x20, A0/A1/A2 tied low.

- GP0 -> LCD CSX
- GP1 -> LCD RESETX
- GP2 -> encoder A
- GP3 -> encoder B
- GP4 -> encoder push
- GP5–GP7 -> spare/test pads

Use interrupt-on-change for encoder inputs.

Shared SPI behaviour

1. Hold BMI088 ACC CS and GYRO CS high.
2. Drive LCD CSX low through MCP23008.
3. Perform ST7701S 9-bit initialization using GPIO35/36.
4. Drive LCD CSX high.
5. Thereafter use the same SCLK/MOSI pair for BMI088 acquisition.

LCD pixel data is always sent over the RGB bus, not SPI.

LCD supply

LCD VDD runs from **+3V3_SYS**. Newhaven specifies 2.5–3.3 V.

Starting RGB timings

The current Newhaven datasheet gives the following **RGB** timing values:

- PCLK: **30 MHz**
- HFP: **50** clocks
- HBP: **50** clocks
- HS pulse: **4** clocks
- VFP: **50** lines
- VBP: **50** lines
- VS pulse: **2** lines

The earlier 18 MHz / 40 / 60 / 20 / 10 / 10 / 6 values were Newhaven's **MIPI** timing table and must not be used as the baseline RGB timing.

These RGB values remain subject to bench verification on the actual panel and carrier PCB.

PCB implementation rule

The first custom PCB is approximately **68 mm diameter** on the existing **60 mm mounting PCD**. It mounts directly to the rear-cover standoffs and replaces the former printed DevKit carrier.

The ESP32-S3-WROOM-1 antenna sits at the PCB edge. Follow Espressif antenna keepout guidance: no copper, traces, fasteners or metal in the antenna zone, and use a board-edge cutout/keepout as required.

A printed PCB fit gauge is generated from the enclosure CAD to verify the mechanical envelope before fabrication.

Authoritative schematic files

- hardware/schematics/CARRIER_PCB_SCHEMATIC.md
- hardware/schematics/carrier-netlist.csv

efis-web-simulator/README.md

Repository source: [efis-web-simulator/README.md](#)

Horizon EFIS Web Simulator

Containerised browser emulator for the Horizon boot/maintenance menus. Its purpose is to exercise OTA firmware and licensing services without physical ESP32 hardware.

Scope

The simulator mirrors the rotary/push boot UX and uses a synthetic device identity. It deliberately does **not** emulate flight sensors or claim to validate flight firmware.

Current menu:

- START EFIS
- FULL TEST
- FAULT LOG
- LICENSE
- FIRMWARE UPDATE

LICENSE exercises status/install/reset flows. FIRMWARE UPDATE exercises Wi-Fi state, manifest checking, download, staged firmware and explicit ACTIVATE & REBOOT. A test panel can inject network/device states and inspect the event log.

Run

```
docker compose up --build
```

Open <http://localhost:8088>.

Service integration

Browser calls the simulator backend only. Configure upstream service URLs in Compose/environment; do not put admin/signing credentials in browser JavaScript.

- OTA_BASE_URL: public OTA origin. Manifest endpoint defaults to `/efis/manifest.json`.
- LICENSE_BASE_URL: customer/device licence service base URL. API paths are intentionally adapters/placeholders until the account/licence service contract is frozen.
- SIM_ALLOW_MUTATIONS: keep false against production services.

Default mode is safe mock mode and works without either upstream.

Controls

Use the on-screen rotary buttons or keyboard: Up/Down = rotate, Enter = short press, Escape = back. State persists in the container only as test state and may be reset.

Safety

Never configure the simulator with the private licence signing key, signer token, OTA admin session secret, payment secrets or real aircraft credentials. It is a client/service test harness, not an admin control plane.

Multi-node offline OTA simulation

The simulator now exposes Horizon and EIU installed versions, release-set caching, local maintenance-network state, transfer/staging and dependency-ordered activation. The default mock release demonstrates EIU 1.7 -> 1.9 before Horizon 2.4 -> 2.5, with Horizon checking the required EIU capability before completing its own activation.

This models control flow only. It is not evidence of physical Wi-Fi, flash, A/B boot or CAN behaviour.

firmware/DEPLOYMENT.md

Repository source: `firmware/DEPLOYMENT.md`

Deploying the ESP32 EFIS firmware

This guide covers building, flashing and monitoring the **ESP32 EFIS** on the project **ESP32-S3-WROOM-1-N16R2** hardware from macOS. The firmware is rooted at `firmware/` and is pinned to **ESP-IDF v5.4.4**.

> **Important:** this is a supplementary/non-primary flight-development instrument. Do not use a newly flashed build in the aircraft until display, sensor, failure-state and power-up behaviour have been bench-tested.

Development shell

The repository provides the standard environment setup script. In every new Terminal session:

```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
source scripts/efis-env.sh
cd "$EFIS_FIRMWARE_DIR"
```

The script pins the known ESP-IDF v5.4.4 Python interpreter, configures `IDF_PATH`, prevents competing ESP-IDF Python environments from being mixed within a CMake build, and makes the installed QEMU binary available when Espressif's normal export omits its path.

For complete first-time Mac installation instructions see `../docs/MACOS_BUILD_AND_QEMU_SETUP.md`.

Hardware connection

The carrier uses ESP32-S3 native USB:

- GPIO19 = USB D-
- GPIO20 = USB D+
- USB-C VBUS = regulated 5 V instrument input
- USB ground = carrier ground

Use a known-good USB data cable and avoid hubs during first bring-up.

Normal hardware build

Normal aircraft-target development uses `sdkconfig.defaults`. It enables the N16R2's physical 2 MB Quad PSRAM and does **not** enable synthetic bench data.

```
cd "$EFIS_FIRMWARE_DIR"
idf.py set-target esp32s3
idf.py build
```

A successful build produces the `esp32_efis` application. Prefer deployment from a commit whose firmware CI build is green.

Before flashing an aircraft-target build, confirm QEMU/simulation configuration has not leaked into the normal build. QEMU has its own `build-qemu` directory and must never be used as the source of a hardware flash.

Find the USB device

```
ls /dev/cu.usbmodem*
```

Use the actual device shown by the Mac, for example `/dev/cu.usbmodem1101`.

If automatic download mode fails, hold **BOOT**, press/release **RESET/EN**, then release **BOOT** and retry.

Flash and monitor

```
idf.py -p /dev/cu.usbmodem1101 flash monitor
```

Replace the port with the actual device. Exit the ESP-IDF monitor with `Ctrl-J`.

For an aircraft-target build, verify the startup log reports:

```
BENCH SIMULATION: OFF
```

If simulation is unexpectedly enabled, stop and do not use that image on hardware.

Clean normal rebuild

After a significant configuration change or interpreter mismatch:

```
rm -rf build
idf.py set-target esp32s3
idf.py build
```

Deleting the build directory is also the safest recovery when CMake reports that the project was configured with a different Python interpreter.

QEMU is not a deployment image

QEMU uses `sdkconfig.qemu.defaults`, enables synthetic data and the virtual display, and explicitly disables external SPIRAM because the emulated target does not provide the physical N16R2 PSRAM device in this configuration. A QEMU image is for desktop simulation only.

A clean emulator build is:

```
rm -rf build-qemu
idf.py -B build-qemu \
  -D SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.qemu.defaults" \
  set-target esp32s3
idf.py -B build-qemu \
  -D SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.qemu.defaults" \
  build
idf.py -B build-qemu qemu --graphics monitor
```

See `../docs/SIMULATION.md` for the simulation safety contract.

Expected physical bring-up checks

On physical hardware, verify the MCP23008 initializes, the ST7701S/RGB display starts correctly, PSRAM allocation succeeds, the backlight behaves correctly, and the BMI088 accelerometer/gyro identities are valid. The BMI088 accelerometer chip ID is `0x1E` and gyro chip ID is `0x0F`.

The three-page UI and control framework now exists, but live sensor integration remains under development. A page must show an unmistakable invalid state whenever its real source is unavailable or stale; a plausible frozen value is unacceptable.

Erase flash

During bench development, stale NVS/partition state can be removed with:

```
idf.py -p /dev/cu.usbmodem1101 erase-flash
```

Then rebuild and reflash. Do not erase casually once calibration data is stored persistently.

Troubleshooting

If the build reports two different ESP-IDF Python paths, pull the latest repository, source `scripts/efis-env.sh`, delete the affected build directory and recreate it. Do not try to continue using a CMake directory configured by another interpreter.

If the physical display remains black, inspect serial logs before changing firmware. Check MCP23008 communication, LCD reset/chip-select, 3.3 V logic supply, TPS61169/backlight power, FFC orientation, ST7701S mode straps and RGB bus continuity.

If QEMU continuously reboots with an assertion in `s_psram_chip_init` / `esp_psram_init`, the QEMU build was created without the emulator PSRAM override. Pull the current repository, delete `build-qemu` and rebuild using both defaults files. `sdkconfig.qemu.defaults` must contain `# CONFIG_SPIRAM is not set`.

Deployment discipline

For every firmware revision intended for hardware testing: commit/push the source, confirm firmware CI, pull the exact commit to the Mac, source `scripts/efis-env.sh`, build locally, flash over native USB, inspect the complete startup log, test all available failure states and record the Git commit installed in the instrument.

The ESP32 EFIS remains experimental and **supplementary/non-primary**. Live AHRS, altitude and heading indications are not considered trustworthy merely because they render convincingly; each sensing path requires calibration, freshness/health monitoring and progressive bench/aircraft comparison testing.

firmware/README.md

Repository source: firmware/README.md

ESP-IDF Firmware

Target: **ESP32-S3-WROOM-1-N16R2**, framework baseline **ESP-IDF v5.4.4**.

The firmware is structured as a **multi-panel supplementary flight instrument** with three 480×480 pages:

1. Artificial Horizon / PFD
2. Classic Altimeter
3. Compass

A short PEC09 encoder press advances to the next panel. A long press of about 0.8 seconds enters/leaves the current panel's settings. Rotation changes Horizon brightness, Altimeter QNH or Compass heading bug. UI settings are persisted in NVS.

Per-panel operating documentation is in [./docs/user-guides/](./docs/user-guides/README.md).

Safety and data validity

instrument_data_t keeps values separate from their validity flags. Renderers must never treat stale, failed or absent sensor data as valid, and the system must not replace failed real data with synthetic values or freeze a plausible indication.

Synthetic bench data is opt-in and defaults OFF. When it is enabled, the data model sets its simulated flag and every firmware-rendered page carries a permanent high-contrast red **SIM** marker.

This remains a **supplementary/non-primary flight-development instrument**.

Source layout

main/	
■■■ app_main.c	physical/QEMU startup and main UI loop
■■■ instrument_ui.c/.h	PEC09 control and NVS-backed settings
■■■ instrument_screens.c/.h	three-panel renderer and validity contract
■■■ instrument_sim.c/.h	explicit synthetic development data
■■■ horizon_renderer.c/.h	horizon graphics
■■■ bmi088.c/.h	BMI088 SPI bring-up
■■■ mcp23008.c/.h	LCD control plus encoder input I/O
■■■ st7701s.c/.h	physical panel controller initialization
■■■ Kconfig.projbuild	simulation/QEMU development options
■■■ idf_component.yml	managed QEMU RGB component dependency

Physical display configuration

- Newhaven NHD-2.1-480480AF-ASXP
- ST7701S
- 480×480 RGB565, 16-bit RGB bus
- 30 MHz pixel clock
- HFP/HBP/HS = 50/50/4
- VFP/VBP/VS = 50/50/2
- two 460,800-byte framebuffers fit within the physical module's 2 MB Quad PSRAM

The ST7701S is configured first over 3-wire/9-bit serial; the same GPIOs are then reused for BMI088 SPI. The project adapts Newhaven's panel sequence from 18-bit to the ST7701S 16-bit VIPF=101 mode; physical-panel verification remains required.

QEMU backend

CONFIG_EFIS_QEMU is emulator-only and depends on CONFIG_EFIS_BENCH_SIMULATION. The QEMU path bypasses physical LCD, MCP23008, encoder, backlight and sensor initialization and renders to Espressif's esp_lcd_qemu_rgb virtual 480×480 RGB565 display.

The normal hardware defaults enable external Quad PSRAM for the N16R2. QEMU builds override this with # CONFIG_SPIRAM is not set in sdkconfig.qemu.defaults; otherwise ESP-IDF asserts in esp_psram_init() before app_main() because the emulated configuration does not provide the physical PSRAM device.

Keep QEMU in the separate build-qemu directory and never flash that image to aircraft hardware.

macOS shell setup

After the one-time ESP-IDF/QEMU installation, use the repository environment script in every new shell:

```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
source scripts/efis-env.sh
cd "$EFIS_FIRMWARE_DIR"
```

The script pins the known ESP-IDF v5.4.4 interpreter at ~/.espressif/tools/python/v5.4.4/venv/bin/python, prevents a second ESP-IDF Python environment from silently taking over an existing CMake build, and adds the installed QEMU binary to PATH when Espressif's export step omits it.

Normal hardware build

```
cd "$EFIS_FIRMWARE_DIR"
idf.py set-target esp32s3
idf.py build
```

Normal sdkconfig.defaults keeps physical PSRAM enabled and bench simulation disabled.

Clean QEMU build and run

```
cd "$EFIS_FIRMWARE_DIR"
rm -rf build-qemu

idf.py -B build-qemu \
-D SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.qemu.defaults" \
set-target esp32s3

idf.py -B build-qemu \
-D SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.qemu.defaults" \
build

idf.py -B build-qemu qemu --graphics monitor
```

See ../docs/MACOS_BUILD_AND_QEMU_SETUP.md, ../docs/SIMULATION.md and DEPLOYMENT.md.

Current implementation direction

The attitude path remains first: configure BMI088 ranges/ODR, acquire and timestamp samples, calibrate gyro bias, apply the aircraft-axis transform, run quaternion AHRS and acceleration-confidence logic, then feed validated live pitch/roll to the Horizon panel.

The Altimeter path uses the selected ported BMP585 architecture and needs live pressure/QNH processing. The Compass architecture uses the selected remote PNI RM3100-CB as the absolute-heading source, with appropriate AHRS fusion and independent validity/freshness monitoring.

Planned boot state machine and maintenance functions

The adopted physical boot UI is controlled by the PEC09 rotary/push encoder and offers **START EFIS** (default), **FULL TEST**, and **FIRMWARE UPDATE**. FULL TEST will run the staged electrical/component harness documented in ../docs/ELECTRICAL_TEST_HARNESS.md. FIRMWARE UPDATE will first establish Wi-Fi through a rotary-operated connection dialog before any OTA manifest request.

Wi-Fi SSID/credentials persist in a dedicated ESP-IDF NVS namespace. Production firmware requires encrypted NVS for credentials; credentials must not be redisplayed after entry or emitted in logs/diagnostics. These boot/test/network functions are design requirements until their firmware implementation and validation status is explicitly advanced.

Persistent operational fault log

Normal-operation hardware and software faults must be captured in a persistent, bounded ESP32 non-volatile fault/event log. The boot menu includes **FAULT LOG**, navigated with the PEC09 rotary/push control. Repeated instances of the same continuing fault are coalesced/count-incremented where practical to limit flash wear rather than written on every poll.

Records include stable fault code, source/subsystem, severity, occurrence information, firmware/build identity and safe diagnostic context. Valid timestamps may be added when a trustworthy time source is available. Wi-Fi credentials and other secrets must never be logged. Log clearing is explicit and confirmed; reboot/update must not erase the history. Runtime failure indications remain immediate and independent of the persistent log.

Product licence subsystem — planned

Provision a local/offline licence verifier using signed licence payloads. Production units have a stable provisioned device/product ID; an authorised off-device tool signs licences with a private key while firmware contains only the corresponding public verification key. Verified licence state is persisted in protected non-volatile storage.

The rotary boot/maintenance menu gains **LICENSE** for Status, Install/Replace and Reset. USB/service installation is the baseline; a later authenticated network path may be added. Reset is explicit/confirmed, clears installed licence state, is logged, and does not alter Wi-Fi credentials. Firmware updates normally preserve the licence. Licence and OTA signing keys are separate.

Licensing must not create plausible-but-invalid flight indications or suppress required failure annunciation. Core versus optionally licensed functions must be explicitly defined before production.

Licence account service architecture — 26 September 2026

ADOPTED / DESIGN STAGED. Add a separate Docker service (working name `efis-account`) alongside OTA/admin services. It owns customer accounts, registered EFIS devices, licence entitlements and payment-provider linkage; it does not hold the licence-signing private key in the public web tier.

Device identity uses a provisioned immutable **EFIS Device ID / serial number** as the primary key. The ESP32 factory/eFuse base MAC may be recorded as a secondary hardware fingerprint and registration aid, but MAC address alone is not the licence identity because interface MACs can be derived/changed and MAC exposure is unnecessary for manual licensing.

Online boot-maintenance flow: **LICENSE** -> Connect Wi-Fi -> identify device -> authenticated account/licence endpoint -> fetch signed licence entitlement -> verify locally -> store in protected NVS -> continue offline. The EFIS must not send a user password; device authentication uses a provisioned device credential/challenge mechanism to be specified. Normal flight startup must not depend on account/payment/network availability.

Offline flow: show Device ID and short registration code/QR-capable text; user obtains a signed licence from the account portal on another device and enters/imports it manually or via USB service. The licence is verified locally with the embedded public verification key.

Payment integration is provider-adaptor based. The account service creates checkout/customer-management requests and consumes verified payment webhooks; payment card data is handled by the payment provider, not stored by EFIS services. Payment status changes entitlements; a separate private signing worker/service generates signed licence payloads. Define explicit grace/revocation policy before subscriptions are enabled.

Suggested containers: `efis-account` API/web portal; PostgreSQL account/device/entitlement store; private `efis-license-signer` with tightly restricted signing-key access; existing `efis-ota` and `efis-ota-admin`. Payment provider secrets live only in server-side secret storage.

hardware/kicad/carrier_rev_a/README.md

Repository source: hardware/kicad/carrier_rev_a/README.md

Carrier PCB Revision A

KiCad 8-compatible project for the ESP32 Artificial Horizon custom carrier.

- 68 mm circular 4-layer PCB
- ESP32-S3-WROOM-1-N16R2
- TPS62162-Q1 3.3 V buck
- MCP23008 control expander
- USB-C 2.0 device/power input
- direct 40-pin Newhaven FFC
- BMI088, encoder and TPS61169 interfaces
- 60 mm mounting-hole PCD

This is a prototype hardware revision for bench validation. It is not flight-qualified hardware.

Testability and production follow-on

Revision A remains a bench PCB, but bring-up should preserve access needed by the bootable electrical test harness: power rails, reset/boot, display/backlight controls, MCP23008/encoder and sensor interfaces. The later production carrier adds protected nominal 12 V aircraft power conversion, encrypted-NVS-capable firmware configuration, GNSS and protected external DATA/EXPANSION interfaces. Do not expose raw ESP32 GPIO as aircraft-voltage-tolerant connections.

hardware/pcb/README.md

Repository source: hardware/pcb/README.md

Custom Carrier PCB — Revision A requirements

This directory records the physical and layout requirements for the custom ESP32-S3 carrier PCB.

Board role

The PCB replaces the ESP32 development board and the former printed electronics carrier. It mounts directly to the four rear-cover standoffs in the instrument enclosure.

Mechanical definition

- nominal board outline: **68.0 mm diameter**
- board thickness: **1.6 mm FR-4** starting point
- four mounting holes: **2.7 mm diameter**
- mounting pattern: **60.0 mm PCD**, holes at 45/135/225/315 degrees
- USB-C connector located near 6 o'clock to align with rear service slot
- ESP32-S3-WROOM-1-N16R2 placed near 12 o'clock with antenna at board edge
- antenna region to use explicit copper/component/metal keepout and board-edge cutout as required by Espressif guidance

A printable mechanical gauge is generated as:

enclosure/stl/ESP32_Artificial_Horizon_Custom_PCB_Fit_Gauge.stl

Print and test the gauge before ordering PCBs.

Preferred layer stack

Four layers:

1. Top — components/signals
2. Inner 1 — continuous GND
3. Inner 2 — power and limited signals
4. Bottom — limited signals / GND fill

Do not route signals through the ESP32 antenna keepout.

Placement zones

12 o'clock — ESP32 module

- ESP32-S3-WROOM-1-N16R2
- antenna facing board edge
- EN/BOOT passives nearby
- local 3.3 V decoupling close to module supply pads

6 o'clock — USB-C

- USB-C receptacle centered on rear service slot
- ESD array immediately behind connector
- CC resistors close to connector
- D+/D- route directly toward GPIO19/20

Power regulator zone

- TPS62162-Q1
- input capacitor, regulator, inductor and output capacitor in one tight switching cell
- place away from BMI088 connector/cradle and ESP32 antenna
- minimize SW copper area

Display connector zone

- 40-pin FFC connector oriented to minimize FFC twist and RGB trace length
- PCLK route short with optional source-series resistor footprint
- RGB data and timing traces referenced to uninterrupted ground

BMI088 connector zone

- located as close as practical to the rigid BMI088 cradle
- no switching regulator inductor/node nearby
- SCLK/MOSI/MISO short and ground referenced

MCP23008 zone

- SMD MCP23008 with local 100 nF decoupling
- short I2C traces
- encoder harness connector nearby if mechanically practical

Routing priorities

1. USB D+/D-
2. 3.3 V power integrity and ground plane
3. LCD PCLK/timing
4. RGB565 bus
5. BMI088 SPI
6. I2C / encoder / control
7. backlight PWM and slow signals

EMI/noise separation

Keep these noisy elements away from the IMU path:

- TPS62162 switch node and inductor
- TPS61169 board/wiring
- LCD PCLK
- high-current backlight loops

Ground-stitch around noisy zones while respecting the antenna keepout.

Connector philosophy

Flight-development assembly must use locking or positively retained connectors. No Dupont jumpers should remain in the assembled instrument.

Revision A interfaces:

- J1 USB-C
- J2 40-pin LCD FFC
- J3 BMI088 harness/carrier connector
- J4 rotary encoder harness
- J5 TPS61169 plug-in backlight-driver interface

Test points

Provide labeled access to at least:

- +5V_USB
- +3V3_SYS
- GND
- EN
- GPIO0/BOOT
- GPIO44 spare/debug
- regulator PG
- SPI SCLK/MOSI/MISO
- BMI088 ACC_CS
- BMI088 GYRO_CS
- I2C SDA/SCL
- LCD PCLK
- backlight PWM

High-speed test pads should be compact and avoid long stubs.

Revision-A exclusions

Do **not** integrate these yet:

- bare TPS61169 LED driver
- BMI088 sensor IC directly on the main PCB
- aircraft 12 V input/transient protection (production requirement now defined; still excluded from Revision A bench PCB)
- GNSS (production architecture must allow active/powered GNSS and USB GNSS receiver options; receiver not yet selected)
- protected external avionics interfaces (production PCB to reserve resources/footprints for RS-232 and optional RS-485/CAN)
- ambient light sensor

Those can follow once the display, USB, regulator and AHRS paths are proven.

Fabrication review checklist

Before releasing Gerbers:

- run ERC and DRC
- verify every WROOM pad against the Espressif footprint
- verify antenna keepout against current Espressif guidance
- verify USB-C footprint and CC orientation
- verify 3.3 V regulator layout against TI recommended placement
- verify 40-pin FFC pin 1 orientation against the Newhaven display
- verify board diameter and 60 mm PCD against printed fit gauge
- verify USB-C connector aligns with rear-cover slot
- verify no component collision with rear-cover standoffs, BMI088 cradle or display carrier
- inspect all strapping pins for unintended pull-ups/pull-downs
- inspect GPIO35–37 remain free from internal-memory conflicts by confirming the exact N16R2 module

The first PCB should be treated as a bench prototype until electrical, thermal, vibration and failure-mode testing are complete.

Production-interface reservations (post-Revision-A)

Revision A remains a bench-development PCB. The eventual production carrier should reserve routing, GPIO/UART resources and board area for:

- nominal 12 V aircraft input with reverse-polarity, transient/surge, filtering and DC/DC conversion;
- separate regulated 5 V and 3.3 V rails;
- GNSS connection supporting the selected active/powered GNSS solution and/or USB GNSS receiver;
- at least one protected RS-232 RX/TX channel for external avionics data;
- optional unpopulated RS-485 and CAN transceiver footprints;
- separate UART allocation for GNSS and external avionics;
- USB-C service/programming;
- internal low-level I2C/GPIO service access rather than exposing unprotected MCU GPIO to aircraft wiring.

A future PilotAware/traffic integration should normally be a **listener/data-consumer** arrangement. Do not place the EFIS in series with, or make it necessary for, an existing transponder/control-head data path.

Production mechanical direction

The current enclosure direction is rear-of-binnacle installation with panel bolts inserted from the front into recessed stainless captive nuts in the enclosure. Minimise enclosure assembly fasteners and use small cap-head Allen hardware where practical. Exact hole pattern, thread size, connector family, enclosure depth and panel geometry remain subject to physical prototype validation.

hardware/schematics/CARRIER_PCB_SCHEMATIC.md

Repository source: hardware/schematics/CARRIER_PCB_SCHEMATIC.md

ESP32-S3 Carrier PCB — schematic definition

This document is the authoritative electrical definition for the first custom carrier PCB used by the ESP32 Artificial Horizon project.

> **Role:** supplementary/non-primary flight-development instrument. This PCB is not certified avionics hardware. It must be bench-tested, thermally tested, vibration-tested and reviewed before aircraft installation.

1. Processor module

U1 — Espressif ESP32-S3-WROOM-1-N16R2

- 16 MB Quad SPI flash
- 2 MB Quad SPI PSRAM
- 3.3 V supply
- standard -40 to +85 °C module class
- 18.0 × 25.5 × approximately 3.1 mm
- integrated PCB antenna

This module is used instead of an ESP32-S3 DevKit. The carrier therefore includes its own USB-C, 3.3 V regulator, reset/boot circuitry and connectors.

The N16R2 is deliberately retained because its Quad PSRAM leaves GPIO35–37 available. Octal-PSRAM R8/R16 variants consume GPIO33–37 and are not compatible with the present pin map.

2. Board mechanical target

First PCB target:

- approximately **68 mm circular PCB**
- four M2.5 mounting holes on **60 mm PCD**
- 1.6 mm FR-4 starting thickness
- four-layer stack preferred
- module antenna at the 12-o'clock edge
- no copper, traces, ground plane, fasteners or metal directly beneath/in front of the module antenna area
- create an edge cutout/keepout beneath the antenna section if needed to satisfy Espressif antenna guidance while remaining inside the instrument body

Recommended four-layer stack:

1. Top — components and signals
2. Inner 1 — uninterrupted GND plane
3. Inner 2 — power plus limited signals
4. Bottom — limited signals / ground fill

The 18 MHz RGB bus should remain short, referenced to continuous ground and routed away from the BMI088 and antenna region.

3. 5 V input and USB-C

J1 — USB-C receptacle, USB 2.0 device / power sink

Connections:

- VBUS -> +5V_USB
- GND -> GND
- D- -> USB protection/series network -> GPIO19
- D+ -> USB protection/series network -> GPIO20
- CC1 -> 5.1 k Ω to GND
- CC2 -> 5.1 k Ω to GND
- shield -> chassis/ground strategy to be finalized after EMI testing; prototype may use direct GND through provisioned 0 Ω / RC footprint

Do not route SuperSpeed pairs; this design is USB 2.0 full-speed only.

USB signal conditioning

Provision:

- **R_USB_DM = 22 Ω** series between GPIO19 and J1 D-
- **R_USB_DP = 22 Ω** series between GPIO20 and J1 D+
- optional DNP capacitor footprint from each USB data line to GND adjacent to U1, as recommended by Espressif
- low-capacitance USB ESD array adjacent to J1

Keep D+/D- short, parallel, length-matched and away from the RGB clock and switching regulator node.

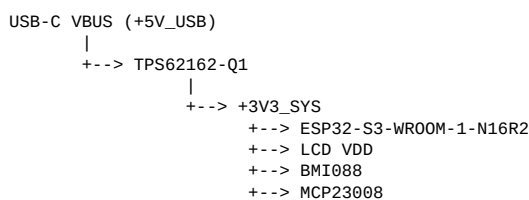
4. 3.3 V regulator

U2 — TI TPS62162-Q1 fixed 3.3 V synchronous buck

Reference choice:

- 3–17 V input
- fixed 3.3 V output
- 1 A continuous rating
- AEC-Q100 automotive-qualified component
- 2.25 MHz-class switching architecture
- power-good output available

Prototype power path:



Starting external components, subject to final TI layout review:

- input ceramic: 10 μ F X7R/X5R plus 100 nF high-frequency bypass
- inductor: 2.2 μ H, saturation/current rating at least 1.5 A preferred
- output ceramic: 22 μ F X7R/X5R plus local 100 nF bypassing at loads
- EN tied high to +5V_USB through provisioned resistor; test pad retained

- PG pulled up to +3V3_SYS and routed to a test pad; firmware monitoring is optional

Place the regulator, inductor and input/output capacitors as a tight cluster. Keep the SW node very small and physically distant from the BMI088 cradle and SPI sensor traces.

5. ESP32 power and reset

U1 supply

- U1 3V3 -> +3V3_SYS
- local decoupling adjacent to module supply entry: 10 μ F + 1 μ F + 100 nF ceramic provision
- all module grounds and exposed ground pad -> solid GND plane with stitching vias

EN / reset

EN must not float.

Starting reset network:

- 10 k Ω pull-up from EN to +3V3_SYS
- 1 μ F EN-to-GND capacitor starting value
- momentary **RESET** switch from EN to GND
- EN test pad

Final RC values must remain compatible with Espressif power-up/reset timing guidance.

BOOT

GPIO0 is reserved for boot mode:

- 10 k Ω pull-up to +3V3_SYS
- momentary **BOOT** switch from GPIO0 to GND
- GPIO0 test pad

Do not attach operational loads to GPIO0, GPIO3, GPIO45 or GPIO46.

6. Display connector

J2 — Molex 54104-4031 or verified 40-pin 0.5 mm FFC equivalent

Panel: Newhaven NHD-2.1-480480AF-ASXP.

The final PCB routes the display directly; the NHD-FFC40 adapter is bench-only.

Power and straps

- pin 1 LED_K -> TPS61169 LED output return
- pin 2 LED_A -> TPS61169 boosted LED output
- pin 3 VDD -> +3V3_SYS
- pin 4 GND -> GND
- pins 5/6 DN0/DP0 -> NC in RGB mode
- pin 7 GND -> GND
- pins 8/9 CN/CP -> follow Newhaven RGB-mode recommendation; no unverified use

- pin 10 GND -> GND
- pin 38 IM0 -> GND
- pin 39 IM1 -> +3V3_SYS
- pin 40 IM2 -> GND

RGB/timing

- pin 11 VS <- GPIO39
- pin 12 HS <- GPIO38
- pin 13 PCLK <- GPIO1
- pin 14 DE <- GPIO2
- pin 15 B0 -> GND
- pins 16–20 B1..B5 <- GPIO4..GPIO8
- pins 21–26 G0..G5 <- GPIO9..GPIO14
- pin 27 R0 -> GND
- pins 28–32 R1..R5 <- GPIO15,16,17,18,21

ST7701S configuration interface

- pin 33 RESETX <- MCP23008 GP1
- pin 34 CSX <- MCP23008 GP0
- pin 35 SCL <- GPIO36
- pin 36 DCX -> DNP/test pad; not required for selected 9-bit 3-wire mode
- pin 37 SDA <- GPIO35

Route RGB data/timing as a compact bus with continuous ground reference. Add optional 22–33 Ω source-series resistor footprints on PCLK and, if signal integrity testing shows need, on other high-edge-rate RGB lines.

7. BMI088 interface

J3 — dedicated BMI088 Shuttle Board connector/carrier interface

Use SPI and separate chip selects:

- +3V3_SYS
- GND
- SCLK <- GPIO36
- MOSI <- GPIO35
- MISO -> GPIO37
- ACC_CS <- GPIO40
- GYRO_CS <- GPIO41

The LCD ST7701S configuration serial bus shares SCLK/MOSI only. During LCD initialization both BMI088 CS lines remain high.

Provide ground adjacent to the SPI signals on the connector where practical.

Keep the BMI088 connector and traces away from:

- TPS62162 switch node/inductor
- TPS61169 backlight switching node
- LCD PCLK trace

- ESP32 antenna zone

8. MCP23008 low-speed GPIO

U3 — Microchip MCP23008

Carrier PCB implementation should use an SMD package rather than the prototype PDIP part.

Connections:

- VDD -> +3V3_SYS
- VSS -> GND
- SDA <-> GPIO47
- SCL <- GPIO48
- INT -> GPIO43
- A0/A1/A2 -> GND for address 0x20
- RESET -> +3V3_SYS through pull-up; provision test/reset pad

Add:

- 4.7 kΩ starting pull-ups from SDA and SCL to +3V3_SYS
- 100 nF local decoupling

GPIO allocation:

- GP0 -> LCD CSX
- GP1 -> LCD RESETX
- GP2 -> encoder A
- GP3 -> encoder B
- GP4 -> encoder push
- GP5–GP7 -> spare/test pads

9. Rotary encoder connector

J4 — Bourns PEC09-class front encoder harness

Provide:

- +3V3_SYS if required for future illuminated/control variant; otherwise NC/DNP
- GND/common
- ENC_A -> MCP23008 GP2
- ENC_B -> MCP23008 GP3
- ENC_SW -> MCP23008 GP4

Use a locking connector rather than loose Dupont pins for the flight-development article.

Debounce will primarily be firmware-controlled, but optional 1 nF–10 nF capacitor footprints may be provisioned after bench testing.

10. Backlight driver

Prototype retains the **Adafruit TPS61169 PID 6354** board as a plug-in module; the first custom ESP32 carrier does not yet absorb the bare TPS61169 IC.

J5 — backlight-driver interface

- +5V_USB
- GND
- PWM <- GPIO42
- LED_A to display FFC pin 2
- LED_K to display FFC pin 1 as required by the driver/output topology

The backlight driver must not draw LED current from the ESP32 3.3 V rail.

A later PCB revision may integrate the bare LED-driver circuit once display proof-of-life and thermal testing are complete.

11. Programming/debug

Primary programming path: native USB on GPIO19/20.

Also provide compact test pads for:

- 3V3
- GND
- EN
- GPIO0
- TXD0 / GPIO43 only if remapped; current GPIO43 is used by MCP23008 INT, so do not assume UART0 pins remain free
- GPIO44 spare/debug
- regulator PG

For early bring-up, add a small 1×4 or Tag-Connect-style debug footprint if space permits, but keep it DNP in the final assembly.

12. Frozen GPIO map

Function	GPIO
---	---
LCD D0..D4 / B1..B5	4,5,6,7,8
LCD D5..D10 / G0..G5	9,10,11,12,13,14
LCD D11..D15 / R1..R5	15,16,17,18,21
LCD PCLK	1
LCD DE	2
LCD HSYNC	38
LCD VSYNC	39
Shared SPI MOSI	35
Shared SPI SCLK	36
BMI088 MISO	37
BMI088 ACC_CS	40
BMI088 GYRO_CS	41
Backlight PWM	42
MCP23008 SDA	47
MCP23008 SCL	48
MCP23008 INT	43
Spare/debug	44
Native USB D-	19
Native USB D+	20

Reserved strapping pins: GPIO0, GPIO3, GPIO45, GPIO46.

13. PCB layout priorities

Priority order:

1. Respect the ESP32 module antenna keepout and board-edge placement.
2. Keep a continuous GND plane under high-speed digital routing except the antenna keepout.
3. Keep the TPS62162 switch node compact and away from the IMU.
4. Keep TPS61169/backlight switching currents away from the IMU and USB.
5. Route USB D+/D- first as a clean differential pair.
6. Route PCLK and RGB bus short and direct to J2.
7. Route BMI088 SPI away from PCLK and power-switching nodes.
8. Stitch ground around board perimeter and noisy zones without violating antenna keepout.
9. Provide accessible test pads without creating long stubs on high-speed nets.

14. First-revision build strategy

Revision A should deliberately remain conservative:

- integrate ESP32-S3 module, USB-C, 3.3 V buck, MCP23008, display FFC and connectors
- retain plug-in Adafruit TPS61169 for the backlight
- retain external Bosch BMI088 Shuttle Board in its rigid mechanical cradle
- retain external front encoder via locking harness

This reduces the number of new circuits that must work simultaneously on the first custom PCB.

15. Bring-up order

1. Inspect bare board for shorts and correct module orientation.
2. Power from current-limited 5 V supply with U1 unpopulated if using staged assembly.
3. Verify +3V3_SYS = 3.3 V and regulator PG.
4. Populate/verify ESP32 module and native USB enumeration.
5. Verify RESET and BOOT controls.
6. Verify MCP23008 at address 0x20.
7. Verify display logic supply and ST7701S reset/configuration.
8. Enable RGB output and static test pattern.
9. Connect BMI088 and verify both SPI devices independently.
10. Connect backlight driver last and raise brightness gradually while monitoring temperature/current.

16. References

Use the latest manufacturer documentation before fabrication:

- Espressif ESP32-S3-WROOM-1 / WROOM-1U datasheet
- Espressif ESP32-S3 Hardware Design Guidelines
- Newhaven NHD-2.1-480480AF-ASXP datasheet
- Bosch BMI088 / Shuttle Board documentation

- TI TPS62162-Q1 datasheet
- Microchip MCP23008 datasheet

Any conflict between this project document and a current manufacturer datasheet must be resolved in favor of the manufacturer datasheet and then reflected back into this repository.

license-signer/BUILD-AND-DEPLOY.md

Repository source: license-signer/BUILD-AND-DEPLOY.md

Licence Signer — Complete Build and Deployment Guide

This is the authoritative build/run procedure for the private efis - license-signer Docker service on the Synology NAS.

Current status: source complete; Synology runtime validation pending.

1. Security boundary

The signer is an internal service. **Do not create a Synology reverse-proxy rule or router port-forward for port 8092.** It binds only to 127.0.0.1:8092.

The Ed25519 private key and service token are secrets. They must never be committed, pasted into documentation, printed into logs, baked into the image or exposed by the public OTA service.

Development keys created by this guide are for integration/testing only. Production signing requires separate controlled key custody.

2. Repository assets

- license-signer/Dockerfile — Python 3.13 non-root image.
- license-signer/compose.yml — hardened Compose service and private-key mount.
- license-signer/requirements.txt — pinned Python dependencies.
- license-signer/app/main.py — FastAPI Ed25519/deterministic-CBOR signer.
- license-signer/.env.example — non-secret configuration template.
- license-signer/.gitignore — excludes environment/key material.
- license-signer/scripts/generate-dev-key.sh — one-time disposable development key generator.
- scripts/build-license-signer.sh — repository-level build/start/health helper.

3. Pull on Synology

From the existing repository checkout:

```
cd /volume1/docker/ESP32-EFIS
git status
git pull --ff-only
```

Do not discard local changes if git status is not clean; reconcile them before pulling.

4. Create signer configuration

```
cd /volume1/docker/ESP32-EFIS/license-signer
cp .env.example .env
```

Edit .env. Replace CHANGE-ME with a long random service-to-service token. Do not display that token in chat, screenshots or logs.

Keep:

```
SIGNER_KEY_ID=lic-2026-01
SIGNER_PRIVATE_KEY_FILE=/run/secrets/efis_license_ed25519.pem
```

The .env file is Git-ignored.

5. Generate DEVELOPMENT signing key

Run once:

```
cd /volume1/docker/ESP32-EFIS/license-signer
chmod +x scripts/generate-dev-key.sh
./scripts/generate-dev-key.sh
```

Expected private/public files are under `license-signer/secrets/`, which is Git-ignored.

The script refuses to overwrite an existing private key. Never solve that refusal by deleting a key unless replacement is deliberate and its consequences are understood.

6. Validate configuration without exposing secrets

```
sudo docker compose config --quiet
```

Do not paste ordinary `docker compose config` output because resolved environment values may reveal secrets.

7. Build

```
sudo docker compose build signer
```

A successful image build does not yet prove signing works.

8. Start/recreate

```
sudo docker compose up -d signer
sudo docker compose ps
```

Expected container: `esp32-efis-license-signer`, running, with host binding `127.0.0.1:8092`.

9. Health test

```
curl -fsS http://127.0.0.1:8092/healthz
echo
```

Expected fields include `status: ok`, service name, configured key ID and `key_file_present: true`.

The health endpoint deliberately does not expose private key material or the service token.

10. Check isolation

Confirm only loopback is published:

```
sudo docker port esp32-efis-license-signer
```

Expected host address is `127.0.0.1`, not `0.0.0.0`.

There must be no public reverse proxy or router rule to 8092.

11. Logs

```
sudo docker compose logs --tail=100 signer
```

Logs must not contain the API token, private key or complete sensitive request credentials.

12. Stop/restart

```
sudo docker compose restart signer
sudo docker compose stop signer
sudo docker compose up -d signer
```

Normal restart must retain the mounted key and key ID.

13. Rebuild after source changes

```
cd /volume1/docker/ESP32-EFIS
git pull --ff-only
cd license-signer
sudo docker compose config --quiet
sudo docker compose build signer
sudo docker compose up -d --force-recreate signer
curl -fsS http://127.0.0.1:8092/healthz
echo
```

14. Repository helper

The repository helper performs build/start/status/health from the repo:

```
cd /volume1/docker/ESP32-EFIS
chmod +x scripts/build-license-signer.sh
sudo ./scripts/build-license-signer.sh
```

If Docker permissions/environment handling on the Synology make the helper unsuitable, use the explicit commands above; they are authoritative.

15. Backup/recovery

For development, loss of the disposable signing key invalidates licences created with that key unless the corresponding public/private test setup is restored. Do not treat the development key as production material.

Before production, define and validate:

- protected signing-key generation;
- encrypted backup/recovery;
- access control;
- key rotation and kid lifecycle;
- compromise response;
- trusted-public-key rollout to EFIS firmware;
- dual-key overlap during rotation.

Never put a production private key in Git or a normal Docker image.

16. Validation gate before account integration

Do not mark the signer runtime VALIDATED until all of these pass:

1. Compose config validation.
2. Clean image build.
3. Container starts non-root.
4. Health endpoint passes and sees the key.
5. Host port is loopback only.
6. Authorized sign request returns an envelope.
7. Unauthorized request is rejected.
8. Returned CBOR decodes correctly.
9. Independent Ed25519 verification succeeds using the public key.
10. One-byte payload/signature corruption is rejected.
11. Wrong public key is rejected.
12. Container restart preserves ability to verify licences issued before restart.
13. Logs contain no signing key/token.

14. No public reverse proxy/router exposure exists.

The automated sign/decode/verify/corruption harness is the next implementation step; until then items 6–12 remain manual/pending.

17. Production gate

Before issuing customer licences:

- replace development key custody with the production-approved mechanism;
- replace/scaffold bearer-only trust with restricted service authentication/network policy;
- integrate only the authorized efis-account service;
- run cross-platform server/ESP32 test vectors;
- validate encrypted/protected NVS on the EFIS;
- test key rotation and recovery;
- complete security review.

See docs/LICENSING.md for the authoritative licensing lifecycle.

license-signer/README.md

Repository source: license-signer/README.md

EFIS Licence Signer

Private internal signing service implementing the adopted Ed25519 + deterministic-CBOR licence design.

Boundary

The service binds to loopback port 8092 by default and must not be exposed through the public reverse proxy/router. The private key is mounted at runtime and is never baked into the image or committed. The container runs non-root, read-only, drops capabilities and enables no-new-privileges.

The bearer token is initial service-to-service scaffolding. Production must additionally restrict network access and use hardened key custody.

For the complete Synology build, deployment, isolation, recovery and validation procedure see BUILD-AND-DEPLOY.md.

Development bring-up

Copy .env.example to .env, replace CHANGE-ME with a strong random service token, then:

```
cd license-signer
./scripts/generate-dev-key.sh
docker compose build
docker compose up -d
curl http://127.0.0.1:8092/healthz
```

Development keys are disposable and excluded from Git. Never use them as production signing keys.

POST /internal/v1/sign is for the authorized account-service backend only, after entitlement/ownership/payment checks.

Status: source implemented; Synology container build, test vectors, account-service integration, production key custody and ESP32 verification remain pending.

ota-server/README.md

Repository source: ota-server/README.md

ESP32 EFIS OTA image server

The OTA server has two deliberately separate services: a **public read-only firmware origin** and a **private firmware administration service**. GitHub remains the source/build history. The Docker server distributes only administrator-published EFIS application images.

Services and trust boundary

```
EFIS -> Internet HTTPS :8448 -> Synology reverse proxy -> 127.0.0.1:8180 -> nginx read-only origin
Admin -> LAN/VPN/SSH/private proxy -> 127.0.0.1:8090 -> OTA Admin -> public/ read-write
```

Never expose host ports 8180 or 8090 directly to the Internet. Public TLS terminates at Synology (or another trusted HTTPS proxy). The admin endpoint requires a private/authenticated access boundary.

Persistent repository

```
public/efis/
■■■ manifest.json           # exactly one PUBLISHED release
■■■ releases/<version>/*.bin # staged/published/archived binaries
■■■ release-metadata/<version>.json
```

The nginx container mounts this read-only. The admin container mounts it read/write.

Build and start on Synology

The repository includes compose .yml for the complete two-container stack. Run these commands from an SSH session on the Synology after cloning/pulling the repository:

```
cd ESP32-EFIS/ota-server
cp .env.example .env           # first deployment only
openssl rand -hex 32           # copy result into ADMIN_SESSION_SECRET in .env
# edit OTA_PUBLIC_BASE_URL in .env to the final HTTPS update hostname

sudo docker compose config
sudo docker compose build --pull
sudo docker compose up -d
sudo docker compose ps
curl -fsS http://127.0.0.1:8180/healthz
curl -fsS http://127.0.0.1:8090/healthz
```

Expected local health responses are ok from the read-only origin and JSON containing status: ok from the admin service. Both published ports are deliberately bound to 127.0.0.1; do not change them to public interfaces. Synology reverse proxy should expose only the read-only origin through HTTPS.

Set OTA_PUBLIC_BASE_URL to the stable public HTTPS hostname and generate ADMIN_SESSION_SECRET with openssl rand -hex 32. Do not commit the real .env.

Rebuild from scratch and functional test

The deployment is reproducible from a clean Git checkout. Preserve the Synology .env separately because it contains the private admin session secret and is deliberately not committed.

Fresh deployment:

```
git clone <repository-url> ESP32-EFIS
cd ESP32-EFIS/ota-server
cp .env.example .env
openssl rand -hex 32
# Put that value in ADMIN_SESSION_SECRET and verify:
# OTA_PUBLIC_BASE_URL=https://granvillehouse.synology.me:8448

sudo docker compose config --quiet
sudo docker compose build --pull
sudo docker compose up -d
```

```
sudo docker compose ps
```

For an existing checkout, pull the current source and run the repeatable harness:

```
cd /volume1/docker/ESP32-EFIS
git pull
cd ota-server
chmod +x scripts/rebuild-and-test.sh
./scripts/rebuild-and-test.sh
```

The runner validates Compose, rebuilds both images, recreates the stack, checks both local health endpoints, runs the isolated admin functional harness, and fetches the locally served public manifest.

The Python harness uses a temporary repository rather than `public/`. It proves that the dashboard renders, upload creates a staged release, staging leaves the published manifest unchanged, the staged binary hash is correct, Publish atomically changes version/build/image URL/SHA-256, and the current published release cannot be deleted. Test version `9.9.9-test` never enters the production repository.

A successful run ends with:

```
ALL OTA ADMIN FUNCTIONAL TESTS PASSED
PASS: rebuild, local health and isolated OTA admin workflow completed.
```

This harness validates server-side release mechanics only. It does not prove Synology external routing/TLS, ESP32 download, A/B flashing, activation, first-boot confirmation or rollback.

Stage a newly built EFIS binary

Staging and publishing are separate. The historical helper filename `publish-release.sh` is retained for compatibility, but it now stages only and prints `STAGED ONLY — NOT PUBLISHED`.

When the binary is already on the Synology/repository filesystem:

```
cd ESP32-EFIS/ota-server
./scripts/publish-release.sh 0.4.2 52 ../firmware/build/esp32_efis.bin 0.4.0 "Release notes"
```

This copies the binary to a versioned directory, calculates SHA-256 and creates per-release metadata. It refuses to overwrite an existing version and **does not alter** `manifest.json`.

If the binary was built on the Mac but the Docker server runs on Synology, transfer it first (example):

```
scp ~/Documents/Xcode/ESP32-EFIS/firmware/build/esp32_efis.bin <nas-user>@<nas-host>:/tmp/esp32_efis.bin
ssh <nas-user>@<nas-host>
cd <path-to>/ESP32-EFIS/ota-server
./scripts/publish-release.sh 0.4.2 52 /tmp/esp32_efis.bin 0.4.0 "Release notes"
```

The exact command supplied for each future EFIS binary should substitute the real version, build, binary path and minimum version rather than copying these example numbers.

Publish for EFIS consumption

Open the **private OTA Admin dashboard** through the Synology LAN-only HTTPS reverse proxy on source port **8447**, forwarding to `http://127.0.0.1:8090`. Review the staged version/build/hash/release notes, then select **Publish**. Publish atomically changes the public manifest. Upload/stage alone never makes a release consumable by EFIS units.

A retained older release may be republished if the advertised release must be withdrawn. The currently published release cannot be deleted.

Public deployment

The validated public endpoint is `https://granvillehouse.synology.me:8448`. Synology reverse proxy forwards HTTPS :8448 to `http://127.0.0.1:8180`; the router forwards TCP 8448 to the Synology. Do not expose the private admin port 8447. If behind CGNAT use a deliberately configured trusted HTTPS tunnel/reverse proxy. Do not expose DSM/Docker administration.

Field path:

```
ESP32 EFIS -> iPhone Personal Hotspot -> Internet -> HTTPS :8448 -> Synology -> nginx OTA origin
```

The phone is a network gateway only. Hotspot credentials stay in EFIS NVS and are never sent to the OTA server.

Verify public consumption

```
curl -fsS https://granvillehouse.synology.me:8448/healthz
curl -fsS https://granvillehouse.synology.me:8448/efis/manifest.json
```

Test from outside the LAN as well. A published release may be automatically **downloaded** by an EFIS configured for auto-download, but activation/reboot always requires explicit **ACTIVATE** & **REBOOT** on the instrument.

Diagnostics

```
sudo docker compose ps
sudo docker compose logs --tail=100 efis-ota
sudo docker compose logs --tail=100 efis-ota-admin
docker inspect --format='{{json .State.Health}}' esp32-efis-ota
curl -v http://127.0.0.1:8180/healthz
curl -v http://127.0.0.1:8090/healthz
```

Security

Do not put GitHub tokens, signing private keys or Wi-Fi credentials in either image. SHA-256 is an integrity check, not publisher authentication. Production firmware still requires signed-image verification. The signing private key remains outside repository, containers and EFIS filesystem. Public manifest/binaries may be readable; secrecy is not a security control.

The current admin application relies on LAN/VPN/private-proxy access as its authentication boundary. Before any public multi-user administration, add proper authentication, CSRF protection, audit records, upload limits and preferably signed-image verification before Publish.

Validation status

IMPLEMENTED / SYNOLOGY RUNTIME + PRIVATE ADMIN + PUBLIC OTA NETWORK PATH VALIDATED. Static origin, private admin container, deployable Compose stack, staged release metadata, explicit Publish and stage-only CLI helper are in source control. Both images have been built and started on the Synology. Local health checks pass on the read-only origin (127.0.0.1:8180) and admin (127.0.0.1:8090). The private Synology HTTPS reverse proxy on source port 8447 has been verified to return the OTA Admin dashboard. The public read-only endpoint <https://granvillehouse.synology.me:8448> has been configured through Synology reverse proxy/router forwarding and externally health-checked. Physical ESP32 OTA validation remains pending. Real ESP32 HTTPS download, A/B flash, activation and rollback remain hardware-unvalidated. See [../docs/OTA_IMAGE_ADMIN.md](#), [../docs/OTA_USER_SCENARIO.md](#), [../docs/PHONE_NETWORK_AND_PUBLIC_OTA.md](#) and [../docs/REMOTE_UPDATES.md](#).

Release sets and offline mobile delivery

The distribution architecture now treats a signed multi-node release set as the deployment unit. Public serving reserves `/efis/release-set.json`; the release-set contains Horizon/EIU/future-node compatibility, capability and sequencing information and is what mobile clients cache for offline maintenance. Horizon remains responsible for local verification and AEF-CAN node sequencing.

`admin/release_resolver.py` is the initial generic compatibility/order resolver. CI exercises resolver, offline-phone and existing OTA simulator tests. The EIU CAN transport simulator now implements the adopted 7-byte FW_DATA, 4 KiB block CRC/ACK and 64 KiB append-only durable checkpoint journal model, including power-loss resume and corrupt/incomplete checkpoint rejection. It remains a software model; physical CAN timing, transceiver behaviour and real flash/bootloader validation are still required.

ota-server/release-set.example.json

Repository source: ota-server/release-set.example.json

```
{
  "schema": "aef-release-set-v1",
  "release": "2026.10.1-example",
  "artifacts": [
    {
      "node_type": "eiu",
      "firmware": "1.9.0",
      "hardware": "E1",
      "aef_can": {"major": 1, "minor": 3},
      "provides": ["ENGINE_DATA_V1", "ENGINE_DATA_V2", "OTA_V1"],
      "requires": []
    },
    {
      "node_type": "horizon",
      "firmware": "2.5.0",
      "hardware": "H1",
      "aef_can": {"major": 1, "minor": 3},
      "provides": ["ENGINE_DATA_V1", "ENGINE_DATA_V2", "OTA_V1"],
      "requires": [
        {"node_type": "eiu", "aef_can_major": 1, "min_minor": 3, "capabilities": ["ENGINE_DATA_V2"]}
      ]
    }
  ],
  "policy": {
    "require_safe_intermediate_states": true,
    "require_rollback_compatibility": true
  }
}
```

protocol/aef-can.yaml

Repository source: protocol/aef-can.yaml

protocol:

name: AEF-CAN

long_name: Aircraft Experimental Flight CAN

version:

major: 1

minor: 0

status: adopted-v1

role: supplementary-non-primary

transport:

baseline: classical-can

bitrate: 500000

identifier_bits: 11

byte_order: little

max_payload_bytes: 8

can_fd_required: false

units:

temperature:

wire_unit: degC

scale: 0.1

pressure:

wire_unit: bar

scale: 0.01

voltage:

wire_unit: V

scale: 0.01

current:

wire_unit: A

scale: 0.01

rotational_speed:

wire_unit: rpm

scale: 1

altitude:

wire_unit: m

scale: 0.1

speed:

wire_unit: m/s

scale: 0.01

fuel_quantity:

wire_unit: L

scale: 0.01

id_ranges:

- { start: 0x000, end: 0x07F, purpose: network_control }
- { start: 0x080, end: 0x0FF, purpose: node_management }
- { start: 0x100, end: 0x17F, purpose: engine }
- { start: 0x180, end: 0x1FF, purpose: electrical }
- { start: 0x200, end: 0x27F, purpose: air_data }
- { start: 0x280, end: 0x2FF, purpose: navigation_gnss }
- { start: 0x300, end: 0x37F, purpose: attitude_motion }
- { start: 0x380, end: 0x3FF, purpose: fuel }
- { start: 0x400, end: 0x47F, purpose: environmental }
- { start: 0x480, end: 0x4FF, purpose: aircraft_state }
- { start: 0x500, end: 0x57F, purpose: alerts_events }
- { start: 0x580, end: 0x5BF, purpose: configuration_service }\n - { start: 0x5C0, end: 0x5FF, purpose: firmware_maintenance }
- { start: 0x600, end: 0x67F, purpose: diagnostics }
- { start: 0x680, end: 0x6FF, purpose: logging_time }
- { start: 0x700, end: 0x77F, purpose: experimental }
- { start: 0x780, end: 0x7FF, purpose: reserved }

sensor_states:

VALID: 0

NOT_INSTALLED: 1

OPEN_CIRCUIT: 2

SHORT_CIRCUIT: 3

OUT_OF_RANGE: 4

STALE: 5

CALIBRATING: 6

SENSOR_FAULT: 7

sensor_ids:

RPM: 1

OIL_PRESSURE: 2

OIL_TEMPERATURE: 3

CHT_1: 16

CHT_2: 17

EGT_1: 32

EGT_2: 33

EGT_3: 34

EGT_4: 35

COOLANT_TEMPERATURE: 48

SUPPLY_VOLTAGE: 64

SUPPLY_CURRENT: 65

INTERNAL_TEMPERATURE: 80

capabilities:

RPM: 0

CHT: 1

EGT: 2

OIL_TEMPERATURE: 3

OIL_PRESSURE: 4

COOLANT_TEMPERATURE: 5

VOLTAGE: 6

CURRENT: 7

DIAGNOSTICS: 8

messages:

- name: NODE_ANNOUNCE_V1

id: 0x090

version: 1

rate_hz: 1

transmit: boot_and_periodic

timeout_ms: 3000

fields:

- { name: protocol_major, offset: 0, type: uint8 }
- { name: protocol_minor, offset: 1, type: uint8 }
- { name: node_type, offset: 2, type: uint8 }
- { name: node_instance, offset: 3, type: uint8 }
- { name: capabilities, offset: 4, type: uint32, note: bitmask }

- name: NODE_HEARTBEAT_V1

id: 0x091

version: 1

rate_hz: 1

timeout_ms: 3000

fields:

- { name: node_type, offset: 0, type: uint8 }
- { name: node_instance, offset: 1, type: uint8 }
- { name: protocol_major, offset: 2, type: uint8 }
- { name: protocol_minor, offset: 3, type: uint8 }
- { name: uptime_s_mod, offset: 4, type: uint32, unit: s }

- name: ENGINE_FAST_V1

id: 0x100

version: 1

rate_hz: 10

timeout_ms: 500

fields:

- { name: rpm, offset: 0, type: uint16, scale: 1, unit: rpm }
- { name: oil_pressure, offset: 2, type: uint16, scale: 0.01, unit: bar }
- { name: oil_temperature, offset: 4, type: int16, scale: 0.1, unit: degC }
- { name: reserved, offset: 6, type: uint16, required_value: 0 }

- name: CHT_V1

id: 0x101

version: 1

rate_hz: 5

timeout_ms: 1000

fields:

- { name: cht_1, offset: 0, type: int16, scale: 0.1, unit: degC }
- { name: cht_2, offset: 2, type: int16, scale: 0.1, unit: degC }
- { name: reserved, offset: 4, type: uint32, required_value: 0 }

- name: EGT_12_V1

id: 0x102

version: 1

rate_hz: 5

timeout_ms: 1000

fields:

- { name: egt_1, offset: 0, type: int16, scale: 0.1, unit: degC }
- { name: egt_2, offset: 2, type: int16, scale: 0.1, unit: degC }
- { name: reserved, offset: 4, type: uint32, required_value: 0 }

- name: EGT_34_V1

id: 0x103

version: 1

rate_hz: 5

timeout_ms: 1000

fields:

- { name: egt_3, offset: 0, type: int16, scale: 0.1, unit: degC }
- { name: egt_4, offset: 2, type: int16, scale: 0.1, unit: degC }
- { name: reserved, offset: 4, type: uint32, required_value: 0 }

- name: ENGINE_COOLING_V1

id: 0x104

version: 1

rate_hz: 5

timeout_ms: 1000

fields:

- { name: coolant_temperature, offset: 0, type: int16, scale: 0.1, unit: degC }
- { name: reserved, offset: 2, type: bytes, length: 6, required_value: 0 }

- name: EIU_ELECTRICAL_V1

id: 0x180

version: 1

rate_hz: 2

timeout_ms: 2000

fields:

- { name: supply_voltage, offset: 0, type: uint16, scale: 0.01, unit: V }

- { name: supply_current, offset: 2, type: int16, scale: 0.01, unit: A }
- { name: reserved, offset: 4, type: uint32, required_value: 0 }

• name: SENSOR_STATUS_V1

id: 0x600

version: 1

rate_hz: 2

timeout_ms: 2000

note: "Generic paged sensor-status frame. One frame describes up to four logical sensors; additional pages use the same CAN ID."

fields:

- { name: node_instance, offset: 0, type: uint8 }
- { name: page, offset: 1, type: uint8, note: "sensor indices page4 through page4+3" }
- { name: sensor_0_id, offset: 2, type: uint8 }
- { name: sensor_0_state, offset: 3, type: uint8, enum: sensor_states }
- { name: sensor_1_id, offset: 4, type: uint8 }
- { name: sensor_1_state, offset: 5, type: uint8, enum: sensor_states }
- { name: sensor_2_id, offset: 6, type: uint8 }
- { name: sensor_2_state, offset: 7, type: uint8, enum: sensor_states }

continuation:

note: "A fourth sensor is intentionally not packed into V1 because node/page provenance consumes two bytes. Pages repeat at the nominal status rate as required."

• name: EIU_HEALTH_V1

id: 0x601

version: 1

rate_hz: 1

timeout_ms: 3000

fields:

- { name: internal_temperature, offset: 0, type: int16, scale: 0.1, unit: degC }
- { name: reset_reason, offset: 2, type: uint8 }
- { name: health_flags, offset: 3, type: uint8, note: bitmask }
- { name: tx_error_count, offset: 4, type: uint16 }
- { name: rx_error_count, offset: 6, type: uint16 }

• name: FW_CONTROL_V1

id: 0x5C0

version: 1

rate_hz: event

fields:

- { name: node_instance, offset: 0, type: uint8 }
- { name: operation, offset: 1, type: uint8, note: "1=BEGIN, 2=FINISH, 3=ACTIVATE, 4=QUERY_TRANSFER" }
- { name: transfer_id, offset: 2, type: uint16 }
- { name: argument, offset: 4, type: uint32, note: "operation-specific; reserved zero unless defined" }

• name: FW_META_V1

id: 0x5C1

version: 1

rate_hz: event

note: "Paged metadata. Complete product/hardware/version/size/digest/signature metadata is transferred before DATA; exact page catalogue remains additive."

fields:

- { name: node_instance, offset: 0, type: uint8 }
- { name: page, offset: 1, type: uint8 }
- { name: data, offset: 2, type: bytes, length: 6 }
- name: FW_DATA_V1

id: 0x5C2

version: 1

rate_hz: transfer

note: "Adopted production transport: one-byte sequence plus seven firmware bytes. Sequence is local to the active 4 KiB transfer block; block CRC and final image verification provide higher-level integrity."

fields:

- { name: sequence, offset: 0, type: uint8, note: "frame sequence within current transfer block; wrap permitted with block byte count providing context" }
- { name: data, offset: 1, type: bytes, length: 7 }
- name: FW_STATUS_V1

id: 0x5C3

version: 1

rate_hz: event

fields:

- { name: node_instance, offset: 0, type: uint8 }
- { name: status, offset: 1, type: uint8, note: "1=READY, 2=ACK, 3=VERIFIED, 4=REJECTED, 5=REBOOT_PENDING, 6=CHECKPOINT, 7=RESUME, 8=NO_TRANSFER" }
- { name: sequence, offset: 2, type: uint16, note: "last accepted or next expected sequence according to status" }
- { name: detail, offset: 4, type: uint16, note: "machine-readable reason code; catalogue TBD" }
- { name: reserved, offset: 6, type: uint16, required_value: 0 }
- name: FW_CHECKPOINT_V1

id: 0x5C4

version: 1

rate_hz: event

note: "Durable resume checkpoint, nominally every 64 KiB. This is distinct from each 4 KiB CRC-verified transfer block. Offset is authoritative; percentage is presentation only."

fields:

- { name: transfer_id, offset: 0, type: uint16 }
- { name: committed_offset, offset: 2, type: uint32, unit: bytes }
- { name: checkpoint_integrity, offset: 6, type: uint16, note: "compact checkpoint-state integrity token; 4 KiB block CRC32 is carried by block-control/status exchange" }

compatibility:

rules:

- "Unknown message IDs are ignored."
- "Existing field meanings are never silently redefined."

- "Minor protocol revisions are additive/backward compatible."
- "Incompatible semantics require a new major version or new message definition."
- "Reserved transmitted bits/bytes are zero and ignored by receivers."
- "Firmware version is independent of protocol version."

protocol/golden-vectors.txt

Repository source: protocol/golden-vectors.txt

AEF-CAN V1 golden vectors

Format: CAN_ID DLC DATA_HEX ; interpretation

0x100 8 06137B00C4030000 ; rpm=4870 oil_pressure=1.23bar oil_temperature=96.4C
0x101 8 FC03B00400000000 ; cht_1=102.0C cht_2=120.0C
0x102 8 841E201F00000000 ; egt_1=781.2C egt_2=796.8C
0x103 8 BC1FC82000000000 ; egt_3=812.4C egt_4=839.2C
0x104 8 6603000000000000 ; coolant_temperature=87.0C
0x180 8 2A05FA0000000000 ; supply_voltage=13.22V supply_current=2.50A
0x600 8 0000010002000300 ; node=0 page=0 RPM/OIL_PRESSURE/OIL_TEMPERATURE all VALID
0x601 8 2C01000000000000 ; internal_temperature=30.0C reset_reason=0 health_flags=0 errors=0
0x091 8 0100010039300000 ; node_type=1 node=0 protocol=1.0 uptime=12345s

README.md

Repository source: README.md

ESP32 EFIS

Experimental **supplementary/non-primary** multifunction electronic flight instrument for a non-certified aircraft, based on ESP32-S3 and a single 2.1-inch round 480×480 high-brightness display.

> **Safety:** experimental development hardware/software. It is not a certified or approved primary flight instrument and must not be relied upon as the sole source of attitude, altitude, heading or position information.

Current development status — 16 September 2026

The software/emulator graphics baseline for Artificial Horizon/PFD, Altimeter and Compass has been accepted in deterministic Espressif QEMU at the real 480×480 framebuffer geometry. Physical hardware validation remains pending; compile or simulation results are not physical validation.

A new **adopted display requirement** adds GNSS position and receiver-reported horizontal accuracy to both the Horizon/PFD and Compass pages. The shared instrument data model and GNSS accuracy classifier are now present in firmware; final 480×480 renderer integration, GNSS receiver selection/acquisition, QEMU acceptance and hardware validation remain pending. See docs/GNSS_DISPLAY.md.

GNSS accuracy colour bands are: green ≤1 m, light green >1–3 m, yellow >3–10 m, orange >10–30 m, red >30 m; no fix/state/invalid is red with coordinates removed or obscured. These are project UI categories based on the receiver's horizontal accuracy estimate, not guaranteed error bounds or certification/integrity limits.

Artificial Horizon / PFD — EXISTING BASELINE ACCEPTED IN QEMU; GNSS OVERLAY PENDING ACCEPTANCE. Accepted attitude behavior remains unchanged. The new lower GNSS strip must not obscure the primary attitude presentation and has independent validity.

Altimeter — ACCEPTED IN QEMU. Existing classic three-pointer, digital altitude, QNH/Kollsman and failure behavior are unchanged.

Compass — EXISTING BASELINE ACCEPTED IN QEMU; GNSS OVERLAY PENDING ACCEPTANCE. Magnetic heading remains independent of GNSS. GNSS course is TRK, never HDG.

BMI088 software pipeline — IMPLEMENTED, COMPILE-VALIDATED, HARDWARE VALIDATION PENDING. Sensor-to-aircraft axis/sign mapping remains intentionally gated on physical commissioning.

BMP585 and RM3100 live acquisition — PLANNED / HARDWARE PENDING. Pressure-to-altitude calculation exists; physical acquisition and magnetic calibration/tilt-compensated heading remain hardware work.

GNSS acquisition — REQUIREMENT ADOPTED / RECEIVER NOT YET FROZEN. The project requires a source that provides valid fix state, WGS84 latitude/longitude and an explicit horizontal accuracy estimate. Satellite count alone must not drive the accuracy colour.

See docs/PROJECT_STATUS.md, docs/GNSS_DISPLAY.md, docs/SENSORS.md, docs/SIMULATION.md and docs/TESTING.md.

Instrument pages

The EFIS has three pages on **one physical round display**, selected by the PEC09 rotary/push control:

1. **Horizon/PFD** — BMI088-based AHRS, pitch/roll and validated auxiliary fields, plus GNSS latitude/longitude and colour-coded position accuracy when valid.
2. **Altimeter** — classic three-pointer presentation using the selected ported BMP585 static-pressure sensor and adjustable QNH.
3. **Compass** — rotating-card presentation using the remotely mounted PNI RM3100-CB absolute-heading source, plus GNSS latitude/longitude and colour-coded position accuracy when valid.

No missing sensor value is replaced by a plausible fallback. Invalid or stale data must be unmistakably invalid.

Hardware baseline

- Espressif **ESP32-S3-WROOM-1-N16R2** — 16 MB Quad flash / 2 MB Quad PSRAM
- project-specific 68 mm carrier PCB, parked pending physical-interface decisions
- TI **TPS62162-Q1** 3.3 V regulator
- Bosch **BMI088 Shuttle Board 3.0** IMU
- Adafruit ported **BMP585 PID 6413** pressure board
- **PNI RM3100-CB** remotely mounted three-axis magnetometer
- **GNSS receiver/interface: to be selected**; must provide explicit fix validity and horizontal accuracy estimate
- one Newhaven **NHD-2.1-480480AF-ASXP** 480×480 round IPS display
- Adafruit **TPS61169** backlight driver
- Microchip **MCP23008** GPIO expansion
- Bourns **PEC09-2320F-T0015** rotary encoder/push switch
- 5 V USB-C development power

See `BOM.md`, `docs/SENSORS.md`, `docs/HARDWARE.md` and `docs/WIRING.md` for authoritative detail. Procurement state does not imply physical validation.

Firmware and development environment

`firmware/` is an **ESP-IDF v5.4.4** project named `esp32_efis`. It contains the physical display/control path, fail-obvious data validity, NVS-backed UI settings, explicit synthetic bench simulation, QEMU display backend, BMI088 acquisition, the initial attitude estimator and the shared GNSS accuracy-quality classifier.

The GNSS classifier deliberately centralizes the adopted bands so Horizon and Compass cannot silently use different thresholds. The instrument data model carries latitude, longitude, horizontal accuracy, satellites used, fix type, valid and stale states. Receiver parsing/acquisition and final on-screen drawing remain separate pending implementation tasks.

On macOS:

```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
source scripts/efis-env.sh
zsh scripts/build-qemu.sh --clean
zsh scripts/run-qemu.sh
```

Physical hardware builds remain paused until hardware is available. A compile alone must never be recorded as physical validation.

Simulator and acceptance

`simulator/` contains the SwiftUI development simulator. QEMU remains authoritative for ESP32 RGB565 rendering. Both simulators must be extended with deterministic GNSS cases covering no fix, stale data and every accuracy colour band before the new overlay can be marked accepted. Synthetic coordinates must remain unmistakably simulation data.

Display baseline

There is **one** Newhaven 2.1-inch 480×480 display. Horizon/PFD, Altimeter and Compass are pages on that same display; they are not separate physical instruments. The panel uses RGB565 and 9-bit serial controller initialization with the existing 30 MHz RGB timing baseline.

GNSS information is deliberately a compact secondary strip. It must not compromise readability of attitude or heading, and real-panel legibility must be checked before the layout is frozen.

Enclosure

The 3 1/8-inch flight-development enclosure is for the single round display and integrated electronics. Physical fitting and final aircraft interfaces remain pending actual hardware/measurements. See docs/ENCLOSURE.md.

Development principles

The BMI088 is rigidly mounted with physically verified aircraft-axis alignment. Pressure, heading, attitude and GNSS have independent validity/freshness handling. GNSS TRK is never magnetic HDG; GS is never IAS. Receiver-reported accuracy is an estimate, not a guarantee, and poor/stale/no-fix position must fail visibly rather than freezing plausible coordinates.

Boot, maintenance and diagnostics — 26 September 2026

The production interaction model now starts with a **rotary/push-controlled boot menu**. **START** EFIS is the default selection; **FULL TEST** launches the electrical/component diagnostic harness; **FIRMWARE UPDATE** enters the maintenance networking/OTA workflow. Rotation changes selection and a short press selects. Long-press Back/Cancel remains subject to physical encoder validation.

Firmware update first opens an explicit Wi-Fi connection dialog. Saved SSID/credentials persist in ESP-IDF NVS; production units require encrypted NVS for credentials and provide **Forget network**. Wi-Fi failure never blocks startup of the known-good EFIS. See docs/OTA_USER_SCENARIO.md, docs/PHONE_NETWORK_AND_PUBLIC_OTA.md and docs/ELECTRICAL_TEST_HARNESS.md.

Persistent fault history

Detected software and hardware faults during normal operation are an adopted persistent-diagnostics requirement. A bounded, flash-wear-conscious non-volatile fault log will survive power cycles and be accessible through **FAULT LOG** on the rotary boot menu. It records stable fault identifiers and useful non-secret diagnostic context while immediate on-screen fail-obvious indications remain authoritative during operation. See docs/ELECTRICAL_TEST_HARNESS.md and docs/PROJECT_STATUS.md.

Product licensing

The production architecture now reserves a signed, offline-verifiable product-licence mechanism. Licences are generated by an authorised off-device tool, installed through maintenance provisioning and verified on the EFIS using a public key; the private signing key is never stored on the instrument. The rotary boot menu will expose **LICENSE** status/install/reset functions. Licence state persists across normal firmware updates, while reset is deliberate, logged and separate from Wi-Fi/factory reset. Licensing of optional functions must never compromise fail-obvious instrument behaviour.

Licence account service architecture — 26 September 2026

ADOPTED / DESIGN STAGED. Add a separate Docker service (working name `efis-account`) alongside OTA/admin services. It owns customer accounts, registered EFIS devices, licence entitlements and payment-provider linkage; it does not hold the licence-signing private key in the public web tier.

Device identity uses a provisioned immutable **EFIS Device ID / serial number** as the primary key. The ESP32 factory/eFuse base MAC may be recorded as a secondary hardware fingerprint and registration aid, but MAC address alone is not the licence identity because interface MACs can be derived/changed and MAC exposure is unnecessary for manual licensing.

Online boot-maintenance flow: **LICENSE** -> Connect Wi-Fi -> identify device -> authenticated account/licence endpoint -> fetch signed licence entitlement -> verify locally -> store in protected NVS -> continue offline. The EFIS must not send a user password; device authentication uses a provisioned device credential/challenge mechanism to be specified. Normal flight startup must not depend on account/payment/network availability.

Offline flow: show Device ID and short registration code/QR-capable text; user obtains a signed licence from the account portal on another device and enters/imports it manually or via USB service. The licence is verified locally with the embedded public verification key.

Payment integration is provider-adapter based. The account service creates checkout/customer-management requests and consumes verified payment webhooks; payment card data is handled by the payment provider, not stored by EFIS services. Payment status changes entitlements; a separate private signing worker/service generates signed licence payloads. Define explicit

grace/revocation policy before subscriptions are enabled.

Suggested containers: efis-account API/web portal; PostgreSQL account/device/entitlement store; private efis-license-signer with tightly restricted signing-key access; existing efis-ota and efis-ota-admin. Payment provider secrets live only in server-side secret storage.

Pressure scope — 26 September 2026

ASI/IAS capability has been removed from scope. The BMP585 uses the aircraft **STATIC** pressure connection for altitude/barometric pressure. The EFIS has no PITOT input or differential-pressure airspeed sensor.

Working product identity — 27 September 2026

The current development brand is **MicroSky Avionics** and the product name is **Horizon**. Customer-facing shorthand is **MicroSky Horizon**. These are working names pending formal commercial/trademark clearance; technical repository/service/device identifiers are not being renamed at this stage. See docs/BRANDING.md.

Customer support chatbot — 27 September 2026

A Dockerized **MicroSky Horizon Support** service is staged under support-service/. It retrieves approved product documentation, records customer interactions/feedback and builds a candidate Q&A queue. Customer conversations never become authoritative automatically: a human reviewer must validate candidates against current product documents before they enter the validated Q&A database. See docs/SUPPORT_SERVICE.md and support-service/README.md.

Server platform — 29 September 2026

The proposed server-side infrastructure baseline is a **£750-class, 64 GB RAM / 1 TB NVMe virtualisation host** running Proxmox VE, with separate production Docker, development/test and monitoring VMs plus an optional k3s lab. Existing Docker Compose services should migrate first without requiring Kubernetes; k3s is a deliberate later evaluation path. This platform supports OTA, accounts/licensing, Horizon Support and related services and is **not part of the flight hardware baseline**. See docs/SERVER_PLATFORM.md.

AEF-CAN aircraft data bus — 30 September 2026

ADOPTED / PROTOCOL STAGED. The project now defines **AEF-CAN (Aircraft Experimental Flight CAN)** as the extensible internal data bus for the proposed Engine Interface Unit (EIU), Horizon display and future aircraft modules. It is deliberately hardware-independent and describes aircraft measurements and validity rather than display presentation.

V1 uses 500 kbit/s Classical CAN with 11-bit identifiers. The current ESP32-S3 TWAI controller requires an external CAN transceiver and is Classical-CAN-only; future CAN FD hardware can be added without redefining the application-level measurement semantics.

The protocol reserves functional CAN-ID ranges for engine, electrical, air data, GNSS/navigation, attitude, fuel, aircraft state, alerts, configuration, diagnostics and logging. It defines canonical wire units, scaled integer encoding, explicit freshness/fault semantics, node discovery/capabilities and major/minor compatibility rules. Producers publish facts; consumers own display units, thresholds and colours.

protocol/aef-can.yaml is the machine-readable source of truth. The long-form rationale and implementation/testing rules are in docs/CAN-PROTOCOL.md. Future C/C++, Swift and Python codecs/test vectors should be generated or verified from the YAML to prevent firmware/simulator/documentation drift.

The AEF-CAN/EIU system remains **secondary, supplementary and non-certified**. No CAN value may remain silently presented as live after its freshness timeout.

simulator/README.md

Repository source: simulator/README.md

ESP32 EFIS iPhone/iPad Simulator

Native SwiftUI design/bench simulator for ESP32 EFIS. It is not a flight-navigation application; displayed flight data and OTA operations are synthetic/simulated.

Scope

QEMU remains authoritative for the actual 480×480 RGB565 instrument renderer. Swift provides interactive instrument controls plus software-only rehearsal of maintenance networking and OTA user behaviour.

The simulator currently provides three areas:

- **Instruments** — accepted Horizon/PFD, Altimeter and Compass representations, manual controls, AUTO FLIGHT, GNSS position/accuracy presentation, validity/failure injection, deterministic acceptance sequences and an automated full demonstration suitable for screen recording.
- **Network** — phone-hotspot SSID/password and HTTPS manifest configuration plus a simulated Phone → Internet → TLS/OTA-server connection test.
- **Software Update** — definitive staged-release consumption flow, persistent auto-download preference, manual download, ACTIVATE & REBOOT, first-boot verification and rollback simulation.

Every synthetic instrument display retains conspicuous simulation marking. The OTA page is explicitly labelled **OTA USER-FLOW SIMULATION**.

Full demonstration / video recording mode

The Instruments screen includes **RUN FULL DEMONSTRATION**. It automatically drives the single-display simulator through the principal instrument and validation functions so a clean demonstration can be recorded without manually operating every control.

The approximately 6 minute 25 second sequence demonstrates:

1. Horizon/PFD level presentation, pitch/roll movement, attitude failure and recovery.
2. Altimeter at representative altitudes, QNH change, >10,000-ft hatching, altitude failure and recovery.
3. Compass presentation, heading bug, rotating heading and heading failure/recovery.
4. GNSS latitude/longitude on Horizon and Compass through Excellent, Good, Fair, Weak, Poor, Stale and No Fix states. Coordinate colour follows the adopted reported-horizontal-accuracy bands; stale/no-fix does not leave plausible frozen coordinates.
5. A short automatic-flight segment.
6. The complete Horizon 12-state, Altimeter 14-state, Compass 12-state and GNSS 7-state acceptance suites.

While the demonstration is running, manual flight-data, settings, GNSS and failure controls are disabled to keep the recording deterministic. A status line shows elapsed/total demonstration time and the current section. **STOP FULL DEMONSTRATION** aborts the sequence. At completion the simulator returns to a valid Horizon/PFD state.

For a recording, start the simulator, open Instruments, begin the operating-system/Xcode Simulator screen recording, press **RUN FULL DEMONSTRATION**, and allow the sequence to complete. The red SIM marking remains visible throughout. The recording demonstrates the Swift simulator; it is not evidence of physical ESP32, sensor, GNSS or aircraft validation.

OTA behaviour represented

The Docker administrator, not the EFIS user, controls what release is **PUBLISHED**. The simulator assumes it discovers only that published manifest.

Automatically download next update defaults ON. In the production design this permits download/verification to the inactive OTA slot only while the EFIS is in the appropriate maintenance/network-update context. It never permits automatic activation.

With auto-download ON:

```
Check published release -> Download/verify -> UPDATE READY -> ACTIVATE & REBOOT -> self-test -> accept/rollback
```

With auto-download OFF:

```
Check -> UPDATE AVAILABLE -> Download -> verify -> UPDATE READY -> ACTIVATE & REBOOT -> self-test -> accept/rollback
```

The simulator includes Later and a deliberate first-boot-failure switch. No real network request, ESP32 flash write, boot partition change or reboot occurs.

Instrument synchronisation

The accepted instrument behaviour includes Horizon pitch/bank direction and fail-obvious ATT FAIL; classic three-pointer Altimeter with digital altitude, progressive >10,000-ft hatching, curved 3-o'clock Kollsman scale and ALT FAIL; rotating-card Compass with N/E/S/W, north wrap, heading bug and HDG FAIL; QNH 950–1050 hPa; heading bug 000–359°; GNSS latitude/longitude and reported horizontal-accuracy quality on Horizon and Compass; and permanent red SIM marking for synthetic data.

The Swift drawing is a native SwiftUI representation, not a pixel-for-pixel copy of the RGB565 framebuffer. Firmware/QEMU acceptance details remain in `../docs/SIMULATION.md` and `../docs/PROJECT_STATUS.md`.

Generate and run

```
cd ~/Documents/Xcode/ESP32-EFIS
git pull
cd simulator
brew install xcodegen          # only if needed
xcodegen generate
open ESP32EFISimulator.xcodeproj
```

Select an iPhone/iPad simulator and Run. The generated `.xcodeproj` is a build product; `project.yml` and Swift sources are maintained artefacts. Target is iOS 17+.

OTA scenario checks

Exercise auto-download ON, auto-download OFF/manual Download, Later, successful ACTIVATE & REBOOT, and first-boot failure/rollback. Also exercise Network configuration independently. A successful Swift scenario validates wording/state logic only; it does not validate ESP32 Wi-Fi, TLS, flash, power-loss recovery or bootloader rollback.

Safety and validation boundary

Simulator values must never become fallback values in an aircraft build. Aircraft firmware retains independent fail-obvious validity handling and must have bench simulation disabled. Swift/QEMU results are never physical sensor or aircraft validation. See `../docs/GNSS_DISPLAY.md`, `../docs/OTA_USER_SCENARIO.md`, `../docs/OTA_IMAGE_ADMIN.md` and `../docs/REMOTE_UPDATES.md`.

Required boot-menu parity — 26 September 2026

The Swift simulator is to mirror the adopted physical boot interaction: **START EFIS** (default), **FULL TEST**, and **FIRMWARE UPDATE**, navigated conceptually by rotary selection + push confirmation. Firmware Update must lead through the Wi-Fi connection dialog before the existing OTA flow. FULL TEST must model the documented component PASS/FAIL/NOT TESTED results and failure codes. Simulator implementation demonstrates workflow only and does not validate physical encoder, Wi-Fi or electrical tests.

support-service/README.md

Repository source: support-service/README.md

MicroSky Horizon Support Service

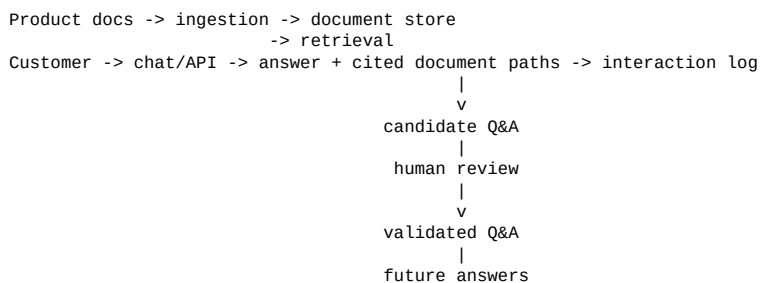
Dockerized support/RAG service for **MicroSky Horizon**.

Principle

The service may learn *what customers ask*, but it does **not autonomously turn model output into product truth**. Customer interactions create candidate Q&A records. A human reviewer checks the answer against versioned product documents before promoting it into the validated Q&A knowledge base.

This prevents a hallucination, misunderstanding or malicious customer message from becoming future support guidance.

Data flow



Current MVP

- FastAPI support API on loopback port 8093
- PostgreSQL 17 persistent store
- version/revision-aware document ingestion
- lexical document retrieval baseline
- interaction history and feedback
- repeated-question aggregation
- candidate Q&A queue
- explicit admin approval into validated Q&A
- conservative fallback when no validated answer exists
- read-only/non-root application container

Generated LLM answers are deliberately not enabled in v0.1. The next phase adds a provider adapter and grounded-answer prompt after the validation workflow and test corpus are in place.

Bring-up

```
cd support-service
cp .env.example .env
# set private DATABASE_URL, POSTGRES_PASSWORD and ADMIN_TOKEN
sudo docker compose config --quiet
sudo docker compose build support
sudo docker compose up -d
curl -fsS http://127.0.0.1:8093/healthz
```

Never commit .env, API keys, customer conversations or database dumps.

Product-document ingestion

The production ingestion job should read approved repository documentation at a specific Git commit, split it into useful sections, and submit each section with path + revision. Do not ingest arbitrary customer text as documentation.

Authoritative sources initially include README, BOM and relevant docs/ product/user/support documents. Internal secrets, signing material, private operational credentials and unrelated development logs must be excluded.

Learning/validation model

1. Store each question, answer, source paths, confidence and optional feedback.
2. Normalize repeated questions into a candidate.
3. Rank candidates by frequency, negative feedback and lack of an approved answer.
4. Reviewer sees question, draft, source passages and document revisions.
5. Reviewer edits/approves/rejects.
6. Only approved content enters `validated_qa`.
7. Revalidate an answer when its supporting product documentation materially changes.

Future versions should add semantic clustering/embeddings, source-line citations, answer-version history, rejection reasons, PII minimization/retention, abuse controls, authentication/RBAC and audit records.

Safety boundary

Horizon is a supplementary/non-primary flight instrument. Support answers concerning installation, operation, limitations, pressure/static systems, electrical interfaces, firmware or flight use must be grounded in current approved documentation and clearly distinguish development concepts from validated instructions.

The chatbot must never silently elevate an old design note, simulation result or customer statement into an approved operating instruction.

Grounded natural-language answers

The service now includes an **OpenAI-compatible chat-completions adapter**. Configure `LLM_BASE_URL`, `LLM_API_KEY` and `LLM_MODEL` privately in `.env`. The API key must never be committed.

For non-exact questions, Horizon support retrieves approved product documents and supplies only that evidence to the LLM. The system prompt requires polished natural-language answers grounded solely in those sources, inline source markers, explicit uncertainty when documentation is insufficient, and correct treatment of provisional/development material. Temperature is deliberately low.

Generated answers are recorded as interactions and Q&A **candidates**. They do not become validated answers until human approval. Exact approved Q&A continues to take precedence. If the LLM is unavailable, the service falls back to retrieved source material rather than fabricating an answer.

The provider interface is deliberately generic so the deployment can use an OpenAI-compatible hosted or local model without coupling the product knowledge architecture to one vendor. Before public deployment add provider-specific privacy/retention review, secret management, token/cost controls, streaming if desired, semantic retrieval and prompt-injection/red-team tests.

Semantic retrieval

The document store now uses PostgreSQL **pgvector**. At ingestion, approved document sections can be embedded through an OpenAI-compatible `/embeddings` endpoint and stored with their Git revision. At question time the question embedding is compared using cosine distance and the closest product-document sections are supplied to the grounded LLM. If the embedding service is absent or fails, retrieval falls back to lexical matching rather than making the support service unavailable.

Configure `EMBEDDING_BASE_URL`, `EMBEDDING_API_KEY`, `EMBEDDING_MODEL` and `EMBEDDING_DIMENSIONS` privately. They may share the LLM provider or point at a separate/local embedding service. Changing embedding dimensionality requires a database migration/re-index rather than simply editing the environment variable on an existing database.

Answer provenance and resolution feedback

The API now exposes an explicit `answer_status`: `validated`, `generated/retrieved but not yet validated`, `invalidated`, or `no validated answer`. Invalidated answers are retained as history and can be surfaced for context, but are visibly warned as not current approved guidance.

Every answer also returns the prompt **“Did this answer resolve your question?”**. The feedback endpoint stores the customer's resolution yes/no separately from the rating. Resolution data ranks knowledge gaps and answer usefulness; it never constitutes technical approval and cannot automatically promote a candidate into `validated_qa`.

Voice and keyboard interaction

Customer clients are designed to offer both keyboard/text and push-to-talk voice interaction. Speech is transcribed to visible/editable text and then uses the same `/v1/ask` pipeline; answers remain visible as text and may also be spoken by TTS. Keyboard remains available for exact identifiers and as a fallback. Raw audio is not retained by default. Answer provenance/status and resolution feedback remain visible regardless of interaction mode. Provider-neutral STT/TTS adapters are the next implementation step.

Customer web chat and speech

A responsive customer UI is now served at `/`. It supports keyboard questions, browser microphone capture, editable STT transcripts, visible answer-status badges, source paths, Yes/No resolution feedback and optional spoken answers. `/v1/transcribe` and `/v1/speech` use provider-neutral OpenAI-compatible audio endpoints configured by STT/TTS environment variables. Raw audio is processed in memory and is not written to the support database by this implementation. Exact technical speech transcripts must be checked by the customer before submission.

Runtime/browser validation is still pending. Production work includes authentication/abuse controls, CSP/security headers, accessibility testing, mobile/Safari microphone validation, provider privacy review, audio format compatibility, streaming/latency controls and automated tests.